Universidad Peruana de Ciencias Aplicadas

Ingeniería de Software

Periodo: 202620

1ACC0238 - Diseño de Experimentos de Ingeniería de Software

NRC: 9090

Docente: Juan Antonio Flores Moroco

Informe de Trabajo Final

Startup: Codexa-Team

Producto: RentiCar

Integrantes

u202314354 - Solís Campos, Estefano Sebastian

U202311086 - Landa Ortiz, Sergio Javier

U20231D400 - Angeles Rojas, Sheila Jasmin

U202019409 - Linares Bernable, Cesar Alejandro

U202313403 - Via Luna, Bruce

2026-02

Registro de Versiones del Informe

Versión	Fecha	Autor	Descripción de modificación
1.0	10/09/2026	Bruce Via Luna	Creacion del documento
1.1	13/09/2026	Todos los intengrantes	Registro de entrevisas
1.2	14/09/2026	Todos los intengrantes	Creacion y pulimiento de Diagramas
1.3	15/09/2026	Todos los intengrantes	Documentacion de despliegues

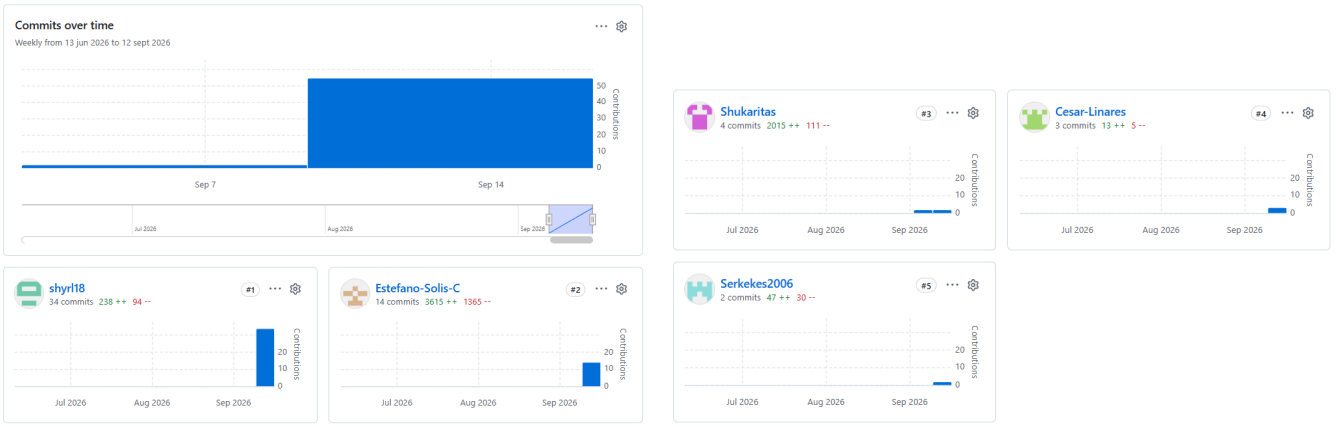
Project Report Collaboration Insights

AUTHORS:

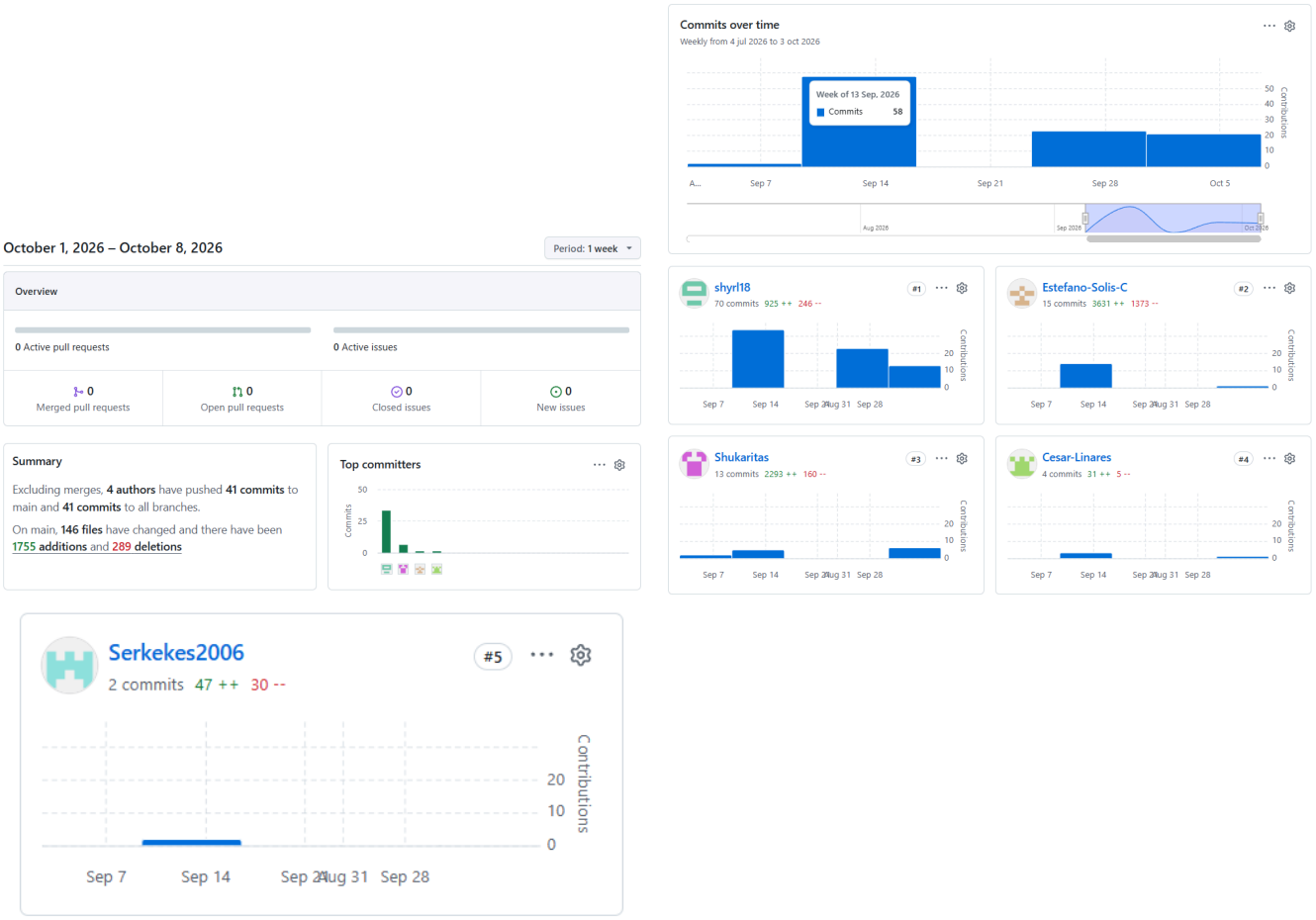
- Bruce Via Luna
- Cesar Alejando Linares Bernable
- Sheila Jasmin Angeles Rojas
- Estefano Sebastian Solis Campos
- Sergio Javier Landa Ortiz

URL del repositorio del reporte
https://github.com/Codexa-Team/Project-Report

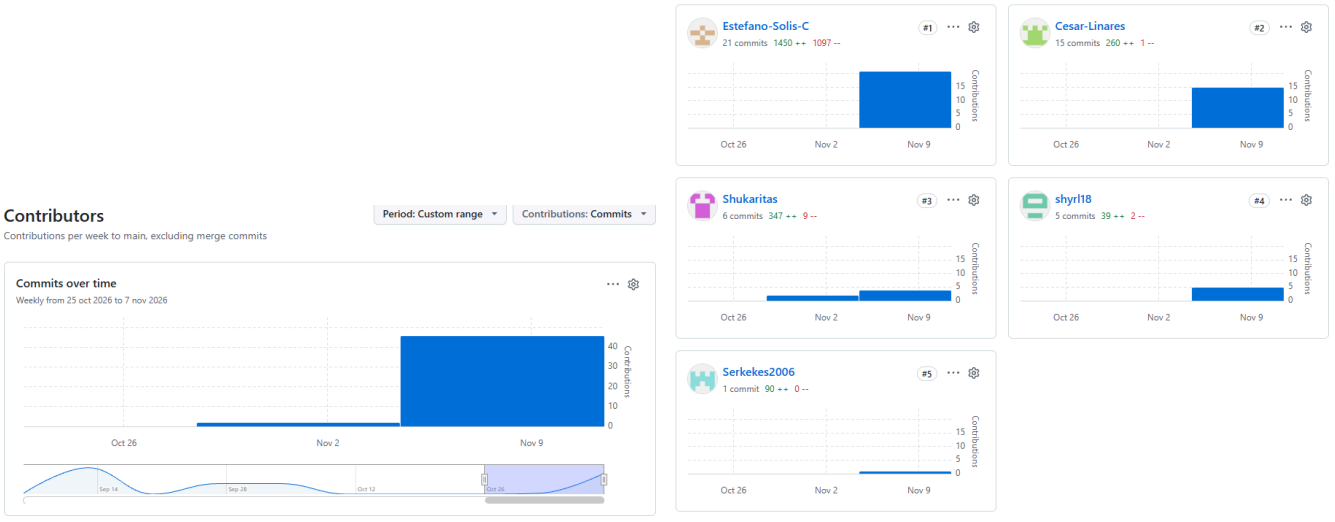
AV1:



TB1:



AV2:



Contenido

- Student Outcome
- Part I: As-Is Software Project
- Capítulo I: Introducción
  - 1.1. Startup Profile
    - 1.1.1. Descripción de la Startup
    - 1.1.2. Perfiles de integrantes del equipo
  - 1.2. Solution Profile
    - 1.2.1. Antecedentes y problemática
    - 1.2.2. Lean UX Process
      - 1.2.2.1. Lean UX Problem Statements
      - 1.2.2.2. Lean UX Assumptions
      - 1.2.2.3. Lean UX Hypothesis Statements
      - 1.2.2.4. Lean UX Canvas

    - 1.3. Segmentos Objetivo

- Capítulo II: Requirements Elicitation & Analysis
    - 2.1. Competidores
        - 2.1.1. Análisis competitivo
        - 2.1.2. Estrategias y tácticas frente a competidores
    - 2.2. Entrevistas
        - 2.2.1. Diseño de entrevistas
        - 2.2.2. Registro de entrevistas
        - 2.2.3. Análisis de entrevistas
    - 2.3. Needfinding
        - 2.3.1. User Personas
        - 2.3.2. User Task Matrix
        - 2.3.3. User Journey Mapping
        - 2.3.4. Empathy Mapping
        - 2.3.5. As-is Scenario Mapping
    - 2.4. Ubiquitous Language
- Capítulo III: Requirements Specification
    - 3.1. To-Be Scenario Mapping
    - 3.2. User Stories
    - 3.3. Product Backlog
    - 3.4. Impact Mapping
- Capítulo IV: Product Design
    - 4.1. Style Guidelines
        - 4.1.1. General Style Guidelines
        - 4.1.2. Web Style Guidelines
        - 4.1.3. Mobile Style Guidelines
            - 4.1.3.1. iOS Mobile Style Guidelines
            - 4.1.3.2. Android Mobile Style Guidelines
    - 4.2. Information Architecture
        - 4.2.1. Organization Systems
        - 4.2.2. Labeling Systems
        - 4.2.3. SEO Tags and Meta Tags
        - 4.2.4. Searching Systems
        - 4.2.5. Navigation Systems
    - 4.3. Landing Page UI Design
        - 4.3.1. Landing Page Wireframe
        - 4.3.2. Landing Page Mock-up
    - 4.4. Mobile Applications UX/UI Design
        - 4.4.1. Mobile Applications Wireframes
        - 4.4.2. Mobile Applications Wireflow Diagrams
        - 4.4.3. Mobile Applications Mock-ups
        - 4.4.4. Mobile Applications User Flow Diagrams
    - 4.5. Mobile Applications Prototyping
        - 4.5.1. Android Mobile Applications Prototyping
        - 4.5.2. iOS Mobile Applications Prototyping
    - 4.6. Web Applications UX/UI Design
        - 4.6.1. Web Applications Wireframes
        - 4.6.2. Web Applications Wireflow Diagrams
        - 4.6.3. Web Applications Mock-ups
        - 4.6.4. Web Applications User Flow Diagrams
    - 4.7. Web Applications Prototyping
    - 4.8. Domain-Driven Software Architecture
        - 4.8.1. Software Architecture Context Diagram
        - 4.8.2. Software Architecture Container Diagrams
        - 4.8.3. Software Architecture Components Diagrams
    - 4.9. Software Object-Oriented Design
        - 4.9.1. Class Diagrams
        - 4.9.2. Class Dictionary
    - 4.10. Database Design
        - 4.10.1. Relational/Non-Relational Database Diagram
- Capítulo V: Product Implementation
    - 5.1. Software Configuration Management
        - 5.1.1. Software Development Environment Configuration
        - 5.1.2. Source Code Management
        - 5.1.3. Source Code Style Guide & Conventions

        - 5.1.4. Software Deployment Configuration
    - 5.2. Product Implementation & Deployment.
        - 5.2.1. Sprint Backlogs.
        - 5.2.2. Implemented Landing Page Evidence
        - 5.2.3. Implemented Frontend-Web Application Evidence
        - 5.2.4. Acuerdo de Servicio - SaaS
        - 5.2.5. Implemented Native-Mobile Application Evidence
        - 5.2.6. Implemented RESTful API and/or Serverless Backend Evidence
        - 5.2.7. RESTful API documentation
        - 5.2.8. Team Collaboration Insights
    - 5.3. Video About-the-Product.

- Capítulo VI: Product Verification & Validation
    - 6.1. Testing Suites & Validation
        - 6.1.1. Core Entities Unit Tests.
        - 6.1.2. Core Integration Tests.
        - 6.1.3. Core Behavior-Driven Development
        - 6.1.4. Core System Tests
    - 6.2. Static testing & Verification
        - 6.2.1. Static Code Analysis
            - 6.2.1.1. Coding standard & Code conventions
            - 6.2.1.2. Code Quality & Code Security
        - 6.2.2. Reviews
    - 6.3. Validation Interviews
        - 6.3.1. Diseño de Entrevistas
        - 6.3.2. Registro de Entrevistas
        - 6.3.3. Evaluaciones según heurísticas
    - 6.4. Auditoría de Experiencias de Usuario
        - 6.4.1. Auditoría realizada
            - 6.4.1.1. Información del grupo auditado
            - 6.4.1.2. Cronograma de auditoría realizada
            - 6.4.1.3. Contenido de auditoría realizada
        - 6.4.2. Auditoría recibida
            - 6.4.2.1. Información del grupo auditor
            - 6.4.2.2. Cronograma de auditoría recibida
            - 6.4.2.3. Contenido de auditoría recibida
            - 6.4.2.4. Resumen de modificaciones para subsanar hallazgos

- Capítulo VII: DevOps Practices
    - 7.1. Continuous Integration
        - 7.1.1. Tools and Practices.
        - 7.1.2. Build & Test Suite Pipeline Components.
    - 7.2. Continuous Delivery
        - 7.2.1. Tools and Practices
        - 7.2.2. Stages Deployment Pipeline Components
    - 7.3. Continuous deployment
        - 7.3.1. Tools and Practices
        - 7.3.2. Production Deployment Pipeline Components
    - 7.4. Continuous Monitoring
        - 7.4.1. Tools and Practices
        - 7.4.2. Monitoring Pipeline Components
        - 7.4.3. Alerting Pipeline Components
        - 7.4.4. Notification Pipeline Components

- Capítulo VIII: Experiment-Driven Development
    - 8.1. Experiment Planning
        - 8.1.1. As-Is Summary
        - 8.1.2. Raw Material: Assumptions, Knowledge Gaps, Ideas, Claims
        - 8.1.3. Experiment-Ready Questions
        - 8.1.4. Question Backlog
        - 8.1.5. Experiment Cards
    - 8.2. Experiment Design
        - 8.2.1. Hypotheses
        - 8.2.2. Domain Business Metrics
        - 8.2.3. Measures
        - 8.2.4. Conditions
        - 8.2.5. Scale Calculations and Decisions
        - 8.2.6. Methods Selection
        - 8.2.7. Data Analytics: Goals, KPIs and Metrics Selection

        - 8.2.8. Web and Mobile Tracking Plan
    - 8.3. Experimentation
        - 8.3.1. To-Be User Stories
        - 8.3.2. To-Be Product Backlog

- Conclusiones
- Bibliografía
- Anexos

Student Outcome

El curso contribuye al cumplimiento del Student Outcome ABET:

ABET – EAC - Student Outcome 4

Criterio: La capacidad de reconocer responsabilidades éticas y profesionales en situaciones de ingeniería y hacer juicios informados, que deben considerar el impacto de las soluciones de ingeniería en contextos globales, económicos, ambientales y sociales.

Criterio específico	Acciones realizadas	Conclusiones
Reconoce responsabilidad ética y profesional en situaciones de ingeniería de software.	Estefano Solis: AV1: Documenté en el informe la gestión y seguridad de cuentas en RentiCar (diferenciación de roles entre arrendador y arrendatario, autenticación y contraseñas protegidas), redacté el Acuerdo de Servicio SaaS sobre la privacidad y uso responsable de la información de los usuarios, y aseguré el citado correcto de las herramientas, imágenes y recursos externos documentados. TB1: Desarrollé e implementé pruebas unitarias (unit tests) para los servicios y componentes clave del backend, garantizando la fiabilidad de la lógica de negocio y estándares de calidad de software. Asimismo, apoyé en la verificación del pipeline de Integración Continua (CI) para asegurar ejecuciones automáticas correctas, y colaboré en la estructuración, redacción y consolidación técnica del informe del proyecto. AV2: Configuré Checkstyle en el backend y corregí el código Java hasta dejarlo sin violaciones ni fallos en las pruebas, además de corregir hallazgos previos en SonarQube y ordenar en inglés la suite E2E con Selenium y Cucumber. También integré Google Analytics 4 en el frontend y audité el informe para usar solo capturas reales, quitar métricas sin respaldo y aclarar lo pendiente. Bruce Via: AV1: Analice e identifique los usuarios objetivos para proporcionarles una solucion basada en la ingenieria de software, redacte y registre entrevistas con ellos para identificar sus necesidades. TB1: Realice en colaboracion con mis compañeros las diferentes pruebas unitarias y de integracion, ademas de la declaracion de pruebas del tipo .feature y organice y designe las actividades de mi compañeros AV2: Analice e identifique los procesos, herramientas y necesidades para los desarrollos continuo y la entrega continua del proyecto junto con sus funciones. Sheyla Rojas: AV1: Durante el desarrollo del proyecto, participé en la realización de la entrevista y en el análisis de la información obtenida, considerando las necesidades y características de los usuarios. Por otro lado, apoyé en el diseño de la versión móvil de la aplicación, procurando que la información presentada fuera clara, accesible y adecuada para los usuarios. TB1: Realice el Mobile Applications UX/UI Design, mobile Applications Prototyping, donde aseguramos los estandares de accesibilidad y la inclusion de usuarios. Asimismo definimos la arquitectura, los componentes y la	AV1: El equipo se centró en la investigación de usuarios mediante múltiples entrevistas para identificar problemas y necesidades antes de definir las funcionalidades clave. Con esta base, propusieron soluciones de software y diseñaron una interfaz móvil accesible y clara. Paralelamente, aseguraron los aspectos técnicos y legales del proyecto documentando la gestión y seguridad de cuentas, redactando el Acuerdo de Servicio SaaS para la privacidad de datos y garantizando la correcta citación de todos los recursos utilizados. TB1: En esta etapa, el equipo demostró un alto nivel de responsabilidad profesional y apego a los estándares de calidad de la industria mediante la adopción de metodologías de prueba formales (TDD con JUnit 5/Mockito y BDD con Karate/Gherkin). Se verificó exhaustivamente la estabilidad del backend y la correcta articulación de los bounded contexts, complementándolo con la automatización del pipeline de Integración Continua (CI con Jenkins) para asegurar compilaciones limpias y confiables. Asimismo, se garantizó la inclusión y accesibilidad en el diseño y desarrollo de la aplicación móvil nativa, manteniendo una documentación técnica transparente y rigurosa en el informe final. AV2: Se fortaleció la calidad, seguridad y continuidad del proyecto mediante la configuración de Checkstyle, SonarQube y Docker, corrigiendo errores y mejorando la mantenibilidad del código. También se implementó Google Analytics 4, se organizaron y validaron las pruebas E2E con Selenium y Cucumber, y se analizaron los procesos relacionados con la integración y entrega continua. Asimismo, se planificaron experimentos mediante hipótesis, criterios de éxito, preguntas y Experiment Cards para validar las suposiciones del negocio. Se realizaron entrevistas para verificar los requerimientos de la solución y se desarrolló integralmente la aplicación móvil del segundo segmento, incluyendo su arquitectura, componentes nativos, interfaces funcionales y sincronización con el backend. Finalmente, se revisaron y consolidaron la documentación y los entregables, garantizando coherencia técnica, responsabilidad profesional y cumplimiento de los estándares de calidad.

Criterio específico	Acciones realizadas	Conclusiones
	especificación del pipeline de Integración Continua (CI), el cual demostraron la responsabilidad profesional necesaria para la correcta ejecución y el alcance de las exigencias de los estandares de calidad AV2: Asegure la responsabilidad profesional y el compromiso con los estándares de calidad mediante la dockerización del entorno y la implementación de SonarQube para el análisis estático, la seguridad y la mantenibilidad del código fuente. Asimismo, se aportaron criterios técnicos y puntos clave en las fases de preview, demostrando las buenas prácticas de desarrollo y la responsabilidad necesaria para garantizar la correcta ejecución del proyecto. Cesar Linares: AV1: Realizar entrevistas a usuarios reales para identificar problemas antes de definir funcionalidades. TB1: Identificó, definió y documentó las herramientas, tecnologías y prácticas empleadas en la implementación de la Integración Continua (CI), abarcando desde frameworks de pruebas unitarias y de integración hasta metodologías como TDD y BDD. Asimismo, determinó el propósito de cada una dentro del proceso de desarrollo, asegurando su alineación con los requisitos funcionales del negocio y con los criterios técnicos de calidad establecidos para el proyecto. AV2: Planifiqué y diseñé los experimentos (secciones 8.1 y 8.2.1), definiendo las hipótesis, los criterios de éxito y las condiciones de control para validar las suposiciones del negocio de manera ética y objetiva. Asimismo, estructuré el backlog de preguntas y las Experiment Cards, asegurando que cada experimento tuviera un propósito claro y un diseño metodológico riguroso para la toma de decisiones informadas. Sergio Landa: AV1: Participe activamente en el proyecto, realizando entrevistas y los analisis de las entrevistas a nuestros usuarios, y recaudando la informacion necesaria en necesidades y caracteristicas para optar por mejoras considerables en el proyecto. TB1: Diseñe, desarrolló y documente en su totalidad la aplicación móvil del proyecto, definiendo las herramientas, tecnologías y prácticas empleadas para su implementación, abarcando desde el diseño de la interfaz y experiencia de usuario (UI/UX) hasta la arquitectura móvil y la integración con servicios backend. Asimismo, determinó el propósito de cada componente dentro del proceso de desarrollo, asegurando su alineación con los requisitos funcionales del negocio y con los criterios técnicos de calidad, rendimiento y usabilidad establecidos AV2: Conduje las entrevistas de investigación y usuario para validar los requerimientos de la solución, y desarrollé por completo la aplicación móvil del proyecto correspondiente al segundo segmento. Implementé la arquitectura y componentes nativos/móviles asegurando la calidad del código, integré las interfaces de usuario funcionales y aseguré la correcta sincronización con el backend. Asimismo, audité la documentación y los entregables del segmento para mantener la coherencia y el rigor técnico en todo el equipo.	
Emite juicios informados considerando el impacto de las soluciones de	Estefano Solis: AV1: Analicé y documenté en el informe el impacto de RentiCar: en lo económico (permitir ingresos a propietarios con autos poco usados y precios justos a arrendatarios), en lo social (fomentar la confianza mutua	AV1: A partir de las entrevistas, el equipo identificó la necesidad de una plataforma centralizada de alquiler de vehículos y documentó el impacto socioeconómico y ambiental del proyecto. Con esta base estratégica, diseñaron una solución digital accesible y escalable —optimizando la landing page y la interfaz móvil— para garantizar transacciones seguras,

Criterio específico	Acciones realizadas	Conclusiones
ingeniería de software en contextos globales, económicos, ambientales y sociales.	con calificaciones y reseñas para evitar tratos informales en redes sociales), en lo ambiental (promover la movilidad compartida aprovechando autos ya existentes) y en lo global (plantear un modelo digital escalable a otras ciudades). TB1: Evalué el impacto de la implementación de pruebas unitarias y el soporte en el pipeline de CI: en lo económico, reduciendo costos de mantenimiento mediante la detección temprana de incidencias; en lo social, garantizando una solución robusta y confiable para los usuarios; en lo ambiental, optimizando los recursos computacionales durante la integración; y en lo global, adhiriéndose a estándares de la industria del software. Además, documenté y sustenté estas consideraciones dentro del informe del proyecto. AV2: Evalué el impacto de la verificación estática, las pruebas E2E y el monitoreo en esta entrega: en lo económico, automatizar el estilo y los flujos críticos reduce el trabajo y errores tardíos; en lo social, probar reservas, vehículos y telemetría entrega una aplicación más clara y confiable; en lo ambiental, usar analítica ligera sin desplegar servidores extra evita consumo innecesario; y en lo global, adoptar Google Java Style, Cucumber y GA4 facilita el mantenimiento del sistema. Bruce Via: AV1: Realice las entrevistas identificando necesidades y situaciones para el planteamiento de una solucion tomando en cuenta aspectos globales y economicos de ambos espectros. TB1: Evalue las capacidades del backend para definir las diferentes pruebas basadas en sus funcionalidades, utilidad y aporte al proyecto dentro de su desarrollo como desarrollador DevOps AV2: Identifique y documente la arquitectura tecnica de los diferentes procesos ademas de comunicar con otros grupos sobre la auditoria dada a nuestros proyectos de forma mutua Sheyla Rojas: AV1: Durante las entrevistas, identifiqué que la falta de una aplicación centralizada para reservar y publicar vehículos genera pérdida de tiempo y mayores costos para los usuarios. Para mitigar este impacto social y económico, adapté el diseño web a dispositivos móviles (iOS y Android). Basándome en la información recopilada, diseñé una interfaz sencilla y accesible que facilita el acceso al servicio, garantizando una experiencia eficiente que resuelve las necesidades detectadas. TB1: Analize y diseñe una interfaz movil que cumpla con toda la infraestructura necesaria para que tenga un impacto socioeconomico positivo en los usuarios al momento de rentar o alquilar un carro. Por otro lado la definición de la arquitectura del pipeline de CI demostró la eficiencia tecnica del proyecto AV2: Participe en el análisis y soporte de la interfaz de Renticar para que el usuario pueda tener una experiencia de alta calidad y le genere un impacto economicamente y socialmente positiva. La incorporación de prácticas de containerización con Docker y la validación continua de la calidad técnica mediante SonarQube, complementadas con revisiones y aportes en preview, evidenciaron la escalabilidad y la robustez de la solución frente a requerimientos de alto estándar.	fomentar la confianza y mejorar la experiencia de los usuarios. TB1: El equipo analizó y sustentó el impacto multidimensional de las soluciones implementadas: en lo económico, la automatización del testing y la integración continua reducen costos operativos y fallos tempranos en producción, mientras que la solución móvil amplía la competitividad y alcance comercial del modelo P2P; en lo social, se entrega un producto accesible, confiable y seguro que fomenta la confianza ciudadana en la movilidad colaborativa; en lo ambiental, se optimizan los recursos computacionales del pipeline y se fomenta la digitalización de contratos para reducir el uso de papel; y en lo global, se adoptaron estándares y arquitecturas escalables que permiten la interoperabilidad y alineación con las mejores prácticas internacionales de ingeniería de software. AV2: Se evaluó el impacto económico, social, ambiental y global de las decisiones técnicas y metodológicas aplicadas al proyecto RentiCar. La automatización de estándares de código, las pruebas E2E, el monitoreo con Google Analytics 4, la contenerización con Docker y el análisis mediante SonarQube permitieron reducir errores, costos de mantenimiento y consumo innecesario de recursos, además de fortalecer la calidad, seguridad y escalabilidad de la solución. También se documentó la arquitectura técnica y se compartieron resultados de auditoría con otros equipos, promoviendo la mejora continua y la adopción de buenas prácticas. La planificación de experimentos y la validación de hipótesis facilitaron la priorización de funcionalidades de alto valor, mientras que las entrevistas con usuarios y el desarrollo de la aplicación móvil contribuyeron a crear una experiencia más accesible, confiable e intuitiva. Finalmente, la consolidación de la documentación técnica y la coordinación del equipo favorecieron el mantenimiento y crecimiento futuro del proyecto.

Criterio específico	Acciones realizadas	Conclusiones
	Cesar Linares: AV1: Realizó una entrevista del segmento 2 (arrendatario de vehículo) y añadió más información para el análisis de las entrevistas. TB1: Evalué cómo prácticas como la Integración Contínua, TDD y BDD generan impactos en distintos contextos: económicamente, al reducir costos de mantenimiento y aumentar la competitividad mediante entregas más rápidas; socialmente, al producir software más confiable, accesible y alineado con las necesidades del negocio; ambientalmente, al optimizar el uso de recursos computacionales y evitar reprocesos; y globalmente, al adoptar estándares internacionales que facilitan la colaboración distribuida y el cumplimiento normativo. AV2: Evalué el impacto de las decisiones de experimentación en el contexto del negocio: en lo económico, la validación de hipótesis permite priorizar inversiones en funcionalidades de alto impacto (como el IoT y la verificación de perfiles), reduciendo el riesgo de desarrollar características de bajo valor; en lo social, los experimentos orientados a la confianza y la usabilidad buscan mejorar la experiencia de los usuarios y fomentar la adopción de la plataforma; en lo ambiental, la selección de herramientas de analítica ligera y la optimización de los experimentos evitan el consumo innecesario de recursos; y en lo global, la adopción de metodologías de experimentación (A/B testing) alinea el proyecto con estándares internacionales de desarrollo de productos digitales. Sergio Landa: AV1: Realice una entrevista para ofrecer mayor cobertura de informacion al proyecto, realice cambios importantes para darle un nuevo enfoque a nuestra landing page, y el as is- to be, que nos ayudara a tomar escenarios posibles para tener mas en claro nuestra proyeccion de proyecto. TB1: Evalué cómo el desarrollo integral de la aplicación móvil genera impactos en distintos contextos: económicamente, al abrir un canal directo y eficiente que aumenta la competitividad y el alcance del negocio; socialmente, al proveer una solución tecnológica intuitiva, accesible y alineada con la necesidad de movilidad de los usuarios; ambientalmente, al fomentar la digitalización de servicios que reduce el uso de recursos físicos y papel; y globalmente, al adoptar estándares internacionales de diseño móvil (UI/UX) y desarrollo que facilitan su escalabilidad y futura distribución en tiendas de aplicaciones. AV2: Evalué el impacto de las entrevistas de usuario, el desarrollo integral de la aplicación móvil y la redacción del informe técnico en esta entrega: en lo económico, la validación temprana con usuarios y la implementación eficiente de las funcionalidades del segundo segmento evitan costos de refactorización tardía; en lo social, estructurar una interfaz móvil intuitiva y confiable garantiza una experiencia accesible y de valor para los usuarios finales; en lo ambiental, optimizar el rendimiento y la arquitectura del desarrollo móvil reduce la sobrecarga computacional innecesaria; y en lo global, consolidar la documentación del informe y la sincronización con el equipo facilita la escalabilidad y el mantenimiento del proyecto a largo plazo.	

Part I: As-Is Software Project

Capítulo I: Introducción

1.1. Startup Profile

En esta sección presentamos a Codexa-Team, nuestra startup tecnológica, así como el sector en el que buscamos incursionar, la problemática identificada y la solución propuesta. Este apartado tiene como finalidad brindar una visión clara del proyecto y de los objetivos que orientan a nuestro equipo.

1.1.1. Descripción de la Startup

Codexa-Team es una startup fundada por estudiantes de la carrera de Ingeniería de Software, orientada al desarrollo de soluciones digitales innovadoras. Para el presente ciclo académico, hemos planteado el desarrollo de una aplicación web denominada RentiCar, una plataforma destinada al alquiler de vehículos en el Perú.

Actualmente, el mercado de alquiler de autos en el país enfrenta limitaciones: procesos poco digitalizados, escasa confianza entre arrendadores y arrendatarios, y la ausencia de un sistema que centralice ofertas de forma segura y sencilla. Frente a esta problemática, RentiCar busca convertirse en un espacio confiable que conecte a propietarios de vehículos con clientes interesados en alquilarlos, garantizando una experiencia rápida, segura y transparente.

Mision:

Facilitar el acceso al alquiler de vehículos en el Perú mediante una plataforma digital segura, confiable y de fácil uso, que conecte de manera transparente a propietarios y clientes.

Vision:

Convertirnos en la plataforma líder de alquiler de vehículos en el Perú y expandirnos hacia Latinoamérica, ofreciendo una experiencia innovadora, eficiente y 100% confiable, que transforme la forma en que las personas acceden a soluciones de movilidad.

1.1.2. Perfiles de integrantes del equipo

Perfil	Foto
Nombre: Bruce Via Luna **Carrera:** Ingeniería de Software **Descripcion:** Soy Bruce Via Luna de código u202313403, entusiasta sobre el control y manejo de bases de datos con conocimientos en C# y C++.	
Nombre: - Cesar Alejando Linares Bernable **Carrera:** Ingenieria de Software **Descripcion:** Soy Cesar Alejandro Linares Bernable, estudiante de la carrera de Ingeniería de Software en la UPC, sede San Miguel, tengo 23 años y estoy cursando el séptimo ciclo académico. Me considero una persona adaptativa, perseverante y comprometida con lo que me propongo. En este proyecto tengo como objetivo buscar múltiples soluciones que beneficien a todo el grupo, por experiencia propia suelo trabajar de manera colaborativa y eficaz.	
Nombre: Landa Ortiz, Sergio Javier **Carrera:** Ingenieria de Software **Descripcion:** Soy un estudiante de software, cursando actualmente el 8vo ciclo, la carrera me interesa mucho ya que siempre aprendo cosas nuevas cada dia, las tecnologias son tantas que nunca te aburres, me considero una persona responsable y me gusta trabajar en equipo	
Nombre: Solis Campos, Estefano Sebastian **Carrera:** Ingenieria de Software **Descripcion:** Soy un estudiante apasionado de la carrera de Ingeniería de Software en la UPC. Me entusiasma aplicar mis conocimientos en el desarrollo de soluciones digitales innovadoras, diseño de experiencia de usuario (UX/UI) y gestión de requerimientos. Comprometido con el rigor técnico, la excelencia operativa y el aprendizaje continuo en ingeniería.	
Nombre: Sheila Jasmin Angeles Rojas **Carrera:** Ingeniería de Software **Descripcion:** Soy Sheila Angeles, estudiante de la carrera de Ingeniería de Software. Me caracterizo por ser una persona responsable, organizada y comprometida con cada tarea que se me asigna.	

1.2. Solution Profile

1.2.1. Antecedentes y problemática

En el Perú, el mercado de alquiler de vehículos se encuentra rezagado frente a otros servicios de movilidad, debido principalmente a la informalidad de las transacciones, la ausencia de plataformas centralizadas confiables y la desconfianza existente entre propietarios y clientes. Actualmente, la mayoría de operaciones se realiza mediante redes sociales, anuncios clasificados o contactos directos, lo que genera serios riesgos de fraude, pérdida de dinero, incumplimiento en los acuerdos y falta de respaldo legal. A esto se suma que, en la mayoría de los casos, los procesos son poco prácticos y requieren múltiples pasos presenciales, dificultando el acceso a un servicio rápido y eficiente.

Si bien existen alternativas en el mercado internacional, su cobertura en el Perú es limitada, costosa o no responde a la realidad del usuario promedio. Esto deja un vacío en el sector, el cual representa una oportunidad para el desarrollo de una solución digital innovadora. Por ello, resulta fundamental plantear una propuesta que brinde seguridad, transparencia y facilidad de uso, y que al mismo tiempo fomente la confianza entre los actores involucrados. Con este proyecto buscamos no solo atender una necesidad inmediata de movilidad, sino también impulsar la digitalización del sector y promover nuevas oportunidades económicas para los propietarios de vehículos.

A continuación, se presenta un análisis detallado de esta problemática empleando la metodología "5W2H":

- **Who (Quién):** Por un lado, se encuentran los propietarios de vehículos, personas que muchas veces tienen un automóvil que no utilizan de forma constante y buscan una manera práctica y segura de generar ingresos adicionales a través de su alquiler. Por otro lado, están los clientes que requieren un vehículo de manera temporal, ya sea por motivos de viaje, trabajo, emergencias o simplemente por no contar con un automóvil propio. Ambos grupos comparten una necesidad común: acceder a un sistema que les permita conectarse de forma confiable y sencilla, sin tener que recurrir a procesos engorrosos o inseguros.
- **What (Qué):** Se identifica la urgencia de contar con una plataforma que funcione como un puente seguro y transparente entre propietarios y clientes. Actualmente, la falta de un servicio centralizado hace que el proceso de alquiler sea desordenado y arriesgado, lo cual limita el crecimiento del sector. Una solución digital debe resolver estos problemas principales: la confianza entre ambas partes, la formalización de los acuerdos y la rapidez en el proceso de búsqueda y alquiler.
- **Where (Dónde):** El problema se concentra principalmente en el Perú, especialmente en las grandes ciudades como Lima, Arequipa y Trujillo, donde existe una mayor demanda de movilidad. Sin embargo, la problemática no es exclusiva de estas zonas; en regiones más pequeñas la necesidad es incluso mayor, ya que los servicios de alquiler tradicionales suelen ser escasos o inexistentes. Esto abre la posibilidad de que, una vez consolidada la solución en el ámbito nacional, se pueda escalar hacia otros países de Latinoamérica que presentan contextos similares.
- **When (Cuándo):** Con el avance de la digitalización y el creciente uso de plataformas para todo tipo de servicios, los usuarios esperan soluciones inmediatas y seguras. Sin embargo, el alquiler de vehículos aún no se ha adaptado a estas expectativas de forma integral en el Perú. Por lo tanto, el momento de implementar una plataforma como RentiCar es ahora, antes de que la brecha entre lo que el usuario espera y lo que el mercado ofrece se amplíe aún más.
- **Why (Por qué):** Los clientes enfrentan altos niveles de incertidumbre al alquilar un vehículo, ya que no cuentan con garantías ni respaldo en caso de inconvenientes. Los propietarios, por su parte, ven limitado su potencial de generar ingresos debido al miedo al incumplimiento de acuerdos o al mal uso de sus vehículos. Una plataforma digital que brinde confianza, formalidad y simplicidad responde directamente a esta necesidad, ofreciendo beneficios tangibles a ambos actores y contribuyendo a mejorar el ecosistema de movilidad en el país.
- **How (Cómo):** Esta plataforma permitirá a los propietarios registrar sus vehículos, establecer condiciones de alquiler y gestionar solicitudes de forma ordenada. Del mismo modo, los clientes podrán buscar, comparar y reservar autos de manera ágil, contando con herramientas de validación de usuarios, contratos digitales y sistemas de monitoreo IoT en tiempo real. De esta manera, se garantizará una experiencia fluida y confiable para ambas partes.
- **How Much (Cuánto):** El modelo de negocio propuesto se basa en un esquema de comisión porcentual por cada transacción formalizada dentro de la plataforma. Esto implica que la barrera de entrada y los costos iniciales de adopción para los usuarios serán bajos, favoreciendo la masificación del servicio. La inversión principal en esta primera etapa se focaliza en el desarrollo tecnológico y en la adquisición de usuarios.

1.2.2. Lean UX Process

1.2.2.1. Lean UX Problem Statements

El proceso de alquiler de vehículos en el Perú se caracteriza por la fragmentación, la falta de transparencia y los altos niveles de desconfianza entre las partes. Los propietarios de automóviles que desean rentabilizar sus vehículos cuando están en desuso carecen de una plataforma formal con verificación estricta y garantías contra siniestros o apropiación ilícita. Por su parte, los arrendatarios que buscan una solución de movilidad temporal se enfrentan a depósitos en efectivo desproporcionados, contratos en papel precarios y vehículos cuyo estado mecánico real y trazabilidad telemática se desconocen.

Nuestro producto busca cerrar esta brecha mediante el desarrollo de una plataforma digital confiable, diseñada para ofrecer procesos de verificación de identidad con roles (`ROLE_RENTER` y `ROLE_OWNER`), contratos digitales, reservas automáticas y monitoreo IoT en tiempo real.

Sabremos que tenemos éxito cuando logremos un incremento continuo en la cantidad de vehículos publicados, reservas confirmadas y una alta tasa de satisfacción en las calificaciones posteriores al alquiler.

1.2.2.2. Lean UX Assumptions

1. **Business Assumptions:** Asumimos que en el Perú existe un mercado desatendido con alta demanda para el alquiler vehicular P2P digitalizado. Los propietarios estarán dispuestos a listar sus vehículos si cuentan con garantías contractuales y monitoreo telemático en tiempo real.
2. **Business Outcome Assumptions:** Asumimos que una plataforma intuitiva y segura incrementará la recurrencia de reservas y permitirá consolidar un flujo sostenible de comisiones por alquiler, posicionando a RentiCar como líder en movilidad colaborativa.
3. **User Assumptions:** Asumimos que tanto arrendadores como arrendatarios cuentan con dispositivos conectados y valoran la inmediatez de la confirmación digital de reservas frente a las llamadas y trámites tradicionales.
4. **User Outcome Assumptions:** Asumimos que los propietarios obtendrán ingresos constantes con tranquilidad patrimonial, y que los arrendatarios accederán a tarifas transparentes y vehículos seguros monitoreados por GPS.
5. **Feature Assumptions:** Asumimos que un MVP enfocado en autenticación segura JWT, catálogo reactivo de vehículos, contratos de reserva con calendario dinámico, monitoreo telemático IoT y sistema de reseñas de 5 estrellas validará fehacientemente la propuesta de valor.

1.2.2.3. Lean UX Hypothesis Statements

- **Hypothesis 1:** Creemos que si proporcionamos un registro con verificación de roles y contratos digitales, los propietarios de vehículos incrementarán su disposición a publicar sus unidades en un 40% en comparación con canales informales.
- **Hypothesis 2:** Creemos que si permitimos a los arrendatarios filtrar vehículos por marca, precio y fechas con cálculo automático de tarifas, el tiempo promedio para completar una reserva se reducirá a menos de 5 minutos.
- **Hypothesis 3:** Creemos que si integramos telemetría IoT en tiempo real (coordenadas GPS, odómetro y nivel de combustible), el 90% de los propietarios reportará mayor tranquilidad respecto a la seguridad de sus unidades durante el alquiler.
- **Hypothesis 4:** Creemos que si implementamos un sistema público de reseñas y calificaciones de 1 a 5 estrellas posterior a cada reserva concluida, se generará un círculo virtuoso de reputación que aumentará la tasa de conversión en un 25%.
- **Hypothesis 5:** Creemos que una Landing Page atractiva y responsiva con soporte multidioma (i18n) captará tráfico orgánico de turistas y profesionales urbanos, impulsando el registro inicial en la plataforma.

1.2.2.4. Lean UX Canvas

1. Business Problem	2. Solution Ideas	3. Business Outcomes

El alquiler vehicular en el Perú predomina en la informalidad, generando desconfianza, riesgo de fraudes, falta de garantías legales y pérdida de tiempo. La ausencia de una solución centralizada y segura limita el desarrollo del mercado P2P.	**- Plataforma web P2P con autenticación JWT y roles diferenciados. - Catálogo con fotos, filtros de marca, precios y disponibilidad en calendario. - Sistema de reservas en línea con cálculo automático de costos. - Monitoreo telemático IoT en tiempo real con mapas interactivos. - Sistema bidireccional de reputación mediante calificaciones de 1 a 5 estrellas.**	**- Incrementar la flota de vehículos activos en Lima. - Maximizar el volumen de reservas completadas exitosamente. - Generar ingresos sostenibles por comisión de transacción. - Establecer a RentiCar como referente de confianza y modernidad.**
4. Users & Customers **Arrendadores:** Dueños de autos que buscan monetizar su vehículo en desuso. **Arrendatarios:** Turistas y profesionales que requieren movilidad temporal rápida, económica y confiable.	**5. User Benefits** **Arrendadores:** Ingresos adicionales, respaldo contractual y supervisión telemática de su auto. **Arrendatarios:** Acceso ágil a vehículos verificados, precios transparentes y sin depósitos abusivos.	**6. Hypotheses** Creemos que al ofrecer una plataforma digital con contratos transparentes y monitoreo IoT, propietarios y clientes migrarán de canales informales a RentiCar.
7. What's the most important thing we need to learn first? Validar si los propietarios perciben suficiente seguridad jurídica y tecnológica (mediante IoT y verificación) para confiar sus vehículos a terceros a través de la plataforma web.		**8. What's the least amount of work to learn it?** Desplegar un MVP web con registro autenticado, catálogo de vehículos con fotos, simulación telemática de ruta y flujo de reserva con confirmación del dueño.

1.3. Segmentos objetivo

Segmento 1 (Arrendador de vehículo):

Dueños de autos que no usan su vehículo con frecuencia y desean generar ingresos extras, estos serían personas de edad entre 25 a 45 años que residen en Lima Metropolitana.

Segmento 2 (Arrendatario de vehículo):

Personas que necesitan un vehículo de manera temporal que tengan edad entre 25 a 45 años con residencia en Lima Metropolitana y estos no tendrían un auto propio o profieran no usarlo todo el tiempo, así buscando una alternativa más económica y flexible frente a taxis o apps de transporte.

Capítulo II: Requirements Elicitation & Analysis

2.1. Competidores

Nombre del Competidor	Descripción
Peru Rent a Car Sitio Web	Empresa peruana tradicional con más de 20 años en el rubro de alquiler vehicular en Lima y principales provincias. Dispone de una flota propia diversa (sedanes, SUVs y minivans) orientada a turistas y clientes corporativos. Su modelo operativo se fundamenta en reservas telefónicas/web asistidas con atención presencial, contratos impresos y depósitos de garantía elevados retenidos en tarjeta de crédito, careciendo de digitalización P2P integral y telemetría en tiempo real accesible para el cliente.
Clase A Rent a Car Sitio Web	Proveedor local de alquiler vehicular enfocado en el sector ejecutivo y turístico de gama media-alta en Lima Metropolitana. Destaca por mantener vehículos modernos y ofrecer atención personalizada vía WhatsApp o vía telefónica. Aunque brinda flexibilidad en contratos temporales, sus procesos de validación de identidad son manuales y presenciales, sin autogestión digital completa ni monitoreo IoT interactivo.
Rento Sitio Web	Plataforma digital pionera en el modelo peer-to-peer (P2P) en el Perú, que conecta a propietarios de autos con usuarios que buscan alquilar por días. Incorpora seguros de cobertura vehicular y verificación básica de usuarios. Si bien es el competidor tecnológico más directo, presenta cobertura limitada fuera de Lima y aún no integra telemetría activa IoT para el seguimiento dinámico de trayectos ni soporte automatizado avanzado para la resolución de disputas contractuales.

2.1.1. Análisis competitivo

Competitive Analysis Landscape

¿Por qué llevar a cabo este análisis?	Comparar las características y funcionalidades clave de nuestra solución con las de la competencia para identificar ventajas competitivas y posibles áreas de mejora.			
Competidores	**RentiCar**	**Peru Rent a Car**	**Clase A Rent a Car**	**Rento**
Perfil	Overview	Plataforma web P2P para conectar dueños de autos con clientes de alquiler. Busca ser rápida, segura y fácil de usar, apostando por digitalización total y confianza.	Empresa tradicional con más de 20 años en el mercado peruano. Ofrece autos, camionetas y vans con seguro, asistencia y soporte para turistas y empresas.	Empresa local de alquiler con sede en Lima, enfocada en brindar vehículos modernos, en buen estado y con contratos flexibles. Reconocida por su trato personalizado.	Primera app P2P de alquiler de autos particulares en Perú. Incluye seguro RIMAC, GPS, monitoreo 24/7 y verificación de usuarios. Opera solo en Lima pero con visión de crecer a nivel nacional.
	Ventaja competitiva	Modelo 100% colaborativo y escalable, con costos más bajos y posibilidad de crecer en número de autos sin necesidad de flota propia.	Reputación consolidada, flota amplia y experiencia. Fiabilidad percibida por clientes corporativos y turistas.	Atención cercana y personalizada. Ideal para clientes que priorizan confianza y flexibilidad de trato.	Innovación tecnológica y respaldo asegurador fuerte. Modelo digital pionero en Perú.
Perfil de Marketing	Mercado objetivo	Jóvenes profesionales, turistas digitales, empresas pequeñas que buscan movilidad flexible y propietarios de autos que quieren generar ingresos extra.	Turistas extranjeros y nacionales, empresas que requieren autos para personal o ejecutivos, familias que buscan viajes seguros.	Residentes de Lima y clientes que buscan un servicio confiable y directo con atención humana.	Dueños de autos que buscan monetizar su vehículo, arrendatarios jóvenes urbanos, millennials y usuarios tecnológicos.
	Estrategias de marketing	Campañas digitales (redes, SEO/SEM), promociones de lanzamiento, alianzas con aseguradoras y fintechs, programas de referidos.	Presencia en buscadores, alianzas con agencias de viajes, ferias de turismo y marketing tradicional.	Publicidad local, confianza por reputación y marketing boca a boca.	Estrategia digital intensiva en redes sociales, marketing de confianza basado en seguridad y respaldo legal.
Perfil de Producto	Productos & Servicios	Alquiler P2P, validación digital de usuarios, registro de autos particulares, seguros, sistema de alertas y soporte técnico.	Alquiler de autos, camionetas, vans y pickups. Incluye seguros, GPS opcional, servicios corporativos y entrega en aeropuertos.	Alquiler de autos modernos, seguros incluidos, contratos adaptables según el cliente, soporte continuo.	Alquiler peer-to-peer, seguro todo riesgo, GPS obligatorio, verificación de identidad y central de monitoreo 24/7.
	Precios y Costos	Precios competitivos, definidos por los propietarios pero regulados por la plataforma. Comisiones más bajas que empresas tradicionales.	Tarifas medias-altas. Reflejan el costo de mantener una flota propia, seguros completos y servicios adicionales.	Tarifas medias. Ajustadas a la atención personalizada y al buen estado de la flota.	Tarifas dinámicas, generalmente más bajas que rent a car tradicionales gracias al modelo P2P.
	Canales de distribución	Pagina Web. Alianzas con aseguradoras y canales digitales.	Oficinas físicas en Lima + página web y contacto telefónico.	Oficina física, reservas por teléfono y página web.	Aplicación móvil y web con reservas 100% digitales.
Análisis SWOT	Fortalezas	Modelo escalable sin necesidad de flota propia, digitalización completa, costos más bajos, foco en experiencia de usuario.	Reputación, infraestructura establecida, atención corporativa, variedad de flota.	Atención personalizada, trato humano, autos modernos y cuidados.	Innovación tecnológica, confianza gracias a seguro RIMAC y GPS, pioneros en P2P.
	Debilidades	Startup nueva sin reputación consolidada, confianza inicial a construir.	Modelo rígido y menos flexible, procesos más burocráticos.	Escala limitada y menos variedad de flota frente a grandes competidores.	Flota aún limitada, dependencia del crecimiento de usuarios particulares.
	Oportunidades	Crecimiento de la economía colaborativa, demanda de digitalización, aumento de turismo y movilidad urbana.	Aumento del turismo post-pandemia, expansión del mercado corporativo.	Crecimiento de clientes que buscan atención diferenciada.	Expansión nacional, crecimiento de usuarios P2P en Perú.
	Amenazas	Competencia consolidada, regulaciones del sector transporte, desconfianza inicial.	Ingreso de apps digitales y startups más ágiles.	Pérdida de clientes frente a plataformas digitales con mayor alcance.	Aparición de nuevos competidores internacionales o nacionales con más capital.

2.1.2. Estrategias y tácticas frente a competidores

	OPORTUNIDADES • **Crecimiento del turismo interno y extranjero en Perú.** • **Tendencia a la digitalización de servicios de movilidad.** • **Aumento de la demanda de opciones económicas y flexibles en transporte.** • **Interés de PYMEs y startups por servicios de movilidad alternativos.**	**AMENAZAS** • **Alta competencia de empresas consolidadas (Peru Rent a Car, Clase A).** • **Competidores emergentes con modelo digital similar (Rento).** • **Riesgo de guerra de precios.** • **Desconfianza cultural hacia plataformas nuevas.**
FORTALEZAS • Plataforma 100% digital con reservas rápidas y seguras. • Flexibilidad en planes de alquiler (por horas, días o suscripción mensual). • Uso de tecnología IoT (GPS, monitoreo en tiempo real). • Enfoque en experiencia del usuario y atención 24/7.	**ESTRATEGIAS FO (Ofensivas)** • Posicionar a RentiCar como la plataforma de alquiler más ágil y flexible para turistas y jóvenes en Lima. • Usar la tecnología IoT como diferenciador innovador, mostrando seguridad y control en tiempo real. • Generar alianzas con hoteles, agencias de turismo y startups para integrar RentiCar en experiencias de viaje y negocios.	**ESTRATEGIAS FA (Defensiva)** • Diferenciarse de grandes competidores mediante la rapidez y simplicidad digital. • Competir con Rento ofreciendo mayor cobertura y disponibilidad de vehículos. • Evitar la guerra de precios destacando valor agregado: seguridad, flexibilidad y soporte 24/7.
DEBILIDADES • Marca nueva sin historial ni reputación consolidada. • Flota inicial limitada frente a grandes competidores. • Recursos financieros más reducidos que empresas consolidadas. • Dependencia de la confianza digital en un mercado aún en transición.	**ESTRATEGIAS DO (Reorientación)** • Reforzar la confianza en la marca mediante certificaciones, seguros y políticas transparentes. • Expandir la flota progresivamente a través de convenios con propietarios particulares y PYMEs. • Aprovechar la digitalización para compensar la falta de trayectoria, mostrando procesos claros, seguros y amigables.	**ESTRATEGIAS DA (Supervivencia)** • Lanzar campañas de marketing educativo sobre la seguridad y beneficios de alquilar en plataformas digitales. • Implementar un programa de fidelización para generar usuarios recurrentes desde el inicio. • Diversificar fuentes de ingreso (planes corporativos, suscripciones, alquiler por horas) para reducir vulnerabilidad frente a la competencia.

2.2. Entrevistas

2.2.1. Diseño de entrevistas

Segmento 1: Dueño de vehiculo

Preguntas de Información General:

- ¿Cuál es tu nombre?
- ¿Cuántos años tienes?
- ¿En qué distrito vives?
- ¿A qué te dedicas actualmente?

Preguntas sobre el alquiler de sus vehículos:

- ¿Que tipo de documento exiges para proceder con el alquiler?
- ¿Qué tipo de vehículo ofreces para el alquiler?
- ¿Cual es la cantidad minima y maxima de tiempo que permites alquilar tu vehiculo?
- ¿Cómo llevas la cuenta de todos tus vehículos, tanto alquilados como sin alquilar?

Preguntas sobre la plataforma:

- ¿Que tipo de plataforma usas para ofrecer tu vehiculo?
- ¿En qué dispositivos accedes a dichas plataformas?
- ¿Consideras que las aplicaciones te dan facilidades para identificar clientes confiables?
- ¿Estarías dispuesto a ofrecer tus vehículos en una nueva plataforma que te permita consultar reseñas de un cliente previo al alquiler?
- ¿Estarías dispuesto a ofrecer tus vehículos en una nueva plataforma que te ofrezca un panel donde registrar tus vehículos y consultar su estado?
- ¿Estarías dispuesto a ofrecer tus vehículos en una nueva plataforma que te permita reseñar a los clientes?

Segmento 2: Usuario que busca alquilar un vehiculo

Preguntas de Información General:

- ¿Cuál es tu nombre?
- ¿Cuántos años tienes?
- ¿En qué distrito vives?

- ¿A qué te dedicas actualmente?

Preguntas sobre el alquiler de sus vehículos:

- ¿Qué tipo de documento te exigen para proceder con el alquiler?
- ¿Que tipo de vehiculo buscas alquilar?
- ¿Que documentos te suelen pedir previo al alquiler del vehiculo?
- ¿Que restricciones se te imponen previo al alquiler del vehiculo?

Preguntas sobre el alquiler de sus vehículos:

- ¿Qué tipo de plataforma usas para buscar vehículos?
- ¿En qué dispositivos accedes a dichas plataformas?
- ¿Consideras que las aplicaciones te dan facilidades para identificar vehículos o arrendadores confiables?
- ¿Estarías dispuesto a migrar a una nueva plataforma que te permita consultar reseñas de arrendadores antes de alquilar?
- ¿Estarías dispuesto a usar una nueva plataforma que te permita reservar un vehículo en unas fechas concretas antes de alquilarlo?
- ¿Estarías dispuesto a usar una nueva plataforma que te permita reseñar a los arrendadores?

2.2.2. Registro de entrevistas

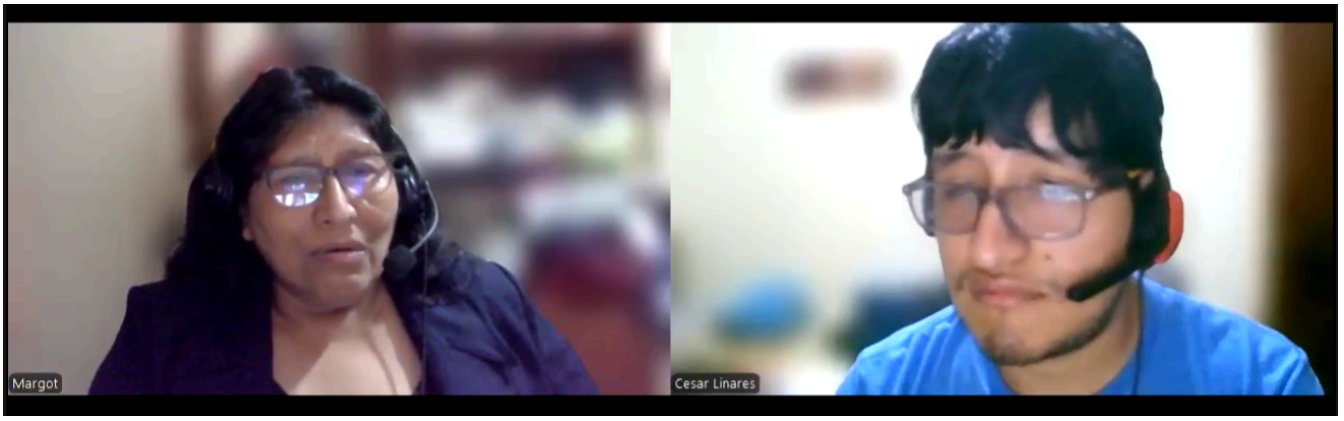
Entrevistas a arrendadores de vehiculos

Entrevista 1	
Entrevistado	Oriana Caceres
Edad	35 años
Distrito	Cercado de Lima
Timing	0:16-5:01
URL	https://youtu.be/B3n8FB8qZX8

Resumen:
Oriana, contadora graduada, reside en el distrito de Cercado de Lima, habla sobre como el tener un auto que no usa mucho le sirve para tener un ingreso adicional al alquilarlo para servicios de uber, le gustaria una alternativa donde su auto sea separado de la identidad del conductor para mejor reputacion y seguridad.

Entrevista 2	
Entrevistado	Margot Bernable
Edad	45 años
Distrito	San Martín de Porres
Timing	0:31-8:34
URL	https://youtu.be/7LyhIITgNnU

Resumen:
Margot Bernable de la Cruz, obstreta de 45 años que reside en el distrito de San Martín de Porres, habla sobre cómo suele alquilar autos económicos o medianos según el uso, y menciona que le piden DNI, licencia de conducir, tarjeta de crédito y voucher, además de restricciones de edad y experiencia para conductores adicionales. Le gustaría una plataforma con más reseñas de arrendadores y detalles del interior del vehículo para alquilar con mayor confianza y seguridad.



Entrevista 3

Entrevistado	Noelia Gomez
Edad	25 años
Distrito	Pueblo Libre
Timing	03:49 - 07:32 minutos (Duración: 3:43 minutos)
URL	https://youtu.be/EWyy7faTG3Y

Resumen:
Comenta su experiencia alquilando su auto personal, un Sedán. Actualmente, gestiona el alquiler a través del calendario de su celular y lo publicita en Facebook Marketplace y grupos de WhatsApp, pero reconoce que estos medios no le brindan facilidades para verificar la confiabilidad de los clientes, teniendo que guiarse por su instinto. Estaría muy dispuesta a usar una nueva plataforma que le permita consultar reseñas de clientes previos para tener más tranquilidad, así como un panel para registrar su vehículo de forma más profesional. Además, considera fundamental poder reseñar a los clientes para fomentar un sistema de confianza y responsabilidad mutua.

Entrevistas a arrendatarios de vehiculos

Entrevista 1

Entrevistado	Sebastian Altamirano
Edad	22 años
Distrito	Cercado de Lima
Timing	0:10-7:41
URL	https://youtu.be/UtvLuMLGYto

Resumen:
Sebastian, estudiante universitario con licencia de conducir, reside en el distrito de Cercado de Lima, habla sobre como se le suelen pedir permisos de mas para poder alquilar un auto ademas de que le es complicado reservar para fechas muy concurridas por lo que le gustaria poder alquilar por modelo y para fechas exactas.

Entrevista 2

Entrevistado	Gino Ábalos Carranza
Edad	20 años
Distrito	Chorrillos
Timing	0:15 - 3:23
URL	https://youtu.be/slelvvtnsms

Resumen:
Gino Ábalos Carranza, arquitecto de 20 años residente en Chorrillos, suele alquilar vehículos sedán para movilizarse y busca sus opciones principalmente en Facebook; aunque señala que actualmente solo le solicitan DNI para el trámite, considera que dicha red social carece de especialización para el servicio, por lo que expresa un total interés en migrar a una plataforma dedicada que integre un sistema de reseñas sobre los arrendadores y le permita agendar reservas para fechas concretas.

Entrevista 3

Entrevistado	Alessandro Egavil
Edad	25 años
Distrito	San Miguel
Timing	0:12 - 3:45

URL	https://youtu.be/1D09YFbTE2s

Resumen:
Alessandro, un joven de 25 años residente en San Miguel, se encuentra constantemente en la búsqueda de carros para alquilar, ya sea para realizar viajes cortos de fin de semana o salidas familiares. Aunque suele buscar opciones en Google o redes sociales, este proceso le resulta frustrante por la desconfianza y el riesgo que le genera dejar un depósito de garantía a desconocidos sin referencias. Por ello, está buscando una alternativa más segura y usaría sin dudar una plataforma o aplicación que le permita leer reseñas y calificaciones de los dueños de los vehículos antes de decidirse por un alquiler.

2.2.3. Análisis de entrevistas

El análisis del primer segmento revela una marcada necesidad de transición desde la informalidad digital hacia una plataforma especializada. Un aspecto central es la alta motivación económica derivada de la subutilización de sus vehículos, los cuales pasan la mayor parte del tiempo estacionados y buscan ser monetizados como ingresos pasivos por perfiles profesionales. No obstante, los propietarios dependen actualmente de métodos rudimentarios e informales, tales como Facebook Marketplace, grupos de WhatsApp y calendarios personales, viéndose obligados a guiarse únicamente por su instinto ante la absoluta ausencia de filtros de verificación y control previos. Esto genera una profunda exposición al riesgo, impulsando una fuerte exigencia por contar con herramientas profesionales de gestión, tales como un panel centralizado de control, sistemas de reseñas bidireccionales para fomentar la responsabilidad mutua y la necesidad de resguardar su seguridad frente a incidentes externos.

Por otro lado, el análisis del segundo segmento expone las deficiencias críticas que experimentan los conductores al buscar vehículos en el mercado actual. Al recurrir a redes sociales y canales informales, se enfrentan a una dualidad frustrante: procesos excesivamente laxos que apenas solicitan un documento de identidad y generan desconfianza general, o bien permisos innecesarios que entorpecen la experiencia. Asimismo, los entrevistados (como Sebastián, Gino y Alessandro) coinciden en las severas complicaciones para asegurar reservas en fechas de alta demanda o temporadas concurridas debido a la falta de estandarización en la oferta. En particular, un punto crítico de fricción destacado por los arrendatarios radica en el elevado riesgo percibido al entregar depósitos de garantía a desconocidos sin garantías ni referencias comprobables. Por ello, existe un marcado consenso e interés en migrar hacia una solución digital especializada que les permita filtrar vehículos por modelos específicos, agendar con precisión para fechas exactas, gestionar depósitos con respaldo y consultar reseñas y calificaciones previas sobre los arrendadores para asegurar una experiencia transparente y confiable.

Al contrastar ambos segmentos, se evidencia que tanto arrendadores como arrendatarios comparten el mismo dolor raíz: operan en un ecosistema altamente informal, inseguro y carente de trazabilidad. Mientras que el propietario busca mitigar el riesgo de daños y validar estrictamente la identidad del conductor, el arrendatario exige certidumbre, agilidad y transparencia en los precios, depósitos y disponibilidad de los modelos. En consecuencia, la oportunidad de valor se centra en consolidar una plataforma unificada que actúe como un filtro de confianza bidireccional. Mediante la integración de módulos de verificación estricta de identidad, reputación basada en calificaciones mutuas y un dashboard especializado de gestión, es posible resolver las fricciones de ambos lados del mercado, transformando un proceso basado en la intuición en una experiencia profesional, segura y escalable.

De acuerdo con las entrevistas y su respectivo análisis, a continuación se presentan los gráficos que resumen los principales puntos de dolor, canales de comunicación y dispositivos utilizados por cada segmento:

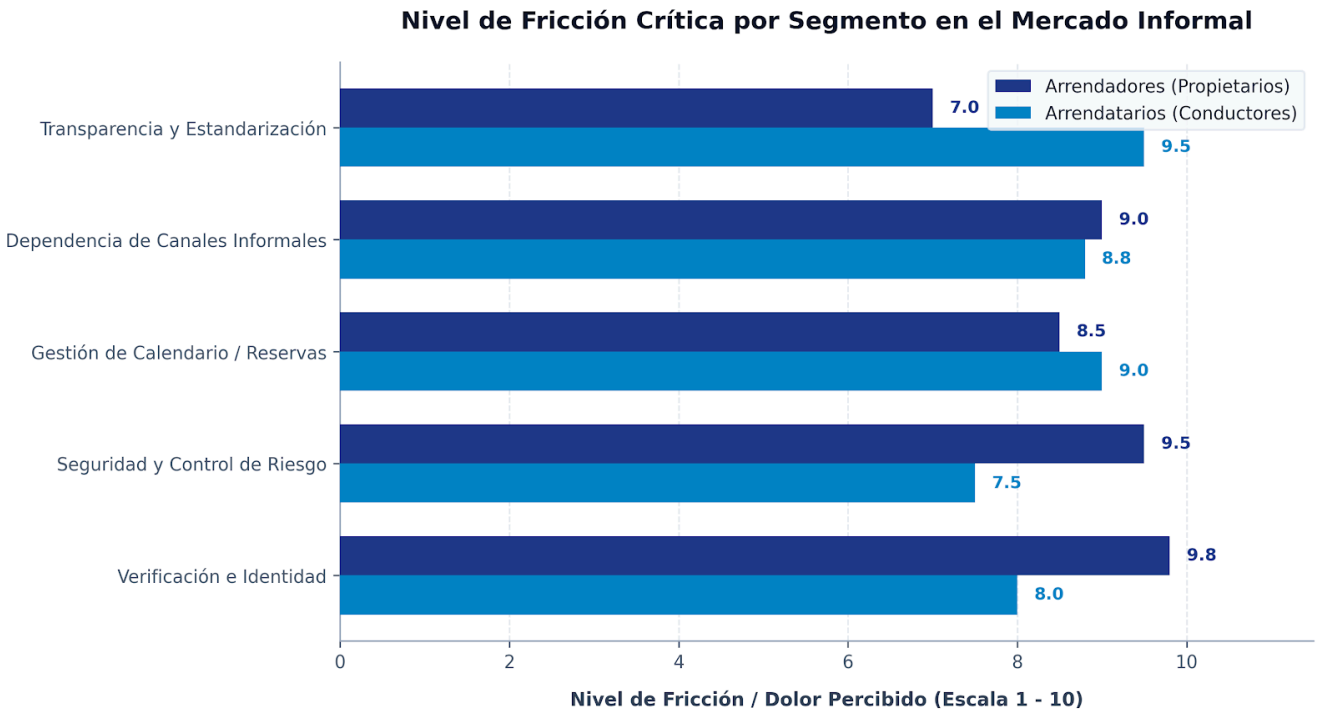


Figura 2.2.3.1: Síntesis de dolores, necesidades y propuesta de valor de los segmentos entrevistados

Ambos segmentos recurren a canales digitales, pero los arrendadores tienen una fuerte dependencia de redes sociales y canales informales, mientras que los arrendatarios combinan redes sociales con plataformas especializadas:

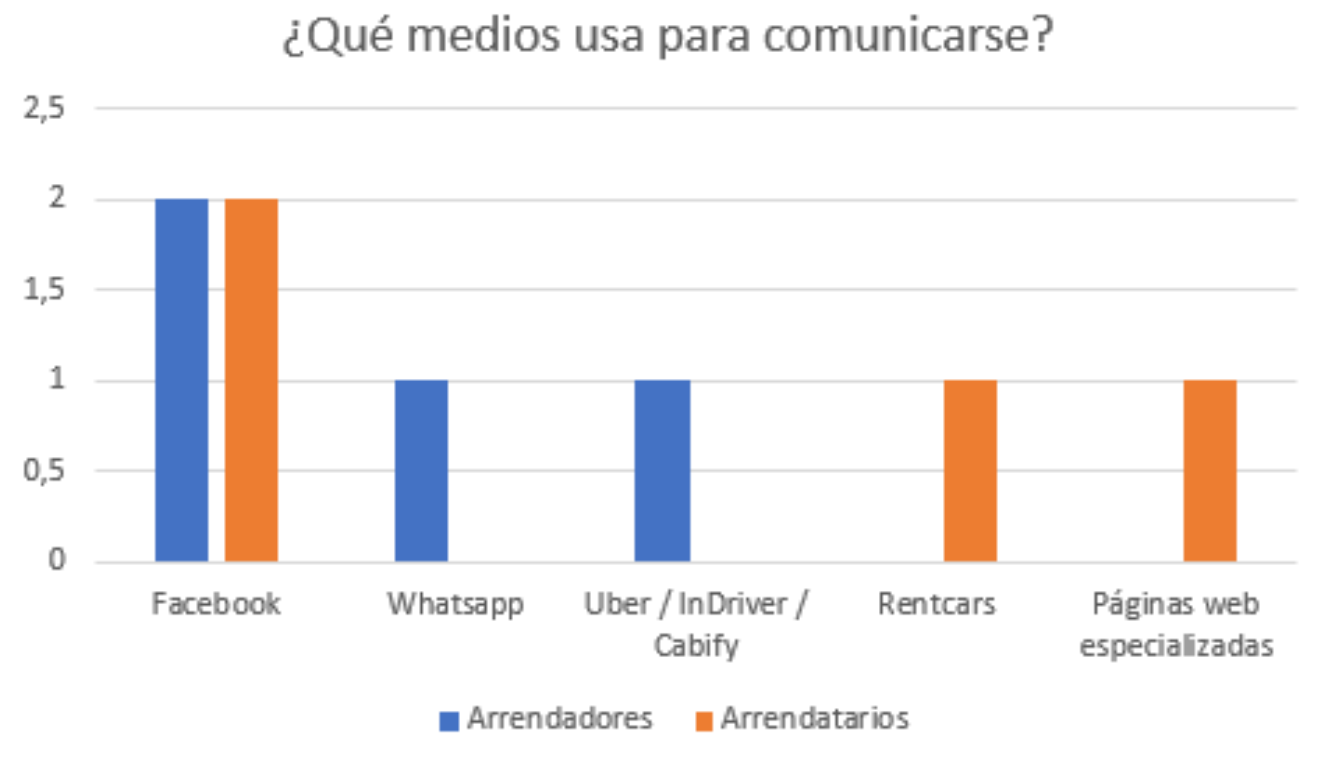


Figura 2.2.3.2: Canales de comunicación empleados por arrendadores y arrendatarios

El celular es el principal dispositivo para ambos segmentos, lo cual justifica y respalda plenamente que la plataforma adopte un enfoque y arquitectura Mobile-First:

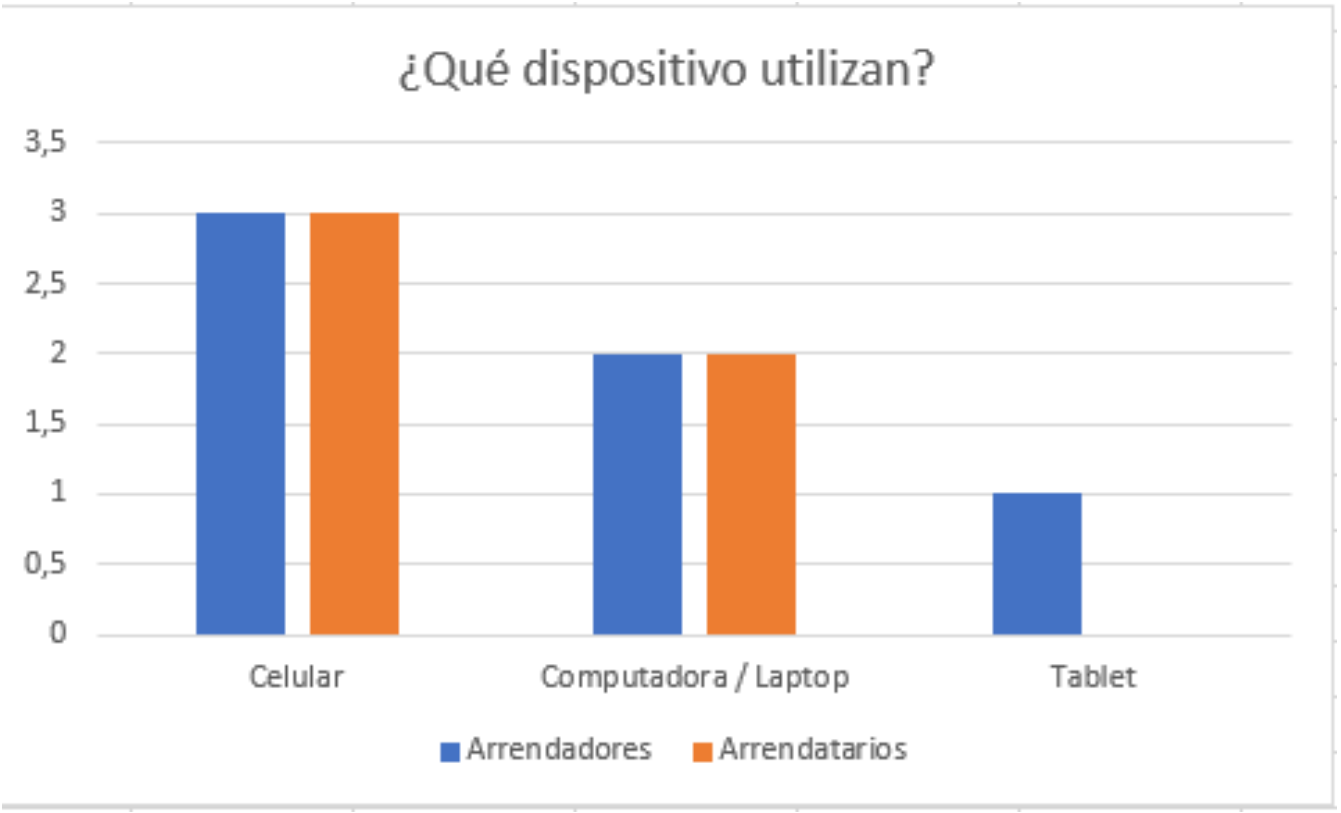


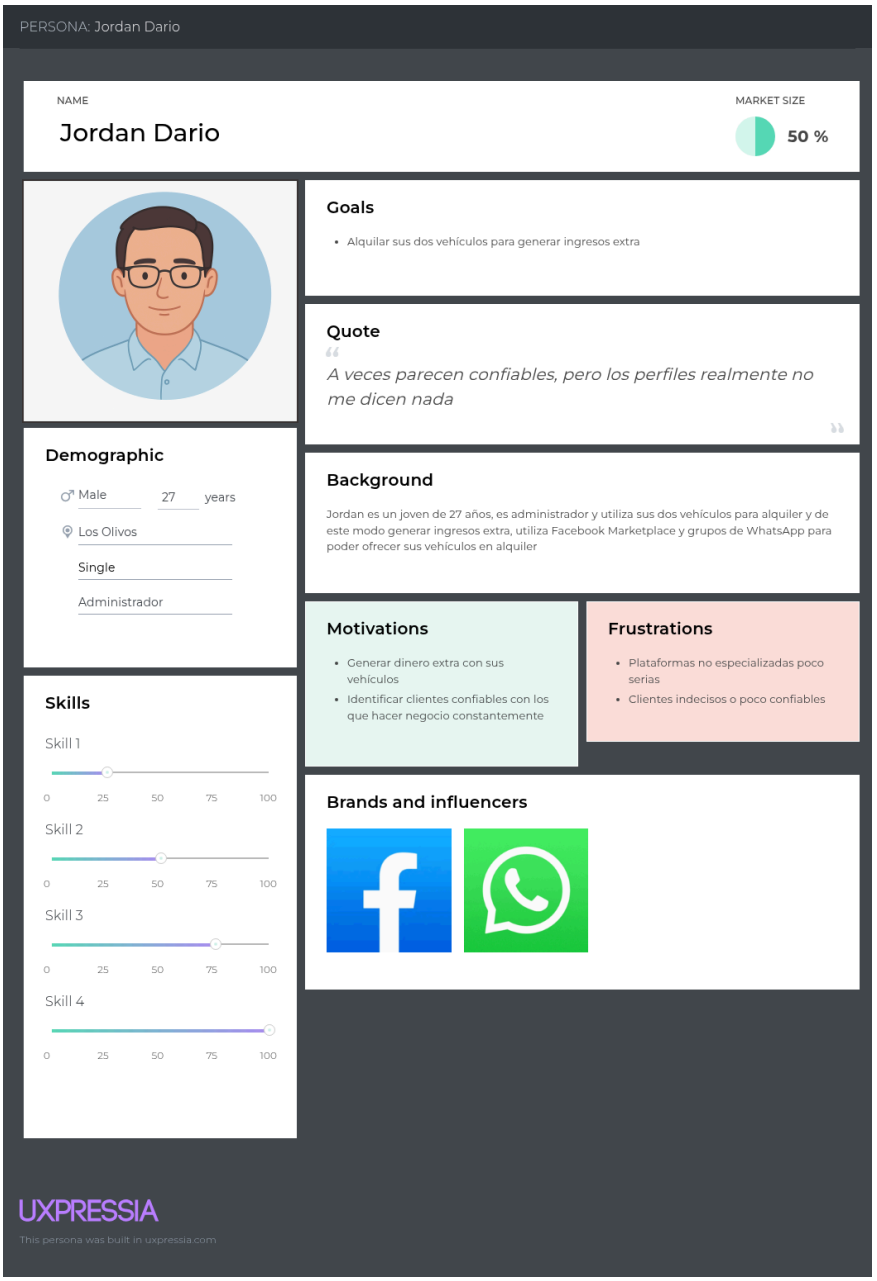
Figura 2.2.3.3: Preferencia de dispositivos de uso identificada en las entrevistas

2.3. Needfinding

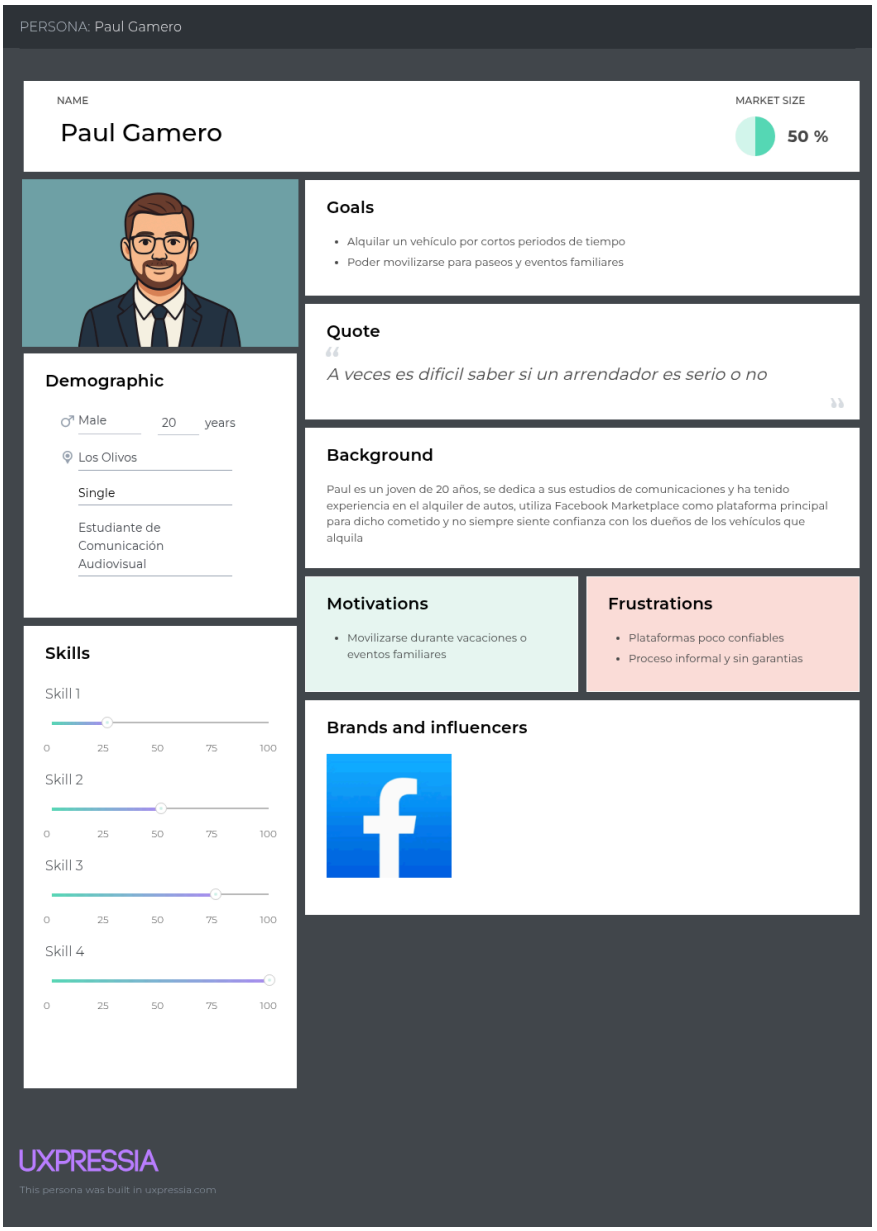
2.3.1. User Personas

Los User Personas son representaciones ficticias pero fundamentadas en los hallazgos de las entrevistas realizadas a los dos segmentos objetivo del proyecto: arrendadores y arrendatarios. Su función es sintetizar las características, motivaciones, necesidades y frustraciones de los usuarios principales, permitiendo orientar el diseño de la aplicación hacia soluciones que respondan a sus expectativas reales. A continuación, se presentan los perfiles construidos con sus respectivos avatares ilustrativos, que servirán como referencia durante el proceso de desarrollo y validación de la plataforma.

Segmento Objetivo 1 (Arrendador de vehículos)



Segmento Objetivo 2 (Arrendatario de vehículos)



2.3.2. User Task Matrix

User Task Matrix	**Arrendador – Frecuencia**	**Arrendador – Importancia**	**Arrendatario – Frecuencia**	**Arrendatario – Importancia**
Publicar vehículo en la plataforma	Siempre	Alta	Nunca	Baja
Gestionar reservas activas	A menudo	Alta	Siempre	Alta
Revisar reseñas de usuarios	Siempre	Alta	A menudo	Alta
Acceso a documentos de garantía	Siempre	Alta	A veces	Media
Consultar estado y ubicación IoT del auto	A menudo	Alta	A veces	Media
Reservar un vehículo	Nunca	Baja	Siempre	Alta

Análisis de la User Task Matrix

El análisis de la matriz de tareas muestra diferencias claras entre los dos segmentos principales. El **arrendador** concentra sus actividades en la **publicación de vehículos, la gestión de reservas y la revisión de reseñas**, todías ellas consideradas de alta importancia, ya que están directamente relacionadas con la protección de su inversión. Asimismo, para este segmento es fundamental contar con acceso a **documentos de garantía** y con un **módulo IoT básico** que permita visualizar el estado y ubicación del vehículo, lo cual incrementa la confianza y reduce el riesgo de fraudes.

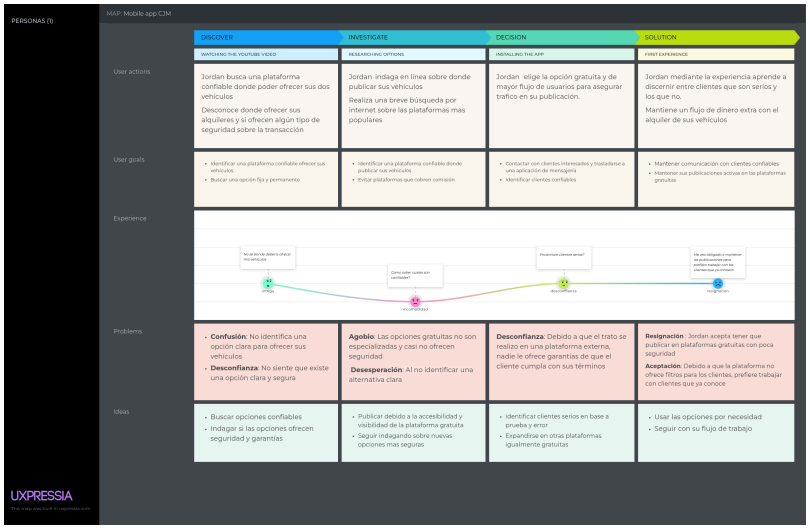
Por su parte, el **arrendatario** enfoca su experiencia en la **reserva de vehículos** y en la posibilidad de **consultar reseñas de arrendadores**, funciones que le otorgan seguridad y transparencia en el proceso. Aunque reconoce la utilidad del acceso a documentos y del indicador IoT, estas tareas no son percibidas como prioritarias frente a la necesidad principal de encontrar y reservar un vehículo de manera confiable.

En conclusión, la matriz evidencia que ambos segmentos comparten la necesidad de un sistema que promueva **transparencia, seguridad y confianza**. El arrendador busca reducir riesgos en la entrega de sus vehículos, mientras que el arrendatario demanda una experiencia clara y confiable al alquilar. Estos hallazgos guían directamente las funcionalidades clave a priorizar en el diseño de la plataforma.

2.3.3. User Journey Mapping

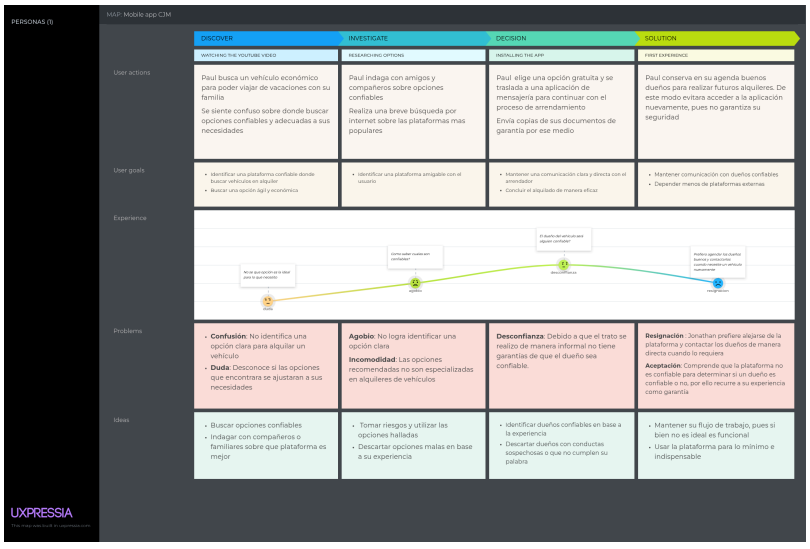
El **User Journey Mapping** permite visualizar de manera detallada la experiencia que atraviesan los usuarios durante la interacción con las plataformas actuales de alquiler de vehículos. Este ejercicio hace posible identificar los puntos de contacto, emociones y frustraciones de cada segmento objetivo, mostrando las debilidades del proceso actual y evidenciando las oportunidades de mejora que justifican la propuesta de valor de la nueva aplicación.

Segmento Objetivo 1 (Arrendador de vehículos)



El recorrido del arrendador refleja un flujo de trabajo que no resulta del todo eficiente ni satisfactorio. Aunque logra captar clientes mediante plataformas gratuitas de amplia difusión como Facebook o WhatsApp, se muestra frustrado por la escasa seguridad y falta de garantías que estas herramientas ofrecen. A pesar de ello, continúa utilizándolas debido a su alcance y practicidad, aunque reconoce la necesidad de migrar a una solución más confiable y profesional.

Segmento Objetivo 2 (Arrendatario de vehículos)

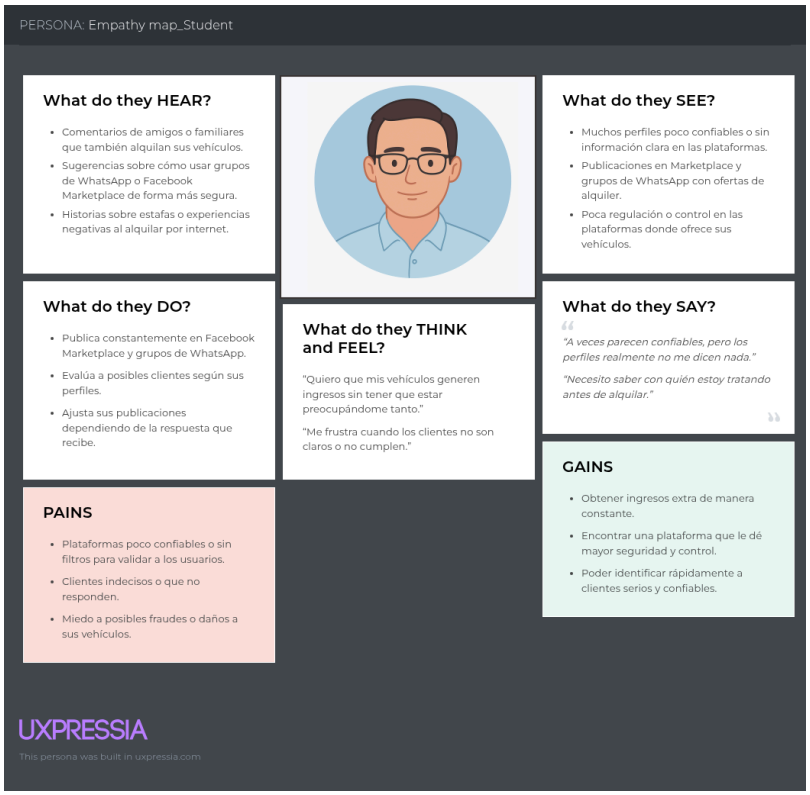


Por el lado del arrendatario, la experiencia está marcada por la limitada disponibilidad de opciones confiables y la inseguridad que transmiten las plataformas gratuitas. Esta situación genera desconfianza y lo obliga a depender de la identificación previa de arrendadores de confianza, reduciendo el uso de dichas plataformas a casos estrictamente necesarios. Esto evidencia que, aunque existe una necesidad de movilidad, la falta de transparencia y seguridad limita la satisfacción y continuidad en el uso de los servicios actuales.

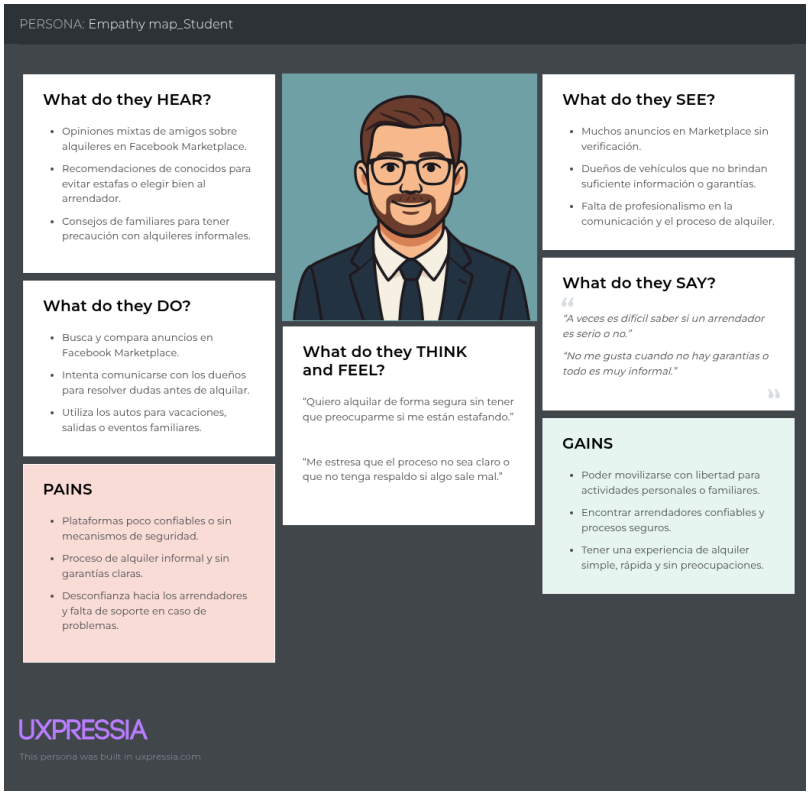
2.3.4. Empathy Mapping

El **Empathy Mapping** es una herramienta que permite comprender de manera profunda a los usuarios, identificando lo que **piensan, sienten, dicen y hacen** durante su experiencia con el servicio. Este recurso facilita visualizar no solo sus necesidades funcionales, sino también sus motivaciones, frustraciones y expectativas, lo que ayuda a orientar el diseño de la solución hacia una experiencia más centrada en el usuario.

Segmento Objetivo 1 (Arrendador de vehículos)



Segmento Objetivo 2 (Arrendatario de vehículos)



2.3.5. As-is Scenario Mapping

El **As-Is Scenario Mapping** es una herramienta fundamental de diseño de experiencia de usuario (UX) que documenta el estado actual y el recorrido empírico que experimentan los usuarios al intentar resolver su necesidad de transporte o alquiler de vehículos antes de la existencia de **RentiCar**. Permite identificar con precisión los puntos de fricción, demoras, sobrecostos y riesgos de seguridad física y financiera presentes en los métodos tradicionales e informales del mercado peruano.

A continuación, se detallan los escenarios As-Is para los dos segmentos objetivo del proyecto:

Segmento Objetivo 1: Arrendatario (Usuario que busca alquilar un auto de forma tradicional o informal)

User Persona representativa: Carlos Mendoza (Profesional independiente que busca un vehículo para viajes de trabajo y ocio en Lima).

Link del Miro: https://miro.com/app/board/uXjVHmc36as=/

Segmento Objetivo 2: Arrendador (Propietario particular que intenta alquilar su auto para obtener ingresos)

User Persona representativa: Ricardo Ramos (Dueño de un sedán que busca rentabilizar su automóvil en días no laborables).

Link del Miro: https://miro.com/app/board/uXjVHmc36as=/

2.4. Ubiquitous Language

Con el fin de establecer un lenguaje común entre los desarrolladores y los usuarios, se definió un conjunto de términos que se emplearán de forma consistente a lo largo del proyecto. Este lenguaje ubicuo evita confusiones conceptuales y facilita la comunicación dentro del equipo, asegurando que todos comprendan los mismos procesos con las mismas palabras.

Términos principales en el dominio de Renticar:

- **Arrendador**: Persona que registra y ofrece su vehículo en la plataforma.
- **Arrendatario**: Usuario que alquila un vehículo registrado en la plataforma.
- **Publicación**: Ficha que contiene los datos del vehículo y sus condiciones de alquiler.
- **Reserva**: Confirmación del alquiler de un vehículo por un periodo determinado.
- **Contrato digital**: Documento electrónico que formaliza las condiciones del alquiler.
- **Reseña**: Evaluación realizada al finalizar la reserva, que permite calificar la experiencia.
- **Estado del vehículo**: Indicador del sistema que señala si el auto está Disponible, En uso o Finalizado.
- **Ubicación IoT**: Dato de localización y estado general del vehículo, mostrado de manera ligera durante la reserva.
- **Panel de gestión**: Espacio central del sistema donde los usuarios consultan publicaciones, reservas y reseñas.

Este lenguaje compartido no solo guía el diseño de la solución, sino que también constituye la base para la documentación, la arquitectura del software y las interacciones con los futuros usuarios, fortaleciendo la alineación entre el negocio y la tecnología.

Capítulo III: Requirements Specification

3.1. To-Be Scenario Mapping

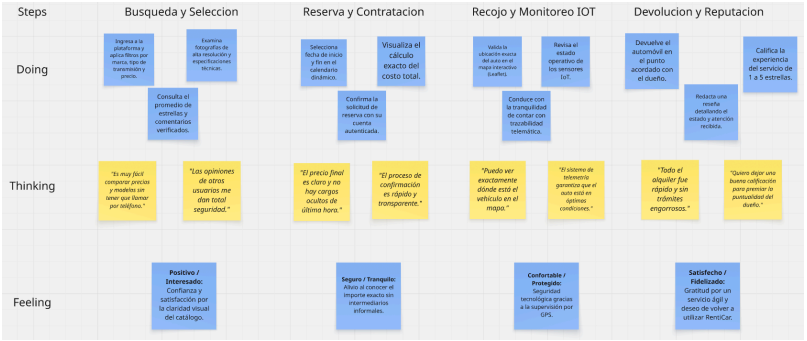
El **To-Be Scenario Mapping** es una herramienta de diseño de experiencia de usuario (UX) que modela el recorrido ideal que experimentará el usuario al interactuar con la plataforma digital **RentiCar**, superando las fricciones, demoras y desconfianzas identificadas en el escenario actual (*As-Is*).

A continuación, se presentan los escenarios To-Be estructurados para los dos segmentos objetivo del proyecto:

Segmento Objetivo 1: Arrendatario (Cliente que busca alquilar un vehículo)

User Persona representativa: Carlos Mendoza (Profesional independiente que requiere movilidad flexible y confiable).

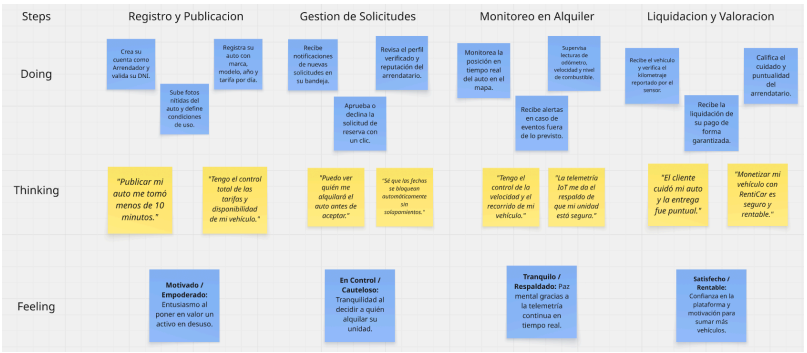
Link del Miro: https://miro.com/app/board/uXjVHmc36as=/



Segmento Objetivo 2: Arrendador (Propietario de vehículo que monetiza su auto)

User Persona representativa: Ricardo Ramos (Dueño de vehículo que busca ingresos adicionales con respaldo legal).

Link del Miro: https://miro.com/app/board/uXjVHmc36as=/



3.2. User Stories

En esta sección se definieron las épicas y la especificación detallada de las **35 User Stories** con sus respectivos criterios de aceptación bajo sintaxis formal Gherkin (**Given-When-Then-And**), verificadas y cumplidas con paridad funcional en el backend (**Spring Boot 3.x**) y el frontend (**Angular 20**).

Epics del Sistema RentiCar

A continuación, se presenta la matriz general de las 8 Épicas que estructuran el desarrollo del producto:

Epic ID	Título de la Épica	Descripción de la Épica
EP-01	**Autenticación y Seguridad de Acceso**	Como usuario (arrendatario o propietario), quiero un sistema de registro, inicio de sesión y gestión de credenciales seguro con encriptación BCrypt y tokens JWT para proteger mi cuenta y privacidad.
EP-02	**Panel de Control del Usuario (Dashboard)**	Como usuario, quiero acceder a un panel centralizado que me brinde un resumen operativo de mis alquileres activos, vehículos recomendados y atajos directos a mis acciones frecuentes.
EP-03	**Gestión de Flota y Publicaciones de Vehículos**	Como propietario, quiero registrar, actualizar el estado, subir fotos y administrar mis vehículos publicados para ponerlos a disposición de los arrendatarios en la plataforma.
EP-04	**Catálogo, Búsqueda y Reserva de Vehículos**	Como arrendatario, quiero explorar el catálogo de vehículos, filtrar por marcas, ordenar por precio y generar solicitudes de alquiler con cálculo automático de tarifas por días.
EP-05	**Monitoreo Telemático IoT y Simulación de Rutas**	Como arrendatario o propietario, quiero visualizar la posición GPS del vehículo en un mapa interactivo (Leaflet), velocidad y nivel de combustible en tiempo real durante el periodo de alquiler.
EP-06	**Sistema de Reputación, Reseñas y Calificaciones**	Como arrendatario, quiero calificar y dejar comentarios sobre el vehículo y la experiencia de alquiler una vez concluido el servicio para orientar a la comunidad y fomentar la confianza.
EP-07	**Administración de Perfil y Preferencias**	Como usuario registrado, quiero consultar y editar mis datos personales de contacto para mantener vigente mi información en el ecosistema RentiCar.
EP-08	**Landing Page Informativa y Captación**	Como visitante, quiero navegar por una página de inicio atractiva y clara con información de valor, catálogo destacado, selector de idioma y formulario de suscripción para conocer el servicio.

Épica EP-01: Autenticación y Seguridad de Acceso

Como usuario (arrendatario o propietario), quiero un sistema de registro, inicio de sesión y gestión de credenciales seguro con encriptación BCrypt y tokens JWT para proteger mi cuenta y privacidad.

User Story US-01: Registro de Usuario

Atributo	Detalle
Story ID	**US-01**
Épica	**EP-01**: Autenticación y Seguridad de Acceso
Usuario	Visitante de la landing page
Prioridad	Alta
Título	**Registro de Usuario**
Descripción	Como visitante, quiero registrar una cuenta con mis datos personales y rol en la plataforma, para poder publicar vehículos o solicitar alquileres de manera segura.
Criterios de Aceptación	**Escenario 1: Registro exitoso de nuevo usuario** **Given** se envía POST /api/v1/authentication/sign-up con datos válidos y rol seleccionado **When** se procesa la petición **Then** el sistema retorna código HTTP 201 Created **And** retorna los datos del perfil registrado con confirmación de cuenta. **Escenario 2: Registro con correo ya existente** **Given** se envía POST /api/v1/authentication/sign-up con un correo previamente registrado **When** se procesa la petición **Then** el sistema retorna error 409 Conflict **And** retorna mensaje de error apropiado indicando duplicidad de credenciales.

User Story US-02: Inicio de Sesión (Log In)

Atributo	Detalle
Story ID	**US-02**
Épica	**EP-01**: Autenticación y Seguridad de Acceso
Usuario	Usuario registrado (arrendatario o propietario)
Prioridad	Alta
Título	**Inicio de Sesión (Log In)**
Descripción	Como usuario registrado, quiero iniciar sesión con mis credenciales, para acceder a mis publicaciones, reservas y funcionalidades protegidas de la plataforma.
Criterios de Aceptación	**Escenario 1: Inicio de sesión exitoso** **Given** se envía POST /api/v1/authentication/sign-in con credenciales válidas **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna el token de autenticación JWT y perfil de usuario. **Escenario 2: Inicio de sesión con credenciales incorrectas** **Given** se envía POST /api/v1/authentication/sign-in con credenciales incorrectas **When** se procesa la petición **Then** el sistema retorna error 401 Unauthorized **And** retorna mensaje de error apropiado.

User Story US-03: Actualizar Contraseña de Usuario

Atributo	Detalle
Story ID	**US-03**
Épica	**EP-01**: Autenticación y Seguridad de Acceso
Usuario	Usuario autenticado
Prioridad	Media
Título	**Actualizar Contraseña de Usuario**
Descripción	Como usuario autenticado, quiero actualizar mi contraseña actual por una nueva, para mantener protegida mi cuenta en la plataforma.
Criterios de Aceptación	**Escenario 1: Actualización exitosa con contraseña actual válida** **Given** se envía PATCH /api/v1/users/{userId}/password con token Bearer JWT y contraseña actual correcta **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna confirmación de actualización exitosa de credenciales. **Escenario 2: Actualización con contraseña actual errónea** **Given** se envía PATCH /api/v1/users/{userId}/password con token Bearer JWT y contraseña actual errónea

Atributo	Detalle
	When se procesa la petición **Then** el sistema retorna error 401 Unauthorized **And** retorna mensaje de error apropiado indicando contraseña actual no válida.

User Story US-04: Eliminación de Cuenta de Usuario

Atributo	Detalle
Story ID	**US-04**
Épica	**EP-01**: Autenticación y Seguridad de Acceso
Usuario	Usuario autenticado
Prioridad	Media
Título	**Eliminación de Cuenta de Usuario**
Descripción	Como usuario autenticado, quiero eliminar mi cuenta de la plataforma, para remover definitivamente mis datos y perfil del sistema.
Criterios de Aceptación	**Escenario 1: Eliminación exitosa de cuenta** **Given** se envía DELETE /api/v1/users/{userId} con token Bearer JWT del usuario **When** se procesa la petición **Then** el sistema retorna código HTTP 204 No Content **And** desactiva la cuenta del usuario en el sistema. **Escenario 2: Intento de eliminación sin autorización** **Given** se envía DELETE /api/v1/users/{userId} sin token de autorización Bearer JWT **When** se procesa la petición **Then** el sistema retorna error 401 Unauthorized **And** retorna mensaje de error apropiado denegando la operación.

Épica EP-02: Panel de Control del Usuario (Dashboard)

Como usuario, quiero acceder a un panel centralizado que me brinde un resumen operativo de mis alquileres activos, vehículos recomendados y atajos directos a mis acciones frecuentes.

User Story US-05: Dashboard del Arrendatario

Atributo	Detalle
Story ID	**US-05**
Épica	**EP-02**: Panel de Control del Usuario (Dashboard)
Usuario	Arrendatario autenticado
Prioridad	Media
Título	**Dashboard del Arrendatario**
Descripción	Como arrendatario, quiero acceder a mi panel principal, para visualizar el estado de mis reservas activas y atajos a mis vehículos contratados.
Criterios de Aceptación	**Escenario 1: Carga exitosa de panel de arrendatario** **Given** se envía GET /api/v1/bookings/my-bookings con token de rol ROLE_ARRENDATARIO **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna el consolidado de reservas activas y pendientes del usuario. **Escenario 2: Acceso a panel sin token de autenticación** **Given** se envía GET /api/v1/bookings/my-bookings sin cabecera Authorization **When** se procesa la petición **Then** el sistema retorna error 401 Unauthorized **And** retorna mensaje de error apropiado solicitando inicio de sesión.

User Story US-06: Dashboard del Propietario

Atributo	Detalle
Story ID	**US-06**
Épica	**EP-02**: Panel de Control del Usuario (Dashboard)
Usuario	Propietario autenticado
Prioridad	Media

Atributo	Detalle
Título	**Dashboard del Propietario**
Descripción	Como propietario, quiero acceder a mi panel principal, para inspeccionar las solicitudes entrantes de alquiler y el estado general de mi flota.
Criterios de Aceptación	**Escenario 1: Carga exitosa de solicitudes de reserva** **Given** se envía GET /api/v1/bookings/my-requests con token de rol ROLE_ARRENDADOR **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna la lista de solicitudes de reserva pendientes para sus vehículos. **Escenario 2: Acceso por usuario no autorizado** **Given** se envía GET /api/v1/bookings/my-requests con token de rol ROLE_ARRENDATARIO **When** se procesa la petición **Then** el sistema retorna error 403 Forbidden **And** retorna mensaje de error apropiado indicando permisos insuficientes.

User Story US-07: Publicaciones en el Dashboard

Atributo	Detalle
Story ID	**US-07**
Épica	**EP-02**: Panel de Control del Usuario (Dashboard)
Usuario	Propietario de vehículo
Prioridad	Media
Título	**Publicaciones en el Dashboard**
Descripción	Como propietario, quiero consultar mis vehículos publicados en el panel, para verificar su disponibilidad inmediata y estado operativo de renta.
Criterios de Aceptación	**Escenario 1: Listado de vehículos propios para propietario** **Given** se envía GET /api/v1/vehicles/my-listings con token Bearer JWT de propietario **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna el listado de vehículos registrados por el arrendador. **Escenario 2: Consulta sin publicaciones registradas** **Given** se envía GET /api/v1/vehicles/my-listings para un propietario sin vehículos **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna lista vacía con botón directo para crear la primera publicación.

User Story US-08: Cancelar Reserva desde el Dashboard

Atributo	Detalle
Story ID	**US-08**
Épica	**EP-02**: Panel de Control del Usuario (Dashboard)
Usuario	Arrendatario
Prioridad	Media
Título	**Cancelar Reserva desde el Dashboard**
Descripción	Como arrendatario, quiero cancelar una solicitud de reserva pendiente desde mi panel, para desistir del alquiler antes de que sea formalizado.
Criterios de Aceptación	**Escenario 1: Cancelación exitosa de reserva pendiente** **Given** se envía PUT /api/v1/bookings/{bookingId}/cancel con reserva en estado PENDING **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna la reserva con estado actualizado a CANCELLED. **Escenario 2: Intento de cancelación de reserva ya completada** **Given** se envía PUT /api/v1/bookings/{bookingId}/cancel con reserva en estado COMPLETED **When** se procesa la petición **Then** el sistema retorna error 400 Bad Request **And** retorna mensaje de error apropiado indicando que no es posible cancelar la reserva.

Épica EP-03: Gestión de Flota y Publicaciones de Vehículos

Como propietario, quiero registrar, actualizar el estado, subir fotos y administrar mis vehículos publicados para ponerlos a disposición de los arrendatarios en la plataforma.

User Story US-09: Gestión de Flota Propia

Atributo	Detalle
Story ID	**US-09**
Épica	**EP-03**: Gestión de Flota y Publicaciones de Vehículos
Usuario	Propietario de vehículo
Prioridad	Media
Título	**Gestión de Flota Propia**
Descripción	Como propietario, quiero acceder a la sección de mi flota vehicular, para administrar de manera organizada todos mis vehículos y publicaciones.
Criterios de Aceptación	**Escenario 1: Carga de flota vehicular del propietario** **Given** se envía GET /api/v1/vehicles/my-listings con token de rol ROLE_ARRENDADOR **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna el listado de vehículos con fotos, tarifas y estado de disponibilidad. **Escenario 2: Acceso por usuario con rol no autorizado** **Given** se envía GET /api/v1/vehicles/my-listings con token de usuario ROLE_ARRENDATARIO **When** se procesa la petición **Then** el sistema retorna error 403 Forbidden **And** retorna mensaje de error apropiado indicando acceso exclusivo a propietarios.

User Story US-10: Crear Publicación de Vehículo

Atributo	Detalle
Story ID	**US-10**
Épica	**EP-03**: Gestión de Flota y Publicaciones de Vehículos
Usuario	Propietario de vehículo
Prioridad	Alta
Título	**Crear Publicación de Vehículo**
Descripción	Como propietario, quiero registrar un nuevo vehículo con datos técnicos y fotografía en formato multipart, para ponerlo en alquiler dentro del catálogo.
Criterios de Aceptación	**Escenario 1: Registro exitoso de nuevo vehículo** **Given** se envía POST /api/v1/vehicles con multipart form data (marca, modelo, año, precio e imagen) **When** se procesa la petición **Then** el sistema retorna código HTTP 201 Created **And** retorna el vehículo registrado con estado inicial AVAILABLE. **Escenario 2: Registro con campos obligatorios vacíos o precio inválido** **Given** se envía POST /api/v1/vehicles con datos incompletos o precio menor o igual a cero **When** se procesa la petición **Then** el sistema retorna error 400 Bad Request **And** retorna mensaje de error apropiado detallando los campos requeridos.

User Story US-11: Visualizar Publicación de Vehículo

Atributo	Detalle
Story ID	**US-11**
Épica	**EP-03**: Gestión de Flota y Publicaciones de Vehículos
Usuario	Propietario y Arrendatario
Prioridad	Alta
Título	**Visualizar Publicación de Vehículo**
Descripción	Como usuario, quiero consultar el detalle de una publicación de vehículo, para revisar especificaciones técnicas, tarifa diaria y disponibilidad operativa.

Atributo	Detalle
Criterios de Aceptación	**Escenario 1: Consulta de publicación existente** **Given** se envía GET /api/v1/vehicles/{vehicleId} con identificador de vehículo válido **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna la ficha técnica, tarifa diaria y datos del propietario. **Escenario 2: Consulta de vehículo inexistente** **Given** se envía GET /api/v1/vehicles/{vehicleId} con identificador no registrado **When** se procesa la petición **Then** el sistema retorna error 404 Not Found **And** retorna mensaje de error apropiado indicando vehículo no encontrado.

User Story US-12: Descarga y Renderizado de Imagen Vehicular

Atributo	Detalle
Story ID	**US-12**
Épica	**EP-03**: Gestión de Flota y Publicaciones de Vehículos
Usuario	Usuario de la plataforma
Prioridad	Alta
Título	**Descarga y Renderizado de Imagen Vehicular**
Descripción	Como usuario, quiero visualizar la fotografía real del vehículo almacenada en la base de datos, para evaluar su estado físico y apariencia antes de rentar.
Criterios de Aceptación	**Escenario 1: Carga exitosa de fotografía vehicular** **Given** se envía GET /api/v1/vehicles/{vehicleId}/image con identificador de vehículo existente **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna el flujo binario de la imagen (JPEG/PNG) con cabecera Content-Type correspondiente. **Escenario 2: Vehículo sin imagen asociada** **Given** se envía GET /api/v1/vehicles/{vehicleId}/image para un auto sin fotografía **When** se procesa la petición **Then** el sistema retorna error 404 Not Found **And** retorna mensaje de error apropiado indicando imagen no disponible.

User Story US-13: Actualizar Datos y Especificaciones del Vehículo

Atributo	Detalle
Story ID	**US-13**
Épica	**EP-03**: Gestión de Flota y Publicaciones de Vehículos
Usuario	Propietario de vehículo
Prioridad	Media
Título	**Actualizar Datos y Especificaciones del Vehículo**
Descripción	Como propietario, quiero modificar las especificaciones, precio o fotografía de mi vehículo, para mantener actualizada la información de mi oferta.
Criterios de Aceptación	**Escenario 1: Actualización exitosa de datos vehiculares** **Given** se envía PUT /api/v1/vehicles/{vehicleId} con multipart form data y token de propietario **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna el vehículo actualizado con las nuevas especificaciones y tarifa. **Escenario 2: Actualización de vehículo inexistente** **Given** se envía PUT /api/v1/vehicles/{vehicleId} con identificador no registrado **When** se procesa la petición **Then** el sistema retorna error 404 Not Found **And** retorna mensaje de error apropiado indicando vehículo no encontrado.

User Story US-14: Eliminar Publicación de Vehículo

Atributo	Detalle
Story ID	**US-14**

Atributo	Detalle
Épica	**EP-03**: Gestión de Flota y Publicaciones de Vehículos
Usuario	Propietario de vehículo
Prioridad	Media
Título	**Eliminar Publicación de Vehículo**
Descripción	Como propietario, quiero dar de baja una publicación de vehículo, para retirarlo definitivamente del catálogo cuando ya no esté destinado a renta.
Criterios de Aceptación	**Escenario 1: Eliminación exitosa de vehículo** **Given** se envía DELETE /api/v1/vehicles/{vehicleId} con token de propietario **When** se procesa la petición **Then** el sistema retorna código HTTP 204 No Content **And** remueve la publicación del catálogo de forma permanente. **Escenario 2: Intento de eliminación sin autorización de propietario** **Given** se envía DELETE /api/v1/vehicles/{vehicleId} sin credenciales de propietario **When** se procesa la petición **Then** el sistema retorna error 401 Unauthorized **And** retorna mensaje de error apropiado impidiendo la baja del vehículo.

Épica EP-04: Catálogo, Búsqueda y Reserva de Vehículos

Como arrendatario, quiero explorar el catálogo de vehículos, filtrar por marcas, ordenar por precio y generar solicitudes de alquiler con cálculo automático de tarifas por días.

User Story US-15: Navegación de Catálogo Público

Atributo	Detalle
Story ID	**US-15**
Épica	**EP-04**: Catálogo, Búsqueda y Reserva de Vehículos
Usuario	Arrendatario y Visitante
Prioridad	Alta
Título	**Navegación de Catálogo Público**
Descripción	Como usuario, quiero explorar el catálogo general de vehículos disponibles, para revisar las alternativas de alquiler en la plataforma.
Criterios de Aceptación	**Escenario 1: Consulta del catálogo público de vehículos** **Given** se envía GET /api/v1/vehicles **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna la lista de vehículos disponibles con sus tarjetas informativas. **Escenario 2: Catálogo sin unidades disponibles** **Given** se envía GET /api/v1/vehicles cuando no existen autos registrados **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna lista vacía con mensaje orientador para reintentar la búsqueda.

User Story US-16: Ordenar Catálogo por Precio

Atributo	Detalle
Story ID	**US-16**
Épica	**EP-04**: Catálogo, Búsqueda y Reserva de Vehículos
Usuario	Arrendatario
Prioridad	Media
Título	**Ordenar Catálogo por Precio**
Descripción	Como arrendatario, quiero ordenar el catálogo de autos por tarifa diaria ascendente o descendente, para ajustar las opciones a mi presupuesto.
Criterios de Aceptación	**Escenario 1: Ordenamiento ascendente por tarifa diaria** **Given** se envía GET /api/v1/vehicles y se aplica ordenamiento por precio menor a mayor **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK

Atributo	Detalle
	And retorna los vehículos ordenados de menor a mayor precio por día. **Escenario 2: Ordenamiento descendente por tarifa diaria** **Given** se envía GET /api/v1/vehicles y se aplica ordenamiento por precio mayor a menor **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna los vehículos ordenados de mayor a menor precio por día.

User Story US-17: Filtrar Catálogo por Marca

Atributo	Detalle
Story ID	**US-17**
Épica	**EP-04**: Catálogo, Búsqueda y Reserva de Vehículos
Usuario	Arrendatario
Prioridad	Alta
Título	**Filtrar Catálogo por Marca**
Descripción	Como arrendatario, quiero filtrar los vehículos por marcas específicas (Toyota, Nissan, etc.), para encontrar rápidamente modelos de mi preferencia.
Criterios de Aceptación	**Escenario 1: Filtrado exitoso por marca específica** **Given** se envía GET /api/v1/vehicles y se aplica filtro por marca 'Toyota' **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna únicamente los vehículos correspondientes a la marca seleccionada. **Escenario 2: Filtrado por marca sin coincidencias** **Given** se envía GET /api/v1/vehicles y se aplica filtro por marca inexistente **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna lista vacía indicando que no hay autos de dicha marca.

User Story US-18: Ver Ficha Detallada de Vehículo

Atributo	Detalle
Story ID	**US-18**
Épica	**EP-04**: Catálogo, Búsqueda y Reserva de Vehículos
Usuario	Arrendatario
Prioridad	Alta
Título	**Ver Ficha Detallada de Vehículo**
Descripción	Como arrendatario, quiero abrir la ficha individual de un auto, para examinar su fotografía, especificaciones mecánicas y procedencia antes de rentar.
Criterios de Aceptación	**Escenario 1: Visualización completa de detalles técnicos** **Given** se envía GET /api/v1/vehicles/{vehicleId} para un vehículo publicado **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna galería de imágenes, características mecánicas, tarifa y estado del auto. **Escenario 2: Visualización de vehículo inexistente** **Given** se envía GET /api/v1/vehicles/{vehicleId} con identificador no existente **When** se procesa la petición **Then** el sistema retorna error 404 Not Found **And** retorna mensaje de error apropiado indicando publicación no disponible.

User Story US-19: Reservar Vehículo con Cálculo de Días

Atributo	Detalle
Story ID	**US-19**
Épica	**EP-04**: Catálogo, Búsqueda y Reserva de Vehículos
Usuario	Arrendatario
Prioridad	Alta

Atributo	Detalle
Título	**Reservar Vehículo con Cálculo de Días**
Descripción	Como arrendatario, quiero seleccionar fechas de inicio y fin de alquiler, para que el sistema calcule automáticamente el costo total por días y cree la solicitud.
Criterios de Aceptación	**Escenario 1: Creación exitosa de reserva por rango de fechas** **Given** se envía POST /api/v1/bookings con vehículo, fechas de inicio y fin válidas **When** se procesa la petición **Then** el sistema retorna código HTTP 201 Created **And** retorna la reserva en estado PENDING con el cálculo automático del precio total. **Escenario 2: Creación de reserva con fechas ya ocupadas** **Given** se envía POST /api/v1/bookings con fechas que se solapan con otra reserva confirmada **When** se procesa la petición **Then** el sistema retorna error 400 Bad Request **And** retorna mensaje de error apropiado indicando fechas no disponibles.

User Story US-20: Confirmar Solicitud de Reserva

Atributo	Detalle
Story ID	**US-20**
Épica	**EP-04**: Catálogo, Búsqueda y Reserva de Vehículos
Usuario	Propietario de vehículo
Prioridad	Alta
Título	**Confirmar Solicitud de Reserva**
Descripción	Como propietario, quiero aceptar una solicitud de reserva pendiente recibida, para formalizar el contrato digital de alquiler con el arrendatario.
Criterios de Aceptación	**Escenario 1: Aprobación de reserva por el propietario** **Given** se envía PUT /api/v1/bookings/{bookingId}/confirm por el dueño del auto **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna la reserva con estado actualizado a CONFIRMED. **Escenario 2: Confirmación por usuario no autorizado** **Given** se envía PUT /api/v1/bookings/{bookingId}/confirm por un usuario que no es el propietario **When** se procesa la petición **Then** el sistema retorna error 401 Unauthorized **And** retorna mensaje de error apropiado indicando permisos insuficientes.

User Story US-21: Rechazar Solicitud de Reserva

Atributo	Detalle
Story ID	**US-21**
Épica	**EP-04**: Catálogo, Búsqueda y Reserva de Vehículos
Usuario	Propietario de vehículo
Prioridad	Alta
Título	**Rechazar Solicitud de Reserva**
Descripción	Como propietario, quiero declinar una solicitud de alquiler cuando no me sea posible atenderla, para liberar las fechas del vehículo en el calendario.
Criterios de Aceptación	**Escenario 1: Rechazo de solicitud por el propietario** **Given** se envía PUT /api/v1/bookings/{bookingId}/reject con token de propietario **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna la reserva con estado REJECTED y libera las fechas en el calendario. **Escenario 2: Rechazo sobre reserva inexistente** **Given** se envía PUT /api/v1/bookings/{bookingId}/reject con identificador no registrado **When** se procesa la petición **Then** el sistema retorna error 404 Not Found **And** retorna mensaje de error apropiado indicando reserva no encontrada.

User Story US-22: Historial de Reservas del Arrendatario

Atributo	Detalle
Story ID	**US-22**
Épica	**EP-04**: Catálogo, Búsqueda y Reserva de Vehículos
Usuario	Arrendatario
Prioridad	Media
Título	**Historial de Reservas del Arrendatario**
Descripción	Como arrendatario, quiero consultar el historial cronológico de todas mis reservas, para revisar contratos pasados, activos y pendientes de aprobación.
Criterios de Aceptación	**Escenario 1: Consulta de reservas del cliente autenticado** **Given** se envía GET /api/v1/bookings/my-bookings con token Bearer JWT **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna el historial cronológico de alquileres pasados, activos y pendientes. **Escenario 2: Cliente sin reservas previas** **Given** se envía GET /api/v1/bookings/my-bookings para usuario nuevo **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna lista vacía con botón de enlace directo al catálogo de vehículos.

Épica EP-05: Monitoreo Telemático IoT y Simulación de Rutas

Como arrendatario o propietario, quiero visualizar la posición GPS del vehículo en un mapa interactivo (Leaflet), velocidad y nivel de combustible en tiempo real durante el periodo de alquiler.

User Story US-23: Consulta de Historial Telemático por Vehículo

Atributo	Detalle
Story ID	**US-23**
Épica	**EP-05**: Monitoreo Telemático IoT y Simulación de Rutas
Usuario	Arrendatario y Propietario
Prioridad	Alta
Título	**Consulta de Historial Telemático por Vehículo**
Descripción	Como usuario involucrado en un alquiler, quiero consultar el historial telemático del vehículo, para analizar su comportamiento en ruta durante el servicio.
Criterios de Aceptación	**Escenario 1: Consulta de telemetría de vehículo** **Given** se envía GET /api/v1/telemetry/vehicle/{vehicleId} con token autorizado **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna la lista de lecturas telemáticas con coordenadas, velocidad y combustible. **Escenario 2: Consulta por usuario no autorizado** **Given** se envía GET /api/v1/telemetry/vehicle/{vehicleId} sin autorización sobre el vehículo **When** se procesa la petición **Then** el sistema retorna error 403 Forbidden **And** retorna mensaje de error apropiado indicando acceso denegado a telemetría.

User Story US-24: Visualizar Última Posición GPS en Mapa Leaflet

Atributo	Detalle
Story ID	**US-24**
Épica	**EP-05**: Monitoreo Telemático IoT y Simulación de Rutas
Usuario	Arrendatario y Propietario
Prioridad	Alta
Título	**Visualizar Última Posición GPS en Mapa Leaflet**
Descripción	Como usuario, quiero consultar el último punto telemático con coordenadas GPS, velocidad y nivel de combustible, para ubicar el auto en tiempo real sobre el mapa Leaflet.

Atributo	Detalle
Criterios de Aceptación	**Escenario 1: Obtención del último registro telemático** **Given** se envía GET /api/v1/telemetry/vehicle/{vehicleId}/latest con reserva activa **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna latitud, longitud, velocidad, combustible y timestamp del último punto. **Escenario 2: Consulta sin lecturas satelitales disponibles** **Given** se envía GET /api/v1/telemetry/vehicle/{vehicleId}/latest y no existen registros telemáticos **When** se procesa la petición **Then** el sistema retorna error 404 Not Found **And** retorna mensaje de error apropiado indicando datos telemáticos no disponibles.

User Story US-25: Ingesta de Telemetría IoT

Atributo	Detalle
Story ID	**US-25**
Épica	**EP-05**: Monitoreo Telemático IoT y Simulación de Rutas
Usuario	Dispositivo IoT / Sensor telemático
Prioridad	Media
Título	**Ingesta de Telemetría IoT**
Descripción	Como módulo telemático del vehículo, quiero transmitir lecturas de sensores por HTTP REST, para que el sistema almacene los datos de velocidad, combustible y posición.
Criterios de Aceptación	**Escenario 1: Recepción e ingesta de paquete telemático** **Given** se envía POST /api/v1/telemetry con payload de coordenadas, velocidad y combustible **When** se procesa la petición **Then** el sistema retorna código HTTP 201 Created **And** persiste la lectura telemática en la base de datos con marca de tiempo. **Escenario 2: Ingesta telemática con datos incompletos** **Given** se envía POST /api/v1/telemetry con payload sin vehículo asociado **When** se procesa la petición **Then** el sistema retorna error 400 Bad Request **And** retorna mensaje de error apropiado rechazando la trama.

User Story US-26: Iniciar Simulación Telemática en Ruta

Atributo	Detalle
Story ID	**US-26**
Épica	**EP-05**: Monitoreo Telemático IoT y Simulación de Rutas
Usuario	Propietario de vehículo
Prioridad	Media
Título	**Iniciar Simulación Telemática en Ruta**
Descripción	Como propietario, quiero iniciar una simulación realista de movimiento vehicular por carreteras reales, para verificar la recepción telemática en tiempo real.
Criterios de Aceptación	**Escenario 1: Inicio exitoso de simulación de ruta** **Given** se envía POST /api/v1/telemetry/simulate/{vehicleId} con token de propietario **When** se procesa la petición **Then** el sistema retorna código HTTP 202 Accepted **And** inicia el hilo de emisión de telemetría periódica a lo largo de la ruta. **Escenario 2: Intento de simulación por usuario no propietario** **Given** se envía POST /api/v1/telemetry/simulate/{vehicleId} con token de arrendatario **When** se procesa la petición **Then** el sistema retorna error 403 Forbidden **And** retorna mensaje de error apropiado indicando acceso restringido al propietario.

User Story US-27: Cálculo de Ruta entre Coordenadas

Atributo	Detalle
Story ID	**US-27**

Atributo	Detalle
Épica	**EP-05**: Monitoreo Telemático IoT y Simulación de Rutas
Usuario	Usuario de la plataforma
Prioridad	Media
Título	**Cálculo de Ruta entre Coordenadas**
Descripción	Como usuario, quiero obtener la geometría de ruta y puntos de navegación entre dos coordenadas, para visualizar el trayecto vehicular planificado sobre el mapa.
Criterios de Aceptación	**Escenario 1: Cálculo exitoso de geometría de ruta** **Given** se envía GET /api/v1/simulation/route con coordenadas de inicio y fin válidas **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna la matriz de coordenadas intermedias calculadas por OpenRouteService. **Escenario 2: Solicitud con coordenadas inválidas** **Given** se envía GET /api/v1/simulation/route con parámetros de latitud o longitud incorrectos **When** se procesa la petición **Then** el sistema retorna error 400 Bad Request **And** retorna mensaje de error apropiado indicando coordenadas inválidas.

Épica EP-06: Sistema de Reputación, Reseñas y Calificaciones

Como arrendatario, quiero calificar y dejar comentarios sobre el vehículo y la experiencia de alquiler una vez concluido el servicio para orientar a la comunidad y fomentar la confianza.

User Story US-28: Publicar Reseña de Alquiler

Atributo	Detalle
Story ID	**US-28**
Épica	**EP-06**: Sistema de Reputación, Reseñas y Calificaciones
Usuario	Arrendatario
Prioridad	Alta
Título	**Publicar Reseña de Alquiler**
Descripción	Como arrendatario, quiero registrar una calificación de 1 a 5 estrellas y un comentario tras un servicio, para compartir mi experiencia en la plataforma.
Criterios de Aceptación	**Escenario 1: Publicación exitosa de reseña** **Given** se envía POST /api/v1/reviews con vehículo, puntuación válida y comentario **When** se procesa la petición **Then** el sistema retorna código HTTP 201 Created **And** retorna la reseña persistida y recalcula el puntaje promedio del vehículo. **Escenario 2: Envío de reseña con puntuación fuera de rango** **Given** se envía POST /api/v1/reviews con puntuación menor a 1 o mayor a 5 **When** se procesa la petición **Then** el sistema retorna error 400 Bad Request **And** retorna mensaje de error apropiado indicando rango de calificación válido.

User Story US-29: Consultar Reseñas por Vehículo

Atributo	Detalle
Story ID	**US-29**
Épica	**EP-06**: Sistema de Reputación, Reseñas y Calificaciones
Usuario	Arrendatario y Visitante
Prioridad	Media
Título	**Consultar Reseñas por Vehículo**
Descripción	Como usuario, quiero leer las opiniones y ver las valoraciones de un vehículo específico, para evaluar su confiabilidad antes de solicitar una reserva.
Criterios de Aceptación	**Escenario 1: Consulta de opiniones de un vehículo** **Given** se envía GET /api/v1/reviews/vehicle/{vehicleId} con identificador de auto **When** se procesa la petición

Atributo	Detalle
	Then el sistema retorna código HTTP 200 OK **And** retorna la lista de reseñas con autor, estrellas y comentarios registrados. **Escenario 2: Vehículo sin reseñas registradas** **Given** se envía GET /api/v1/reviews/vehicle/{vehicleId} para un auto sin calificaciones **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna lista vacía indicando ausencia de reseñas para el vehículo.

User Story US-30: Consultar Mis Reseñas Publicadas

Atributo	Detalle
Story ID	**US-30**
Épica	**EP-06**: Sistema de Reputación, Reseñas y Calificaciones
Usuario	Arrendatario
Prioridad	Media
Título	**Consultar Mis Reseñas Publicadas**
Descripción	Como arrendatario, quiero consultar el historial de todas las reseñas que he emitido, para dar seguimiento a mis opiniones y valoraciones.
Criterios de Aceptación	**Escenario 1: Historial de valoraciones del usuario** **Given** se envía GET /api/v1/reviews/my-reviews con token de rol ROLE_ARRENDATARIO **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna el consolidado de comentarios y calificaciones emitidas por el arrendatario. **Escenario 2: Consulta sin autenticación activa** **Given** se envía GET /api/v1/reviews/my-reviews sin token Bearer JWT **When** se procesa la petición **Then** el sistema retorna error 401 Unauthorized **And** retorna mensaje de error apropiado solicitando inicio de sesión.

Épica EP-07: Administración de Perfil y Preferencias

Como usuario registrado, quiero consultar y editar mis datos personales de contacto para mantener vigente mi información en el ecosistema RentiCar.

User Story US-31: Visualización y Modificación de Perfil

Atributo	Detalle
Story ID	**US-31**
Épica	**EP-07**: Administración de Perfil y Preferencias
Usuario	Usuario registrado
Prioridad	Media
Título	**Visualización y Modificación de Perfil**
Descripción	Como usuario registrado, quiero consultar y editar mis datos personales de perfil, para mantener actualizada mi información de contacto en el sistema.
Criterios de Aceptación	**Escenario 1: Actualización exitosa de datos de perfil** **Given** se envía PATCH /api/v1/users/{userId} con nuevos datos personales y token Bearer JWT **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** retorna el perfil de usuario actualizado con confirmación de cambios guardados. **Escenario 2: Intento de actualización de perfil ajeno** **Given** se envía PATCH /api/v1/users/{userId} con identificador de otro usuario **When** se procesa la petición **Then** el sistema retorna error 401 Unauthorized **And** retorna mensaje de error apropiado impidiendo la modificación de cuentas ajenas.

Épica EP-08: Landing Page Informativa y Captación

Como visitante, quiero navegar por una página de inicio atractiva y clara con información de valor, catálogo destacado, selector de idioma y formulario de suscripción para conocer el servicio.

User Story US-32: Acceso a Landing Page e Información del Servicio

Atributo	Detalle
Story ID	**US-32**
Épica	**EP-08**: Landing Page Informativa y Captación
Usuario	Visitante
Prioridad	Alta
Título	**Acceso a Landing Page e Información del Servicio**
Descripción	Como visitante, quiero acceder a la Landing Page pública, para conocer la propuesta de valor de RentiCar y las ventajas del alquiler colaborativo.
Criterios de Aceptación	**Escenario 1: Carga inicial de la Landing Page** **Given** el visitante accede a la URL principal de la Landing Page **When** se procesa la petición **Then** el sistema retorna código HTTP 200 OK **And** renderiza la página de inicio con la propuesta de valor, hero banner y enlaces de registro. **Escenario 2: Solicitud de recurso inexistente en la plataforma** **Given** el visitante intenta acceder a una ruta inexistente en la plataforma web **When** se procesa la petición **Then** el sistema retorna código HTTP 404 Not Found **And** muestra la página de error personalizada PageNotFound con botón de retorno al inicio.

User Story US-33: Catálogo Popular y Testimonios en Landing Page

Atributo	Detalle
Story ID	**US-33**
Épica	**EP-08**: Landing Page Informativa y Captación
Usuario	Visitante
Prioridad	Media
Título	**Catálogo Popular y Testimonios en Landing Page**
Descripción	Como visitante, quiero ver una muestra del catálogo destacado y testimonios de usuarios en la página de inicio, para comprobar la calidad del servicio antes de registrarme.
Criterios de Aceptación	**Escenario 1: Visualización de catálogo destacado y opiniones** **Given** el visitante desciende hacia la sección de destacados en la Landing Page **When** se procesa la acción **Then** el sistema retorna código HTTP 200 OK **And** despliega las tarjetas de vehículos populares y testimonios de arrendatarios y dueños. **Escenario 2: Interacción con tarjeta de vehículo destacado** **Given** el visitante selecciona un vehículo de la sección de destacados **When** se procesa la acción **Then** el sistema redirige hacia la aplicación web en /login **And** solicita inicio de sesión o registro para continuar con el alquiler.

User Story US-34: Cambio de Idioma en la Plataforma

Atributo	Detalle
Story ID	**US-34**
Épica	**EP-08**: Landing Page Informativa y Captación
Usuario	Visitante y Usuario
Prioridad	Media
Título	**Cambio de Idioma en la Plataforma**
Descripción	Como usuario, quiero cambiar el idioma de la plataforma entre español e inglés, para navegar en mi lengua de preferencia mediante el componente LanguageSwitcher.
Criterios de Aceptación	**Escenario 1: Selección de idioma inglés** **Given** el usuario interactúa con el componente LanguageSwitcher y selecciona 'EN' **When** se procesa la acción **Then** el sistema aplica el paquete de internacionalización i18n **And** actualiza todos los textos de la interfaz al idioma inglés de manera inmediata.

Atributo	Detalle
	Escenario 2: Retorno a idioma español predeterminado **Given** la interfaz se encuentra en inglés y el usuario selecciona 'ES' **When** se procesa la acción **Then** el sistema aplica el paquete de idioma en español **And** restablece todos los textos y descripciones a su versión original en español.

User Story US-35: Formulario Call to Action (Suscripción)

Atributo	Detalle
Story ID	**US-35**
Épica	**EP-08**: Landing Page Informativa y Captación
Usuario	Visitante interesado
Prioridad	Baja
Título	**Formulario Call to Action (Suscripción)**
Descripción	Como visitante interesado, quiero ingresar mi correo en el formulario de contacto o suscripción de la Landing Page, para recibir noticias, promociones y novedades sobre lanzamientos.
Criterios de Aceptación	**Escenario 1: Envío exitoso de correo para suscripción** **Given** el visitante ingresa una dirección de correo válida en el formulario de suscripción y presiona enviar **When** se procesa la acción **Then** el sistema valida el formato de correo electrónico **And** muestra mensaje de confirmación de suscripción exitosa a novedades. **Escenario 2: Envío de correo con formato inválido** **Given** el visitante ingresa un texto con formato de correo incorrecto **When** se procesa la acción **Then** el sistema activa la validación del formulario **And** muestra aviso de error solicitando el ingreso de una dirección de correo válida.

3.3. Product Backlog

A continuación, se presenta la priorización del Product Backlog general para la plataforma **RentiCar**, estimando el esfuerzo relativo de desarrollo de cada User Story en Story Points y documentando el título y la descripción completa de cada requerimiento funcional:

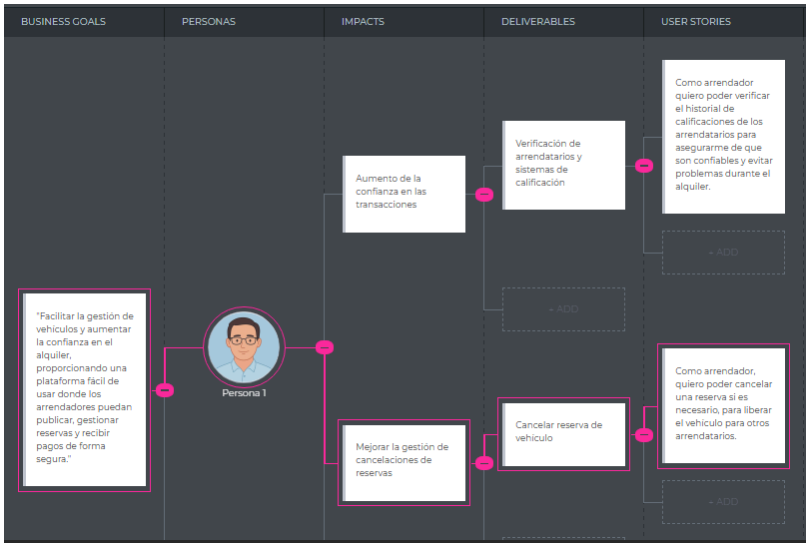
# Orden	StoryID	Título	Descripción	Story Points (1/3/5/8)
1	US-01	Registro de Usuario	Como visitante, quiero registrar una cuenta con mis datos personales y rol en la plataforma, para poder publicar vehículos o solicitar alquileres de manera segura.	3
2	US-02	Inicio de Sesión (Log In)	Como usuario registrado, quiero iniciar sesión con mis credenciales, para acceder a mis publicaciones, reservas y funcionalidades protegidas de la plataforma.	3
3	US-03	Actualizar Contraseña de Usuario	Como usuario autenticado, quiero actualizar mi contraseña actual por una nueva, para mantener protegida mi cuenta en la plataforma.	3
4	US-04	Eliminación de Cuenta de Usuario	Como usuario autenticado, quiero eliminar mi cuenta de la plataforma, para remover definitivamente mis datos y perfil del sistema.	2
5	US-05	Dashboard del Arrendatario	Como arrendatario, quiero acceder a mi panel principal, para visualizar el estado de mis reservas activas y atajos a mis vehículos contratados.	3
6	US-06	Dashboard del Propietario	Como propietario, quiero acceder a mi panel principal, para inspeccionar las solicitudes entrantes de alquiler y el estado general de mi flota.	3
7	US-07	Publicaciones en el Dashboard	Como propietario, quiero consultar mis vehículos publicados en el panel, para verificar su disponibilidad inmediata y estado operativo de renta.	3
8	US-08	Cancelar Reserva desde el Dashboard	Como arrendatario, quiero cancelar una solicitud de reserva pendiente desde mi panel, para desistir del alquiler antes de que sea formalizado.	3
9	US-09	Gestión de Flota Propia	Como propietario, quiero acceder a la sección de mi flota vehicular, para administrar de manera organizada todos mis vehículos y publicaciones.	2
10	US-10	Crear Publicación de Vehículo	Como propietario, quiero registrar un nuevo vehículo con datos técnicos y fotografía en formato multipart, para ponerlo en alquiler dentro del catálogo.	5
11	US-11	Visualizar Publicación de Vehículo	Como usuario, quiero consultar el detalle de una publicación de vehículo, para revisar especificaciones técnicas, tarifa diaria y disponibilidad operativa.	3

# Orden	StoryID	Título	Descripción	Story Points (1/3/5/8)
12	US-12	Descarga y Renderizado de Imagen Vehicular	Como usuario, quiero visualizar la fotografía real del vehículo almacenada en la base de datos, para evaluar su estado físico y apariencia antes de rentar.	3
13	US-13	Actualizar Datos y Especificaciones del Vehículo	Como propietario, quiero modificar las especificaciones, precio o fotografía de mi vehículo, para mantener actualizada la información de mi oferta.	3
14	US-14	Eliminar Publicación de Vehículo	Como propietario, quiero dar de baja una publicación de vehículo, para retirarlo definitivamente del catálogo cuando ya no esté destinado a renta.	2
15	US-15	Navegación de Catálogo Público	Como usuario, quiero explorar el catálogo general de vehículos disponibles, para revisar las alternativas de alquiler en la plataforma.	5
16	US-16	Ordenar Catálogo por Precio	Como arrendatario, quiero ordenar el catálogo de autos por tarifa diaria ascendente o descendente, para ajustar las opciones a mi presupuesto.	2
17	US-17	Filtrar Catálogo por Marca	Como arrendatario, quiero filtrar los vehículos por marcas específicas (Toyota, Nissan, etc.), para encontrar rápidamente modelos de mi preferencia.	3
18	US-18	Ver Ficha Detallada de Vehículo	Como arrendatario, quiero abrir la ficha individual de un auto, para examinar su fotografía, especificaciones mecánicas y procedencia antes de rentar.	5
19	US-19	Reservar Vehículo con Cálculo de Días	Como arrendatario, quiero seleccionar fechas de inicio y fin de alquiler, para que el sistema calcule automáticamente el costo total por días y cree la solicitud.	5
20	US-20	Confirmar Solicitud de Reserva	Como propietario, quiero aceptar una solicitud de reserva pendiente recibida, para formalizar el contrato digital de alquiler con el arrendatario.	5
21	US-21	Rechazar Solicitud de Reserva	Como propietario, quiero declinar una solicitud de alquiler cuando no me sea posible atenderla, para liberar las fechas del vehículo en el calendario.	3
22	US-22	Historial de Reservas del Arrendatario	Como arrendatario, quiero consultar el historial cronológico de todas mis reservas, para revisar contratos pasados, activos y pendientes de aprobación.	3
23	US-23	Consulta de Historial Telemático por Vehículo	Como usuario involucrado en un alquiler, quiero consultar el historial telemático del vehículo, para analizar su comportamiento en ruta durante el servicio.	5
24	US-24	Visualizar Última Posición GPS en Mapa Leaflet	Como usuario, quiero consultar el último punto telemático con coordenadas GPS, velocidad y nivel de combustible, para ubicar el auto en tiempo real sobre el mapa Leaflet.	8
25	US-25	Ingesta de Telemetría IoT	Como módulo telemático del vehículo, quiero transmitir lecturas de sensores por HTTP REST, para que el sistema almacene los datos de velocidad, combustible y posición.	5
26	US-26	Iniciar Simulación Telemática en Ruta	Como propietario, quiero iniciar una simulación realista de movimiento vehicular por carreteras reales, para verificar la recepción telemática en tiempo real.	5
27	US-27	Cálculo de Ruta entre Coordenadas	Como usuario, quiero obtener la geometría de ruta y puntos de navegación entre dos coordenadas, para visualizar el trayecto vehicular planificado sobre el mapa.	3
28	US-28	Publicar Reseña de Alquiler	Como arrendatario, quiero registrar una calificación de 1 a 5 estrellas y un comentario tras un servicio, para compartir mi experiencia en la plataforma.	3
29	US-29	Consultar Reseñas por Vehículo	Como usuario, quiero leer las opiniones y ver las valoraciones de un vehículo específico, para evaluar su confiabilidad antes de solicitar una reserva.	2
30	US-30	Consultar Mis Reseñas Publicadas	Como arrendatario, quiero consultar el historial de todas las reseñas que he emitido, para dar seguimiento a mis opiniones y valoraciones.	3
31	US-31	Visualización y Modificación de Perfil	Como usuario registrado, quiero consultar y editar mis datos personales de perfil, para mantener actualizada mi información de contacto en el sistema.	3
32	US-32	Acceso a Landing Page e Información del Servicio	Como visitante, quiero acceder a la Landing Page pública, para conocer la propuesta de valor de RentiCar y las ventajas del alquiler colaborativo.	3
33	US-33	Catálogo Popular y Testimonios en Landing Page	Como visitante, quiero ver una muestra del catálogo destacado y testimonios de usuarios en la página de inicio, para comprobar la calidad del servicio antes de registrarme.	3
34	US-34	Cambio de Idioma en la Plataforma	Como usuario, quiero cambiar el idioma de la plataforma entre español e inglés, para navegar en mi lengua de preferencia mediante el componente LanguageSwitcher.	5
35	US-35	Formulario Call to Action (Suscripción)	Como visitante interesado, quiero ingresar mi correo en el formulario de contacto o suscripción de la Landing Page, para recibir noticias, promociones y novedades sobre lanzamientos.	2

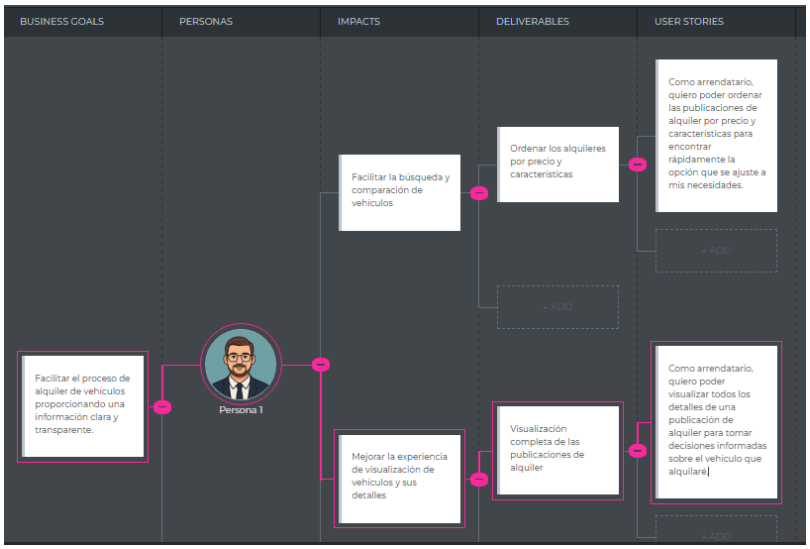
3.4. Impact Mapping

El **Impact Mapping** es una técnica de planificación estratégica que permite vincular los **objetivos del proyecto** con los **actores involucrados**, los **impactos esperados** y las **entregas o funcionalidades necesarias** para alcanzarlos. Esta herramienta ayuda a mantener la trazabilidad entre lo que se quiere lograr y las acciones que se deben implementar, asegurando que cada funcionalidad aporte valor real al sistema. En el caso de *Renticar*, el Impact Mapping se utiliza para alinear los intereses de arrendadores y arrendatarios con los objetivos del producto, priorizando la seguridad, la transparencia y la confianza como ejes principales de la plataforma.

Segmento Objetivo 1 (Arrendador de vehículos)



Segmento Objetivo 2 (Arrendatario de vehículos)



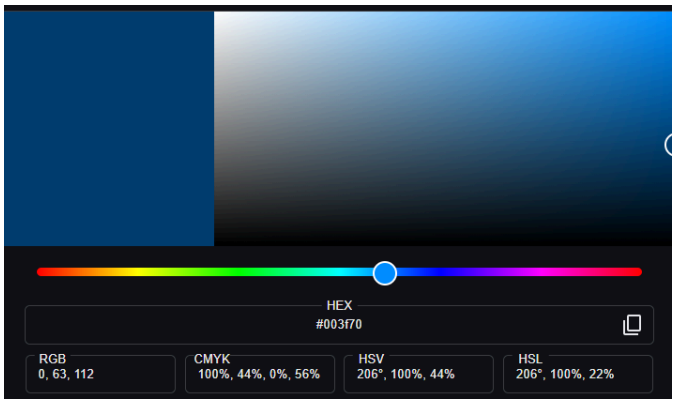
Capítulo IV: Product Design

4.1. Style Guidelines

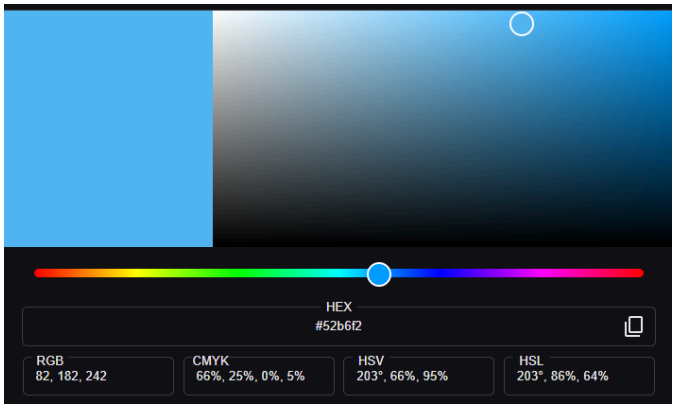
En esta sección se establecen los lineamientos de diseño orientados a garantizar la coherencia tanto visual como funcional del producto. Dichos lineamientos abarcan elementos de la identidad corporativa, la tipografía, la paleta de colores, el uso de íconos, así como los componentes interactivos —como botones— y su relación directa con la experiencia del usuario.

4.1.1. General Style Guidelines

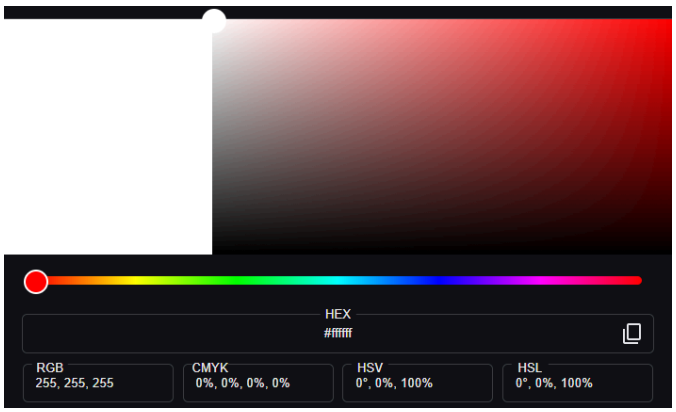
- **Branding**:
    - **Color Primario**: #003F70 (Azul oscuro) – Utilizado en el fondo, elementos destacados y menús. Este color representa seguridad y confianza, ideal para una plataforma de alquiler de vehículos.



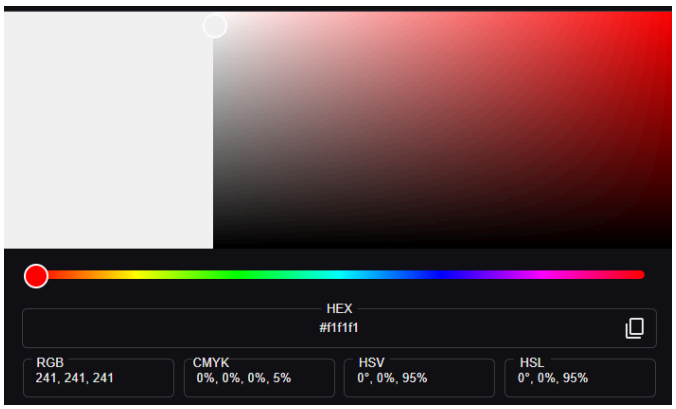
- **Color Secundario**: #52B6F2 (Azul claro) – Es el color de los botones principales, como "Sign up" y "Search". Este color añade un toque de frescura y accesibilidad, además de hacer que las llamadas a la acción sean visualmente atractivas.



- **Color de Fondo**: #FFFFFF (Blanco) – Se utiliza para el fondo general de la página, proporcionando claridad y asegurando que los elementos visuales no se sobrecarguen.

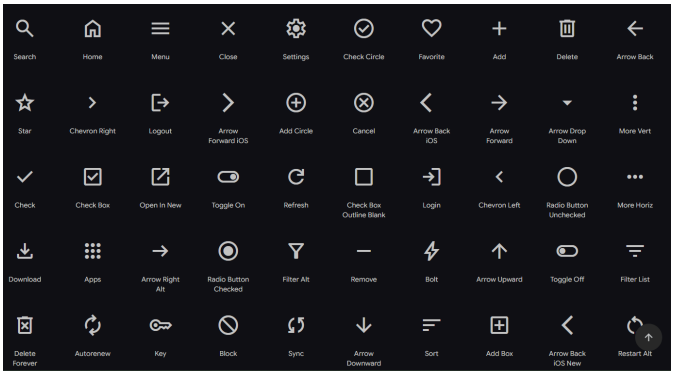


- **Colore Secundarios de Contraste**: #F1F1F1 (Gris claro) para las áreas de soporte visual, como formularios o tarjetas de contenido. Además, #1F2A44 (gris oscuro) se usa para textos secundarios y menús.

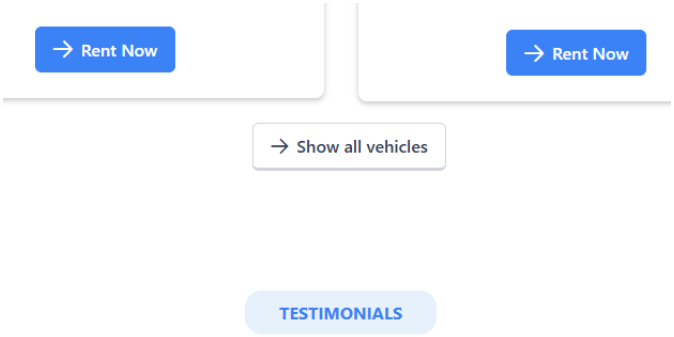


- **Typography**:
  Se ha elegido la tipografía Poppins debido a su claridad, modernidad y legibilidad. Las distintas variaciones de peso permiten crear una jerarquía visual efectiva.
  - **ExtraBold**: Usado para encabezados y titulos.
  - **Semibold**: Para subtítulos o elementos secundarios.
  - **Regular**: Para el cuerpo del texto.
  - **Light**: Para textos menos relevantes o secundarios.

- **Icons**:
  Los íconos son lineales y minimalistas, alineados con el estilo moderno de la plataforma. Se utilizan para una comunicación clara y rápida, permitiendo a los usuarios identificar fácilmente las acciones disponibles.



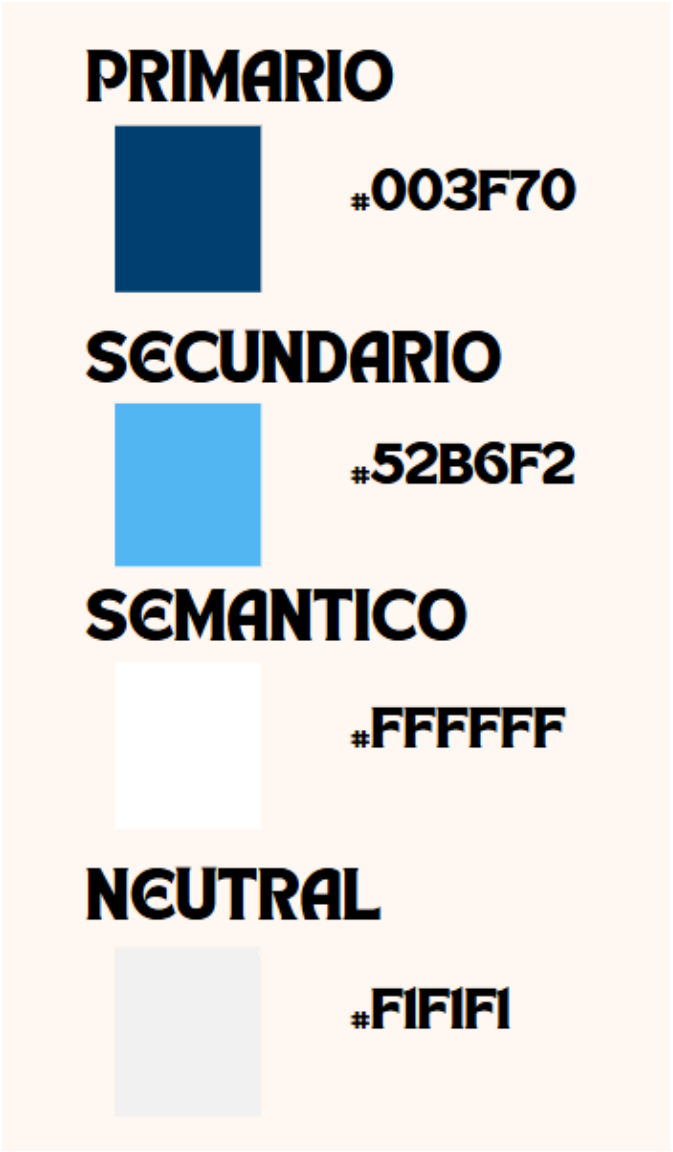
- **Buttons**:
  Los botones deben ser altamente visibles y accesibles. Usamos colores vibrantes como #52B6F2 (Azul claro) para las acciones principales y bordes redondeados para mejorar la interacción en dispositivos táctiles.



- **Spacing and Layout**:
  Se establece un sistema de espaciado uniforme, lo que garantiza la claridad visual y un flujo de contenido bien organizado. El espacio adecuado entre los elementos facilita la navegación sin que la interfaz se sienta sobrecargada.

4.1.2. Web Style Guidelines

- **Responsive Design**: Todo el diseño está optimizado para ser completamente responsivo. Esto garantiza que los elementos sean accesibles y fáciles de interactuar en cualquier dispositivo, desde móviles hasta escritorios.
- **Color Accessibility**: Los colores están seleccionados con criterios de contraste adecuados, cumpliendo con los estándares de accesibilidad (WCAG). Todo el texto será legible y fácil de leer, con alternativas de alto contraste para usuarios con discapacidades visuales.



- **Hover and Active States**: Los botones, enlaces e iconos incluyen estados de "hover" y "active" claramente definidos, lo cual proporciona retroalimentación visual a los usuarios. Esto es esencial para mejorar la experiencia de usuario y evitar errores.
- **Performance Optimization**: Se priorizan diseños ligeros que cargan rápidamente, utilizando imágenes y recursos optimizados. La eficiencia en la carga es crucial para mantener la fluidez del sitio en un entorno de uso rápido.
- **Navigation and Interactivity**: La navegación está diseñada para ser intuitiva, con accesos directos a funciones clave y un uso mínimo de submenús. Los elementos interactivos incluyen retroalimentación visual y táctil para mejorar la experiencia del usuario.
- **Consistent Component Usage**: Se utilizan componentes reutilizables y consistentes en toda la interfaz web, como botones, tarjetas y formularios. Esto garantiza una experiencia de usuario cohesiva y facilita el mantenimiento y la escalabilidad del código.

4.1.3. Mobile Style Guidelines

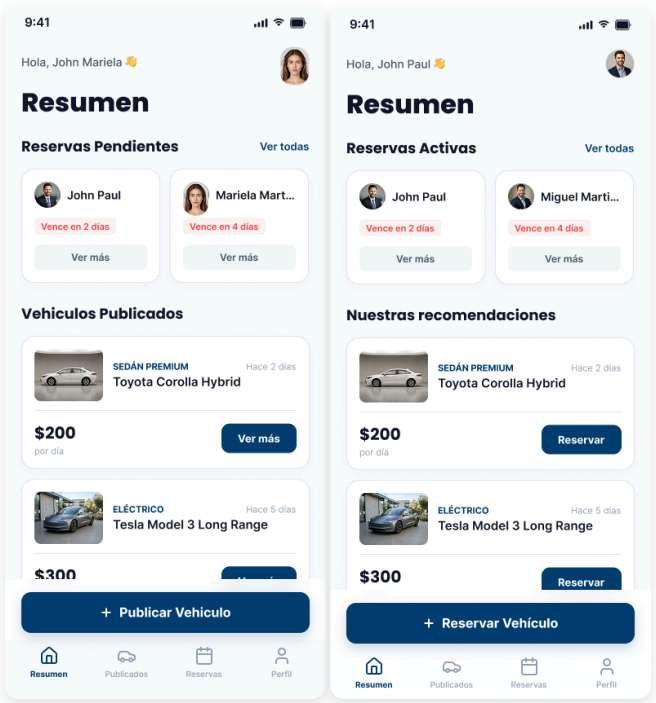
Para las versiones móviles de Renticar, los lineamientos de diseño mantienen una identidad visual entre iOS y Android, conservando la paleta cromática principal de las cuales solo se integro un ligero cambios(como el #2E4050 y #52B6F2). Asismismo, se sigue manteniendo la misma tipografía, la iconografía y los componentes principales de la plataforma. La interfaz se adapta a las características de cada sistema operativo mediante ajustes en la distribución, las dimensiones y los elementos de navegación, garantizando una experiencia consistente, intuitiva y accesible en dispositivos móviles.Todo ello con el objetivo de facilitar la navegación y el uso de las funcionalidades tanto para arrendadores como para arrendatarios.

4.1.3.1. iOS Mobile Style Guidelines

- **Layout y Retícula Base:** La arquitectura visual sigue un flujo vertical continuo sobre una retícula modular de 8pt, complementada con márgenes de seguridad laterales de 16pt a 20pt. Soporta de forma nativa los gestos del sistema, permitiendo al usuario deslizar desde el borde izquierdo hacia la derecha .
- **Navegación e Interfaz Dinámica por Rol:**
  - **Navigation Bar y Large Titles:** El encabezado superior presenta títulos alineados a la izquierda, utilizando la tipografía Poppins ExtraBold para mantener la identidad visual de Renticar. El título cambia dinámicamente según la sección seleccionada en la barra de navegación inferior: Resumen, Publicados o Mis vehículos, Reservas o Solicitudes, y Perfil, de acuerdo con el tipo de usuario.

- **Adaptabilidad de las 4 Pestañas Principales: Resumen:** Muestra un panel personalizado según el perfil activo. Para el Arrendador, consolida métricas de ingresos y estado del negocio; para el Arrendatario, despliega accesos rápidos a la búsqueda y sugerencias sobre otros vehiculos.
- Tab Bar Inferior : a barra de navegación permanece fija en la parte inferior de la pantalla y contiene cuatro accesos principales. La pestaña seleccionada se distingue mediante el color azul (#52B6F2) aplicado al icono y, cuando corresponda, a sus elementos de texto.
- **Acción Principal:** La acción prioritaria de publicación (para Arrendador) o reserva (para Arrendatario) se integra como un botón principal fijo en la zona inferior sobre el área segura, con bordes redondeados de 12pt y altura de 50pt.

- **Interacción y Accesibilidad:**
  - **Respuesta de Opacidad y Háptica:** Disminución leve en la opacidad del componente al ser presionado, combinada con vibración de respuesta táctil mediante el motor háptico al confirmar acciones o alternar vistas.
  - **Áreas de Toque:** Superficie de contacto interactiva mínima de 44×44 pt para todos los componentes de la interfaz.



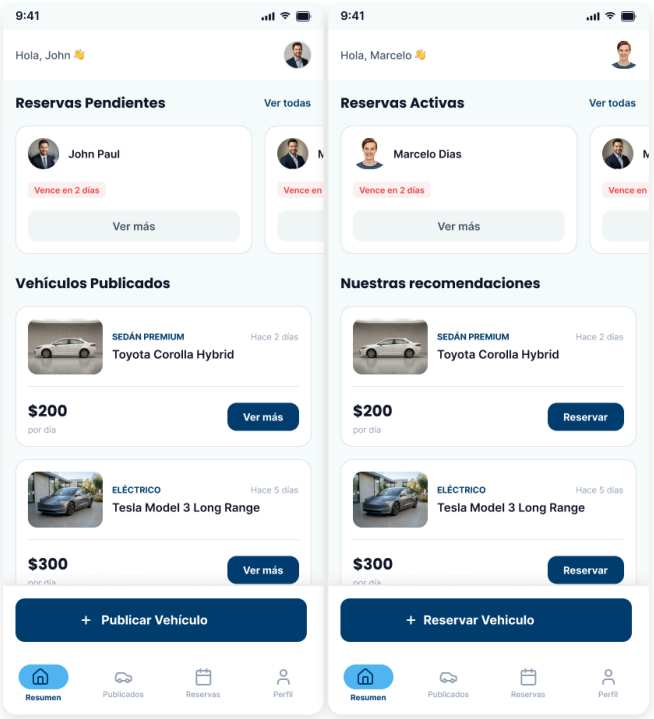
Link del Figma: https://www.figma.com/design/ZYZ81d4uAuktz4Xr7l7MVJ/Dise%C3%B1o-Movil-Pantalla?node-id=2-4606&t=O9RoGjXSUGKxsjO3-1

4.1.3.2. Android Mobile Style Guidelines

- **Layout y Retícula Base:** La interfaz se distribuye en una estructura de columna única vertical adaptativa para el perfil de Arrendador o Arrendatario, organizada estrictamente sobre una retícula base de 8dp. Se establecen márgenes laterales fijos de 16dp para evitar colisiones táctiles, manteniendo los espaciados internos en múltiplos de 8dp (8dp, 16dp y 24dp) para asegurar simetría visual.

- **Navegación e Interfaz Dinámica por Rol:**
  - **Estructura del Resumen:** Organiza visualmente las secciones clave del negocio mediante un carrusel de desplazamiento horizontal.
  - **NavigationBar (Inferior):** Componente de navegación persistente ubicado en el borde inferior de la interfaz, con una altura fija de 80dp y fondo blanco (#FFFFFF). El componente se estructura en cuatro destinos de navegación, definidos según el tipo de usuario: Resumen, Vehículos, Reservas y Perfil; o Resumen, Mis vehículos, Solicitudes y Perfil. La pestaña activa se resalta mediante un indicador visual en forma de cápsula u óvalo de color azul claro (#52B6F2) sobre los iconos en estilo relleno (Filled) en azul primario (#003F70).
  - **Extended Floating Action Button :** Botón flotante de acción primaria exclusivo para el usuario arrendador, ubicado sobre el área segura inferior. Presenta formato de cápsula con bordes redondeados de 16dp, fondo azul primario #003F70, e integra el ícono "+" y el texto correspondiente a la acción principal del módulo (Publicar Vehiculo).

- **Interacción y Accesibilidad:**
  - **Efecto Ripple (Onda):** Toda interacción táctil sobre botones, tarjetas o elementos de navegación activa una animación de onda circular difusa que se expande desde el punto de contacto, ofreciendo retroalimentación visual inmediata.
  - **Áreas de Toque:** Cumplimiento estricto de una superficie de contacto interactiva mínima de 48×48 dp para todos los componentes navegables, garantizando la accesibilidad táctil en el dispositivo.



Link del Figma: https://www.figma.com/design/ZYZ81d4uAuktz4Xr7l7MVJ/Dise%C3%B1o-Movil-Pantalla?node-id=2-4606&t=O9RoGjXSUGKxsjO3-1

4.2. Information Architecture

La arquitectura de información define cómo se organiza, etiqueta, busca y navega en la aplicación web. Este apartado busca garantizar que *Renticar* sea intuitiva y accesible para arrendadores y arrendatarios, mejorando la experiencia de usuario y asegurando la eficiencia en los procesos.

4.2.1. Organization Systems

El sistema de organización de *Renticar* se basa en una estructura **jerárquica y funcional**, con secciones principales que responden a las necesidades detectadas en los segmentos objetivos.

- **Arrendador:** Publicaciones, gestión de vehículos, reservas, reseñas.
- **Arrendatario:** Búsqueda de autos, reservas, reseñas, perfil.
- **Común:** Dashboard, perfil de usuario, landing page informativa.
  Esta organización facilita que cada usuario encuentre de manera rápida las funciones más relevantes según su rol.

4.2.2. Labeling Systems

Se utilizarán etiquetas claras y consistentes, priorizando un lenguaje sencillo y directo, evitando tecnicismos. Algunos ejemplos:

- **"Mis Publicaciones"** → sección donde el arrendador gestiona sus vehículos.
- **"Reservar Ahora"** → acción directa para el arrendatario.
- **"Dashboard"** → punto central para acceder a la información principal.
  El etiquetado estará alineado con las convenciones de usabilidad y las necesidades de accesibilidad, garantizando comprensión rápida.

4.2.3. SEO Tags and Meta Tags

Para mejorar la visibilidad en buscadores, se incorporarán **etiquetas SEO y metadatos optimizados**:

- **Meta Title:** "Renticar – Plataforma confiable para alquiler de vehículos con IoT en Perú".
- **Meta Description:** "Renticar conecta arrendadores y arrendatarios de forma segura, con gestión digital, reseñas verificadas y soporte IoT para mayor transparencia."
- **Keywords:** "alquiler de autos Perú, rentar vehículos seguros, plataforma IoT alquiler".
  Estas prácticas permiten posicionar el producto en los primeros resultados de búsqueda y atraer usuarios interesados.

4.2.4. Searching Systems

El sistema de búsqueda permitirá a los arrendatarios localizar vehículos de manera eficiente mediante filtros y ordenamientos:

- **Filtros principales:** Marca, modelo, precio, ubicación.
- **Filtros secundarios:** Tipo de vehículo (sedán, SUV, hatchback), disponibilidad, kilometraje permitido.
- **Ordenamientos:** Precio (menor-mayor), valoración de usuarios, cercanía geográfica.
  Se prioriza un diseño que reduzca el tiempo de búsqueda y que brinde resultados relevantes de manera inmediata.

4.2.5. Navigation Systems

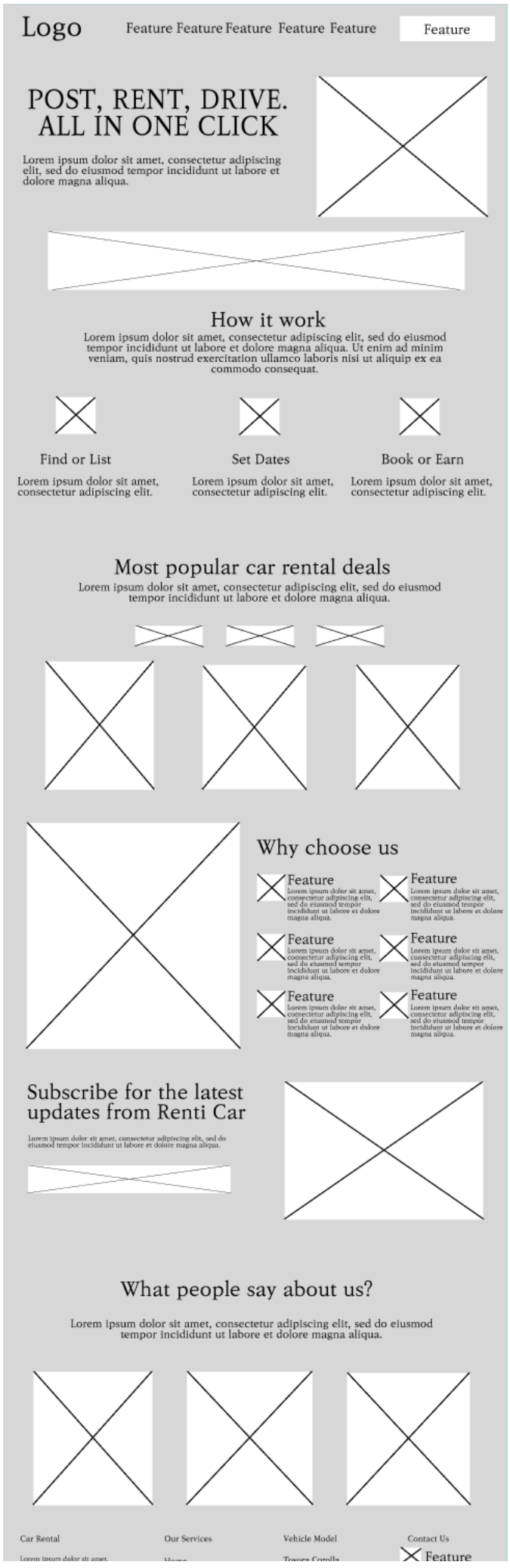
El sistema de navegación seguirá un modelo **híbrido**:

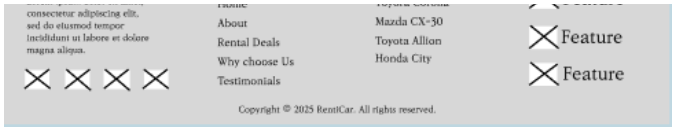
- **Menú principal superior (Toolbar):** acceso directo a Dashboard, Publicaciones, Navegación, Perfil.
- **Menú secundario contextual:** disponible dentro de cada sección, con accesos rápidos a acciones específicas (ej. “Nueva Publicación”, “Cancelar Reserva”).
- **Breadcrumbs:** para ubicar al usuario dentro de la aplicación.
- **Navegación responsiva:** optimizada para dispositivos móviles, asegurando accesibilidad en smartphones y tablets.

Este esquema garantiza que los usuarios puedan moverse de manera intuitiva por el sistema, encontrando siempre las funciones más relevantes según su objetivo.

4.3. Landing Page UI Design

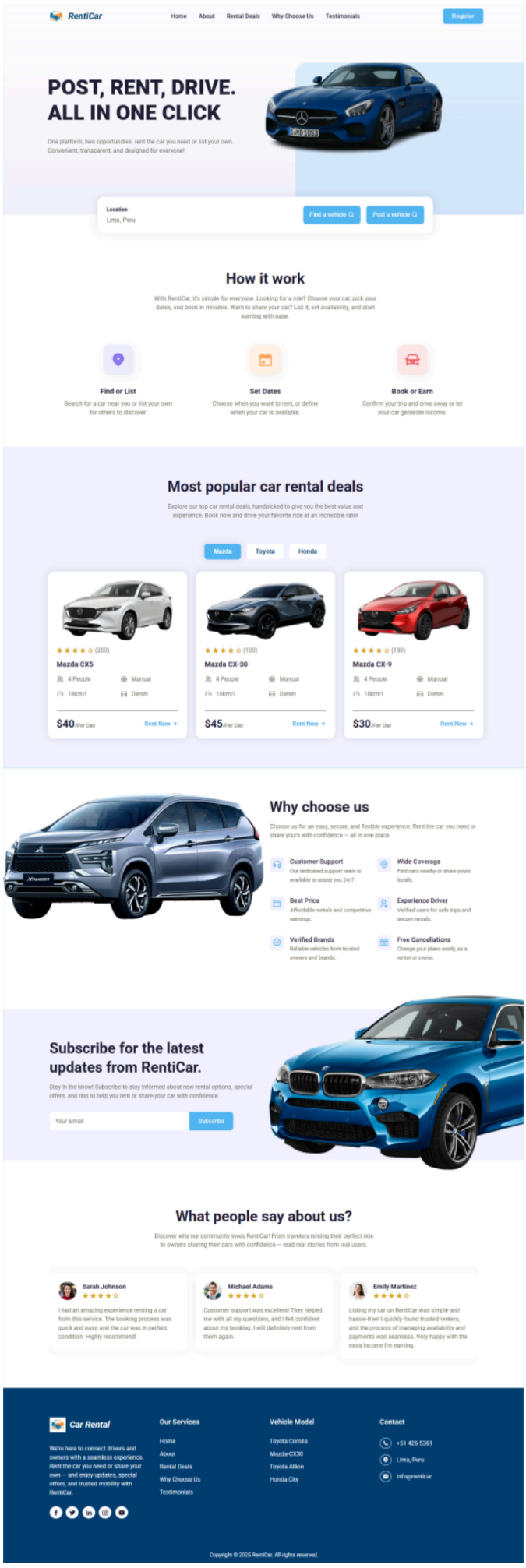
4.3.1. Landing Page Wireframe


Logo
Feature Feature Feature Feature Feature
Feature
POST, RENT, DRIVE.
ALL IN ONE CLICK
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
How it work
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.
Find or List
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Set Dates
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Book or Earn
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Most popular car rental deals
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Why choose us
Feature
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Feature
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Feature
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Feature
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Feature
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Feature
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Subscribe for the latest updates from Renti Car
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
What people say about us?
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Car Rental
Our Services
Vehicle Model
Contact Us
Feature



Link del Figma: https://www.figma.com/design/PChWJ99signLAO8NcRCRrY/Landing-RentiCar?node-id=2461-182&t=FAwGoGSyUJDdEzex-1

4.3.2. Landing Page Mock-up


RentiCar
Home
About
Rental Deals
Why Choose Us
Testimonials
Register
POST, RENT, DRIVE.
ALL IN ONE CLICK
One platform, two opportunities: rent the car you need or list your own. Convenient, transparent, and designed for everyone!
Location
Lima, Peru
Find a vehicle
Post a vehicle
How it work
With RentiCar, it's simple for everyone. Looking for a ride? Choose your car, pick your dates, and book in minutes. Want to share your car? List it, set availability, and start earning with ease.
Find or List
Search for a car near you or list your own for others to discover.
Set Dates
Choose when you want to rent, or define when your car is available.
Book or Earn
Confirm your trip and drive away or let your car generate income.
Most popular car rental deals
Explore our top car rental deals, handpicked to give you the best value and experience. Book now and drive your favorite ride at an incredible rate!
Mazda
Toyota
Honda
Mazda CX5
4 People
Manual
18km/l
Diesel
$40 /Per Day
Rent Now
Mazda CX-30
4 People
Manual
18km/l
Diesel
$45 /Per Day
Rent Now
Mazda CX-9
4 People
Manual
18km/l
Diesel
$30 /Per Day
Rent Now
Why choose us
Choose us for an easy, secure, and flexible experience. Rent the car you need or share yours with confidence — all in one place.
Customer Support
Our dedicated support team is available to assist you 24/7.
Wide Coverage
Find cars nearby or share yours locally.
Best Price
Affordable rentals and competitive earnings.
Experience Driver
Verified users for safe trips and secure rentals.
Verified Brands
Reliable vehicles from trusted owners and brands.
Free Cancellations
Change your plans easily, as a renter or owner.
Subscribe for the latest updates from RentiCar.
Stay in the know! Subscribe to stay informed about new rental options, special offers, and tips to help you rent or share your car with confidence.
Your Email
Subscribe
What people say about us?
Discover why our community loves RentiCar! From travelers renting their perfect ride to owners sharing their cars with confidence — read real stories from real users.
Sarah Johnson
I had an amazing experience renting a car from this service. The booking process was quick and easy, and the car was in perfect condition. Highly recommend!
Michael Adams
Customer support was excellent! They helped me with all my questions, and I felt confident about my booking. I will definitely rent from them again.
Emily Martinez
Listing my car on RentiCar was simple and hassle-free! I quickly found trusted renters, and the process of managing availability and payments was seamless. Very happy with the extra income I'm earning.
Car Rental
We're here to connect drivers and owners with a seamless experience. Rent the car you need or share your own — and enjoy updates, special offers, and trusted mobility with RentiCar.
Our Services
Home
About
Rental Deals
Why Choose Us
Testimonials
Vehicle Model
Toyota Corolla
Mazda CX30
Toyota Allion
Honda City
Contact
+51 426 5361
Lima, Peru
info@renticar
Copyright © 2025 RentiCar. All rights reserved.

Link del Figma: https://www.figma.com/design/PChWJ99signLAO8NcRCRrY/Landing-RentiCar?node-id=2461-182&t=FAwGoGSyUJDdEzex-1

4.4. Mobile Applications UX/UI Design

Esta sección define la propuesta de diseño para la experiencia móvil de RentiCar, enfocada en simplificar la interacción desde dispositivos de pantalla reducida. La interfaz organiza las funciones de acuerdo con las necesidades de arrendadores o arrendatarios, priorizando accesos rápidos, navegación táctil y una presentación clara de la información para realizar las principales tareas de forma ágil.

4.4.1. Mobile Applications Wireframes

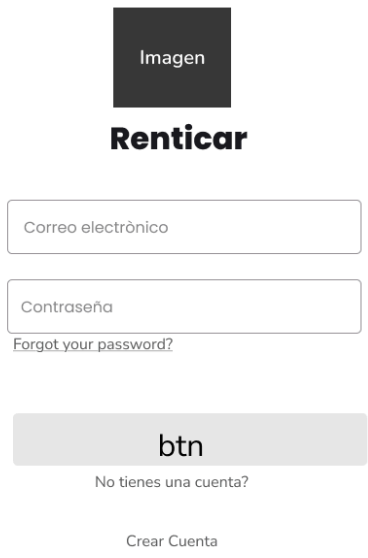
Los wireframes de la versión móvil de RentiCar se plantean como una guía estructural para definir la distribución de contenido, componentes y controles dentro de la interfaz. Se prioriza una navegación sencilla e intuitiva, considerando las dimensiones de la pantalla y la interacción táctil para facilitar acciones como explorar vehículos, gestionar reservas y administrar publicaciones.

Enlace al proyecto en Figma: Wireframes

1. Iniciar Sesión / Registrarse

Pantallas destinadas al acceso y creación de cuentas en RentiCar. La interfaz mobile presenta formularios simplificados y adaptados a la interacción táctil, permitiendo seleccionar el tipo de usuario (arrendador o arrendatario) durante el proceso de registro.

1. Inicio de Sesión



2. Registro de Arrendatario

3. Registro de Arrendador

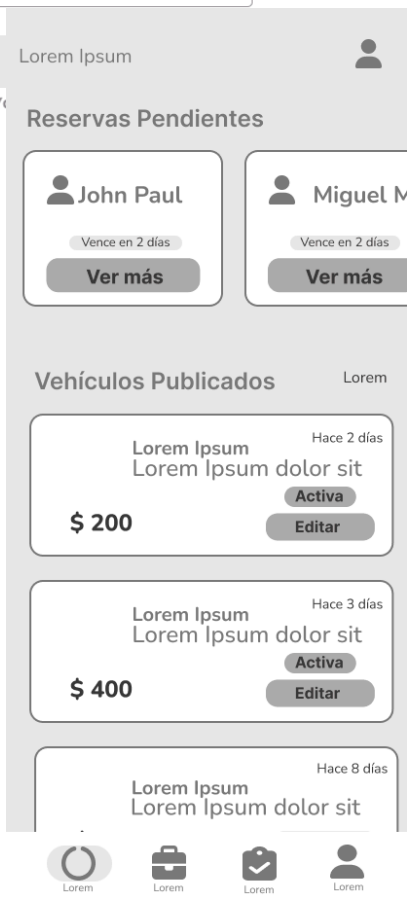
4. Recuperación de Contraseña

2. Dashboard del Arrendador

La versión mobile del dashboard concentra en una sola vista las funciones más relevantes para el alquiler de sus vehículos. Permite gestionar vehículos, consultar solicitudes, revisar reservas y acceder rápidamente a la información de su cuenta mediante una distribución optimizada para pantallas reducidas.

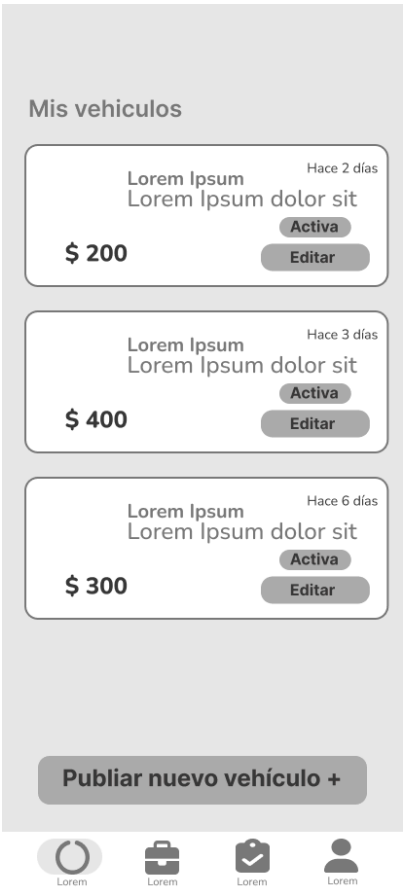
1. Resumen

Vista principal que muestra el estado general de los vehículos publicados, solicitudes y reservas activas.



2. Mis vehículos

Permite consultar y administrar los vehículos registrados, incluyendo la gestión de sus publicaciones.



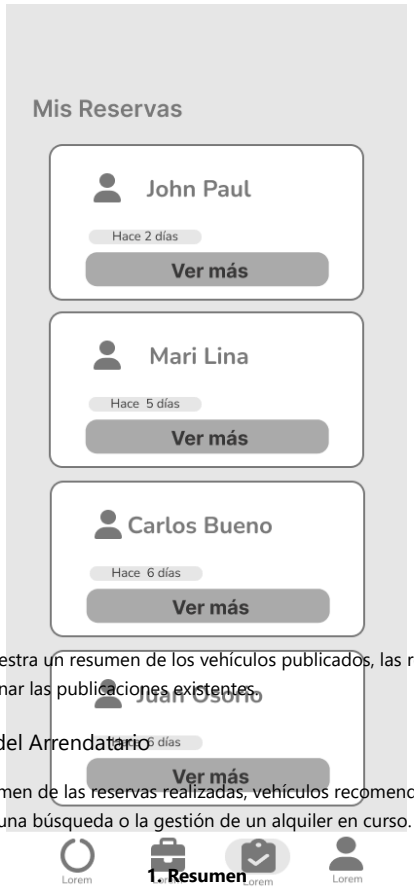
---

3. Solicitudes

Centraliza las solicitudes de reserva recibidas, facilitando su consulta y gestión.

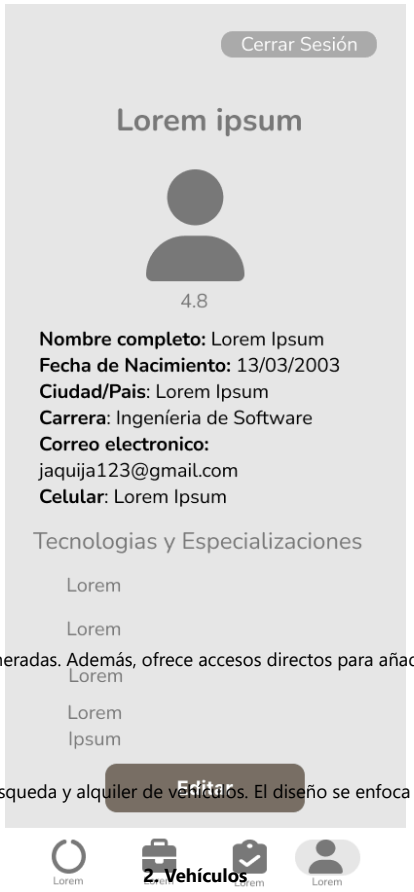
4. Perfil

Permite visualizar y actualizar la información personal y los datos asociados a la cuenta del arrendador.



1. Resumen

Vista principal que muestra recomendaciones, vehículos disponibles y el estado de las reservas activas.



2. Vehículos

Permite explorar y consultar los vehículos disponibles, facilitando la búsqueda según las necesidades del arrendatario.

El dashboard muestra un resumen de los vehículos publicados, las reservas activas y las ganancias generadas. Además, ofrece accesos directos para añadir un nuevo vehículo y gestionar las publicaciones existentes.

3. Dashboard del Arrendatario

Presenta un resumen de las reservas realizadas, vehículos recomendados y accesos rápidos para la búsqueda y alquiler de vehículos. El diseño se enfoca en facilitar la continuación de una búsqueda o la gestión de un alquiler en curso.

3. Reservas

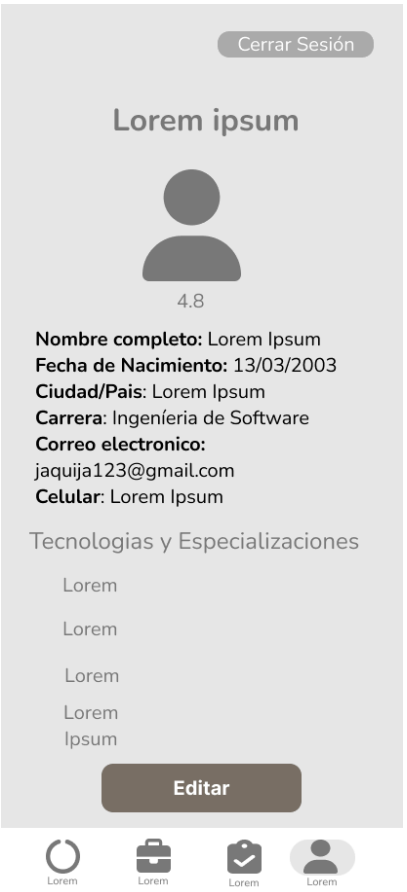
Centraliza las reservas realizadas, permitiendo consultar su estado, detalles y

4. Perfil

Permite visualizar y actualizar la información personal y los datos asociados a la

gestionar los alquileres activos.

cuenta del arrendatario.



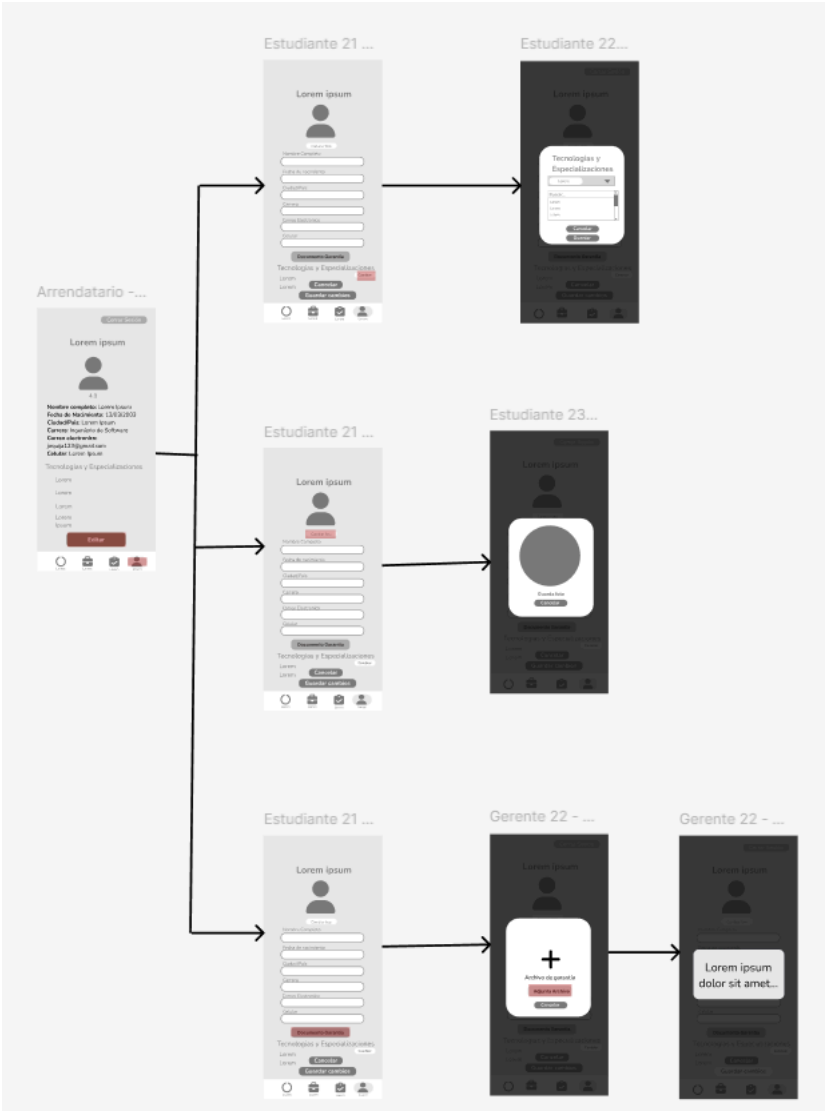
4.4.2. Mobile Applications Wireflow Diagrams

Los wireflows representan las rutas que siguen los usuarios para completar las principales tareas dentro de la aplicación mobile. Aunque los flujos mantienen la misma lógica funcional, su diseño se adapta a pantallas reducidas, navegación táctil y accesos rápidos, buscando que cada proceso sea claro, sencillo y fluido.

Enlace al proyecto en Figma: Wireflow

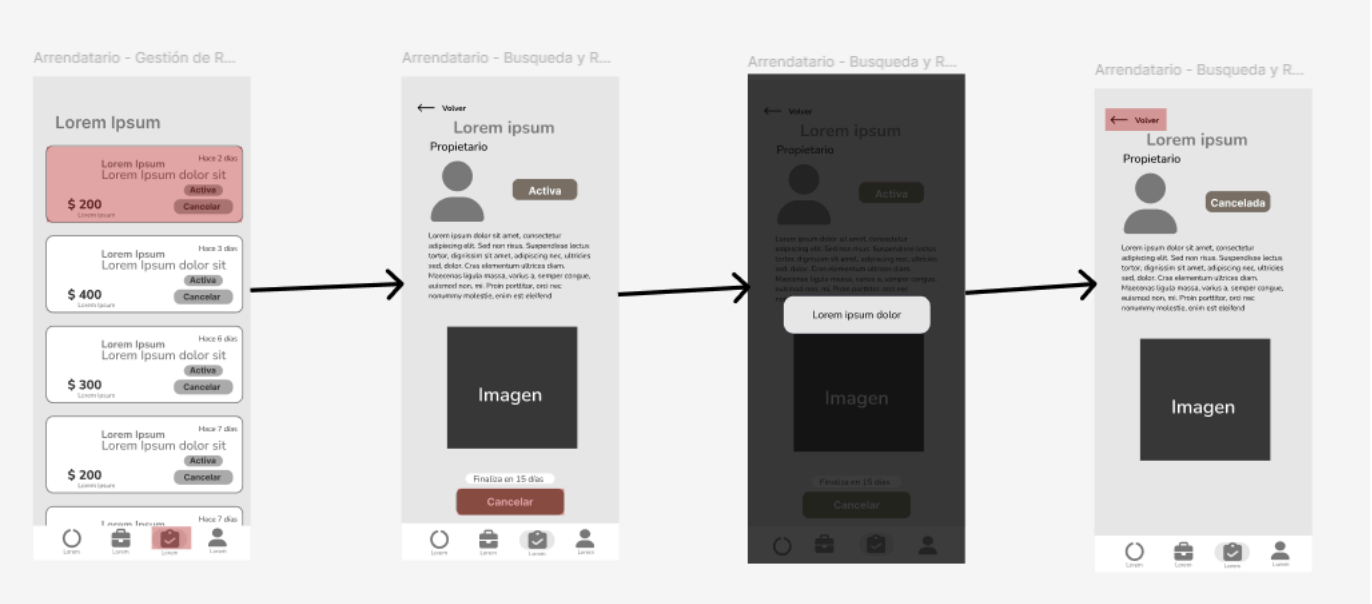
1. Segmento 1: Arrendatario

User Goal: Como arrendatario, quiero gestionar mi perfil desde mi celular, pudiendo editar mi información personal, actualizar mi foto y cargar los documentos necesarios para validar mi cuenta.



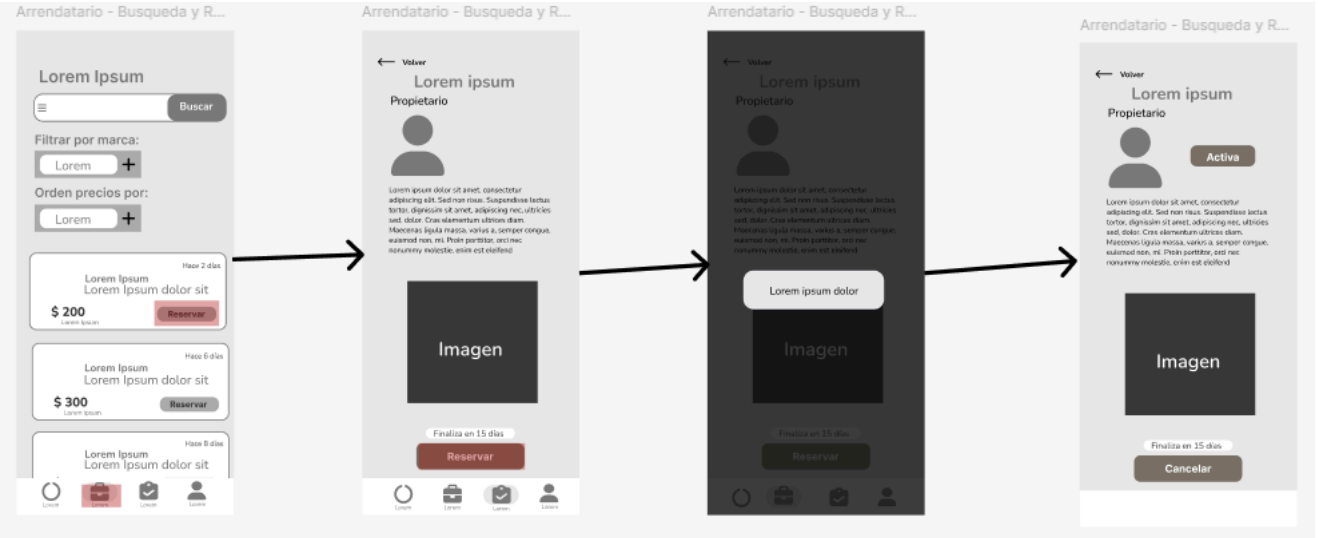
- Wireflow

User Goal: Como arrendatario, quiero gestionar mis reservas desde el celular, pudiendo consultar sus detalles, cancelarlas y dejar una reseña cuando finalice el alquiler.



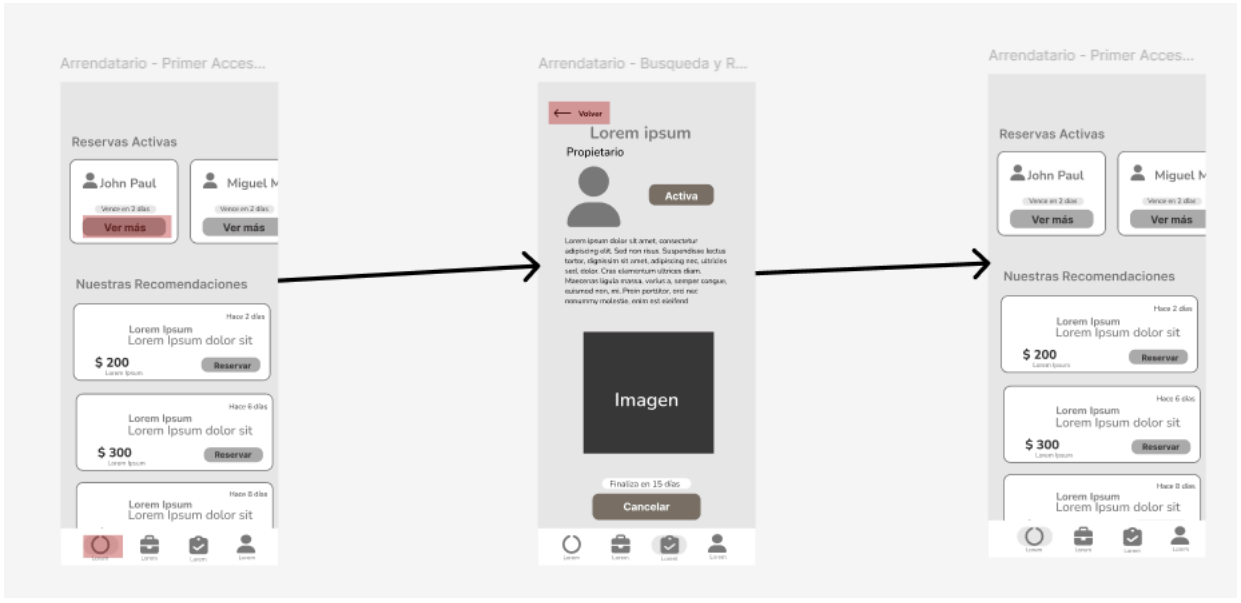
- Wireflow

User Goal: Como arrendatario, quiero buscar vehículos desde mi celular utilizando filtros, consultar sus características y completar una reserva de manera rápida y sencilla.

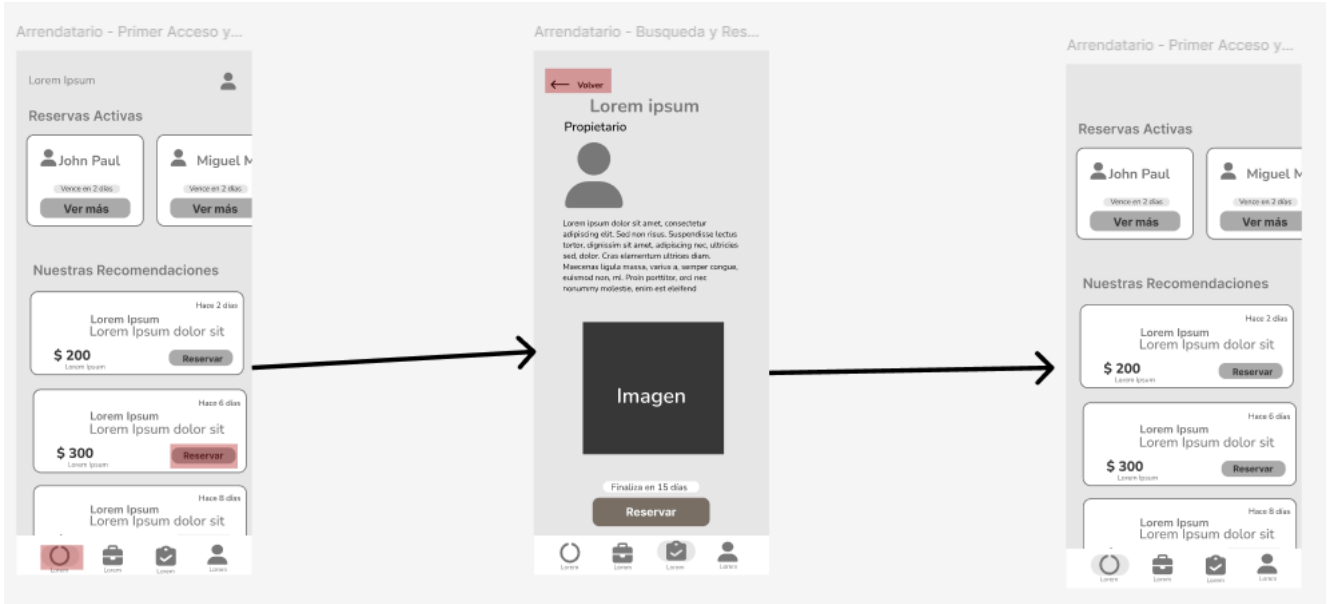


- Wireflow

User Goal: Como arrendatario, quiero acceder rápidamente desde mi dashboard mobile a mis reservas activas para consultar sus detalles o cancelarlas cuando sea necesario.

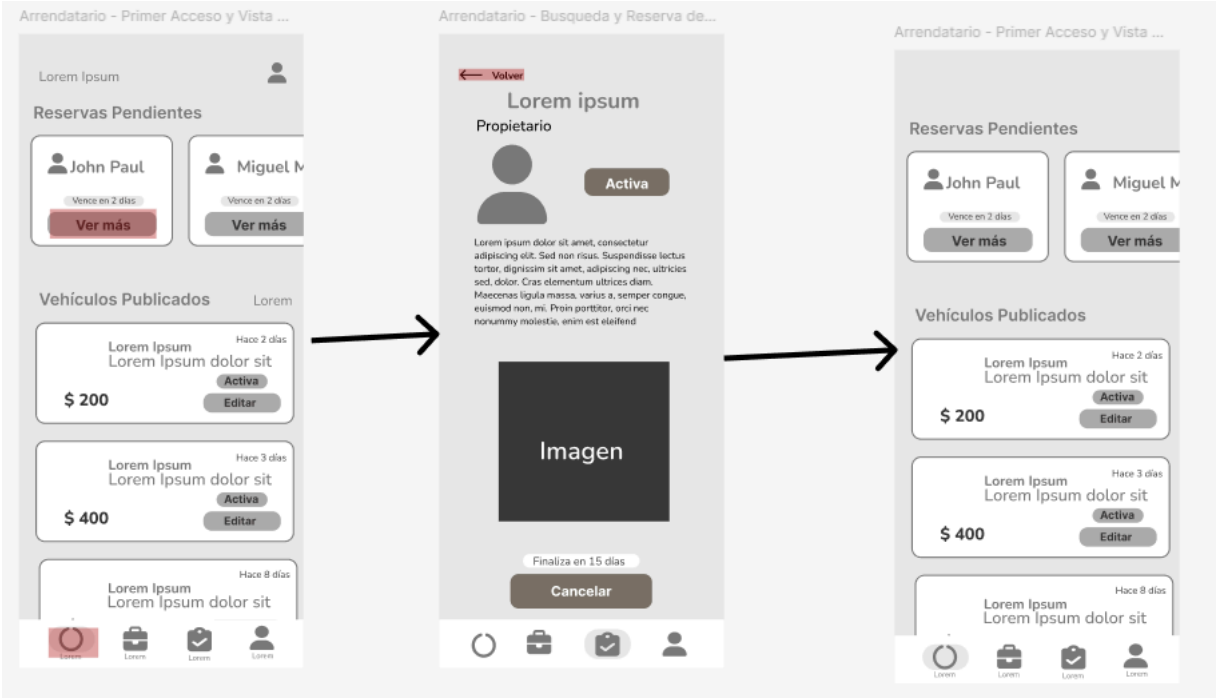


User Goal: Como arrendatario, quiero explorar vehículos recomendados desde mi dashboard mobile y realizar una reserva de forma rápida.

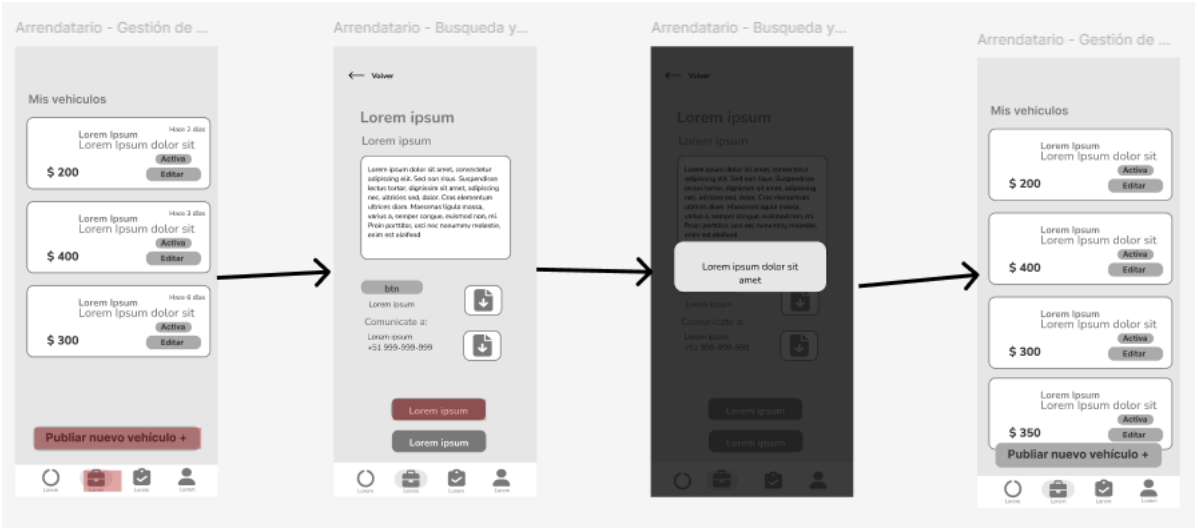


2. Segmento 2: Arrendador

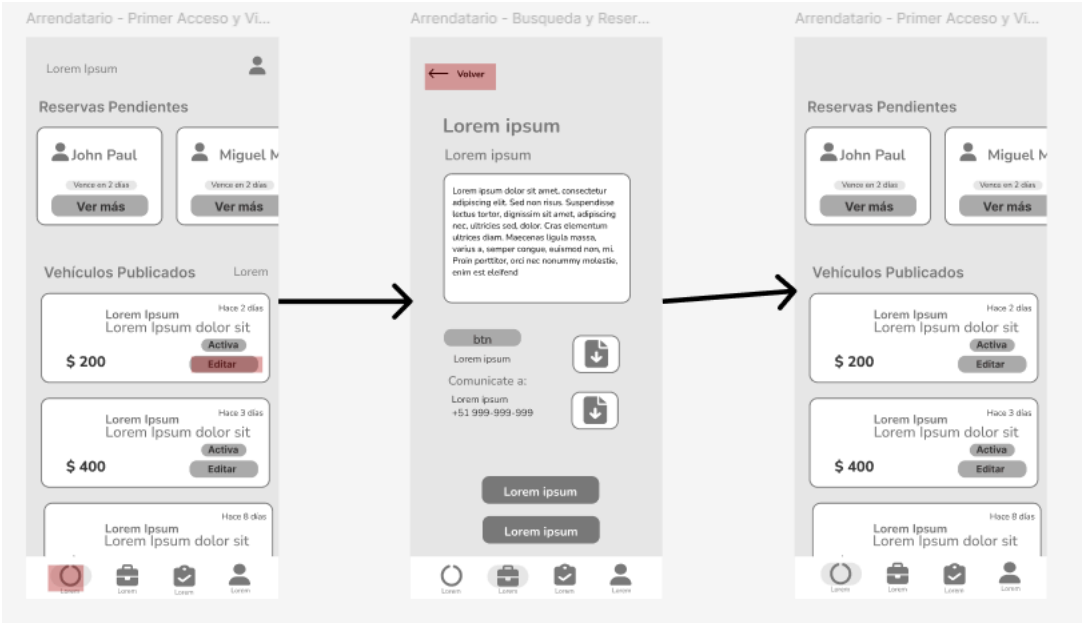
User Goal: Como arrendador, quiero consultar desde mi celular un resumen de mi actividad, incluyendo mis vehículos publicados y las solicitudes de reserva pendientes.



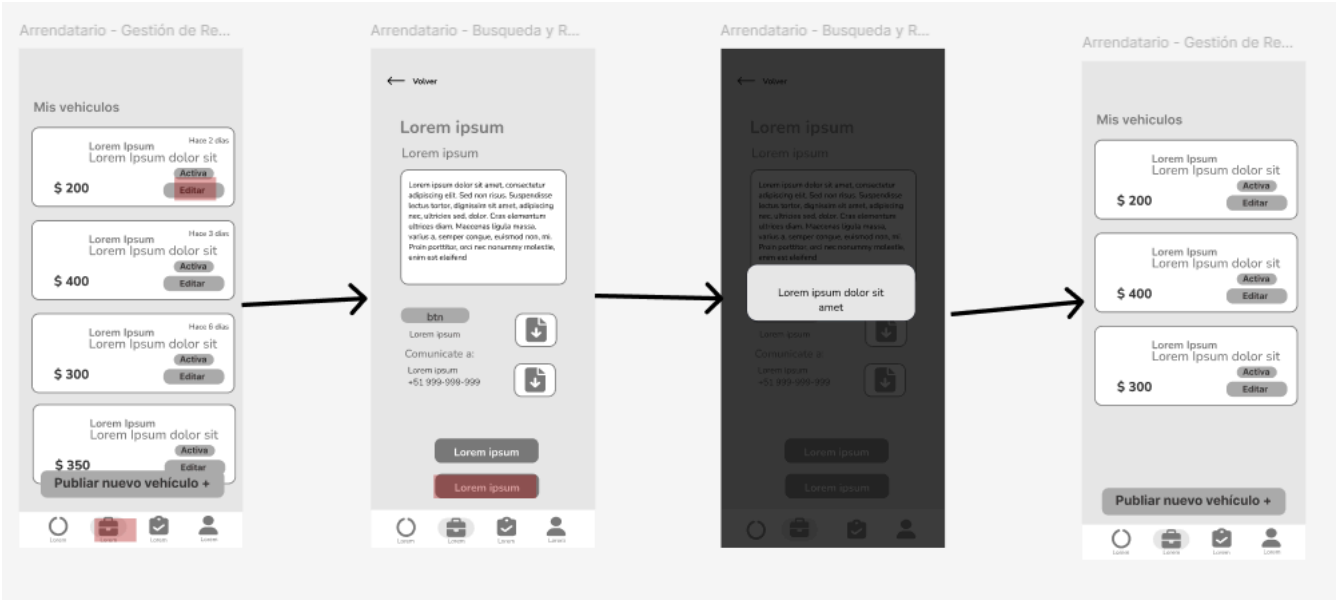
User Goal (Crear): Como arrendador, quiero registrar un nuevo vehículo desde mi celular de forma rápida y sencilla para comenzar a alquilarlo.



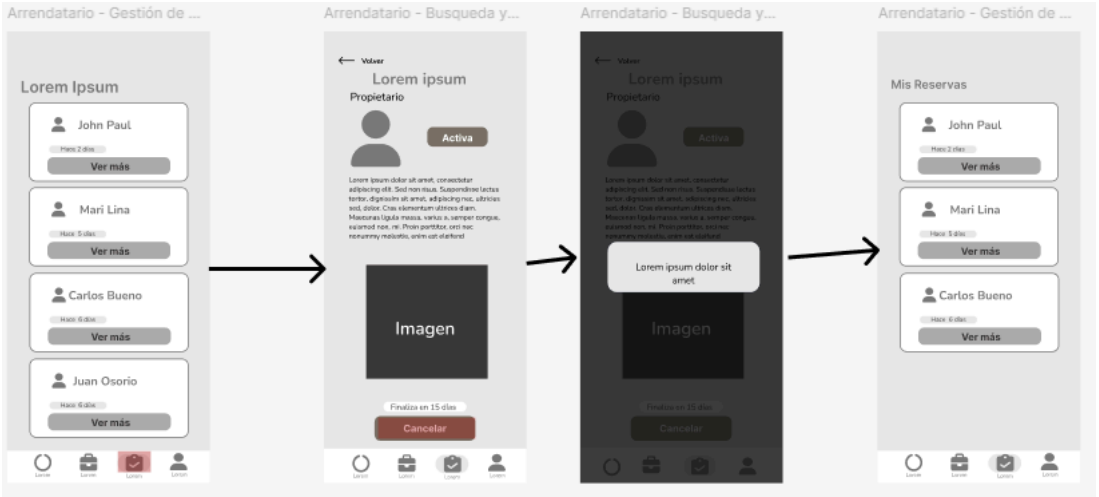
User Goal (Editar): Como arrendador, quiero modificar desde mi celular los datos de mis vehículos publicados, como el precio o la descripción, para mantenerlos actualizados.



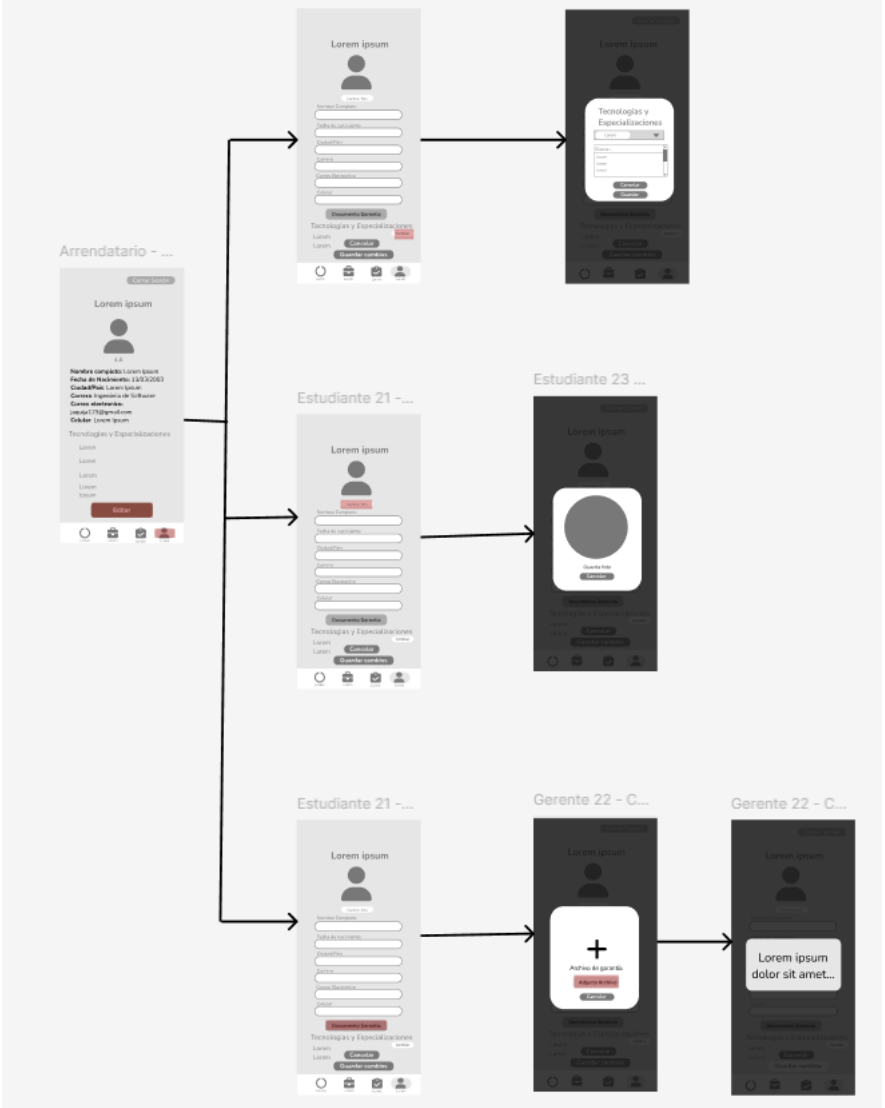
User Goal (Eliminar): Como arrendador, quiero retirar fácilmente un vehículo que ya no se encuentre disponible para alquiler.



User Goal: Como arrendador, quiero gestionar desde mi celular las solicitudes de alquiler recibidas, pudiendo aceptarlas o rechazarlas de manera sencilla.



User Goal: Como arrendador, quiero actualizar desde mi celular mi información de contacto y documentos de garantía para mantener mi perfil actualizado y brindar confianza a los clientes.



4.4.3. Mobile Applications Mock-ups

En esta sección se presentan los mock-ups de alta fidelidad correspondientes a la aplicación mobile de **RentiCar**, diseñados en Figma respetando la guía de estilos, paleta de colores y componentes visuales definidos previamente. Las interfaces mantienen las mismas funcionalidades de la aplicación web, pero se encuentran adaptadas a pantallas de menor tamaño y a la interacción táctil.

Los mock-ups se encuentran agrupados según los principales flujos de interacción de los dos segmentos clave: Arrendatarios y Arrendadores, además de los módulos transversales de autenticación y gestión de acceso.

Enlace al proyecto en Figma: MockUps

4.4.3.1. Módulo de Autenticación y Gestión de Acceso (IAM)

Este módulo centraliza las interfaces de registro, inicio de sesión seguro y recuperación de credenciales para ambos perfiles de usuario. En la versión mobile, los formularios se presentan de manera simplificada para facilitar la interacción mediante dispositivos táctiles.

- **Pantalla de Inicio de Sesión (Sign In):**
  Permite a los usuarios registrados acceder a su cuenta ingresando su correo y contraseña. La interfaz está optimizada para dispositivos móviles y mantiene las opciones principales de acceso y validación.

- **Pantalla de Registro para Arrendatarios (Tenant Register):**
  Formulario adaptado a dispositivos móviles donde los clientes ingresan sus datos personales, correo, teléfono y contraseña para crear su cuenta de arrendatario.

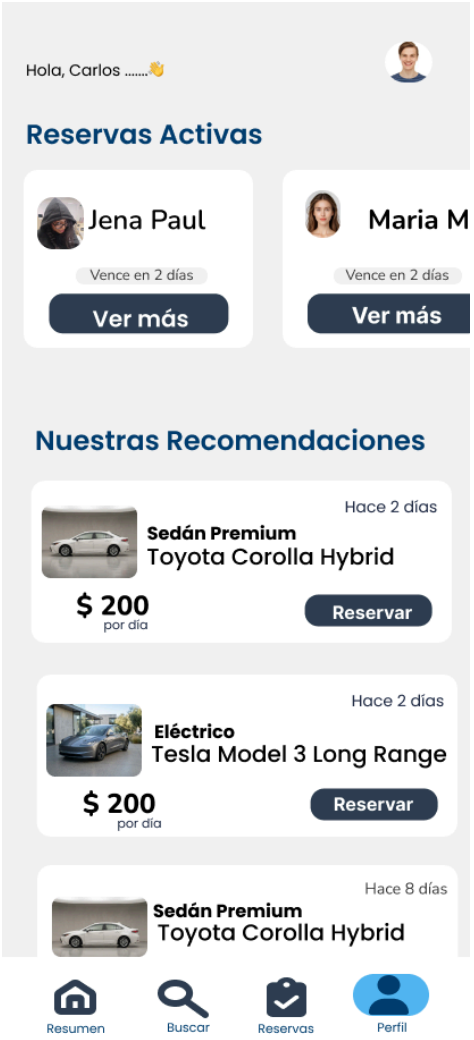
- **Pantalla de Registro para Arrendadores (Hirer Register):**
  Interfaz orientada a propietarios de vehículos que desean publicar sus unidades. El formulario se organiza verticalmente para facilitar el ingreso de datos desde el dispositivo móvil.

- **Pantalla de Recuperación de Contraseña:**
  Permite ingresar el correo registrado para iniciar el proceso de recuperación de contraseña mediante una interfaz sencilla y adaptada a pantallas reducidas.

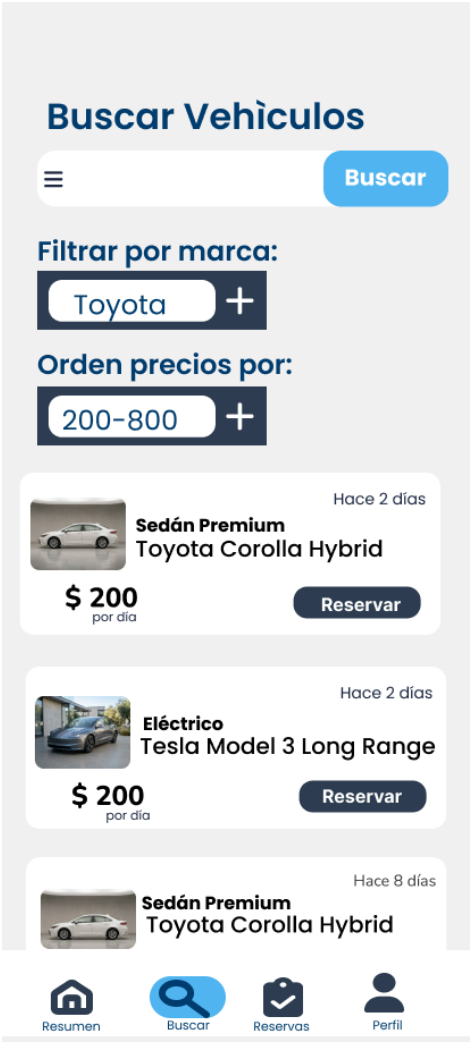
4.4.3.2. Módulo de Experiencia del Arrendatario (Resumen, Búsqueda, Reservas y Perfil)

Interfaces diseñadas para que el arrendatario pueda gestionar las principales funciones de alquiler desde su dispositivo móvil. El módulo permite consultar el estado de sus reservas, explorar vehículos, aplicar filtros, gestionar sus alquileres y actualizar la información de su perfil.

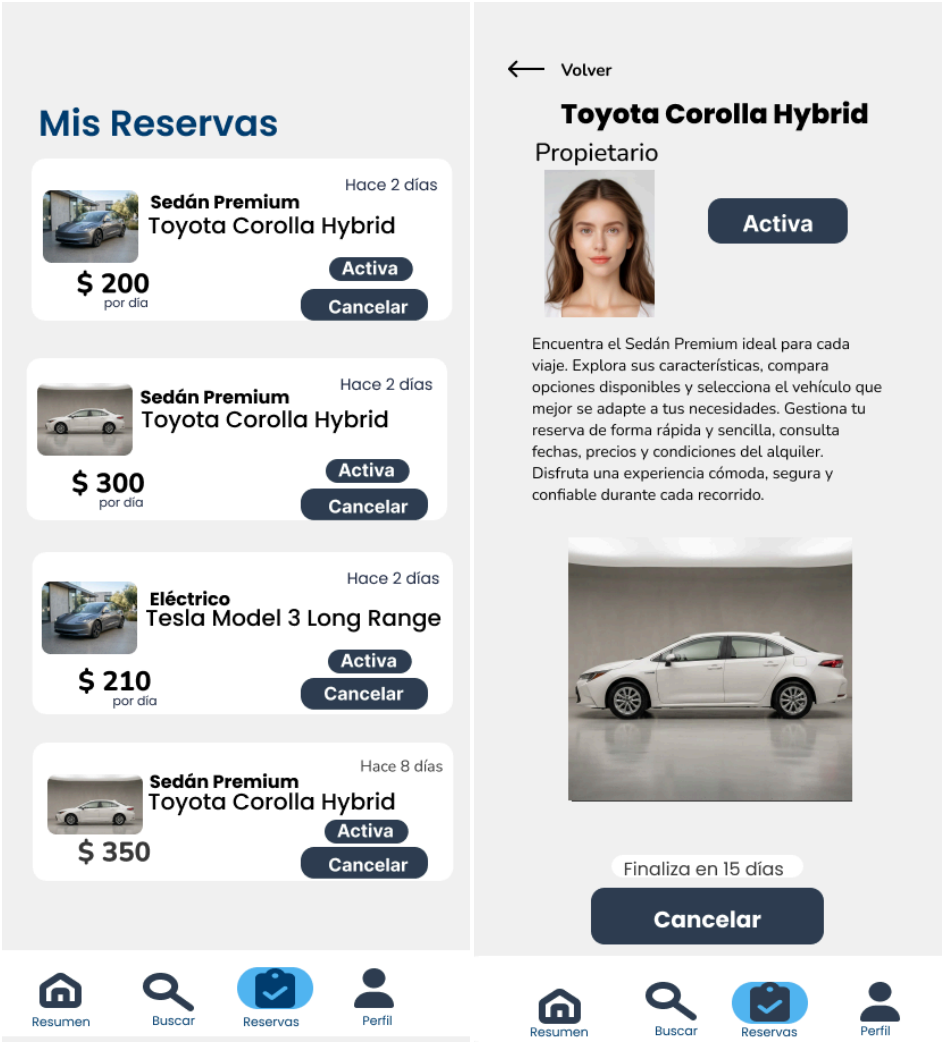
- **Resumen y Recomendaciones:**
  Vista principal del arrendatario que presenta un resumen de sus reservas activas y recomendaciones de vehículos proporcionadas por el sistema, permitiendo acceder rápidamente a las opciones de alquiler.



- **Búsqueda de Vehículos y Filtros:**
  Permite explorar los vehículos disponibles y utilizar filtros para personalizar la búsqueda según características como tipo de vehículo, precio u otras preferencias del arrendatario.



- **Gestión de Reservas:**
  Pantalla destinada a consultar las reservas realizadas y gestionar los alquileres activos. Permite visualizar los detalles de cada reserva y realizar modificaciones o acciones disponibles sobre ellas.

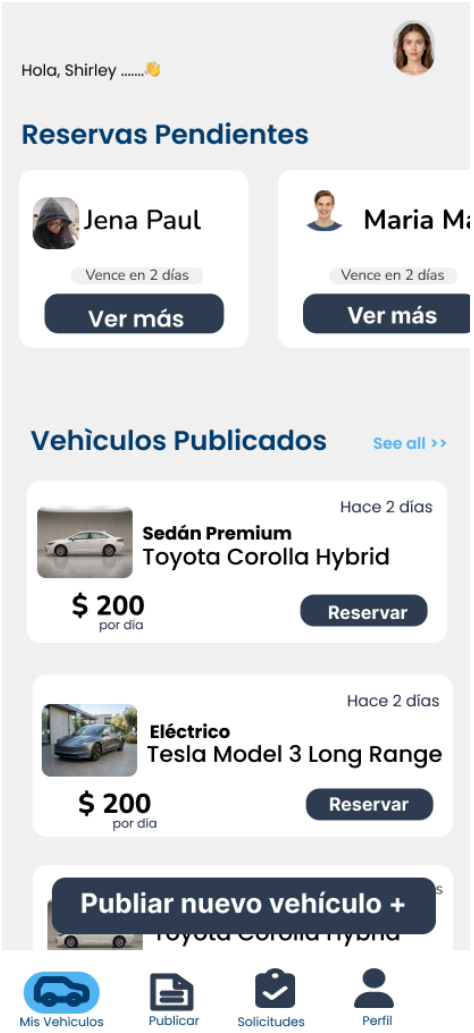


- **Perfil del Arrendatario:**
  Permite visualizar y actualizar la información personal asociada a la cuenta, facilitando la gestión del perfil directamente desde el dispositivo móvil.

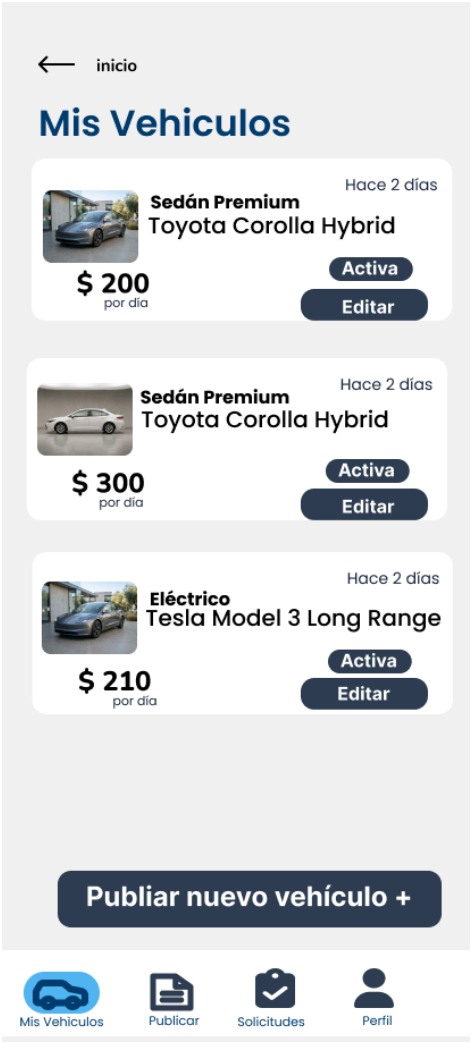
4.4.3.3. Módulo de Experiencia del Arrendador (Gestión de Flota y Solicitudes)

Interfaces especializadas para que los propietarios puedan administrar sus vehículos, gestionar solicitudes de alquiler y actualizar su información personal desde un dispositivo móvil.

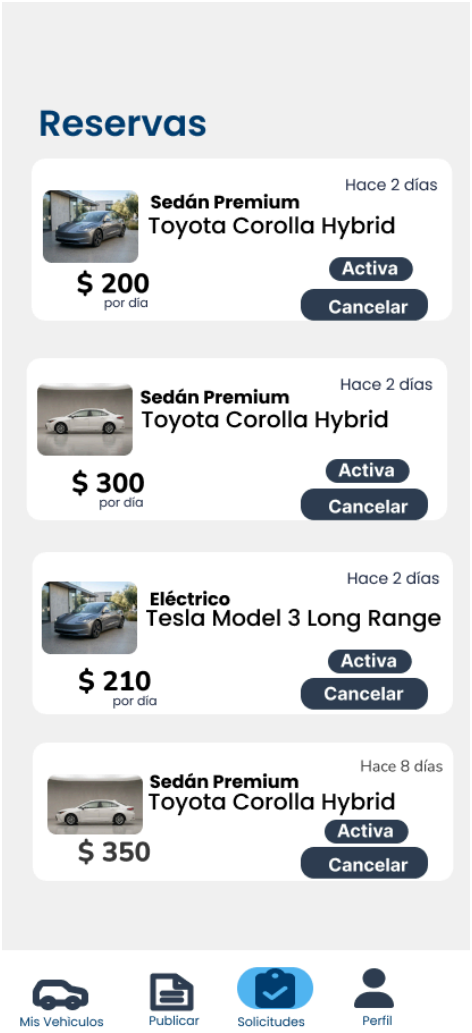
- **Primer Acceso y Vista General (Dashboard del Arrendador):**
  Dashboard mobile que presenta de manera resumida los ingresos, vehículos activos y reservas en curso, además de proporcionar accesos rápidos a las principales funciones de gestión.

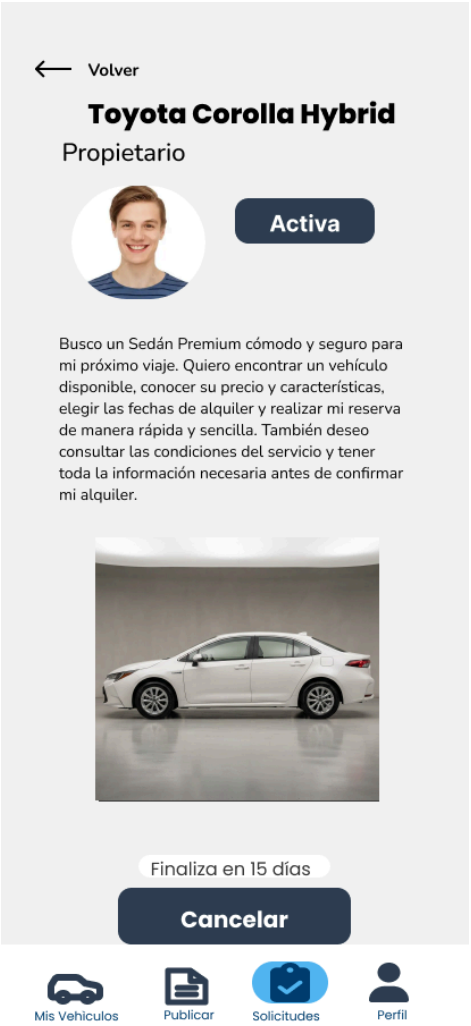


- **Gestión y Publicación de Vehículos:**
  Permite consultar la flota registrada y administrar las publicaciones de los vehículos. Desde esta sección, el arrendador puede agregar nuevos vehículos, modificar sus datos, actualizar su disponibilidad y gestionar las publicaciones existentes.


inicio
Mis Vehiculos
Hace 2 días
Sedán Premium
Toyota Corolla Hybrid
$ 200
por día
Activa
Editar
Hace 2 días
Sedán Premium
Toyota Corolla Hybrid
$ 300
por día
Activa
Editar
Hace 2 días
Eléctrico
Tesla Model 3 Long Range
$ 210
por día
Activa
Editar
Publiar nuevo vehículo +
Mis Vehiculos
Publicar
Solicitudes
Perfil

- **Administración de Solicitudes de Reserva:**
  Pantalla mobile que centraliza las solicitudes recibidas. El arrendador puede consultar la información principal del arrendatario, revisar las fechas solicitadas y gestionar cada solicitud aceptándola o rechazándola.


Reservas
Hace 2 días
Sedán Premium
Toyota Corolla Hybrid
$ 200
por día
Activa
Cancelar
Hace 2 días
Sedán Premium
Toyota Corolla Hybrid
$ 300
por día
Activa
Cancelar
Hace 2 días
Eléctrico
Tesla Model 3 Long Range
$ 210
por día
Activa
Cancelar
Hace 8 días
Sedán Premium
Toyota Corolla Hybrid
$ 350
Activa
Cancelar
Mis Vehiculos
Publicar
Solicitudes
Perfil



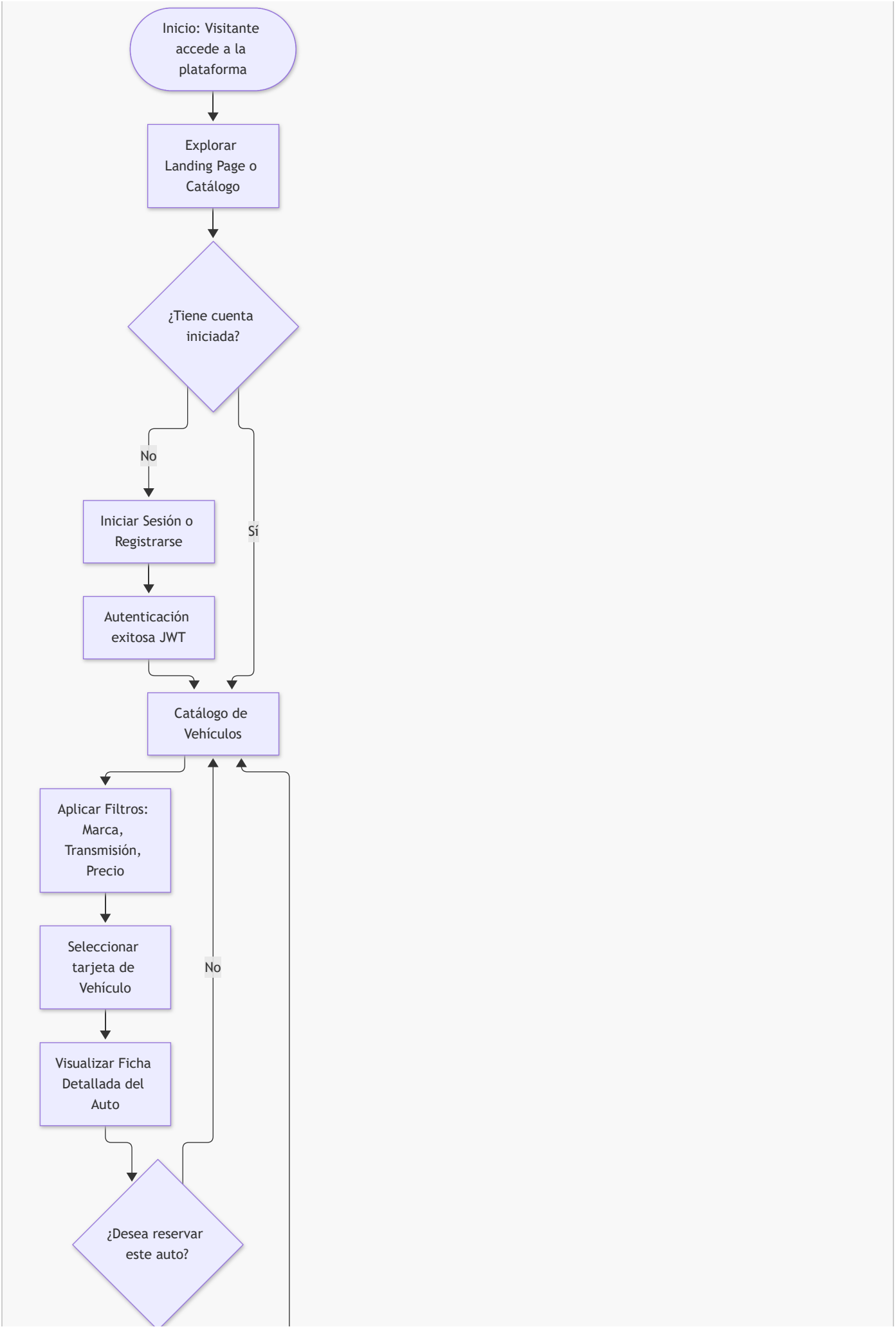
- **Edición de Perfil de Propietario:**
  Formulario optimizado para dispositivos móviles que permite actualizar la información personal, el número telefónico y los documentos asociados a la cuenta del arrendador.

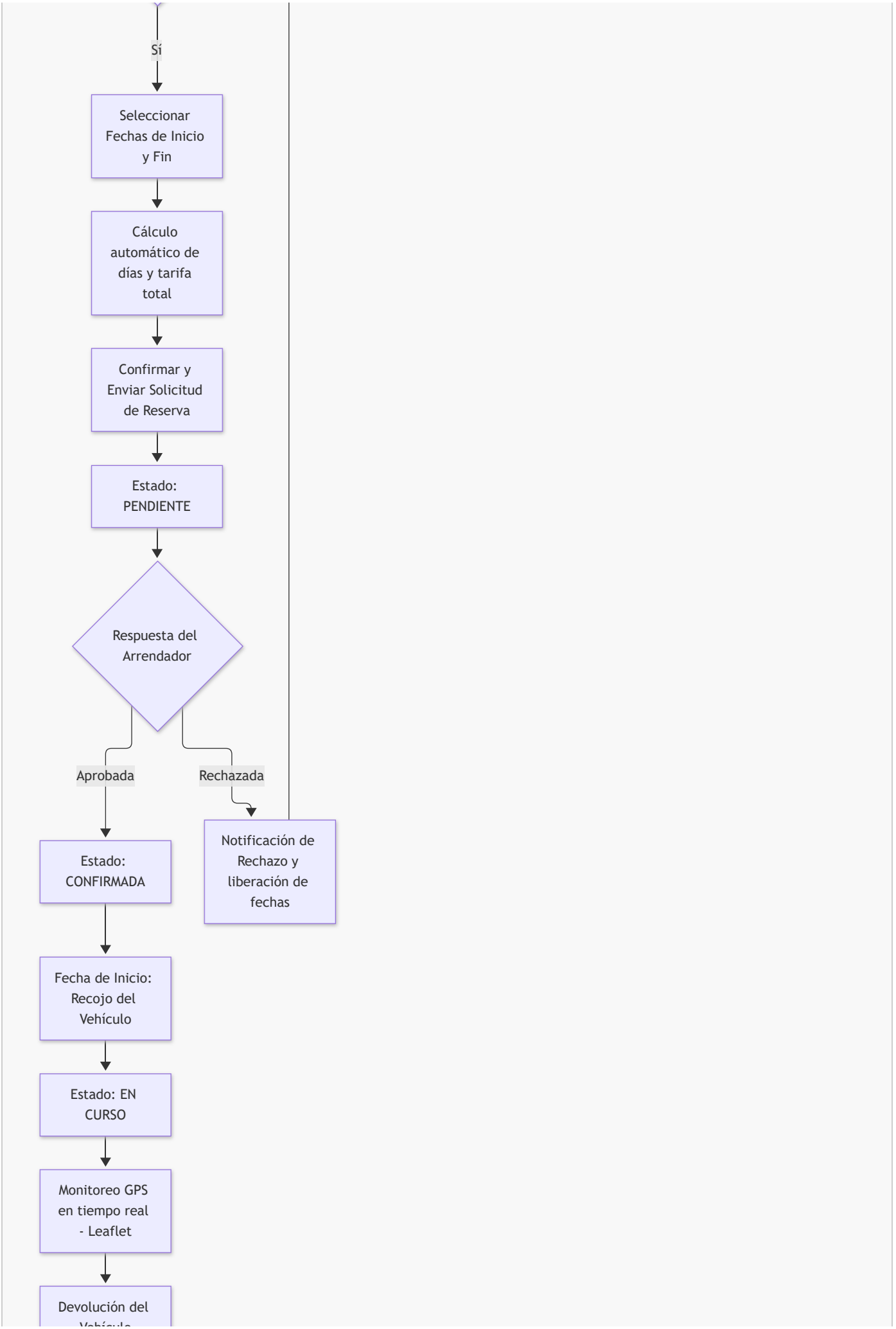
4.4.4. Mobile Applications User Flow Diagrams

En esta sección se modelan los **diagramas de flujo de usuario (User Flow Diagrams)** que describen las rutas de interacción, puntos de decisión y transiciones de estado de los usuarios dentro de la aplicación mobile RentiCar. Los flujos mantienen la misma lógica funcional, adaptando la interacción a pantallas reducidas, navegación táctil y accesos rápidos.

4.4.4.1. Flujo de Usuario: Arrendatario (Búsqueda, Reserva, Monitoreo y Calificación)

El siguiente diagrama representa el recorrido del arrendatario dentro de la aplicación mobile, desde su acceso inicial hasta la finalización y calificación del servicio de alquiler:


Inicio: Visitante accede a la plataforma
Explorar Landing Page o Catálogo
¿Tiene cuenta iniciada?
No
Sí
Iniciar Sesión o Registrarse
Autenticación exitosa JWT
Catálogo de Vehículos
Aplicar Filtros: Marca, Transmisión, Precio
Seleccionar tarjeta de Vehículo
Visualizar Ficha Detallada del Auto
¿Desea reservar este auto?
No

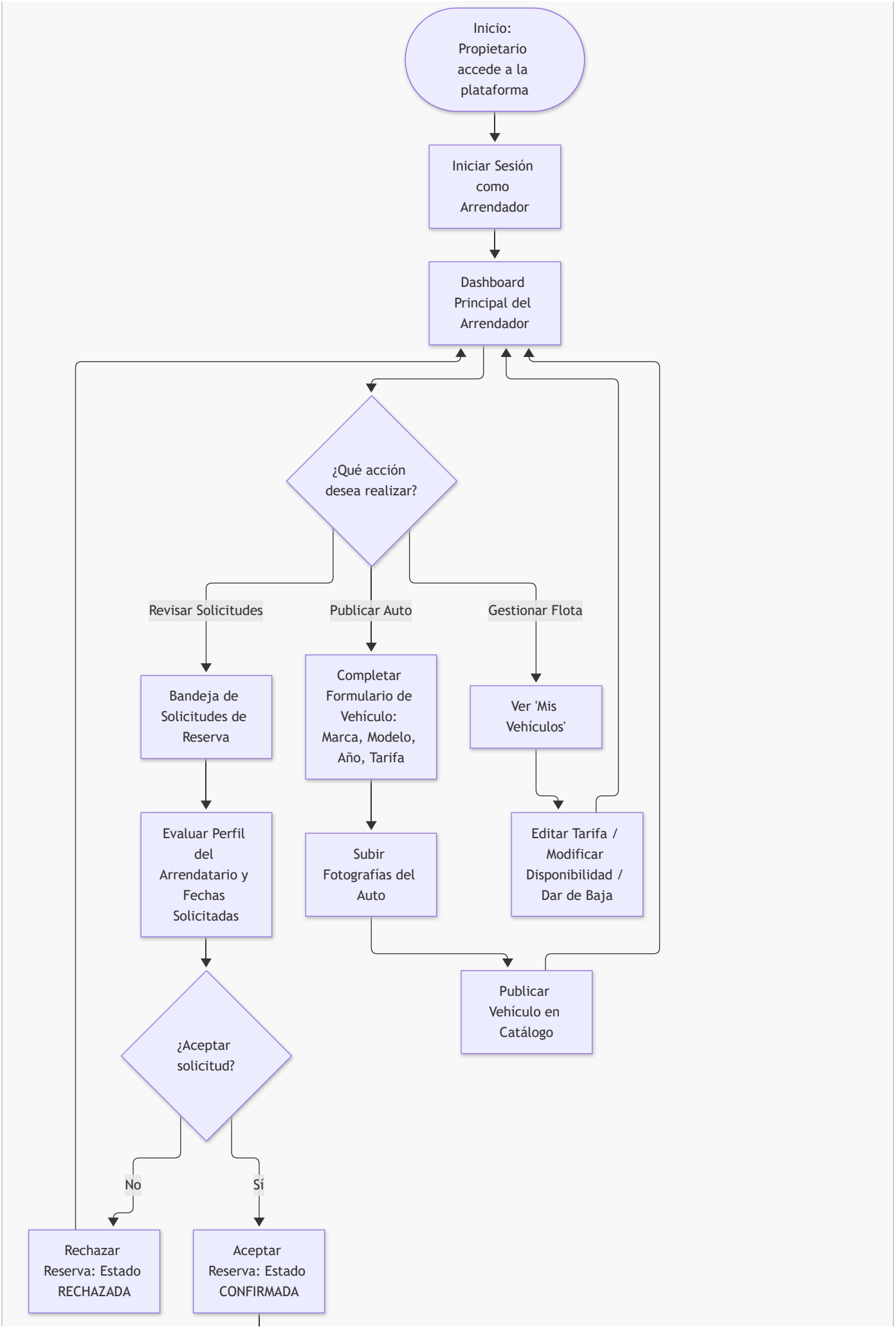

Sí
Seleccionar Fechas de Inicio y Fin
Cálculo automático de días y tarifa total
Confirmar y Enviar Solicitud de Reserva
Estado: PENDIENTE
Respuesta del Arrendador
Aprobada
Rechazada
Estado: CONFIRMADA
Notificación de Rechazo y liberación de fechas
Fecha de Inicio: Recojo del Vehículo
Estado: EN CURSO
Monitoreo GPS en tiempo real - Leaflet
Devolución del

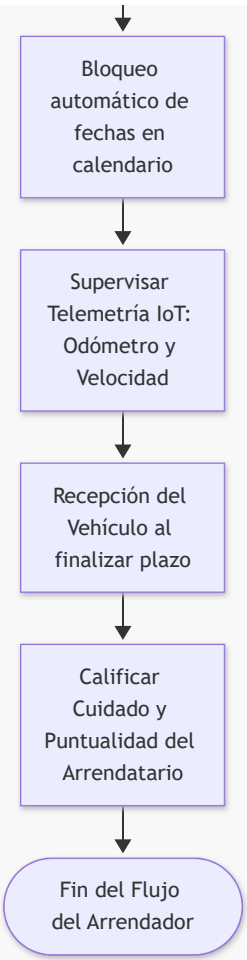
Descripción del flujo del Arrendatario:

1. **Descubrimiento y Navegación:** El cliente ingresa y navega por el catálogo público. Para concretar una reserva, el sistema valida su sesión activa o le solicita autenticarse con JWT.
2. **Filtrado y Selección:** El usuario puede refinar los resultados por marca, caja de cambios y tarifa diaria. Al seleccionar un vehículo, visualiza especificaciones completas e imágenes.
3. **Reserva y Presupuesto:** Al elegir el rango de fechas en el calendario interactivo, el sistema calcula de forma reactiva el importe total. Tras confirmar, la solicitud se envía al propietario con estado `PENDIENTE`.
4. **Seguimiento y Uso IoT:** Tras la aprobación del dueño (`CONFIRMADA`), el alquiler pasa a `EN CURSO` en la fecha pactada. El cliente tiene acceso al mapa interactivo de telemetría IoT.
5. **Cierre y Reputación:** Finalizado el alquiler (`FINALIZADA`), se activa el diálogo para emitir su valoración y comentario, enriqueciendo la reputación pública del vehículo.

4.4.4.2. Flujo de Usuario: Arrendador (Publicación de Flota, Aprobación y Supervisión)

El siguiente diagrama representa el recorrido del arrendador dentro de la aplicación mobile para publicar vehículos, gestionar solicitudes de reserva y supervisar sus unidades:


Inicio: Propietario accede a la plataforma
Iniciar Sesión como Arrendador
Dashboard Principal del Arrendador
¿Qué acción desea realizar?
Revisar Solicitudes
Publicar Auto
Gestionar Flota
Bandeja de Solicitudes de Reserva
Completar Formulario de Vehículo: Marca, Modelo, Año, Tarifa
Ver 'Mis Vehículos'
Evaluar Perfil del Arrendatario y Fechas Solicitadas
Subir Fotografías del Auto
Editar Tarifa / Modificar Disponibilidad / Dar de Baja
Publicar Vehículo en Catálogo
¿Aceptar solicitud?
No
Sí
Rechazar Reserva: Estado RECHAZADA
Aceptar Reserva: Estado CONFIRMADA



Descripción del flujo del Arrendador:

1. **Autenticación y Panel de Control:** El arrendador accede a su dashboard donde monitorea el rendimiento de sus publicaciones y el resumen de ingresos.
2. **Gestión de Publicaciones:** Puede registrar nuevos vehículos suministrando datos técnicos, tarifa por jornada y fotos. Las publicaciones pueden editarse o pausarse en cualquier momento (siempre que no tengan una reserva en curso).
3. **Bandeja de Solicitudes:** El propietario recibe notificaciones de solicitudes de reserva. Puede inspeccionar las fechas requeridas y el perfil verificado del arrendatario para tomar una decisión informada.
4. **Supervisión de Alquiler:** Una vez confirmada la reserva y entregada la unidad, el arrendador dispone de telemetría IoT para supervisar odómetro y velocidad.
5. **Cierre de Ciclo:** Tras recibir el auto de vuelta, el arrendador evalúa al cliente, fortaleciendo el ecosistema de confianza de RentiCar.

4.5. Mobile Applications Prototyping

En esta sección se presenta el prototipo interactivo de alta fidelidad de la aplicación móvil Renticar, desarrollado para los sistemas operativos iOS y Android. La propuesta busca ofrecer una experiencia de usuario intuitiva, accesible y consistente, manteniendo una identidad visual común y adaptando determinados elementos de la interfaz a las convenciones de cada plataforma. El diseño se fundamenta en los principios del diseño centrado en el usuario (UCD), considerando las necesidades de los arrendatarios, quienes buscan y reservan vehículos, y de los arrendadores, responsables de publicar y administrar sus unidades. Asimismo, se contemplan las características de interacción de cada sistema operativo, procurando mantener una estructura de navegación coherente, componentes reutilizables y dimensiones táctiles adecuadas para facilitar el uso de la aplicación.

Principales aspectos funcionales y de diseño del prototipo

- **Navegación adaptada a cada plataforma:** La aplicación incorpora cuatro secciones principales: Resumen, Vehículos o Mis vehículos, Reservas o Solicitudes y Perfil. La denominación y el contenido de estas secciones varían según el rol del usuario.
- **Contenido personalizado según el rol del usuario:** La interfaz adapta la información y las acciones disponibles a las necesidades de cada perfil. En el caso de los arrendatarios, la pantalla de resumen prioriza las reservas activas, las recomendaciones de vehículos y los accesos a la búsqueda. Para los arrendadores, presenta información sobre los vehículos publicados, la actividad del negocio y las opciones para administrar sus unidades.
- **Consistencia visual y reutilización de componentes:** Ambas versiones comparten criterios de diseño para la tipografía, la iconografía, las tarjetas, los campos de formulario, los botones y la distribución de los contenidos.
- **Priorización de las acciones principales:** Las acciones destacadas responden a las tareas más importantes de cada perfil. Para los arrendadores, se prioriza la publicación de vehículos mediante un botón visible que incorpora el icono «+» y el texto «Publicar vehículo». Para los arrendatarios, se facilita el acceso a la consulta y reserva de vehículos mediante botones integrados en las tarjetas y en las pantallas correspondientes. La ubicación de estos elementos considera la navegación y las áreas seguras de cada plataforma.

- **Interacción y accesibilidad táctil:** El prototipo contempla áreas táctiles mínimas de 44 × 44 pt para iOS y 48 × 48 dp para Android, con el propósito de facilitar la selección de los elementos interactivos.

4.5.1. Android Mobile Applications Prototyping

El prototipo para iOS presenta una interfaz limpia y minimalista, orientada a facilitar la navegación y el acceso a las funcionalidades principales de Renticar. La estructura incorpora una barra de navegación inferior fija con cuatro pestañas, cuyo estado activo se distingue mediante el color del icono (#52B6F2) y sus elementos asociados. Respecto a la interacción, se consideran áreas táctiles mínimas de 44 × 44 pt, cambios de opacidad como respuesta a las pulsaciones y retroalimentación háptica cuando resulte apropiado. Estas decisiones buscan proporcionar una experiencia de uso clara y familiar para los usuarios de dispositivos Apple, sin comprometer la consistencia visual de la aplicación.

🎥 Demostración en Video del Prototipo Web:
Ver demostración en video del prototipo interactivo Androi

4.5.2. iOS Mobile Applications Prototyping

El prototipo para Android conserva la estructura funcional, la paleta cromática y los componentes principales de Renticar, adaptando su presentación a los patrones de interacción de esta plataforma. La barra de navegación inferior dispone de cuatro destinos principales sobre un fondo blanco y una altura aproximada de 80 dp. Para comunicar el estado seleccionado, se incorpora una cápsula de color azul claro (#52B6F2) detrás del icono activo, mientras que el icono utiliza el azul oscuro (#003F70), reforzando el contraste y la continuidad con la identidad de la marca. La distribución de los elementos sigue una retícula de 8 dp, con márgenes laterales de 16 dp y espaciados internos de 8, 16 y 24 dp. Esta organización favorece la alineación de los componentes y mantiene una distribución visual ordenada.

🎥 Demostración en Video del Prototipo Web:
Ver demostración en video del prototipo interactivo iOs

4.6. Web Applications UX/UI Design

En esta sección se presenta la propuesta visual y de interacción para la aplicación web RentiCar. El diseño se ha centrado en los dos segmentos objetivos identificados: arrendadores (dueños de vehículos) y arrendatarios (clientes que buscan alquilar). La prioridad ha sido crear una experiencia de usuario intuitiva, segura y eficiente, aplicando principios de diseño inclusivo y una arquitectura de información clara para facilitar la navegación y el cumplimiento de los objetivos de cada usuario.

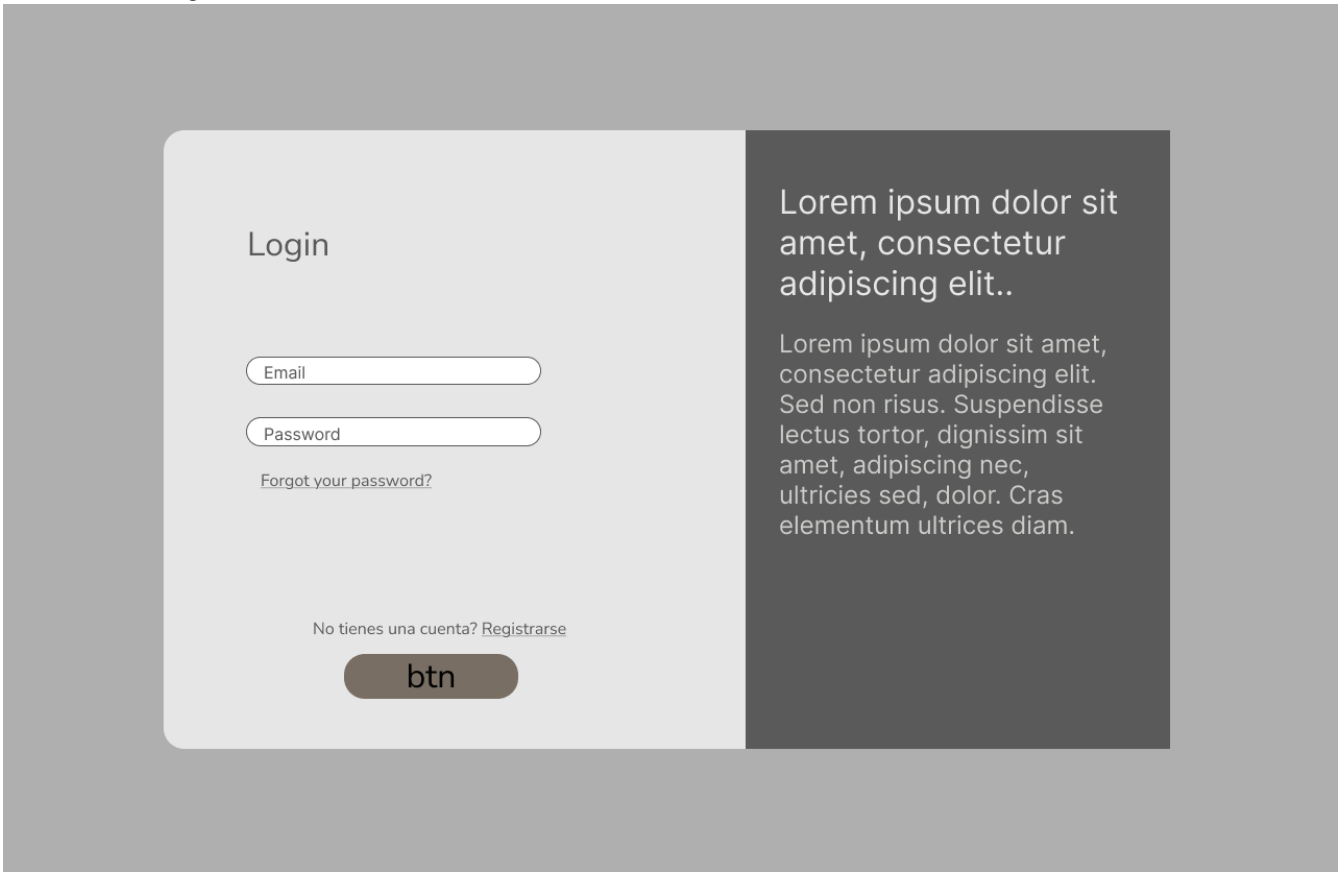
4.6.1. Web Applications Wireframes

Los wireframes de RentiCar se han diseñado para establecer la estructura fundamental de la aplicación, priorizando la funcionalidad y la jerarquía de la información antes de aplicar el estilo visual. Se ha buscado una disposición limpia y organizada que guíe a los usuarios a través de las tareas clave, como publicar un vehículo o reservar uno. El diseño es adaptable para garantizar una experiencia consistente tanto en navegadores de escritorio como en dispositivos móviles.

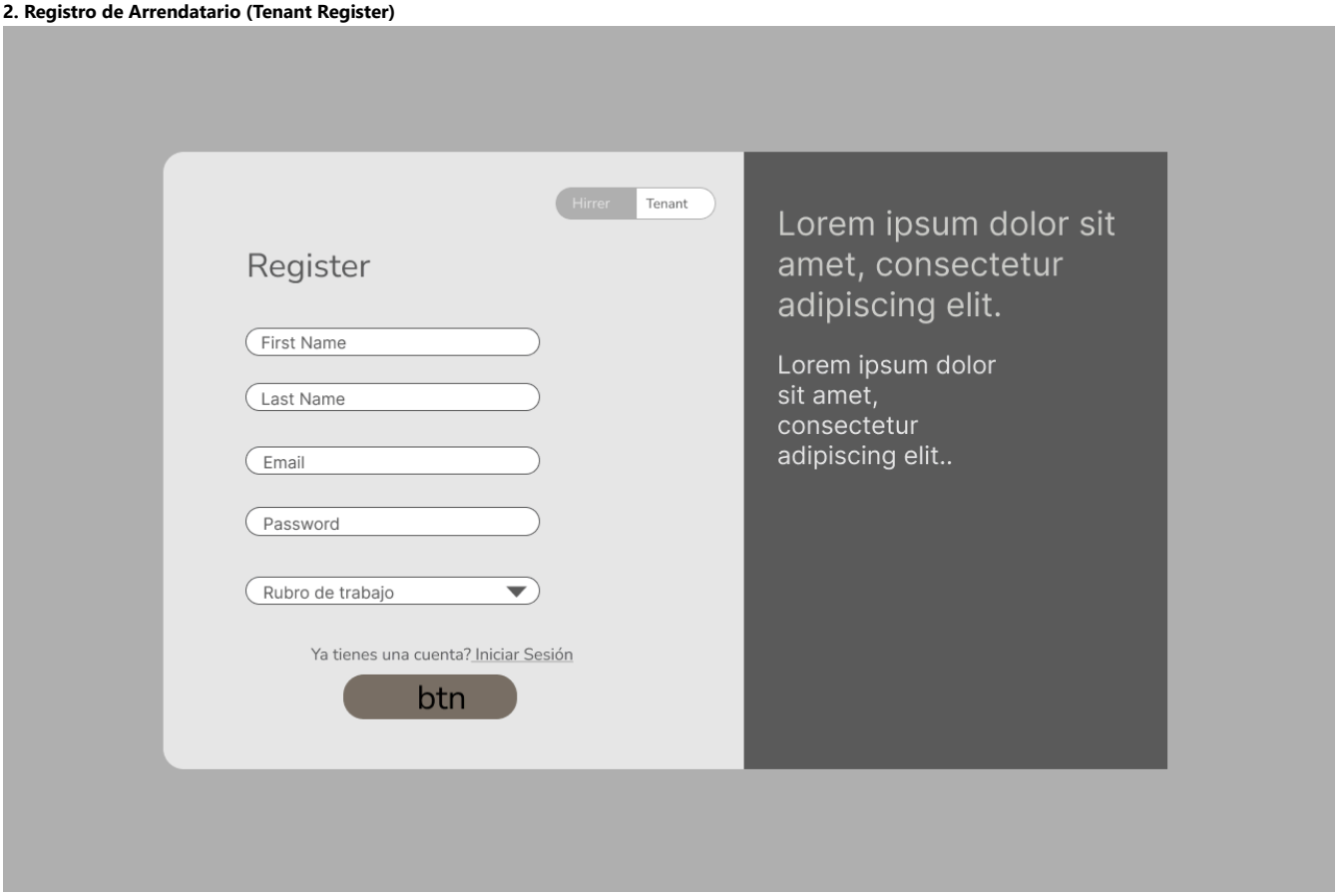
Enlace al proyecto en Figma: https://www.figma.com/design/ICtIq3D2cmTwZMzqJPUxF7/RentiCar?node-id=2-4606&t=mjT7rIE8qUSnwcfL-1

1. *Iniciar Sesión / Registrarse* Permite a los usuarios crear una cuenta o acceder a la plataforma. El formulario de registro solicita información básica y permite elegir el tipo de perfil (arrendador o arrendatario) para personalizar la experiencia desde el inicio.

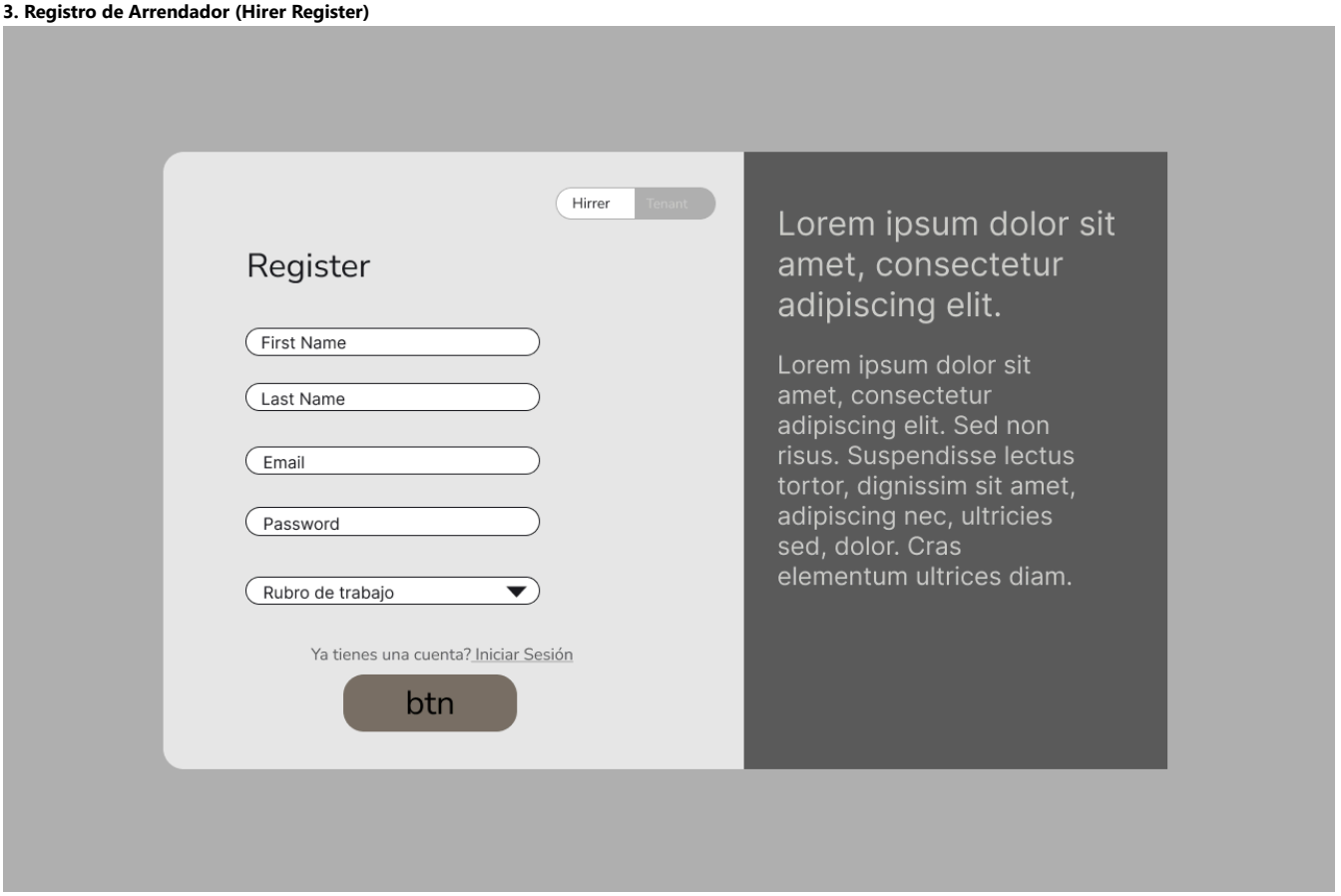
1. Inicio de Sesión (Sign In)



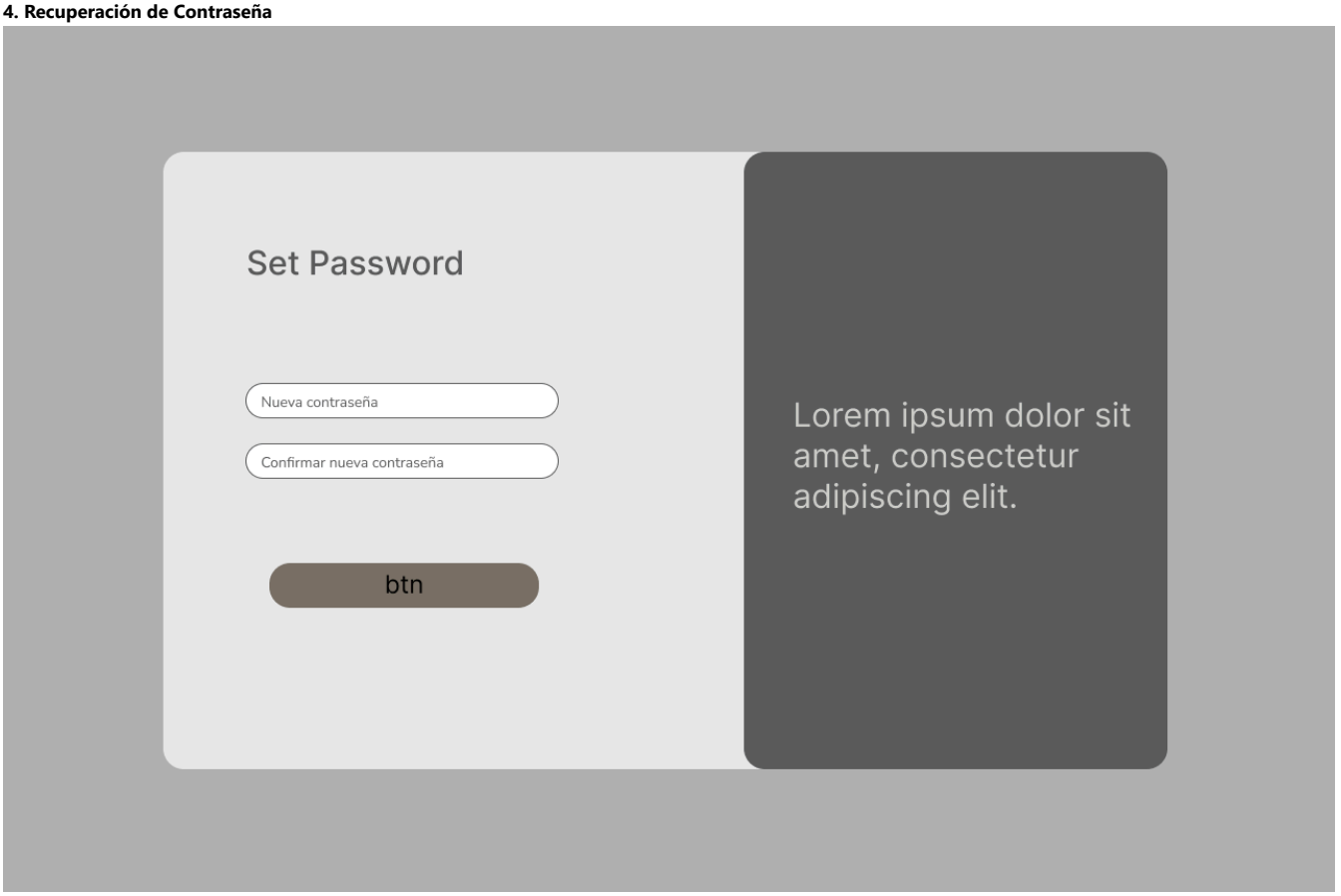
2. Registro de Arrendatario (Tenant Register)



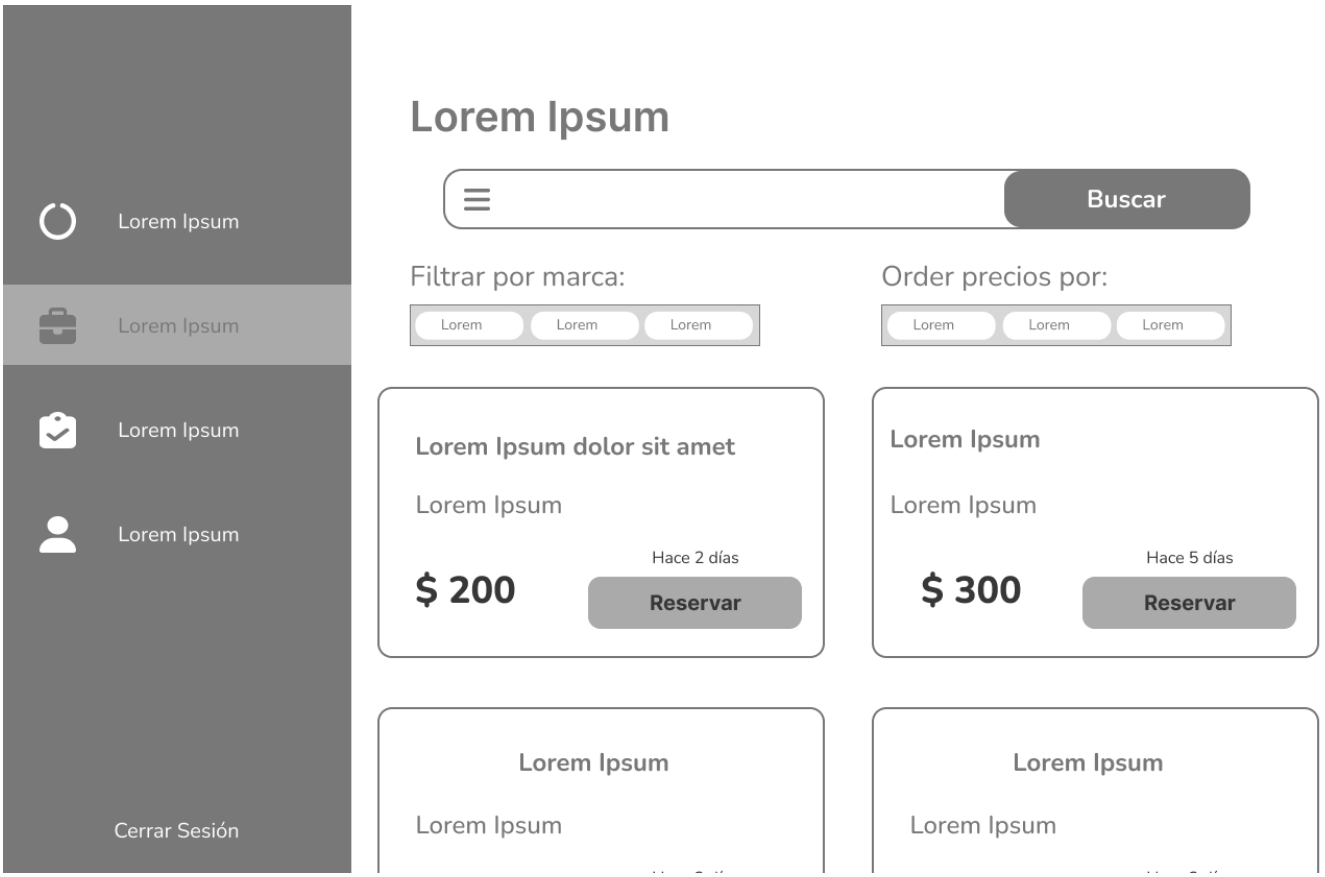
3. Registro de Arrendador (Hirer Register)

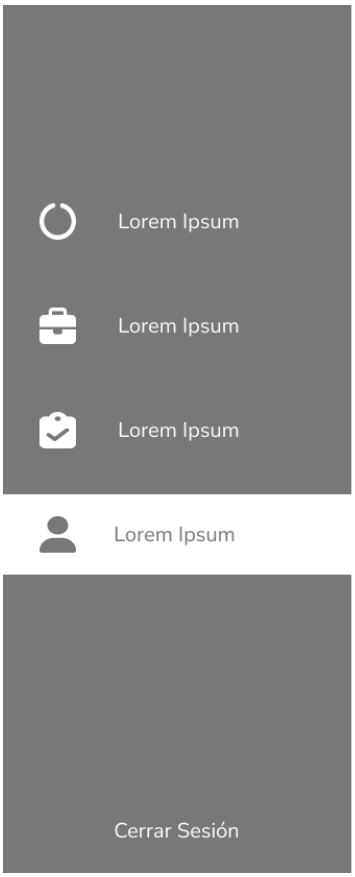


4. Recuperación de Contraseña



2. *Dashboard del Arrendador* Muestra un resumen de los vehículos publicados, las reservas activas y las ganancias generadas. Ofrece accesos directos para añadir un nuevo vehículo y gestionar las publicaciones existentes.





Mi Lorem Ipsum

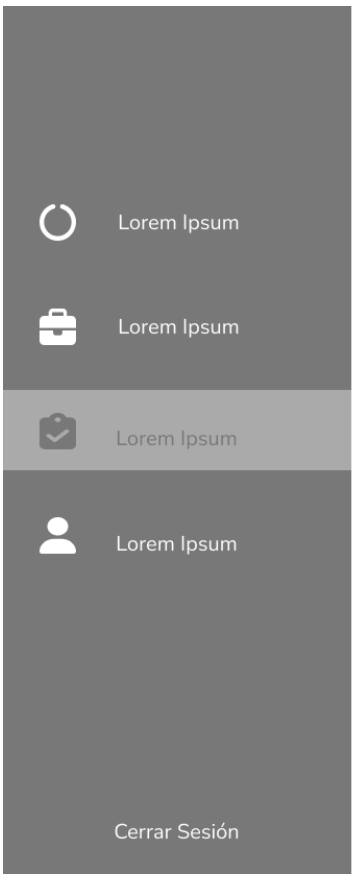
Nombre completo: Lorem Ipsum
Fecha de Nacimiento: 13/03/2003
Ciudad/Pais: Lorem Ipsum
Carrera: Ingeníeria de Software
Correo electronico: jairquijada123@gmail.com
Celular: Lorem Ipsum

4.8

Tecnologias y Especializaciones

Lorem Lorem Lorem Lorem Lorem Ipsum

Editar Perfil



Lorem Ipsum

Lorem Ipsum dolor sit amet
Lorem Ipsum
Activa
$ 200
Cancelar

Lorem Ipsum dolor sit amet
Lorem Ipsum
Activa
$ 200
Cancelar

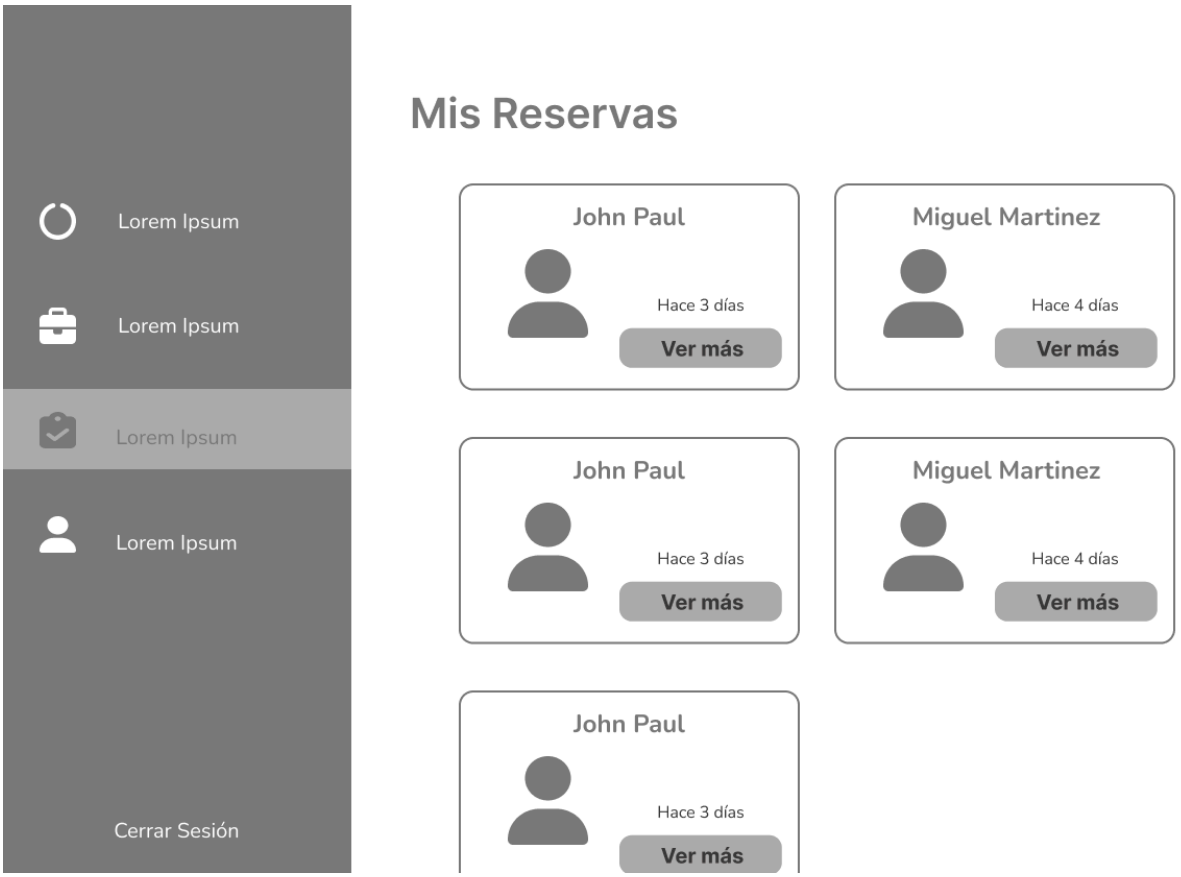
Lorem Ipsum dolor sit amet
Lorem Ipsum
Activa
$ 230
Cancelar

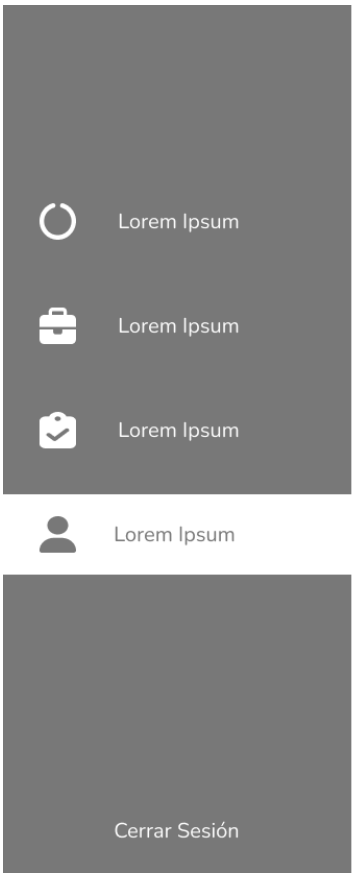
Lorem Ipsum dolor sit amet
Lorem Ipsum
Activa
$ 300
Cancelar

Lorem Ipsum dolor sit amet

Lorem Ipsum dolor sit amet

3. *Dashboard del Arrendatario* Presenta un resumen de las reservas realizadas, vehículos recomendados y accesos rápidos para buscar nuevas opciones. El diseño se enfoca en facilitar la continuación de una búsqueda o la gestión de un alquiler en curso.





Mi Lorem Ipsum

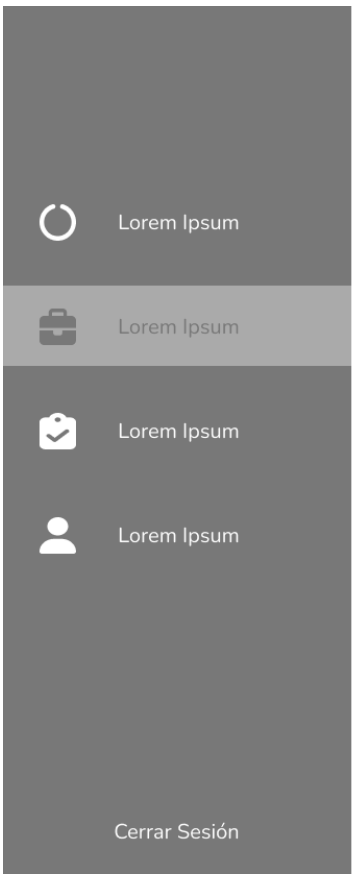
Nombre completo: Lorem Ipsum
Fecha de Nacimiento: 13/03/2003
Ciudad/Pais: Lorem Ipsum
Carrera: Ingeníeria de Software
Correo electronico: jairquijada123@gmail.com
Celular: Lorem Ipsum

4.8

Tecnologias y Especializaciones

Lorem Lorem Lorem Lorem Lorem Ipsum

Editar Perfil



Mis vehiculos

Publiar nuevo vehículo

Lorem Ipsum dolor sit amet
Lorem Ipsum
Activa
$ 200
Editar

Lorem Ipsum dolor sit amet
Lorem Ipsum
Activa
$ 200
Editar

Lorem Ipsum dolor sit amet
Lorem Ipsum
Activa
$ 230
Editar

Muestra un resumen de los vehículos publicados, las reservas activas y las ganancias generadas. Ofrece accesos directos para añadir un nuevo vehículo y gestionar las publicaciones existentes.

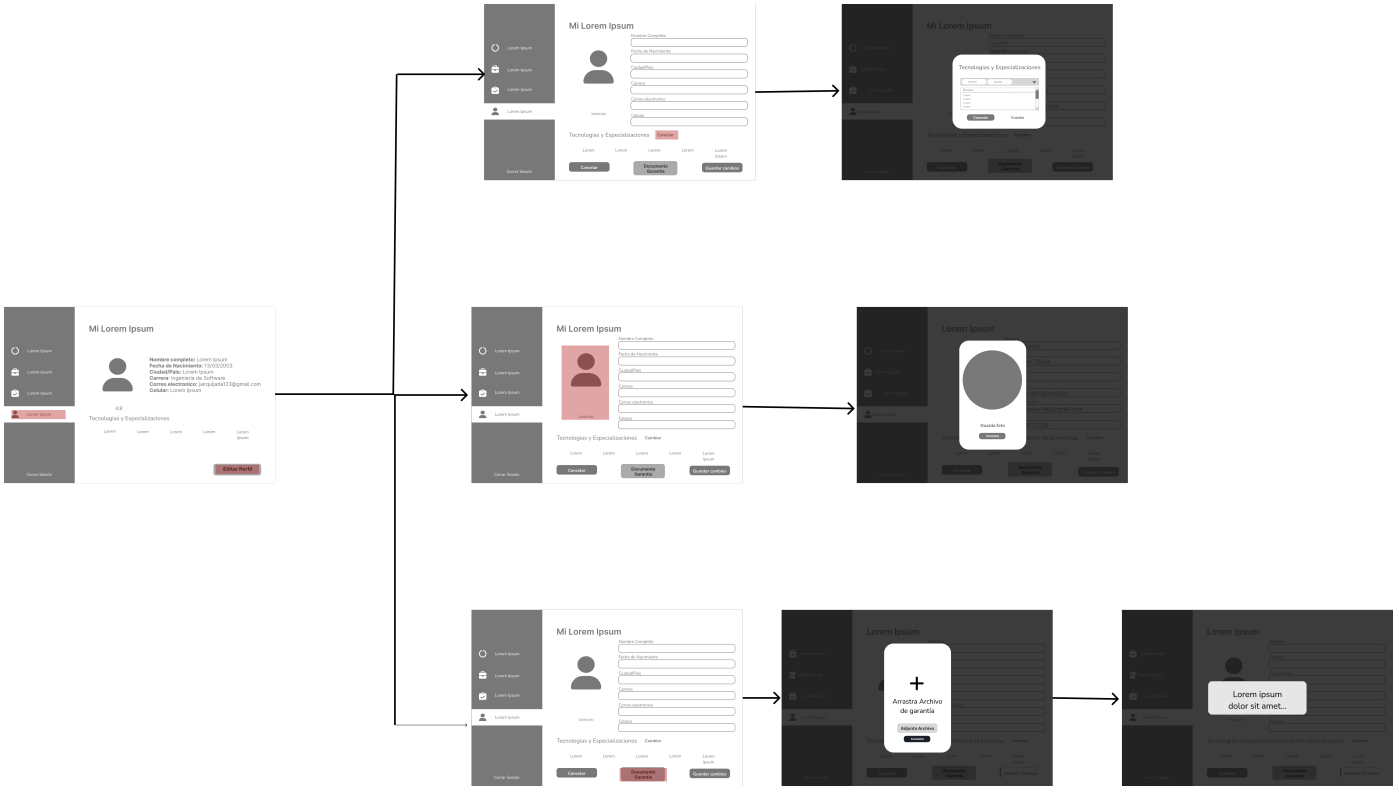
4.6.2. Web Applications Wireflow Diagrams

Los wireflows diagraman las rutas que los usuarios siguen para completar tareas específicas dentro de la aplicación. Se ha creado un wireflow para cada objetivo principal de nuestros User Personas, asegurando que cada paso sea lógico y sin fricciones.

---

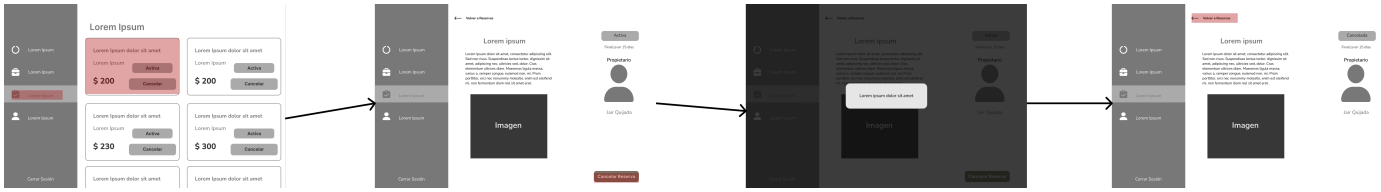
1. Segmento 1: Arrendatarios

User Goal: Como usuario, quiero gestionar mi perfil de forma integral, pudiendo editar mi información personal, cambiar mi foto y cargar los documentos necesarios para validar mi cuenta.



- Wireflow

User Goal: Como arrendatario, quiero administrar el ciclo de vida completo de mis reservas, desde ver sus detalles y cancelarlas, hasta dejar una reseña una vez finalizado el alquiler.

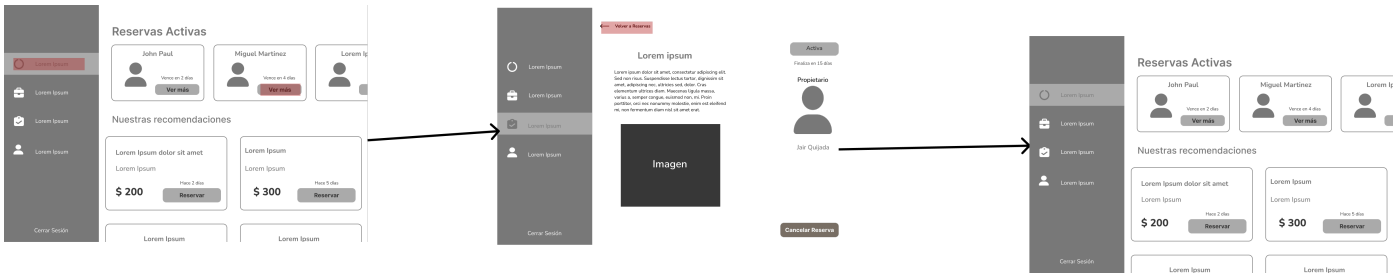


- Wireflow

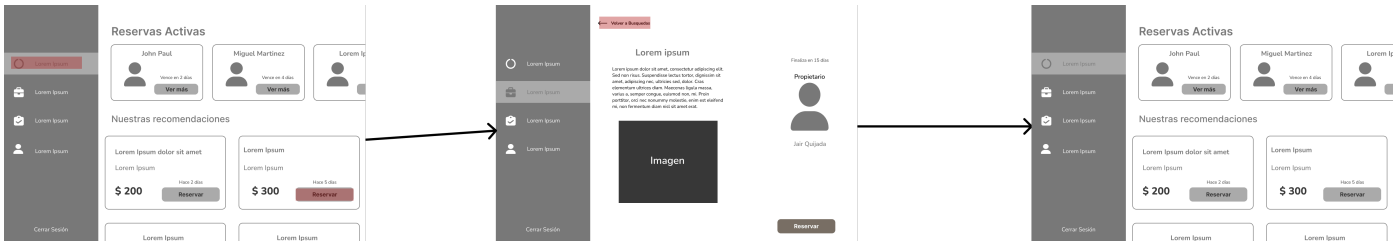
User Goal: Como arrendatario, quiero buscar un vehículo con filtros específicos, ver sus detalles en profundidad y completar el proceso de reserva de manera fluida.

- Wireflow

User Goal: Como arrendatario, quiero acceder desde mi dashboard a la gestión de mis reservas activas para poder ver sus detalles o cancelarlas si es necesario.

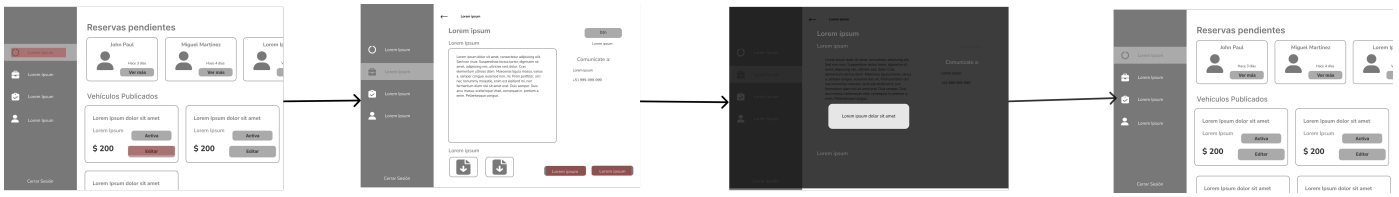


User Goal: Como arrendatario, quiero poder explorar las recomendaciones de vehículos directamente desde mi dashboard y realizar una reserva de forma rápida y sencilla.

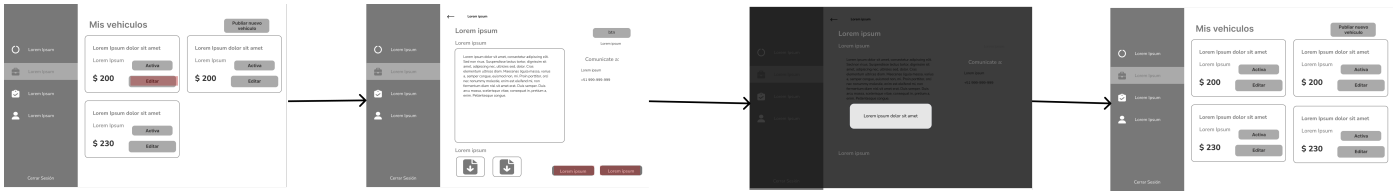


2. Segmento 2: Arrendadores

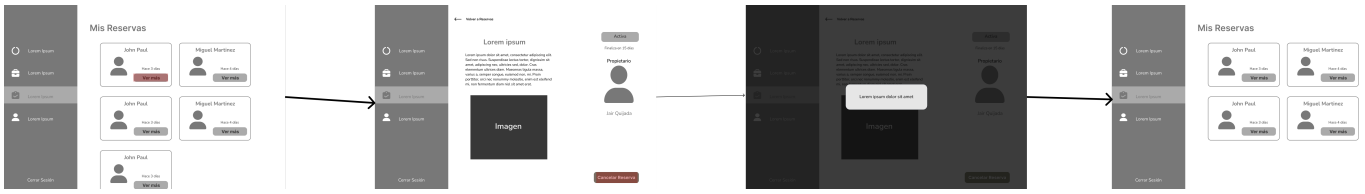
User Goal: Como arrendador, quiero ver un resumen rápido del estado de mi negocio, incluyendo mis vehículos publicados y las reservas pendientes, tan pronto como inicie sesión.



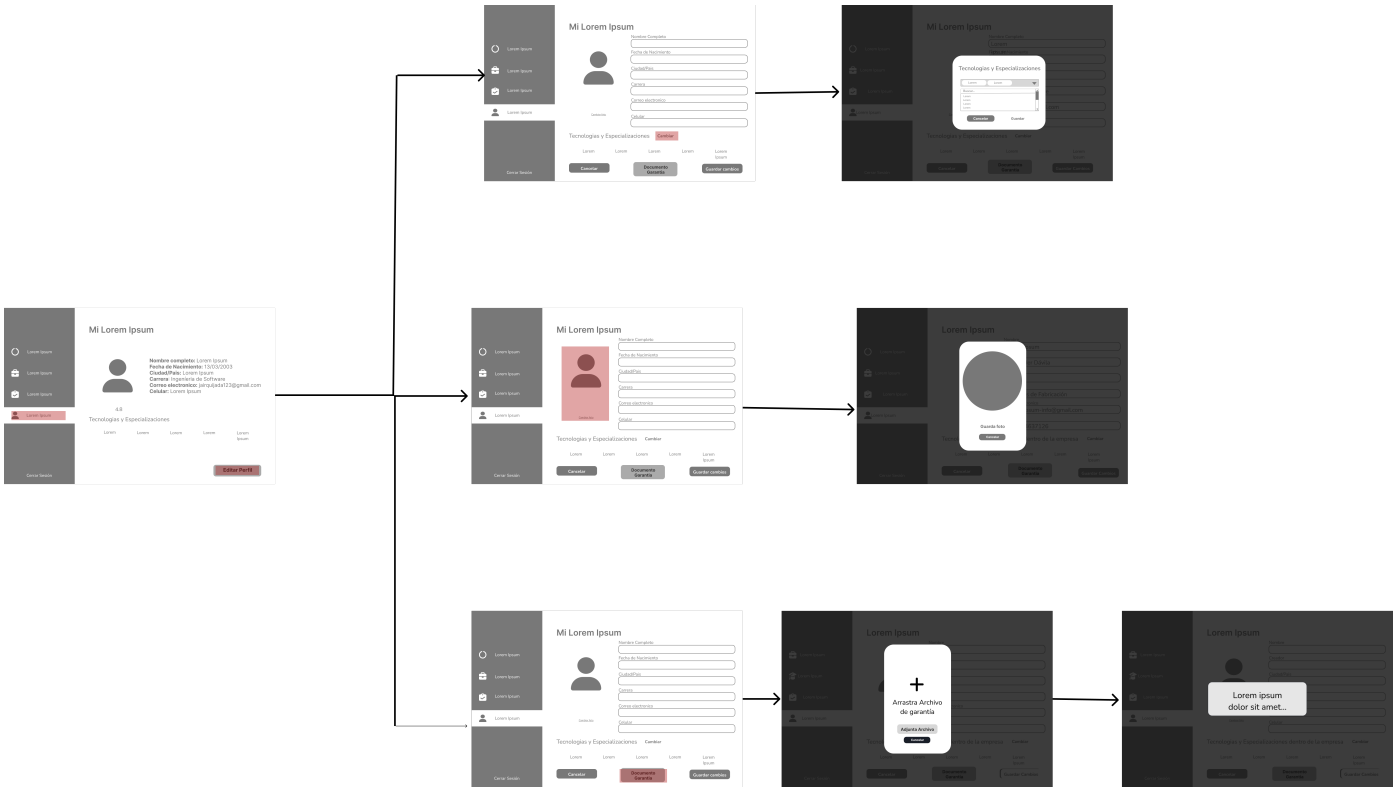
User Goal (Crear): Como arrendador, quiero añadir un nuevo vehículo a la plataforma de forma rápida y sencilla para empezar a alquilarlo. *User Goal (Editar):* Como arrendador, quiero poder modificar fácilmente los detalles de mis publicaciones existentes, como el precio o la descripción, para mantener la información actualizada. *User Goal (Eliminar):* Como arrendador, quiero retirar de la plataforma los vehículos que ya no están disponibles para alquilar, con una simple acción de eliminación.



User Goal: Como arrendador, quiero gestionar las solicitudes de alquiler de mis vehículos de manera eficiente, pudiendo aceptarlas o rechazarlas desde un panel de control.



User Goal: Como arrendador, quiero mantener mi perfil actualizado con mi información de contacto y documentos de garantía para generar confianza en los clientes.



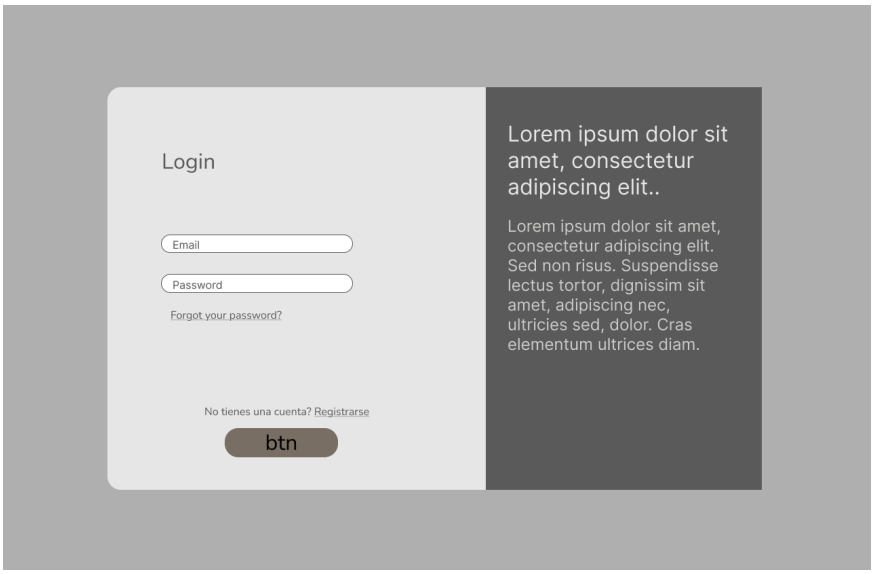
4.6.3. Web Applications Mock-ups

En esta sección se presentan los mock-ups de alta fidelidad correspondientes a la aplicación web de **RentiCar**, diseñados en Figma respetando la guía de estilos, paleta de colores y componentes visuales definidos previamente. Las interfaces se encuentran agrupadas según los flujos de interacción de los dos segmentos clave: Arrendatarios y Arrendadores, además de los módulos transversales de autenticación y telemetría.

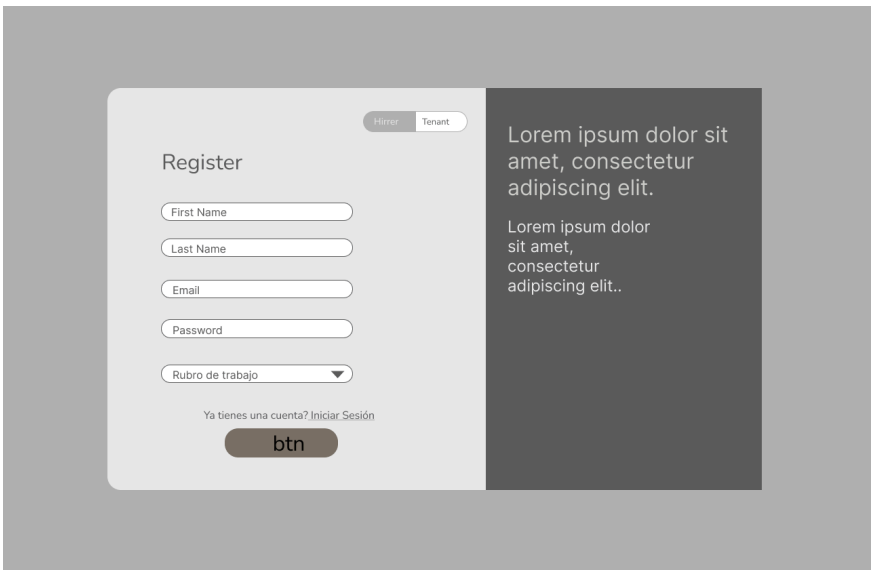
4.6.3.1. Módulo de Autenticación y Gestión de Acceso (IAM)

Este módulo centraliza las interfaces de registro, inicio de sesión seguro y recuperación de credenciales para ambos perfiles de usuario.

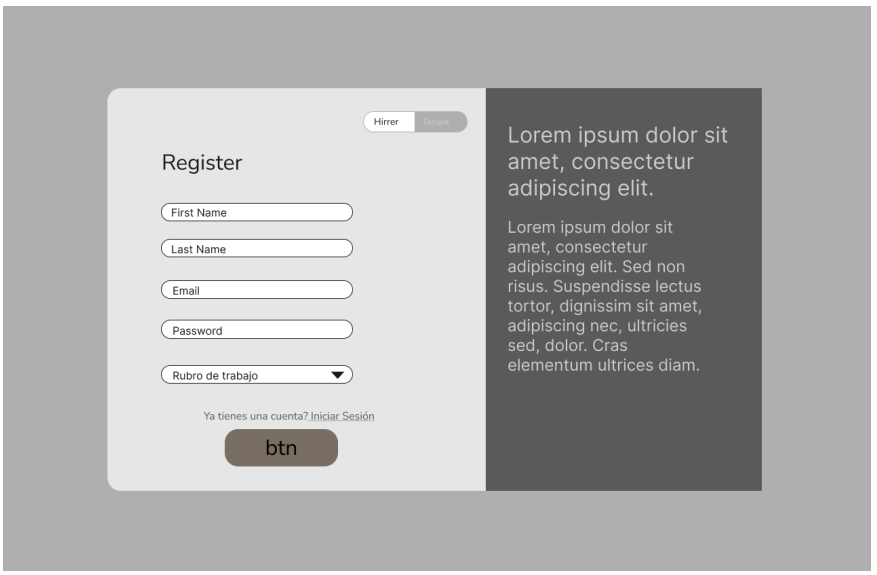
- **Pantalla de Inicio de Sesión (Sign In):**
  Permite a los usuarios registrados acceder a su sesión ingresando su correo y contraseña, con validaciones en tiempo real y opción de recordar credenciales.



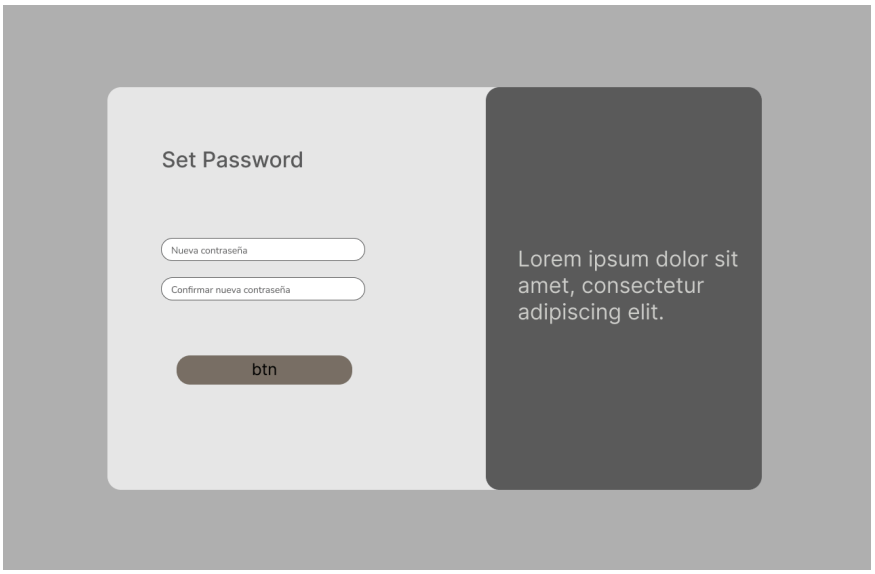
- **Pantalla de Registro para Arrendatarios (Tenant Register):**
  Formulario intuitivo donde los clientes ingresan sus datos personales, correo, teléfono y contraseña para crear su cuenta de arrendatario.



- **Pantalla de Registro para Arrendadores (Hirrer Register):**
  Interfaz orientada a propietarios de vehículos que desean publicar unidades, solicitando datos de contacto y verificación de identidad.



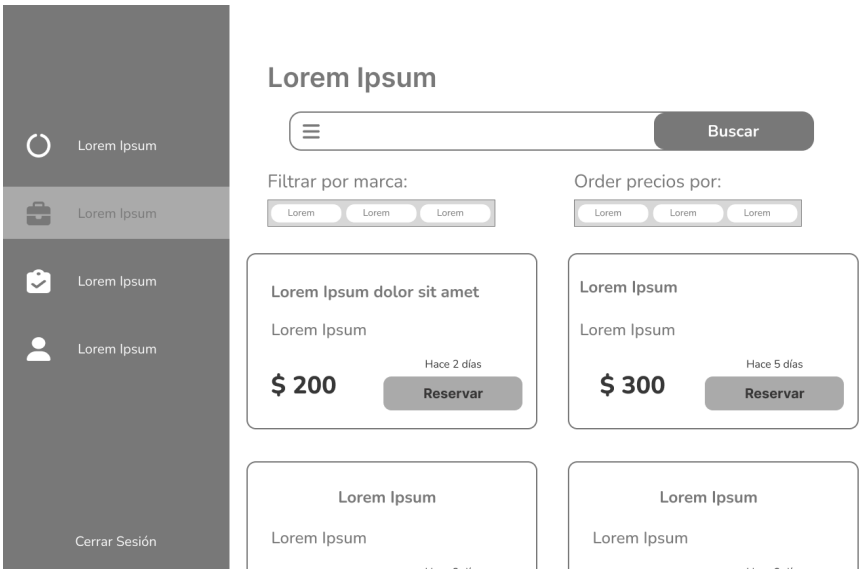
- **Pantalla de Recuperación de Contraseña:**
  Permite ingresar el correo registrado para recibir un enlace seguro de restablecimiento de contraseña mediante token de vigencia limitada.



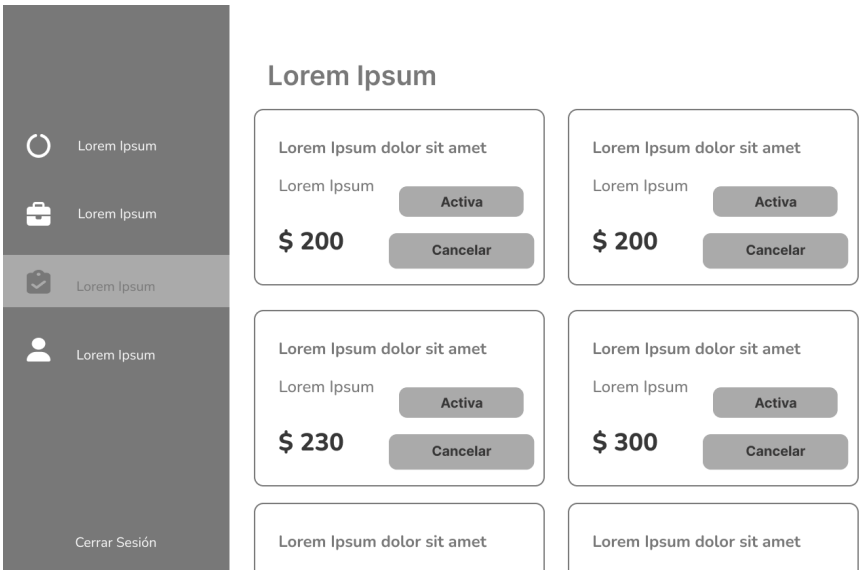
4.6.3.2. Módulo de Experiencia del Arrendatario (Búsqueda, Reserva y Reseñas)

Interfaces diseñadas para que el arrendatario explore el catálogo, configure sus fechas de alquiler, dé seguimiento a sus solicitudes y califique el servicio.

- **Búsqueda y Reserva de un Vehículo:**
  Vista principal de catálogo con cuadrícula de autos, filtros avanzados por marca, rango de precio, tipo de combustible y botón directo para iniciar la reserva.



- **Gestión de Reservas del Arrendatario:**
  Panel interactivo que consolida las reservas activas, pendientes y concluidas del usuario, mostrando estado actual e información del auto.



- **Detalle de Reserva y Comprobante:**
  Ficha descriptiva que desglosa el cálculo de días reservados, tarifa pactada por jornada, importe total en soles y datos de contacto del propietario.

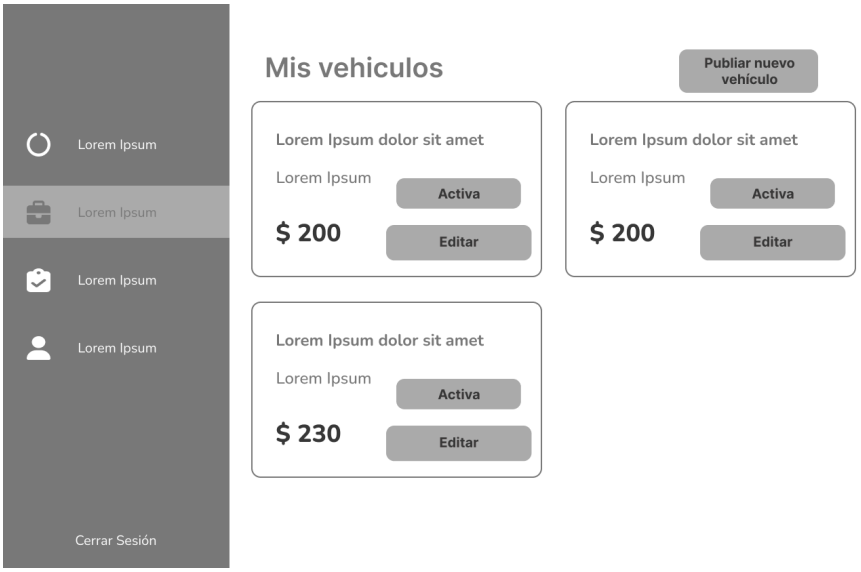
- **Finalización y Calificación de un Alquiler:**
  Modal que se activa al culminar el periodo de alquiler, permitiendo puntuar con estrellas (1 a 5) y redactar una reseña pública sobre la experiencia.

4.6.3.3. Módulo de Experiencia del Arrendador (Gestión de Flota y Solicitudes)

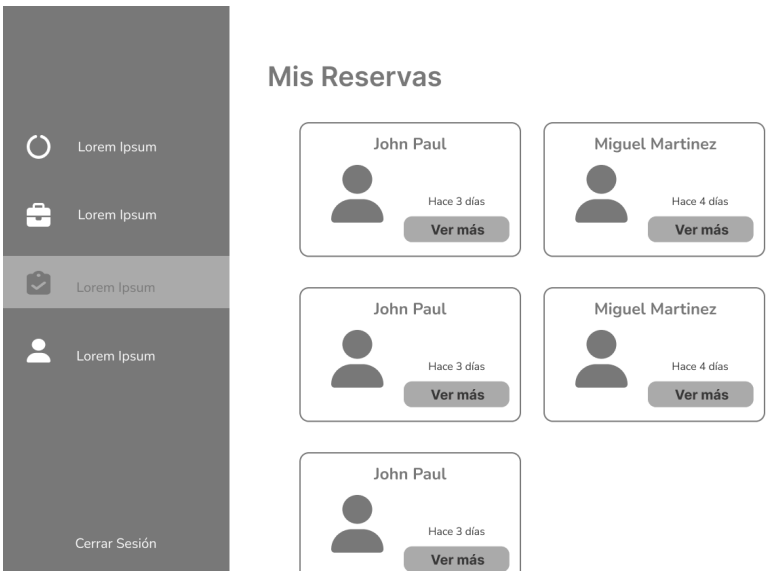
Interfaces especializadas para que los propietarios gestionen sus publicaciones, administren reservas entrantes y configuren su perfil comercial.

- **Primer Acceso y Vista General (Dashboard del Arrendador):**
  Tablero de control con métricas de ingresos, cantidad de vehículos activos, reservas en curso y accesos rápidos a publicaciones.

- **Gestión y Publicación de Vehículos:**
  Listado de la flota registrada del arrendador, con opciones para agregar un nuevo auto, editar tarifa diaria, pausar disponibilidad o retirar publicaciones.



- **Administración de Solicitudes de Reserva:**
  Bandeja de solicitudes entrantes donde el propietario revisa la identidad del solicitante, fechas deseadas y puede aceptar o rechazar el alquiler.



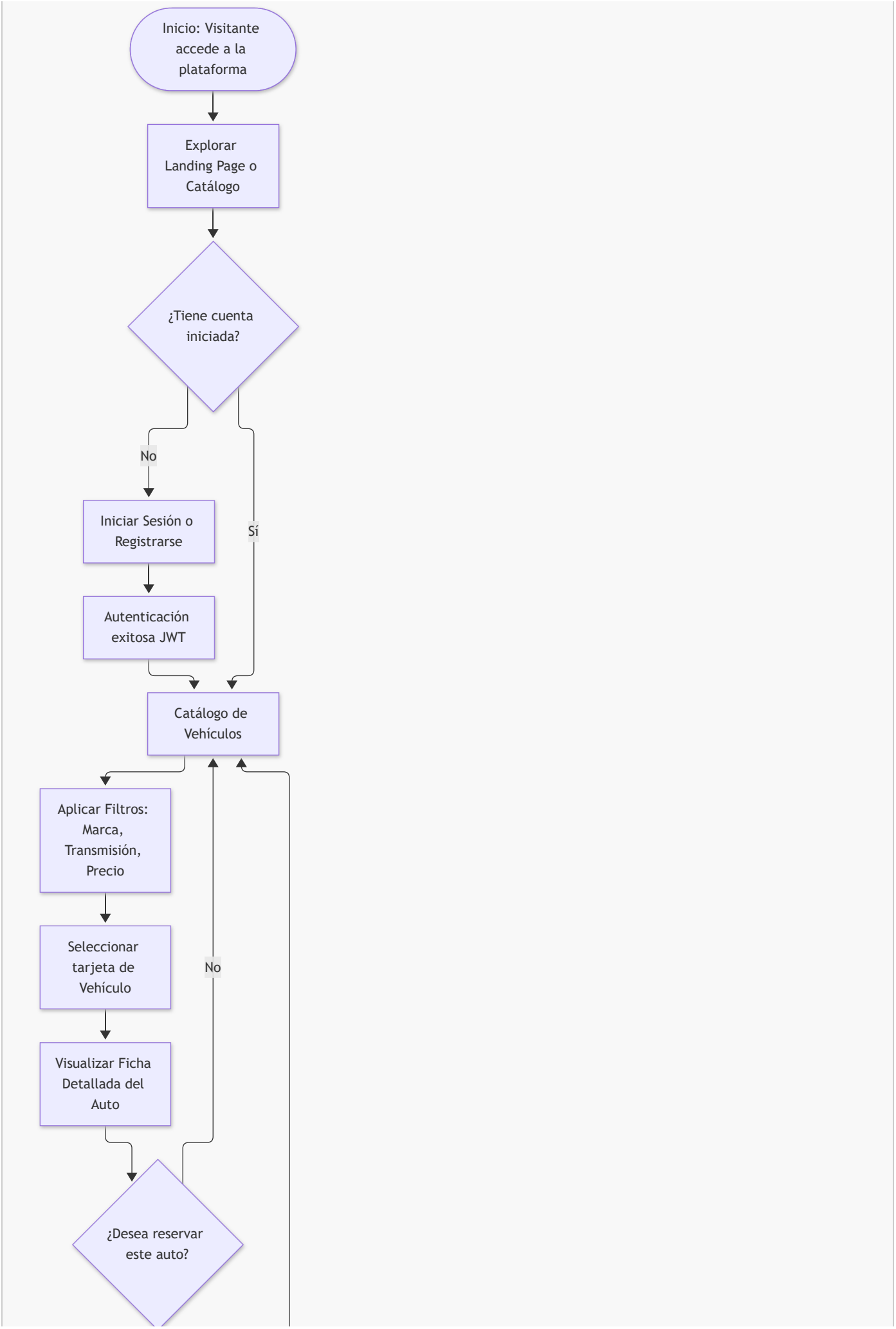
- **Edición de Perfil de Propietario:**
  Formulario para actualizar información personal, número telefónico de contacto y carga de documentos de garantía.

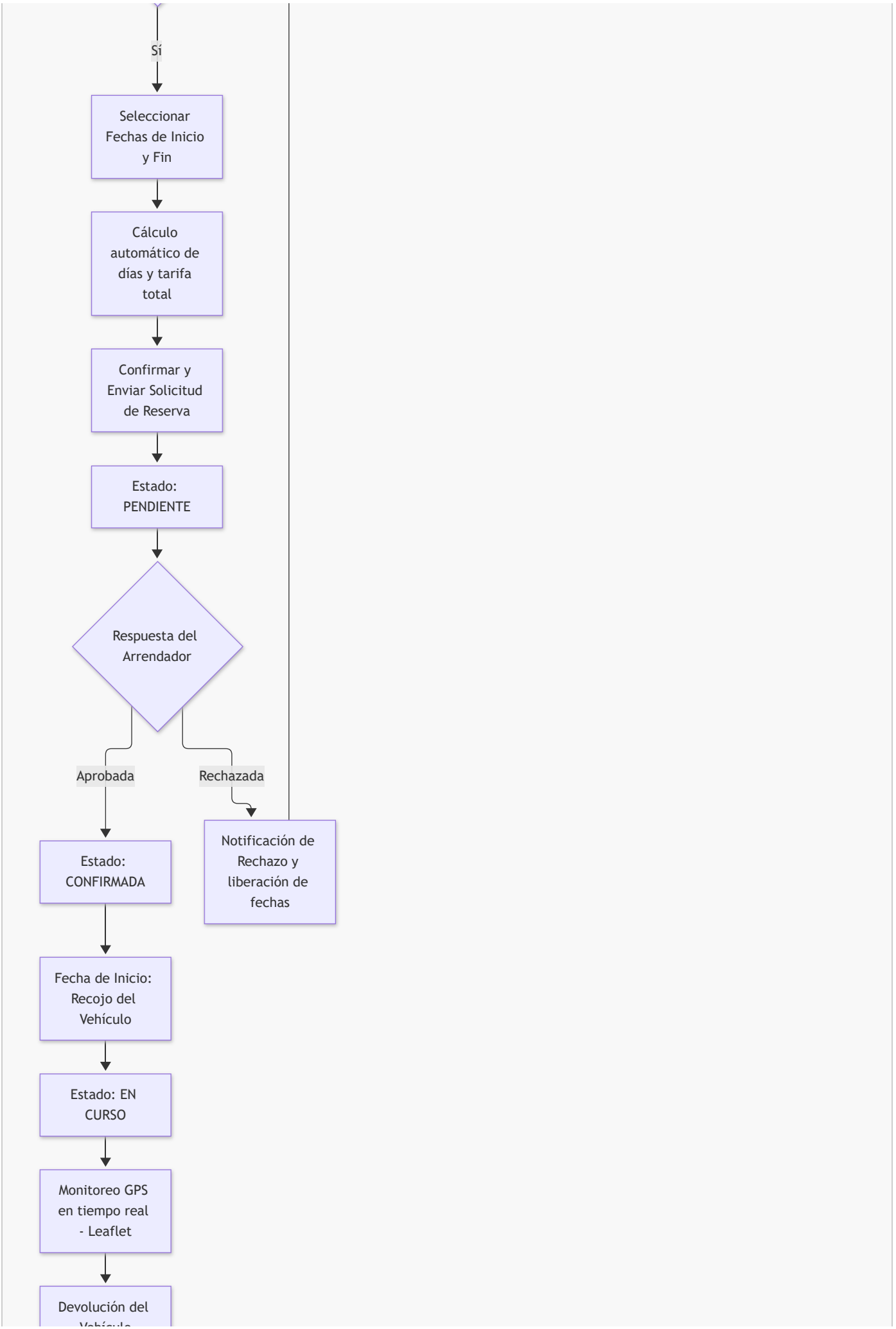
4.6.4. Web Applications User Flow Diagrams

En esta sección se modelan los **diagramas de flujo de usuario (User Flow Diagrams)** que describen las rutas de interacción, puntos de decisión y transiciones de estado de los usuarios a través de la aplicación web RentiCar.

4.6.4.1. Flujo de Usuario: Arrendatario (Búsqueda, Reserva, Monitoreo y Calificación)

El siguiente diagrama ilustra el recorrido completo de un cliente desde su acceso inicial a la plataforma hasta la conclusión y calificación del servicio de alquiler:


Inicio: Visitante accede a la plataforma
Explorar Landing Page o Catálogo
¿Tiene cuenta iniciada?
No
Sí
Iniciar Sesión o Registrarse
Autenticación exitosa JWT
Catálogo de Vehículos
Aplicar Filtros: Marca, Transmisión, Precio
Seleccionar tarjeta de Vehículo
Visualizar Ficha Detallada del Auto
¿Desea reservar este auto?
No

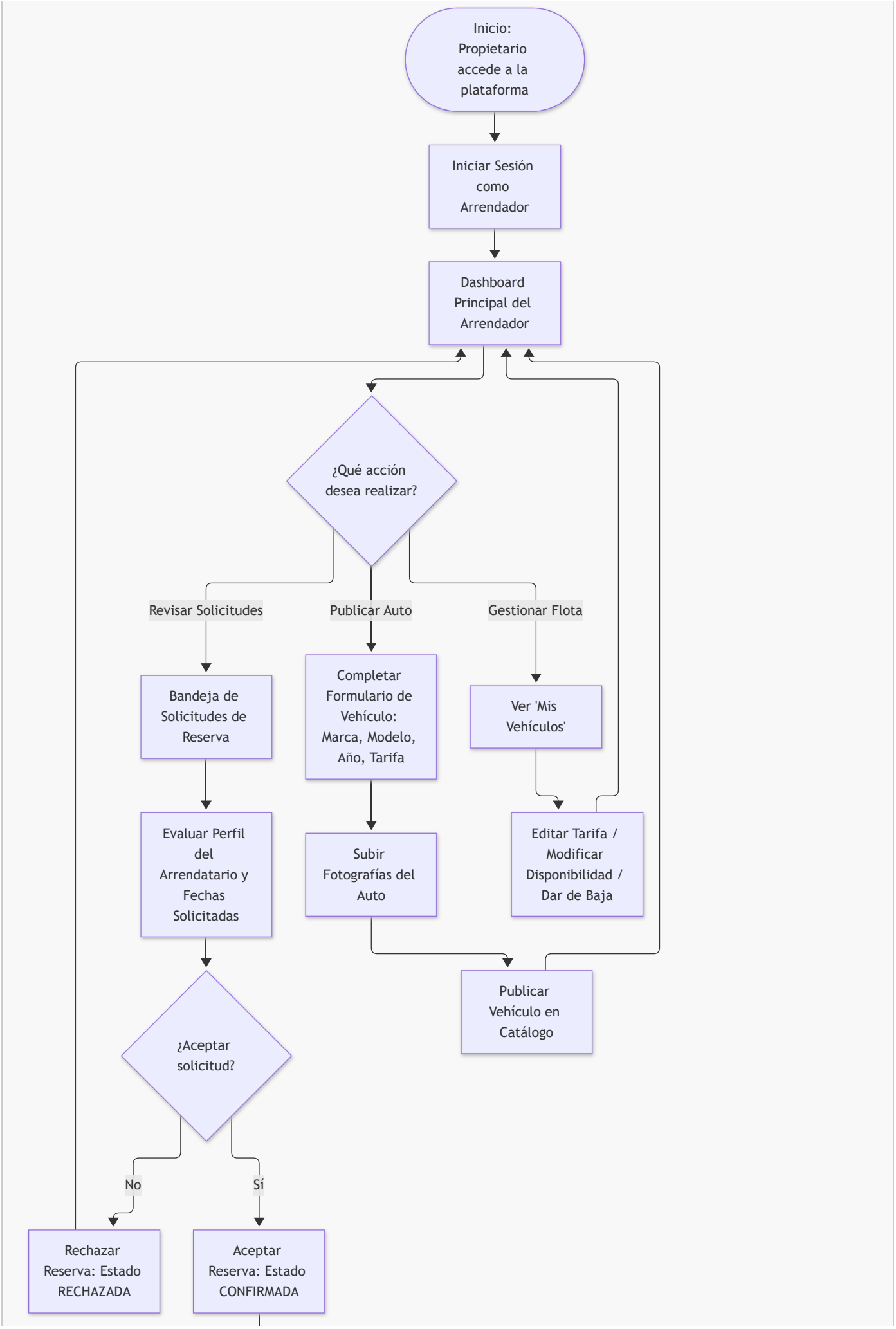

Sí
Seleccionar Fechas de Inicio y Fin
Cálculo automático de días y tarifa total
Confirmar y Enviar Solicitud de Reserva
Estado: PENDIENTE
Respuesta del Arrendador
Aprobada
Rechazada
Estado: CONFIRMADA
Notificación de Rechazo y liberación de fechas
Fecha de Inicio: Recojo del Vehículo
Estado: EN CURSO
Monitoreo GPS en tiempo real - Leaflet
Devolución del

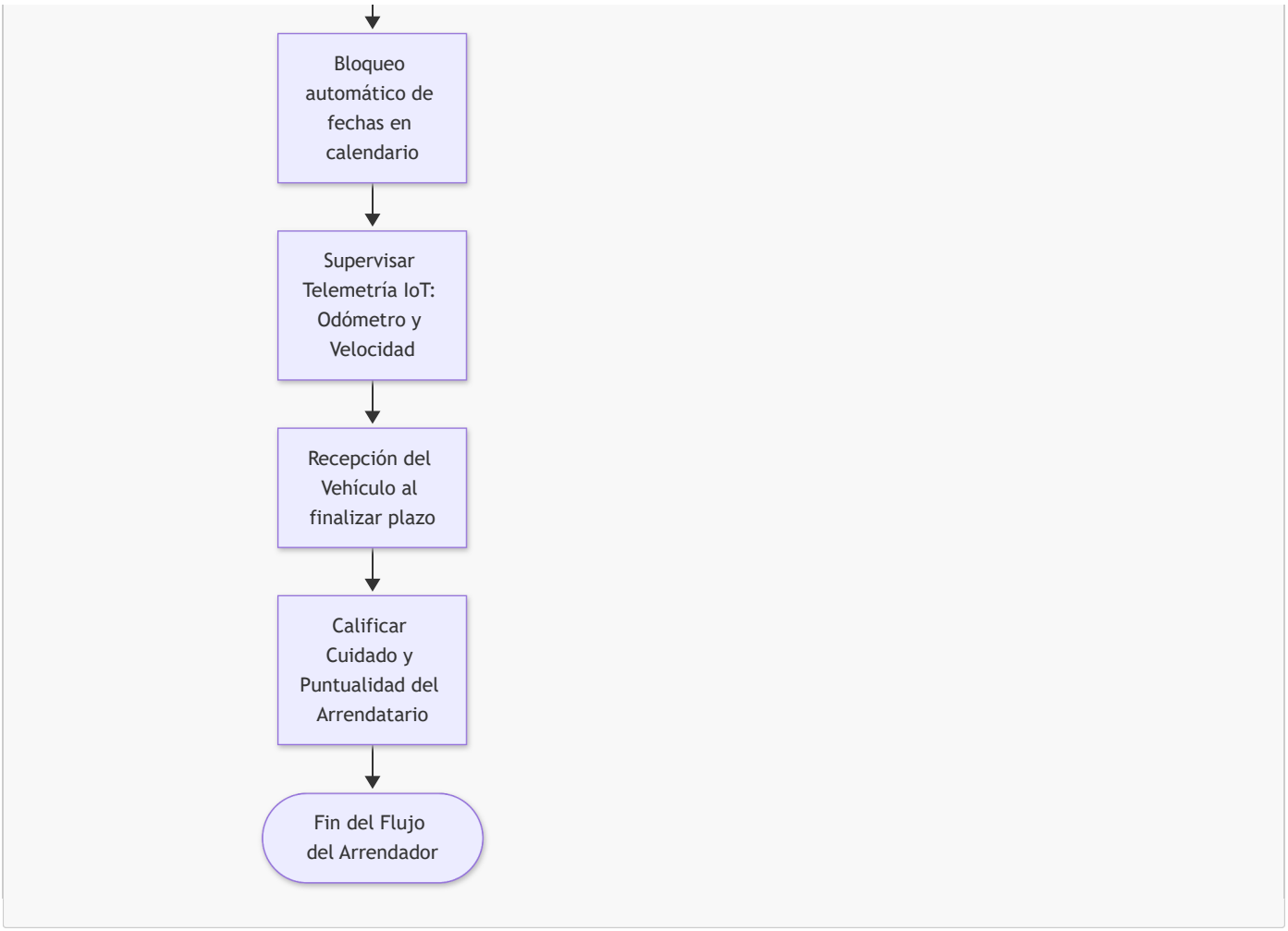
Descripción del flujo del Arrendatario:

1. **Descubrimiento y Navegación:** El cliente ingresa y navega por el catálogo público. Para concretar una reserva, el sistema valida su sesión activa o le solicita autenticarse con JWT.
2. **Filtrado y Selección:** El usuario puede refinar los resultados por marca, caja de cambios y tarifa diaria. Al seleccionar un vehículo, visualiza especificaciones completas e imágenes.
3. **Reserva y Presupuesto:** Al elegir el rango de fechas en el calendario interactivo, el sistema calcula de forma reactiva el importe total. Tras confirmar, la solicitud se envía al propietario con estado `PENDIENTE`.
4. **Seguimiento y Uso IoT:** Tras la aprobación del dueño (`CONFIRMADA`), el alquiler pasa a `EN CURSO` en la fecha pactada. El cliente tiene acceso al mapa interactivo de telemetría IoT.
5. **Cierre y Reputación:** Finalizado el alquiler (`FINALIZADA`), se activa el diálogo para emitir su valoración y comentario, enriqueciendo la reputación pública del vehículo.

4.6.4.2. Flujo de Usuario: Arrendador (Publicación de Flota, Aprobación y Supervisión)

El siguiente diagrama detalla el recorrido del propietario para publicar un vehículo, gestionar solicitudes de reserva y dar seguimiento a sus unidades:


Inicio: Propietario accede a la plataforma
Iniciar Sesión como Arrendador
Dashboard Principal del Arrendador
¿Qué acción desea realizar?
Revisar Solicitudes
Publicar Auto
Gestionar Flota
Bandeja de Solicitudes de Reserva
Completar Formulario de Vehículo: Marca, Modelo, Año, Tarifa
Ver 'Mis Vehículos'
Evaluar Perfil del Arrendatario y Fechas Solicitadas
Subir Fotografías del Auto
Editar Tarifa / Modificar Disponibilidad / Dar de Baja
Publicar Vehículo en Catálogo
¿Aceptar solicitud?
No
Sí
Rechazar Reserva: Estado RECHAZADA
Aceptar Reserva: Estado CONFIRMADA



Descripción del flujo del Arrendador:

1. **Autenticación y Panel de Control:** El arrendador accede a su dashboard donde monitorea el rendimiento de sus publicaciones y el resumen de ingresos.
2. **Gestión de Publicaciones:** Puede registrar nuevos vehículos suministrando datos técnicos, tarifa por jornada y fotos. Las publicaciones pueden editarse o pausarse en cualquier momento (siempre que no tengan una reserva en curso).
3. **Bandeja de Solicitudes:** El propietario recibe notificaciones de solicitudes de reserva. Puede inspeccionar las fechas requeridas y el perfil verificado del arrendatario para tomar una decisión informada.
4. **Supervisión de Alquiler:** Una vez confirmada la reserva y entregada la unidad, el arrendador dispone de telemetría IoT para supervisar odómetro y velocidad.
5. **Cierre de Ciclo:** Tras recibir el auto de vuelta, el arrendador evalúa al cliente, fortaleciendo el ecosistema de confianza de RentiCar.

4.7. Web Applications Prototyping

En esta sección se presenta el **prototipo interactivo de alta fidelidad** de la aplicación web RentiCar, desarrollado y validado mediante la herramienta **Figma** y posteriormente implementado sobre el framework **Angular** con arquitectura basada en componentes reutilizables y Angular Material.

El diseño del prototipo se fundamenta en principios de diseño centrado en el usuario (UCD), garantizando una curva de aprendizaje mínima tanto para personas que alquilan por primera vez como para propietarios que administran múltiples vehículos.

Principales aspectos funcionales validados en el prototipo:

1. **Flujo de Navegación Responsivo:** Adaptabilidad fluida desde resoluciones de escritorio (1920x1080) hasta dispositivos móviles (375x812), con navegación mediante barra lateral colapsable y barra superior contextual.
2. **Interactividad en Formularios:** Validaciones reactivas inmediatas de contraseñas seguras, correos con formato correcto y selectores de fecha con restricciones lógicas (la fecha de devolución no puede ser anterior a la de recogida).
3. **Visualización de Telemetría IoT:** Integración interactiva con mapas vectoriales vía **Leaflet**, simulando la posición GPS en vivo del vehículo alquilado y tableros de métricas de odómetro y velocidad.
4. **Sistema de Feedback y Notificaciones:** Notificaciones tipo snackbar/toast para confirmar operaciones críticas (reserva enviada, auto publicado, cambios guardados en el perfil).

El prototipo funcional navegable y la demostración de los flujos de usuario se encuentran documentados y disponibles en el siguiente recurso audiovisual:

Demostración en Video del Prototipo Web:
Ver demostración en video del prototipo interactivo en Microsoft Stream

---

4.8. Domain-Driven Software Architecture

A continuación, se presenta la arquitectura de software del sistema **RentiCar**, estructurada bajo los principios de **Domain-Driven Design (DDD)** y documentada mediante el modelo de arquitectura **C4 (Contexto, Contenedores, Componentes y Código)**. Este modelo permite visualizar la descomposición del sistema en diferentes niveles de abstracción, garantizando un diseño desacoplado, mantenible y escalable.

4.8.1. Software Architecture Context Diagram

El diagrama de contexto define los límites del sistema **RentiCar**, ilustrando su relación con los diferentes perfiles de usuarios y los sistemas externos con los que interactúa dentro de su entorno operativo.

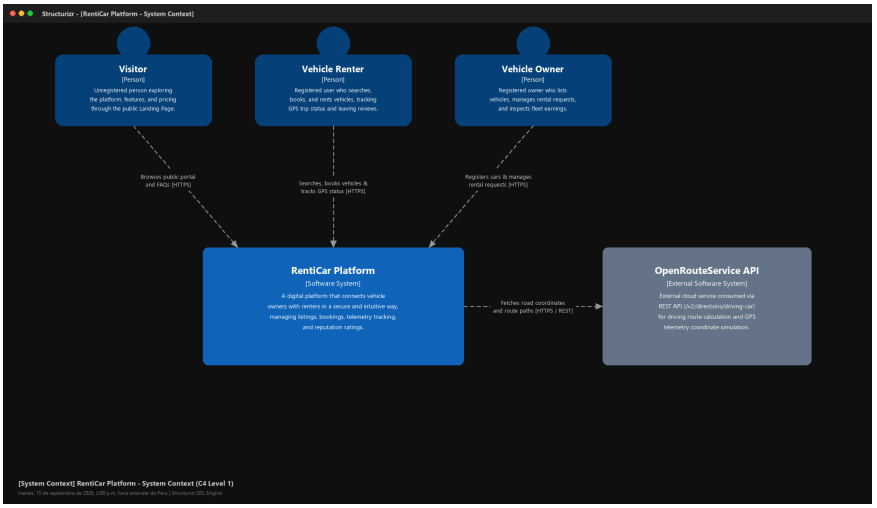


Figura 4.8.1: Diagrama de Contexto del Sistema RentiCar (Nivel 1 C4)

Descripción de Elementos del Diagrama de Contexto:

- **Personas (Usuarios):**
  - **Arrendatario (Customer/Renter):** Usuario que explora el catálogo de autos, aplica filtros, solicita reservas de vehículos por fechas determinadas, monitorea el auto en ruta vía GPS y califica el servicio recibido.
  - **Arrendador (Vehicle Owner/Lessor):** Propietario que registra y publica sus vehículos, establece tarifas de alquiler, gestiona la admisión de solicitudes y supervisa la telemetría y kilometraje de su flota.
- **Sistema de Software Central (RentiCar Platform):** Plataforma web colaborativa P2P que centraliza las operaciones de alquiler, reservas, verificación de usuarios y monitoreo telemático.
- **Sistemas Externos Integrados:**
  - **Servicio de Mapas y Geolocalización (OpenStreetMap / Leaflet):** Provee la infraestructura de renderizado cartográfico para el seguimiento en tiempo real del vehículo.
  - **Dispositivos IoT / Simulador de Telemetría:** Módulos integrados en los vehículos que transmiten coordenadas geográficas, velocidad y nivel de combustible hacia la API mediante protocolos HTTP/REST.
  - **Servicio de Notificaciones y Correo:** Encargado de despachar notificaciones transaccionales y confirmaciones de reserva.

4.8.2. Software Architecture Container Diagrams

El diagrama de contenedores desglosa la solución en sus unidades de ejecución independientes de software, detallando los roles tecnológicos, protocolos de red y bases de datos empleadas.

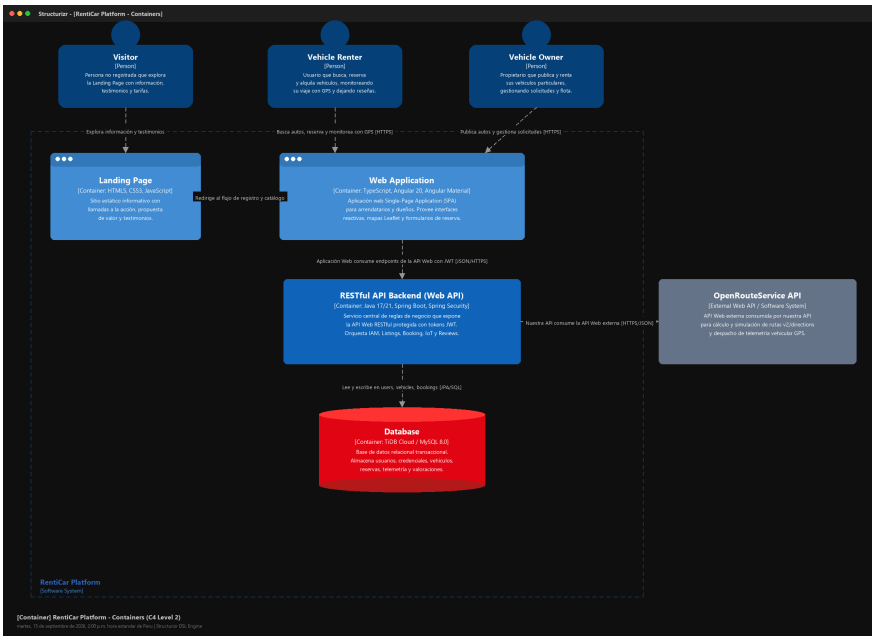


Figura 4.8.2: Diagrama de Contenedores del Sistema RentiCar (Nivel 2 C4)

Descripción de Contenedores de la Solución:

- **Landing Page (Sitio Web Público):**
  Sitio web público estático desarrollado en **HTML5, CSS3 y JavaScript moderno**, optimizado para conversión, presentación de propuesta de valor, opiniones y llamado a la acción (CTAs) para visitantes interesados.
- **Web Application (Single-Page Application - SPA):**
  Desarrollada en **Angular 20** con arquitectura basada en componentes standalone, TypeScript, Angular Material, Tailwind CSS y **Leaflet** para mapas interactivos. Proporciona la interfaz gráfica reactiva para Arrendatarios y Arrendadores, comunicándose con la API mediante peticiones asíncronas HTTPS / JSON con autenticación JWT.
- **RESTful API Backend (Aplicación de Servicios Central):**
  Construida en **Java 17 / 21** empleando el framework **Spring Boot 3.x**. Estructurada bajo Domain-Driven Design (DDD) y arquitectura hexagonal por Bounded Contexts, expone endpoints REST seguros protegidos con Spring Security y tokens JWT emitidos con HMAC-SHA256.
- **Database (Base de Datos Relacional Transaccional):**
  Implementada sobre **TiDB Cloud (compatible con MySQL 8.0)** accesible mediante Spring Data JPA e Hibernate por el puerto 4000. Aloja las entidades de negocio (usuarios, credenciales, vehículos, contratos de reserva, telemetría y valoraciones) garantizando transaccionalidad ACID e integridad referencial estricta.
- **OpenRouteService API (External Software System):**
  Servicio externo en la nube consumido por el backend a través de su API REST (`/v2/directions/driving-car`) para el cálculo de matrices de ruta y simulación de coordenadas GPS telemáticas en tiempo real.
- **Consumo de la API por la Aplicación Web:**
  La **Web Application** es el contenedor cliente que consume de forma directa y reactiva los endpoints de la **RESTful API Backend** mediante peticiones HTTP asíncronas sobre TLS/HTTPS serializadas en JSON y autenticadas con Bearer JWT, soportando todos los casos de uso de arrendatarios y propietarios.

4.8.3. Software Architecture Components Diagrams

En esta sección se presentan los **diagramas de componentes (Nivel 3 del modelo C4)** descompuestos de manera individual para cada uno de los **Bounded Contexts** que conforman el backend de **RentiCar** desarrollado en **Spring Boot**. Cada diagrama modela la interacción entre los controladores REST, los servicios de aplicación (Command y Query Handlers), el modelo de dominio y los adaptadores de infraestructura y persistencia.

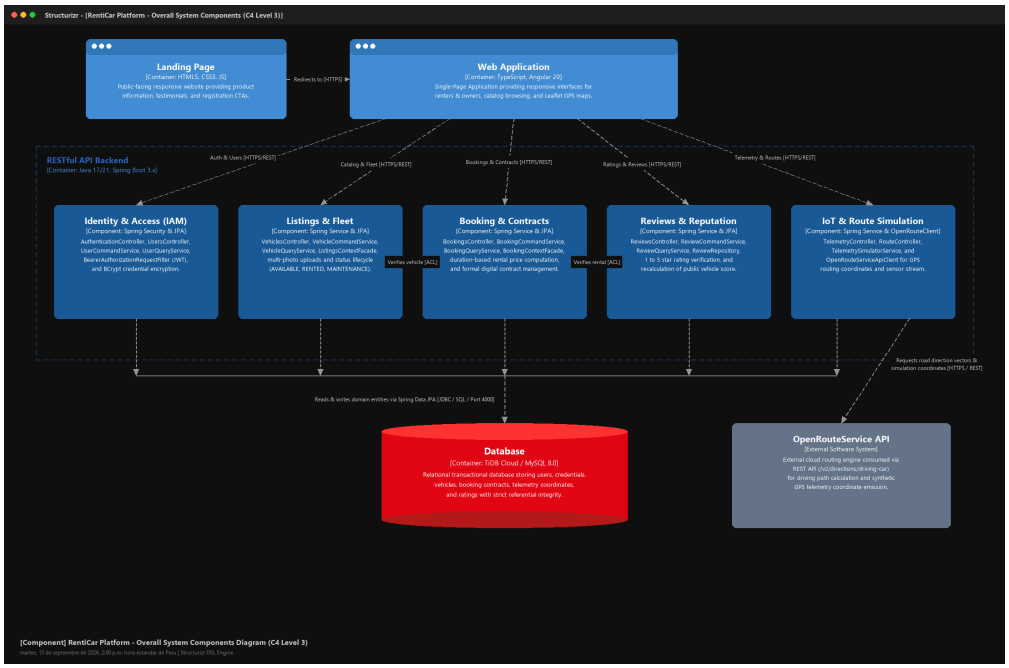


Figura 4.8.3: Diagrama General de Componentes de la Plataforma RentiCar

---

4.8.3.1. Components Diagram: Identity & Access Management (IAM) Bounded Context

Gestiona el registro de usuarios, autenticación mediante tokens JWT, encriptación de credenciales con BCrypt y administración de perfiles según roles (`ROLE_RENTER` y `ROLE_OWNER`).

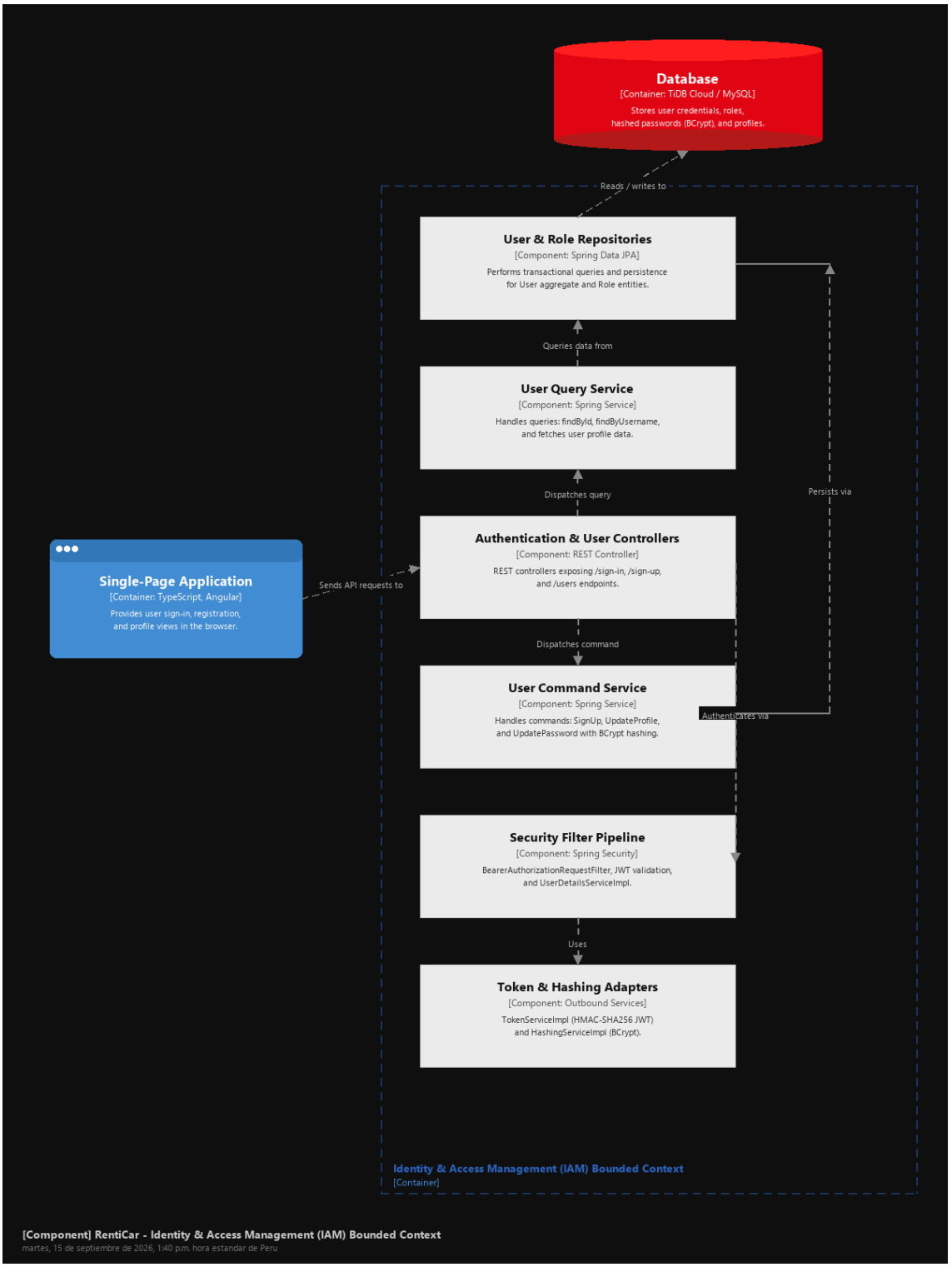


Figura 4.8.3.1: Diagrama de Componentes C4 (Nivel 3) - IAM Bounded Context (Structurizr Style)

Catálogo de Componentes del IAM Bounded Context:

Componente	Capa	Responsabilidad
`AuthenticationController`	Interfaces	Expone los endpoints públicos de registro (`/sign-up`) e inicio de sesión (`/sign-in`).
`UsersController`	Interfaces	Permite consultar información de usuarios y actualizar contraseñas o datos de perfil.
`BearerAuthorizationRequestFilter`	Seguridad	Intercepta peticiones HTTP para extraer y validar tokens Bearer JWT en las cabeceras.
`UserCommandServiceImpl`	Aplicación	Orquesta los casos de uso transaccionales de alta, modificación de perfil y reseteo de claves.
`UserQueryServiceImpl`	Aplicación	Ejecuta consultas de solo lectura por identificador de usuario o dirección de correo.
`UserRepository` / `RoleRepository`	Infraestructura	Interfaces Spring Data JPA para la persistencia transaccional en las tablas `users` y `roles`.

4.8.3.2. Components Diagram: Listings & Fleet Management Bounded Context

Controla la publicación de vehículos por parte de los propietarios, especificaciones técnicas (marca, modelo, año, precio diario), carga fotográfica y ciclo de vida de disponibilidad.

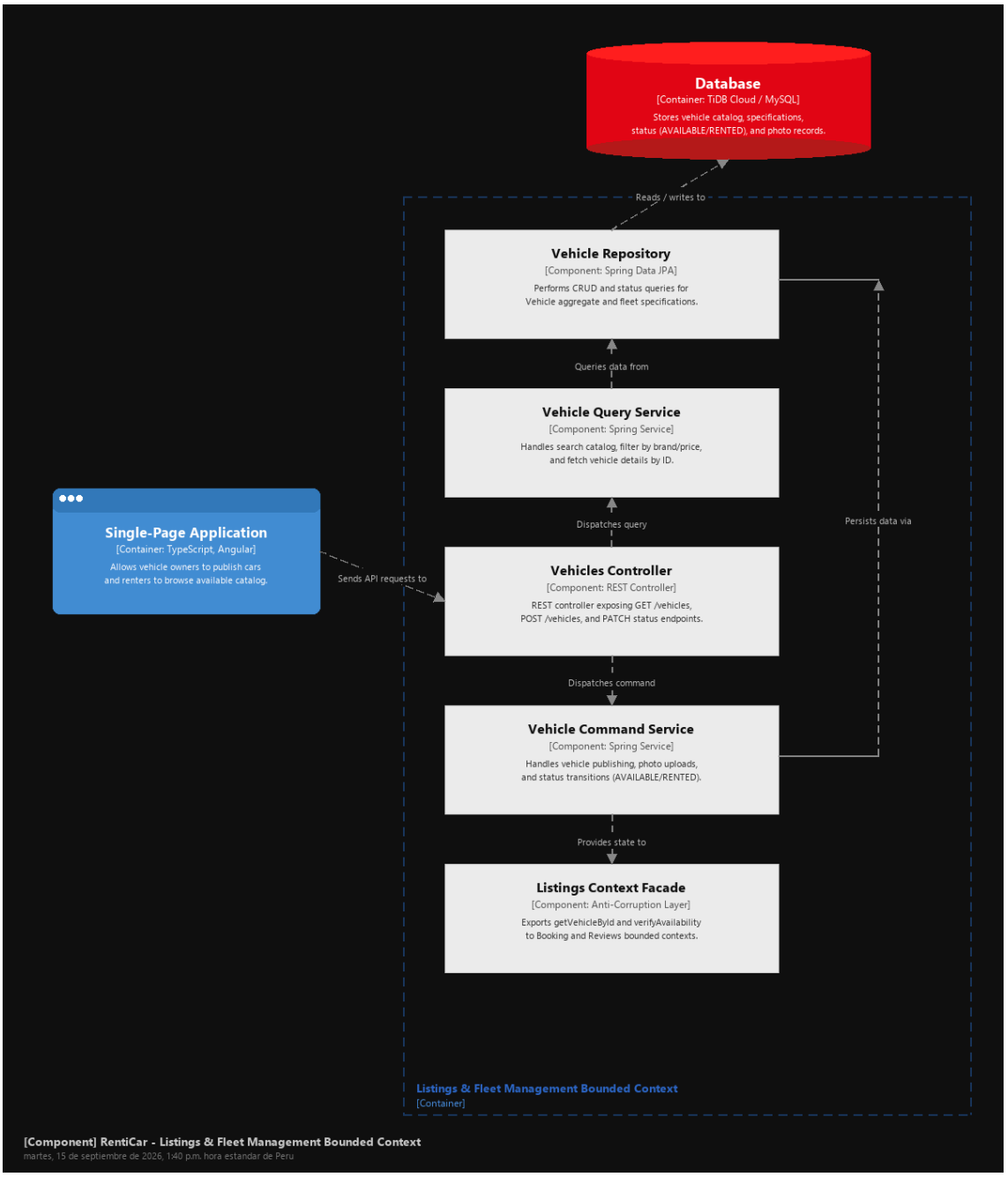


Figura 4.8.3.2: Diagrama de Componentes C4 (Nivel 3) - Listings & Fleet Management Bounded Context (Structurizr Style)

Catálogo de Componentes del Listings Bounded Context:

Componente	Capa	Responsabilidad
`VehiclesController`	Interfaces	Maneja endpoints para listar el catálogo público, registrar vehículos, subir imágenes y cambiar estado.
`VehicleCommandServiceImpl`	Aplicación	Valida reglas de negocio para crear, actualizar especificaciones, almacenar fotos binarias y dar de baja publicaciones.
`VehicleQueryServiceImpl`	Aplicación	Recupera listados de vehículos aplicando filtros por estado operativo (`AVAILABLE`) y por identificador de propietario.
`ListingsContextFacade`	ACL	Expone métodos de consulta de disponibilidad y pertenencia vehicular hacia los contextos de Booking e IoT.
`VehicleRepository`	Infraestructura	Gestiona las operaciones CRUD y consultas derivadas sobre la tabla `vehicles`.

4.8.3.3. Components Diagram: Booking & Digital Contract Bounded Context

Administra las solicitudes de alquiler formuladas por los arrendatarios, el cálculo automático de tarifas por días de contrato, la confirmación o rechazo por el arrendador y las cancelaciones.

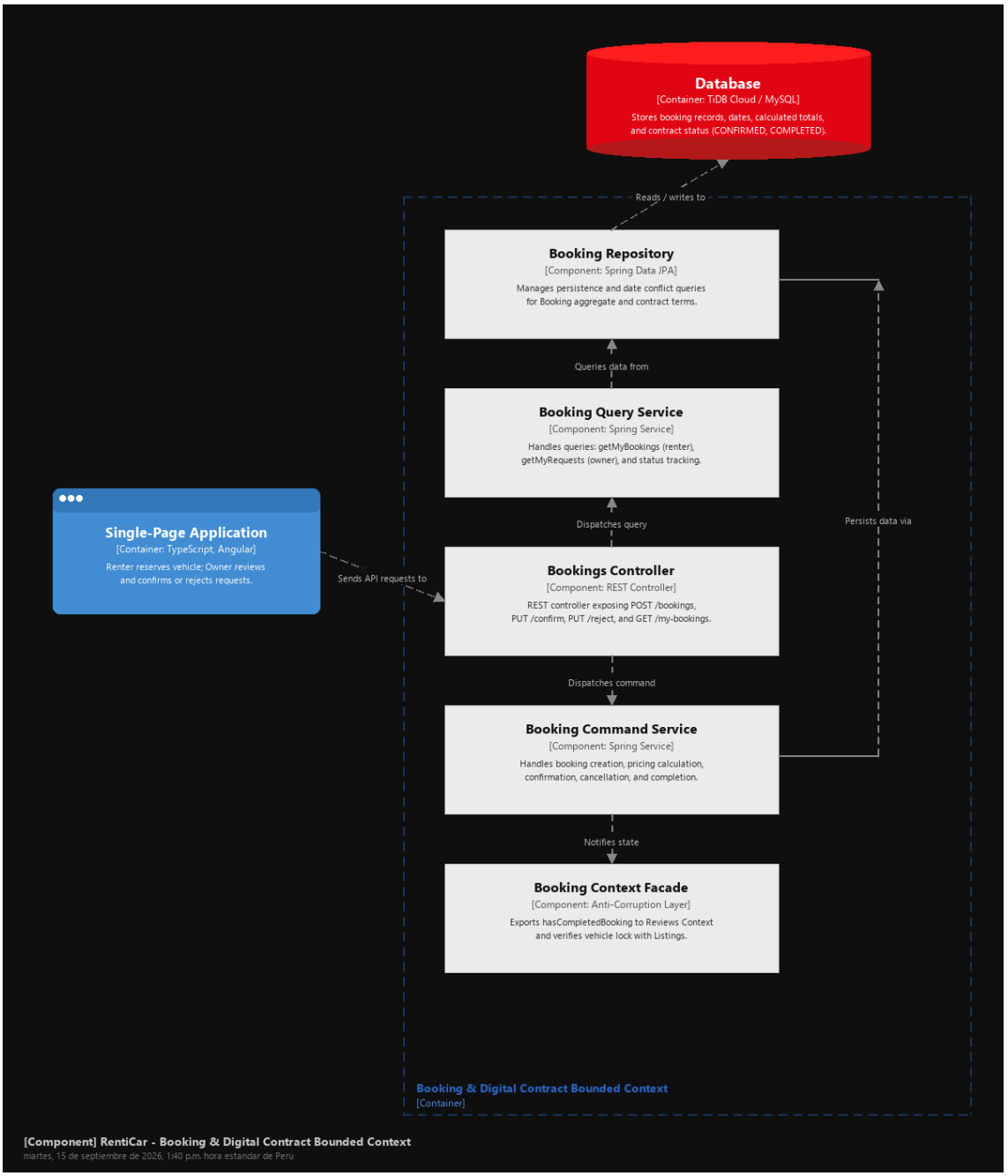


Figura 4.8.3.3: Diagrama de Componentes C4 (Nivel 3) - Booking & Digital Contract Bounded Context (Structurizr Style)

Catálogo de Componentes del Booking Bounded Context:

Componente	Capa	Responsabilidad
`BookingsController`	Interfaces	Expone los endpoints de creación de reserva, confirmación, rechazo, cancelación y consulta de bandejas.
`BookingCommandServiceImpl`	Aplicación	Ejecuta transiciones de estado contractuales, calcula el precio total por días y evita reservas solapadas.
`BookingQueryServiceImpl`	Aplicación	Obtiene el historial de reservas para el arrendatario y la bandeja de solicitudes entrantes para el arrendador.
`BookingContextFacade`	ACL	Provee a los contextos de Reviews e IoT la verificación de si un alquiler fue completado satisfactoriamente.
`BookingRepository`	Infraestructura	Repositorio Spring Data JPA con consultas JPQL para filtrado por fechas, usuario y vehículo.

4.8.3.4. Components Diagram: IoT Monitoring & Route Simulation Bounded Context

Procesa las señales telemáticas transmitidas por sensores vehiculares (coordenadas GPS, odómetro, velocidad y nivel de combustible) y la simulación interactiva de trayectorias mediante OpenRouteService.

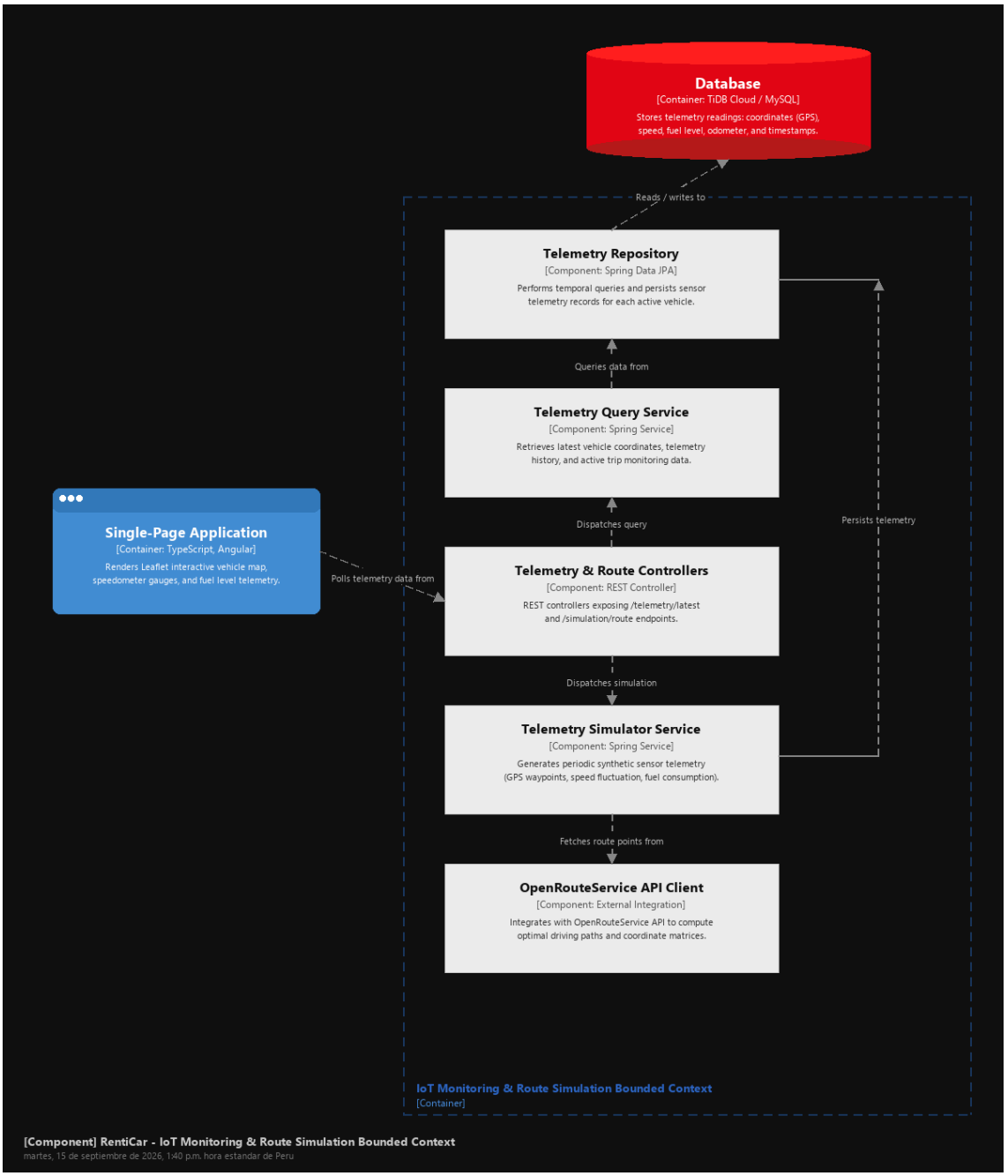


Figura 4.8.3.4: Diagrama de Componentes C4 (Nivel 3) - IoT Monitoring & Route Simulation Bounded Context (Structurizr Style)

Catálogo de Componentes del IoT Bounded Context:

Componente	Capa	Responsabilidad
`TelemetryController`	Interfaces	Provee endpoints para registrar telemetría y consultar la última posición telemática de un auto.
`RouteController`	Interfaces	Expone los endpoints de simulación de rutas dinámicas consumidos por el componente Leaflet en frontend.
`OpenRouteServiceApiClient`	Infraestructura	Cliente HTTP REST que consulta la API de OpenRouteService para obtener coordenadas secuenciales de rutas.
`TelemetrySimulatorService`	Aplicación	Simula el desplazamiento físico de vehículos emitiendo coordenadas periódicas hacia la persistencia.
`TelemetryRepository`	Infraestructura	Repositorio JPA para almacenar y consultar lecturas de GPS, velocidad y combustible por vehículo.

4.8.3.5. Components Diagram: Reviews & Reputation Bounded Context

Gestiona las evaluaciones y comentarios redactados por clientes una vez completado su alquiler, recalculando dinámicamente el promedio de satisfacción pública del vehículo.

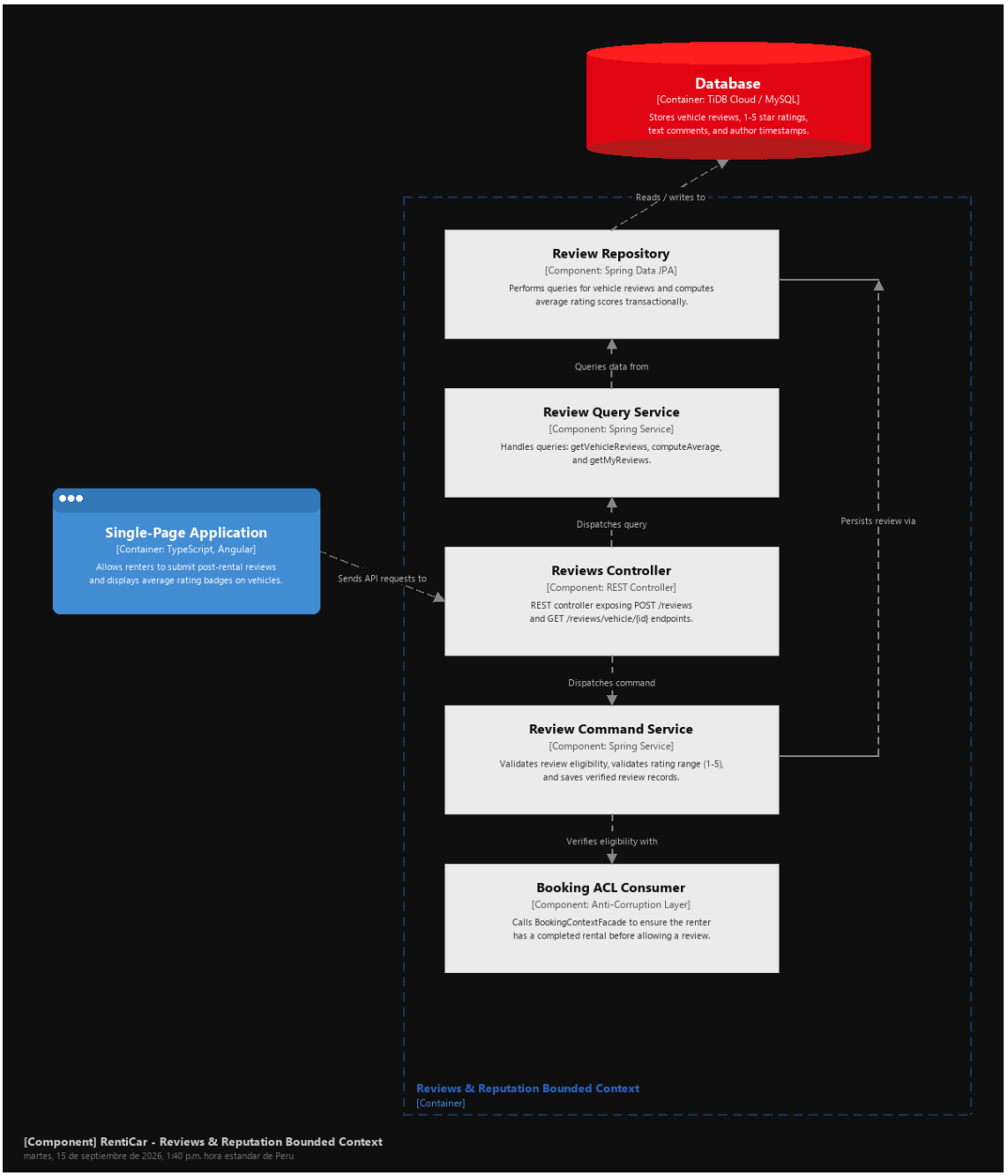


Figura 4.8.3.5: Diagrama de Componentes C4 (Nivel 3) - Reviews & Reputation Bounded Context (Structurizr Style)

Catálogo de Componentes del Reviews Bounded Context:

Componente	Capa	Responsabilidad
`ReviewsController`	Interfaces	Expone los endpoints para registrar una nueva reseña y consultar opiniones por vehículo o usuario.
`ReviewCommandServiceImpl`	Aplicación	Valida que el usuario posea un alquiler completado (vía ACL) y que no haya calificado previamente el servicio.
`ReviewQueryServiceImpl`	Aplicación	Consulta el listado histórico de calificaciones y calcula el promedio ponderado de satisfacción.
`ReviewRepository`	Infraestructura	Repositorio Spring Data JPA para la persistencia y lectura de opiniones en la tabla `reviews`.

4.9. Software Object-Oriented Design

El diseño orientado a objetos de **RentiCar** unifica todos los módulos y Bounded Contexts de la plataforma bajo los principios tácticos de **Domain-Driven Design (DDD)** y arquitectura hexagonal, garantizando alta cohesión, bajo acoplamiento e integridad referencial y de dominio.

4.9.1. Class Diagrams

A continuación, se presenta el **Diagrama Completo y Unificado de Clases del Sistema RentiCar (UML 2.5)**, modelado a nivel profesional y alineado a la arquitectura backend real con separación estricta **CQRS (Command Query Responsibility Segregation)** y límites transaccionales definidos por Bounded Contexts:

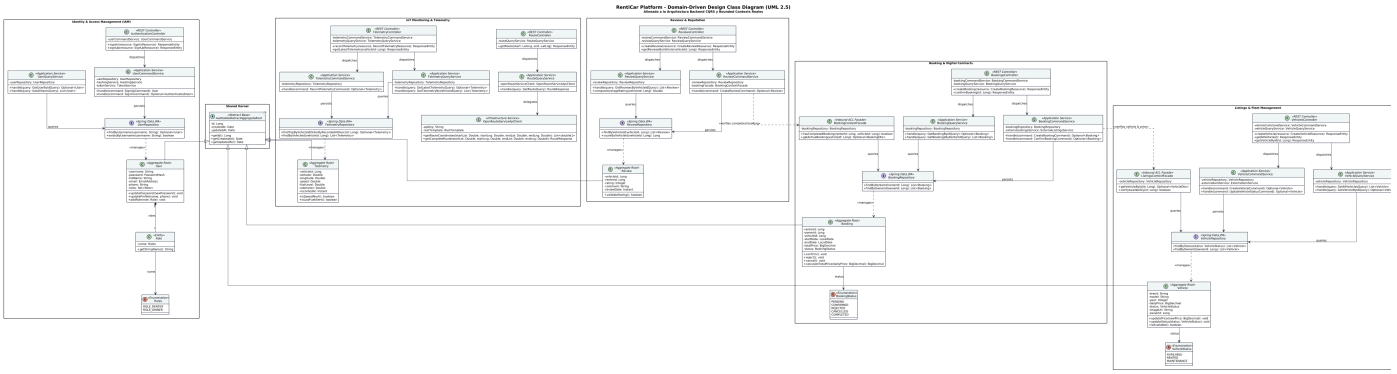


Figura 4.9.1: Diagrama Maestro de Clases del Dominio de RentiCar (UML 2.5 - All Bounded Contexts)
Ver diagrama de clases en resolución completa (4096px)

Criterio y Principios Arquitectónicos Clave del Diagrama

En el backend (**Java / Spring Boot 3.x**) se aplica una arquitectura moderna basada en **Domain-Driven Design (DDD)** y **CQRS (Command Query Responsibility Segregation)**:

1. **Separación de Responsabilidad entre Comandos y Consultas (CQRS):**
   - Las operaciones de **escritura/mutación** (crear, actualizar, eliminar, confirmar) son atendidas exclusivamente por los servicios `*CommandService`.
   - Las operaciones de **lectura** (listar, buscar por ID, filtrar) son atendidas por los servicios `*QueryService`.
   - Los controladores REST inyectan ambos servicios según los endpoints expuestos (ej. `VehiclesController` despacha a `VehicleCommandService` y a `VehicleQueryService`).
2. **Límites de los Bounded Contexts y Referencias por ID:**
   - En DDD, los agregados de diferentes contextos no mantienen referencias directas por punteros de memoria; se desacoplan utilizando **identificadores escalares** (`ownerId: Long`, `vehicleId: Long`, `renterId: Long`).
   - Se eliminaron las asociaciones directas cruzadas entre agregados para preservar la autonomía transaccional y evitar dependencias cíclicas.
3. **Capa Anticorrupción (ACL - Anti-Corruption Layer) mediante Facades:**
   - La integración inter-contexto se realiza formalmente a través de interfaces de integración: **Inbound ACL Facades** (`ListingsContextFacade`, `BookingContextFacade`).
   - Por ejemplo, `BookingCommandService` consulta la disponibilidad y tarifa del vehículo a través de `ListingsContextFacade`, y `ReviewCommandService` verifica la culminación del alquiler mediante `BookingContextFacade`.
4. **Repositorios Vinculados a sus Aggregate Roots:**
   - Todos los repositorios Spring Data JPA (`UserRepository`, `VehicleRepository`, `BookingRepository`, `TelemetryRepository`, `ReviewRepository`) expresan formalmente su dependencia de administración: `Repository ..> AggregateRoot : <<manages>>`.
5. **Shared Kernel y Capa Base de Auditoría:**
   - Todos los Aggregate Roots (`User`, `Vehicle`, `Booking`, `Telemetry`, `Review`) heredan de la clase base abstracta `AuditableAbstractAggregateRoot<T>`, asegurando los campos automáticos `id: Long`, `createdAt: Date` y `updatedAt: Date`.

Guía de Notación UML Utilizada (Para Sustentación)

Símbolo UML	Tipo de Relación	Justificación Técnica en el Proyecto
`AuditBase <`	`-- Vehicle`	**Herencia / Generalización**
`User "1" *-- "*" Role`	**Composición**	Una entidad `Role` existe como parte del ciclo de vida del usuario dentro de IAM.
`Controller --> Service`	**Asociación Dirigida**	Inyección de dependencias para el despacho de comandos y consultas.
`Service --> Repository`	**Asociación Dirigida**	El servicio de aplicación utiliza el repositorio JPA para persistencia y consultas.
`Repository ..> Entity`	**Dependencia (`<<manages>>`)**	El repositorio administra el ciclo de vida de la entidad correspondiente al Aggregate Root.
`CommandService ..> Facade`	**Dependencia de Integración (ACL)**	Comunicación desacoplada entre Bounded Contexts a través de contratos de interfaz.
`Service --> ExternalClient`	**Delegación de Infraestructura**	Conexión directa hacia el cliente HTTP de integración con servicios web externos (OpenRouteService).

Flujo de Datos para una Operación Típica (Ejemplo: Booking)

1. El cliente envía un JSON al endpoint `POST /api/v1/bookings`.
2. `BookingsController` transforma el recurso en `CreateBookingCommand` y lo despacha a `BookingCommandService`.

3. `BookingCommandService` invoca `ListingsContextFacade.verifyAvailability(vehicleId)`.
4. Si el vehículo está disponible, `BookingCommandService` instancia la entidad de dominio `Booking`, calcula el precio total y la envía a `BookingRepository.save()`.
5. La reserva queda registrada en estado transaccional `PENDING` o `CONFIRMED`.

4.9.2. Class Dictionary

El siguiente diccionario de clases consolida la totalidad de entidades, agregados, objetos de valor, repositorios, servicios de aplicación y controladores del sistema **RentiCar**, detallando su visibilidad, atributos, métodos, tipos de datos y descripciones operativas en estricto apego al estándar UML 2.5:

Bounded Context	Clase / Componente	Atributo / Método	Descripción
Shared Kernel	`AuditableAbstractAggregateRoot<T>`	`-id`	Identificador único autoincremental de la entidad
Shared Kernel	`AuditableAbstractAggregateRoot<T>`	`-createdAt`	Marca temporal de creación del registro
Shared Kernel	`AuditableAbstractAggregateRoot<T>`	`-updatedAt`	Marca temporal de última modificación
Shared Kernel	`AuditableAbstractAggregateRoot<T>`	`+getId()`	Retorna la clave primaria del agregado
Shared Kernel	`AuditableAbstractAggregateRoot<T>`	`+getCreatedAt()`	Retorna fecha de auditoría de creación
Shared Kernel	`AuditableAbstractAggregateRoot<T>`	`+getUpdatedAt()`	Retorna fecha de auditoría de actualización
IAM	`User`	`-username`	Nombre de usuario o credencial única de acceso
IAM	`User`	`-password`	Contraseña cifrada con algoritmo BCrypt
IAM	`User`	`-fullName`	Nombres y apellidos completos del usuario
IAM	`User`	`-email`	Dirección de correo electrónico verificada
IAM	`User`	`-phone`	Número de contacto telefónico
IAM	`User`	`-roles`	Colección de roles de seguridad asociados al usuario
IAM	`User`	`+updatePassword()`	Modifica la clave cifrada de acceso
IAM	`User`	`+updateProfile()`	Actualiza nombre y número telefónico del perfil
IAM	`User`	`+addRole()`	Asigna un nuevo rol de acceso (`ROLE_RENTER` / `ROLE_OWNER`)
IAM	`Role`	`-name`	Nombre del rol en el sistema
IAM	`Role`	`+getStringName()`	Retorna la representación en texto del rol para Spring Security
IAM	`Roles`	`ROLE_RENTER`	Valor constante: Perfil de cliente que busca alquilar vehículos
IAM	`Roles`	`ROLE_OWNER`	Valor constante: Perfil de propietario que publica y renta autos
IAM	`UserRepository`	`+findByUsername()`	Consulta un usuario por su nombre de cuenta
IAM	`UserRepository`	`+existsByUsername()`	Verifica si ya existe una cuenta con el nombre indicado
IAM	`UserCommandService`	`+handle(SignUpCommand)`	Procesa el registro de una nueva cuenta y encripta contraseña
IAM	`UserCommandService`	`+handle(SignInCommand)`	Valida credenciales y genera sesión autenticada
IAM	`UserQueryService`	`+handle(GetUserByIdQuery)`	Recupera el perfil de un usuario a partir de su ID
IAM	`UserQueryService`	`+handle(GetAllUsersQuery)`	Lista la totalidad de usuarios registrados
IAM	`AuthenticationController`	`+signIn()`	Endpoint REST `POST /api/v1/authentication/sign-in`
IAM	`AuthenticationController`	`+signUp()`	Endpoint REST `POST /api/v1/authentication/sign-up`
Listings	`Vehicle`	`-brand`	Marca del fabricante automotriz
Listings	`Vehicle`	`-model`	Modelo de la unidad vehicular

Bounded Context	Clase / Componente	Atributo / Método	Descripción
Listings	`Vehicle`	`-year`	Año de fabricación del automóvil
Listings	`Vehicle`	`-dailyPrice`	Tarifa diaria de alquiler en PEN
Listings	`Vehicle`	`-status`	Estado de disponibilidad operativa de la unidad
Listings	`Vehicle`	`-imageUrl`	Enlace o identificador de la fotografía vehicular
Listings	`Vehicle`	`-ownerId`	Clave foránea escalar del propietario arrendador
Listings	`Vehicle`	`+updatePrice()`	Modifica la tarifa diaria validando importe mayor a cero
Listings	`Vehicle`	`+updateStatus()`	Modifica el estado del vehículo en el catálogo
Listings	`Vehicle`	`+isAvailable()`	Retorna verdadero si el automóvil está libre para reserva
Listings	`VehicleStatus`	`AVAILABLE, RENTED, MAINTENANCE`	Estados operativos admitidos para un vehículo en la flota
Listings	`VehicleRepository`	`+findByStatus()`	Consulta vehículos filtrados por su estado operativo
Listings	`VehicleRepository`	`+findByOwnerId()`	Consulta los vehículos pertenecientes a un propietario específico
Listings	`VehicleCommandService`	`+handle(CreateVehicleCommand)`	Registra una nueva publicación de vehículo en el catálogo
Listings	`VehicleCommandService`	`+handle(UpdateVehicleStatusCommand)`	Actualiza el estado de disponibilidad del vehículo
Listings	`VehicleQueryService`	`+handle(GetAllVehiclesQuery)`	Retorna el catálogo completo de vehículos disponibles
Listings	`VehicleQueryService`	`+handle(GetVehicleByIdQuery)`	Recupera la ficha técnica detallada de un vehículo por su ID
Listings	`ListingsContextFacade`	`+getVehicleById()`	Fachada ACL para exponer datos de vehículo a otros contextos
Listings	`ListingsContextFacade`	`+verifyAvailability()`	Fachada ACL para validar si el auto puede ser reservado
Listings	`VehiclesController`	`+createVehicle()`	Endpoint REST `POST /api/v1/vehicles`
Listings	`VehiclesController`	`+getAllVehicles()`	Endpoint REST `GET /api/v1/vehicles`
Listings	`VehiclesController`	`+getVehicleById()`	Endpoint REST `GET /api/v1/vehicles/{vehicleId}`
Booking	`Booking`	`-renterId`	Identificador escalar del cliente solicitante
Booking	`Booking`	`-ownerId`	Identificador escalar del dueño del vehículo
Booking	`Booking`	`-vehicleId`	Identificador escalar del automóvil contratado
Booking	`Booking`	`-startDate`	Fecha de inicio del periodo de alquiler
Booking	`Booking`	`-endDate`	Fecha de culminación del alquiler
Booking	`Booking`	`-totalPrice`	Monto total liquidado por la reserva
Booking	`Booking`	`-status`	Estado del ciclo de vida de la reserva
Booking	`Booking`	`+confirm()`	Formaliza la reserva transicionándola a `CONFIRMED`
Booking	`Booking`	`+reject()`	Declina la solicitud transicionándola a `REJECTED`
Booking	`Booking`	`+cancel()`	Desiste del alquiler transicionándolo a `CANCELLED`
Booking	`Booking`	`+calculateTotalPrice()`	Multiplica la tarifa por día por la cantidad de días
Booking	`BookingStatus`	`PENDING, CONFIRMED, REJECTED, CANCELLED, COMPLETED`	Estados transaccionales de la reserva digital
Booking	`BookingRepository`	`+findByRenterId()`	Consulta reservas efectuadas por un arrendatario
Booking	`BookingRepository`	`+findByOwnerId()`	Consulta reservas recibidas por un propietario
Booking	`BookingCommandService`	`+handle(CreateBookingCommand)`	Orquesta la creación de reserva validando con ListingsFacade
Booking	`BookingCommandService`	`+handle(ConfirmBookingCommand)`	Aprueba y confirma la reserva formalizando el contrato
Booking	`BookingQueryService`	`+handle(GetBookingByIdQuery)`	Consulta los datos y estado de una reserva específica

Bounded Context	Clase / Componente	Atributo / Método	Descripción
Booking	`BookingQueryService`	`+handle(GetBookingsByRenterIdQuery)`	Retorna el historial de reservas de un cliente
Booking	`BookingContextFacade`	`+hasCompletedBooking()`	Fachada ACL: valida si un usuario completó un alquiler previo
Booking	`BookingContextFacade`	`+getActiveBooking()`	Fachada ACL: consulta la reserva en curso de un auto
Booking	`BookingsController`	`+createBooking()`	Endpoint REST `POST /api/v1/bookings`
Booking	`BookingsController`	`+confirmBooking()`	Endpoint REST `PUT /api/v1/bookings/{bookingId}/confirm`
IoT	`Telemetry`	`-vehicleId`	Identificador escalar del vehículo monitoreado
IoT	`Telemetry`	`-latitude`	Coordenada geográfica de latitud GPS
IoT	`Telemetry`	`-longitude`	Coordenada geográfica de longitud GPS
IoT	`Telemetry`	`-speed`	Velocidad instantánea en km/h
IoT	`Telemetry`	`-fuelLevel`	Porcentaje de combustible restante (0 a 100%)
IoT	`Telemetry`	`-odometer`	Kilometraje total acumulado del automóvil
IoT	`Telemetry`	`-recordedAt`	Marca temporal exacta de la captura telemática
IoT	`Telemetry`	`+isSpeedAlert()`	Evalúa si la velocidad supera el umbral seguro (>100 km/h)
IoT	`Telemetry`	`+isLowFuelAlert()`	Evalúa si el nivel de combustible es crítico (<15%)
IoT	`TelemetryRepository`	`+findTopByVehicleIdOrderByRecordedAtDesc()`	Obtiene la última lectura telemática registrada de un auto
IoT	`TelemetryRepository`	`+findByVehicleId()`	Obtiene la serie histórica de lecturas telemáticas
IoT	`TelemetryCommandService`	`+handle(RecordTelemetryCommand)`	Ingesta y persiste una nueva medición de sensores IoT
IoT	`TelemetryQueryService`	`+handle(GetLatestTelemetryQuery)`	Retorna el estado telemático más reciente de un auto
IoT	`TelemetryQueryService`	`+handle(GetTelemetryByVehicleIdQuery)`	Retorna el historial de telemetría de un vehículo
IoT	`RouteQueryService`	`+handle(GetRouteQuery)`	Consulta la geometría de ruta a OpenRouteServiceApiClient
IoT	`OpenRouteServiceApiClient`	`-apiKey`	Clave de autenticación para consumir la API de OpenRouteService
IoT	`OpenRouteServiceApiClient`	`-restTemplate`	Cliente HTTP de Spring para llamadas REST externas
IoT	`OpenRouteServiceApiClient`	`+getRouteCoordinates()`	Resuelve la polilínea vial de coordenadas entre dos puntos geográficos
IoT	`OpenRouteServiceApiClient`	`+getCompleteRoute()`	Resuelve la ruta completa con distancia métrica y duración estimada
IoT	`TelemetryController`	`+recordTelemetry()`	Endpoint REST `POST /api/v1/telemetry`
IoT	`TelemetryController`	`+getLatestTelemetry()`	Endpoint REST `GET /api/v1/telemetry/vehicle/{vehicleId}/latest`
IoT	`RouteController`	`+getRoute()`	Endpoint REST `GET /api/v1/simulation/route`
Reviews	`Review`	`-vehicleId`	Identificador escalar del automóvil evaluado
Reviews	`Review`	`-renterId`	Identificador escalar del arrendatario que emite la reseña
Reviews	`Review`	`-rating`	Calificación cuantitativa en estrellas (1 a 5)
Reviews	`Review`	`-comment`	Comentario cualitativo sobre la experiencia de uso
Reviews	`Review`	`-reviewDate`	Fecha y hora de emisión de la reseña
Reviews	`Review`	`+validateRating()`	Valida que la puntuación se encuentre en el rango permitido
Reviews	`ReviewRepository`	`+findByVehicleId()`	Retorna todas las calificaciones y opiniones de un vehículo
Reviews	`ReviewRepository`	`+countByVehicleId()`	Cuenta el número total de reseñas emitidas para un auto

Bounded Context	Clase / Componente	Atributo / Método	Descripción
Reviews	`ReviewCommandService`	`+handle(CreateReviewCommand)`	Registra reseña verificando culminación con BookingFacade
Reviews	`ReviewQueryService`	`+handle(GetReviewsByVehicleIdQuery)`	Consulta el listado de reseñas de un vehículo
Reviews	`ReviewQueryService`	`+computeAverageRating()`	Calcula el promedio de estrellas de un auto en el catálogo
Reviews	`ReviewsController`	`+createReview()`	Endpoint REST `POST /api/v1/reviews`
Reviews	`ReviewsController`	`+getReviewsByVehicle()`	Endpoint REST `GET /api/v1/reviews/vehicle/{vehicleId}`

4.10. Database Design

El diseño de la base de datos relacional de **RentiCar** ha sido modelado para garantizar la consistencia transaccional, normalización en Tercera Forma Normal (3FN) e integridad referencial entre todas las entidades del negocio, alineándose directamente con las entidades y agregados definidos en el diseño orientado a objetos del dominio.

4.10.1. Relational Database Diagram

A continuación se presenta el esquema relacional global del sistema **RentiCar**, detallando las tablas principales, claves primarias autoincrementales, claves foráneas, restricciones de unicidad y cardinalidades:

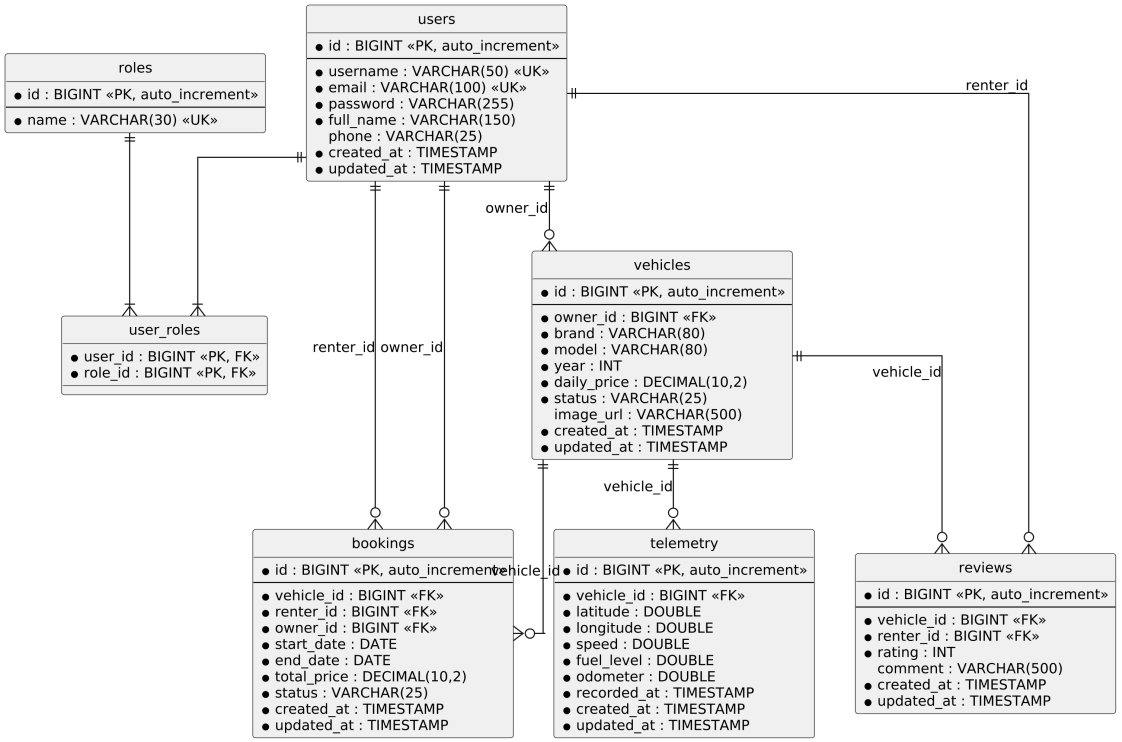


Figura 4.10.1: Diagrama Entidad-Relación Relacional de RentiCar

Ver diagrama relacional de base de datos en alta resolución

4.10.2. Database Data Dictionary (Diccionario de Datos)

El siguiente diccionario de datos describe detalladamente las tablas, atributos, tipos de datos, restricciones y relaciones gestionadas por **JPA / Hibernate** en la base de datos **TiDB Cloud (MySQL 8.0)** de RentiCar:

Tabla: `users`

Almacena las cuentas y credenciales de acceso de los usuarios de la plataforma (arrendadores y arrendatarios).

Columna	Tipo SQL	Longitud	PK	FK	Not Null	Descripción
`id`	`BIGINT`	20	Sí	No	Sí	Identificador autoincremental único del usuario.
`username`	`VARCHAR`	50	No	No	Sí	Nombre de usuario o credencial única de acceso (Unique Key).
`email`	`VARCHAR`	100	No	No	Sí	Correo electrónico verificado del usuario (Unique Key).
`password`	`VARCHAR`	255	No	No	Sí	Hash BCrypt seguro de la contraseña para autenticación.

Columna	Tipo SQL	Longitud	PK	FK	Not Null	Descripción
full_name	VARCHAR	150	No	No	Sí	Nombres y apellidos completos del titular de la cuenta.
phone	VARCHAR	25	No	No	No	Número telefónico de contacto del usuario.
created_at	TIMESTAMP	-	No	No	Sí	Estampa de auditoría de creación del registro.
updated_at	TIMESTAMP	-	No	No	Sí	Estampa de auditoría de última actualización.

Tabla: `roles`

Catálogo maestro de roles del sistema para el control de autorización basado en roles (RBAC).

Columna	Tipo SQL	Longitud	PK	FK	Not Null	Descripción
id	BIGINT	20	Sí	No	Sí	Clave primaria autoincremental del rol.
name	VARCHAR	30	No	No	Sí	Nombre representativo del rol (ROLE_RENTER, ROLE_OWNER) (Unique Key).

Tabla: `user_roles`

Tabla intermedia que resuelve la relación N:M entre `users` y `roles`.

Columna	Tipo SQL	Longitud	PK	FK	Not Null	Descripción
user_id	BIGINT	20	Sí	Sí	Sí	Clave foránea que referencia a users.id.
role_id	BIGINT	20	Sí	Sí	Sí	Clave foránea que referencia a roles.id.

Tabla: `vehicles`

Contiene el catálogo de vehículos publicados por los propietarios (arrendadores) en la plataforma.

Columna	Tipo SQL	Longitud	PK	FK	Not Null	Descripción
id	BIGINT	20	Sí	No	Sí	Clave primaria autoincremental del vehículo.
owner_id	BIGINT	20	No	Sí	Sí	Clave foránea del propietario arrendador (users.id).
brand	VARCHAR	80	No	No	Sí	Marca o fabricante de la unidad vehicular.
model	VARCHAR	80	No	No	Sí	Modelo específico del automóvil.
year	INT	4	No	No	Sí	Año de fabricación del vehículo.
daily_price	DECIMAL(10,2)	10,2	No	No	Sí	Tarifa de alquiler diaria en nuevos soles (PEN).
status	VARCHAR	25	No	No	Sí	Estado operativo de la unidad (AVAILABLE, RENTED, MAINTENANCE).
image_url	VARCHAR	500	No	No	No	URL pública de la fotografía del vehículo en almacenamiento en la nube.
created_at	TIMESTAMP	-	No	No	Sí	Estampa de auditoría de creación del registro.
updated_at	TIMESTAMP	-	No	No	Sí	Estampa de auditoría de última actualización.

Tabla: `bookings`

Registra las transacciones y contratos de alquiler formalizados entre arrendatarios y vehículos.

Columna	Tipo SQL	Longitud	PK	FK	Not Null	Descripción
id	BIGINT	20	Sí	No	Sí	Clave primaria autoincremental de la reserva.
vehicle_id	BIGINT	20	No	Sí	Sí	Clave foránea que referencia al vehículo reservado (vehicles.id).
renter_id	BIGINT	20	No	Sí	Sí	Clave foránea del arrendatario cliente (users.id).
owner_id	BIGINT	20	No	Sí	Sí	Clave foránea del arrendador propietario (users.id).
start_date	DATE	-	No	No	Sí	Fecha pactada de inicio del servicio de alquiler.
end_date	DATE	-	No	No	Sí	Fecha pactada de culminación del servicio de alquiler.
total_price	DECIMAL(10,2)	10,2	No	No	Sí	Importe económico total liquidado por los días de alquiler.
status	VARCHAR	25	No	No	Sí	Estado del ciclo transaccional (PENDING, CONFIRMED, REJECTED, CANCELLED, COMPLETED).
created_at	TIMESTAMP	-	No	No	Sí	Estampa de auditoría de creación de la reserva.
updated_at	TIMESTAMP	-	No	No	Sí	Estampa de auditoría de confirmación o actualización de estado.

Tabla: `telemetry`

Almacena las capturas continuas de telemetría emitidas por los dispositivos IoT vehiculares.

Columna	Tipo SQL	Longitud	PK	FK	Not Null	Descripción
id	BIGINT	20	Sí	No	Sí	Clave primaria autoincremental de la lectura telemática.
vehicle_id	BIGINT	20	No	Sí	Sí	Clave foránea del vehículo monitoreado (vehicles.id).
latitude	DOUBLE	-	No	No	Sí	Coordenada geográfica de latitud GPS.
longitude	DOUBLE	-	No	No	Sí	Coordenada geográfica de longitud GPS.
speed	DOUBLE	-	No	No	Sí	Velocidad instantánea de desplazamiento en km/h.
fuel_level	DOUBLE	-	No	No	Sí	Nivel residual de combustible o carga eléctrica (0 a 100%).
odometer	DOUBLE	-	No	No	Sí	Kilometraje total acumulado reportado por el odómetro del auto.
recorded_at	TIMESTAMP	-	No	No	Sí	Marca temporal de captura en el sensor IoT vehicular.
created_at	TIMESTAMP	-	No	No	Sí	Estampa de recepción y persistencia en el backend.
updated_at	TIMESTAMP	-	No	No	Sí	Estampa de auditoría de última actualización del registro.

Tabla: reviews

Registra las valoraciones y comentarios emitidos tras concluir satisfactoriamente una reserva.

Columna	Tipo SQL	Longitud	PK	FK	Not Null	Descripción
id	BIGINT	20	Sí	No	Sí	Clave primaria autoincremental de la reseña.
vehicle_id	BIGINT	20	No	Sí	Sí	Clave foránea del vehículo calificado (vehicles.id).
renter_id	BIGINT	20	No	Sí	Sí	Clave foránea del cliente arrendatario calificador (users.id).
rating	INT	2	No	No	Sí	Puntuación numérica entera de satisfacción (1 a 5 estrellas).
comment	VARCHAR	500	No	No	No	Comentario cualitativo sobre la experiencia de uso del vehículo.
created_at	TIMESTAMP	-	No	No	Sí	Estampa de auditoría de registro de la valoración.
updated_at	TIMESTAMP	-	No	No	Sí	Estampa de auditoría de última actualización de la reseña.

Capítulo V: Product Implementation

5.1. Software Configuration Management

5.1.1. Software Development Environment Configuration

Desarrollo del reporte: **Github**
Planificación y reunion: **Discord**
Comunicación: **WhatsApp**

Requirement Management

Utilizamos UXPressia para poder desarrollar los mapas, los user personas y lean UX canvas

Product UX/UI Design

Utilizamos Figma para el desarrollo de mock ups tanto de la landing page como de la web app

Software Deployment

Trabajamos con Github Pages para el despliegue de la Landing Page

GitHub Pages

5.1.2. Source Code Management

Producto	Tipo de Entregable	URL Oficial del Repositorio
Landing Page	Sitio Web Estático y Promocional	https://github.com/Codexa-Team/Landing-Page
Web Application	Frontend Single-Page Application (Angular 20)	https://github.com/Codexa-Team/Front-End
RESTful API Backend	Backend API & DDD Bounded Contexts (Spring Boot 3.x)	https://github.com/Codexa-Team/Backend
Project Report	Informe Académico de Ingeniería de Software	https://github.com/Codexa-Team/Project-Report

Para el desarrollo de todos los productos de Codexa-Team hemos empleado GitFlow con las siguientes restricciones:

- **Main**: Rama principal del proyecto. Contiene la versión estable y lista para producción.
- **Develop**: Rama de desarrollo. Aquí se integran las nuevas características y correcciones antes de ser fusionadas a la rama principal.
- **Feature**: Rama para el desarrollo de nuevas características. Se crea a partir de la rama de desarrollo y se fusiona nuevamente en ella una vez se complete el mismo
- **Fix**: Rama para correcciones de errores. Se crea a partir de la rama de desarrollo y se fusiona nuevamente en ella una vez completada la corrección.

Además, para el registro de progreso y colaboración se emplearán los conventional commits con su respectivo formato:

- feat: Nueva funcionalidad
- fix: Corrección de errores
- docs: Cambios en la documentación
- style: Cambios de formato o estilo
- refactor: Cambios en el código que no afectan la funcionalidad

5.1.3. Source Code Style Guide & Conventions

Para el desarrollo de la Landing Page de Renticar se establecieron convenciones de estilo y organización del código que aseguran consistencia, legibilidad y mantenibilidad del proyecto. Estas prácticas fueron aplicadas en todos los archivos del repositorio.

HTML

- **Use Lowercase Element Names**:
  En la landing de *Renticar*, todos los elementos HTML están en minúsculas.

```
<body>
  <header>
    <nav class="nav__header">...</nav>
  </header>
</body>
```

- **Use Lowercase Attribute Names**:
  Los atributos están en minúsculas como `src`, `alt`, `href`.

```
<img src="assets/chapter05/renticar-brand-logo.png" alt="logo" class="logo-dark" />
```

- **Use Double Quotes for Attribute Values**:
  Todos los atributos usan comillas dobles.

```
<a href="#about">About</a>
```

- **Omit Type Attributes for Style Sheets and Scripts**:
  No se incluyen atributos `type` innecesarios en CSS o JS.

```
<link rel="stylesheet" href="styles.css" />
<script src="main.js"></script>
```

CSS

- **Use Variables for Reusability**:
  Se emplean variables globales en `:root` para definir colores y tipografías.

```
:root {
  --primary-color: #52b6f2;
  --primary-color-dark: #003f70;
  --text-dark: #221e36;
}
```

- **ID and Class Name Style**:
  Se sigue una convención cercana a BEM (`bloque__elemento`) para mantener claridad.

```
.nav__header { ... }
.section__container { ... }
```

- **Use Shorthand Properties**:
  Se emplean propiedades abreviadas para mayor eficiencia.

```
.btn {
  padding: 0.75rem 1.5rem;
  font: 1rem "Roboto", sans-serif;
  border: none;
}
```

- **Organized Structure with Comments**:
  El archivo `styles.css` incluye comentarios para identificar secciones clave.

```
/* Header styles */
.header__content { ... }
```

JavaScript

- **Use `const` and `let`**:
  Se utilizan `const` para selectores fijos y `let` cuando el valor puede cambiar.

```
const menuBtn = document.getElementById("menu-btn");
const navLinks = document.getElementById("nav-links");
```

- **Use Event Listeners for Interactions**:
  Las interacciones se implementan con `addEventListener`, evitando código inline.

```
menuBtn.addEventListener("click", (e) => {
  navLinks.classList.toggle("open");
});
```

- **Animations with ScrollReveal**:
  Se emplea la librería `ScrollReveal` para dar dinamismo a los elementos.

```
ScrollReveal().reveal(".header__content h1", {
  distance: "50px",
  duration: 1000,
  delay: 1000,
});
```

- **Tabs Implementation**:
  La sección "Deals" se maneja con tabs dinámicos utilizando dataset.

```
tabs.addEventListener("click", (e) => {
  if (item.dataset.id === e.target.dataset.id) {
    item.classList.add("active");
  }
});
```

Convenciones observadas en el proyecto:

1. Estructura de archivos:

- index.html: página principal con meta tags optimizados y enlaces a hojas de estilo y librerías externas.
- styles.css: hoja de estilos centralizada con variables globales (--primary-color, --max-width).
- main.js: archivo JavaScript para interacción (menús, animaciones, etc.).
- /assets: carpeta destinada a recursos estáticos (logo, imágenes).
- README.md: guía básica del proyecto.

2. HTML (index.html):

- Uso correcto de y lang="en".
- Inclusión de meta tags esenciales (charset, viewport).
- Uso de etiquetas semánticas (header, nav, ul, li).
- Convención de clases siguiendo un estilo cercano a BEM: nav__header, nav__logo, section__container.

3. CSS (styles.css):

- Definición de variables globales en :root para colores, tipografía y ancho máximo.
- Reset inicial (* { margin: 0; padding: 0; box-sizing: border-box; }).
- Estructura organizada por secciones: contenedores, encabezados, botones.
- Comentarios descriptivos en bloques clave (ej. /* Estilos para header */).
- Convenciones de clases consistentes (section__header, section__description).

4. JavaScript (main.js):

- Uso de IDs (menu-btn, nav-links) para interacción de elementos de navegación.
- Separación clara de lógica en funciones.
- Comentarios ligeros para mejorar la comprensión del código.

5. Control de versiones (Git):

   Organización recomendada de commits con mensajes breves y descriptivos. Ejemplo:

   - feat: agregar sección de testimonios
   - fix: corregir responsive en navbar
   - style: aplicar variables CSS globales

Estas convenciones permiten que el código sea uniforme y entendible por cualquier miembro del equipo, reduciendo la curva de aprendizaje y asegurando que el proyecto sea escalable y mantenible en futuras iteraciones.

5.1.4. Software Deployment Configuration

Para el despliegue de *Renticar* se implementó una configuración basada en **Vercel**, vinculada directamente con el repositorio del proyecto alojado en **GitHub**. Esta estrategia garantiza la integración continua, la disponibilidad pública del sistema y la facilidad de actualización en cada commit.

Configuración de despliegue aplicada:

- **Entorno local de desarrollo:**
  - El proyecto puede ejecutarse en cualquier navegador moderno (Chrome, Edge, Firefox).
  - Para revisiones rápidas, basta con abrir el archivo `index.html`.
  - Para pruebas locales más completas, se recomienda levantar un servidor ligero con:

```
npx live-server
```

- **Repositorio en GitHub:**
  - El proyecto se mantiene en un repositorio público/privado en GitHub.
  - La rama principal (`main`) contiene el código estable del proyecto.
  - Cada commit en `main` dispara automáticamente un redeploy en Vercel.
- **Despliegue en Vercel:**
  - Se configuró la integración de GitHub con Vercel para automatizar el deployment.
  - En cada push al repositorio, Vercel ejecuta el build y genera un enlace actualizado de la Landing Page.
  - El resultado es una **Landing Page v1 pública y accesible** mediante un dominio generado por Vercel.
- **Estructura de archivos desplegados:**
  - `index.html`: página principal.
  - `styles.css`: hoja de estilos centralizada.
  - `main.js`: archivo de interactividad y animaciones.
  - `/assets`: carpeta con imágenes y recursos estáticos.
  - `README.md`: documentación básica del proyecto.
- **Validación del despliegue:**
  - Se verificó la carga correcta de recursos (CSS, JS, imágenes) en la versión desplegada.
  - Pruebas de compatibilidad en diferentes dispositivos (PC, tablet, smartphone).
  - Validación de responsividad en pantallas móviles.
  - Confirmación de que las animaciones, tabs y menú responsive funcionen en la versión online.

El uso de GitHub + Vercel permitió una configuración de despliegue **sencilla, automatizada y confiable**, asegurando que *Renticar* esté disponible en línea de manera inmediata después de cada actualización en el repositorio. Esto cumple con lo requerido en la TB1 y establece un flujo de trabajo profesional alineado con las prácticas modernas de desarrollo web.

5.2. Product Implementation & Deployment

En este apartado se documenta la ejecución del ciclo de vida del desarrollo ágil del software durante los diferentes Sprints del proyecto, las evidencias de despliegue en entornos productivos cloud, la documentación interactiva de los servicios web (Swagger/OpenAPI), y los indicadores de colaboración y productividad del equipo de desarrollo.

5.2.1. Sprint Backlogs

A continuación, se presentan las tablas de ejecución de los dos sprints desarrollados para la construcción de la plataforma **RentiCar**, detallando las User Stories asignadas, el desglose de tareas técnicas (work-items), estimación en horas-hombre, responsables individuales (distribuidos equitativamente entre los 5 miembros del equipo: Bruce Vía, César Linares, Sergio Landa, Estefano Solís y Sheila Angeles) y estado final:

Sprint #	**Sprint 1**						
User Story		**Work-Item/task**					
ID	**Title**	**Id**	**Title**	**Description**	**Estimation(Hrs)**	**Assigned To**	**Status**
US-01	Registro de Usuario	T001	Creacion de usuario	Crea un perfil para el registro del usuario	4	Bruce Via	Done
		T002	Seleccion de rol	Al crear el usuario este tiene la capacidad de elegir el rol que desea y acceder a las funcionalidades adecuadas	5	Estefano Solis	Done
US-07	Publicaciones en el Dashboard	T0011	Ver Publicaciones disponibles	Toda publicacion publicada se mostrara en el dashboard correspondiendte con acceso a dicha publciacion	4	Sergio Landa	Done
		T0012	Separar publicaciones por roles	Separa las secciones exclusivas para los roles especificos	5	Cesar Linares	Done
		T0013	Separacion de dashboard	Muestra un dashboard diferente para cada rol con sus publciaciones respectivas	4	Sheila Angeles	Done

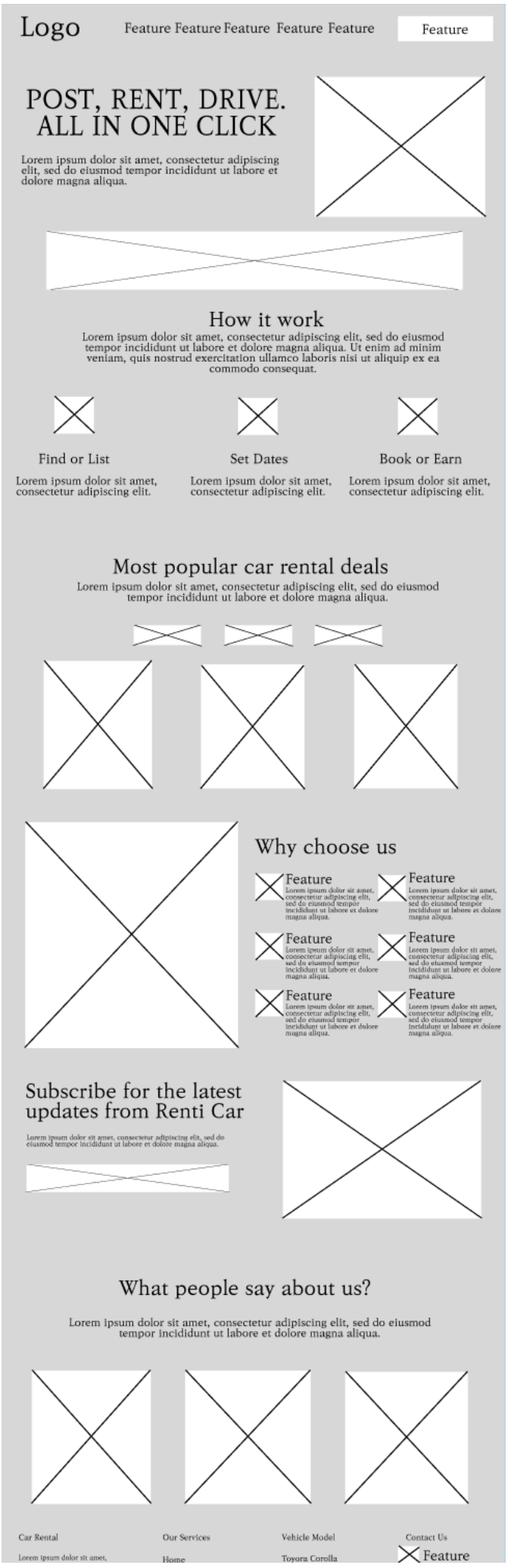
Sprint #	**Sprint 1**						
US-20	Confirmar Solicitud de Reserva	T011	Reservar correctamente un vehiculo	Al momento de realizar la rerva y se cumplan los requisitos se confirma y se inicia el proceso	7	Estefano Solis	Done
US-26	Iniciar Simulación Telemática en Ruta	T021	Visualizar la simulacion	Ver la ubicacion de auto en un mapa	8	Sheila Angeles	Done
		T022	Visualizar simulacion	Muestra al auto moviendose por el mapa de Lima	8	Bruce Via	Done
TS-01	Pruebas de API - Registro de Usuarios (`sign-up.feature`)	T1	Automatización de registro por roles	Pruebas Karate para registro de Arrendatario y Arrendador con asignación de roles	4	Bruce Via	Done
		T2	Pruebas de validación y duplicidad	Pruebas Karate para rechazo por correo duplicado (409) y sintaxis inválida (400)	4	Cesar Linares	Done
TS-02	Pruebas de API - Autenticación e Inicio de Sesión (`sign-in.feature`)	T3	Automatización de inicio de sesión	Pruebas Karate para login exitoso y verificación de Bearer Token JWT	4	Sergio Landa	Done
		T4	Pruebas negativas de credenciales	Pruebas Karate para contraseña errónea (401) y usuario no existente (404/401)	3	Estefano Solis	Done
TS-03	Pruebas de API - Gestión de Roles (`roles.feature`)	T5	Verificación de catálogo de roles	Pruebas Karate para consultar roles y validar ROLE_ARRENDADOR y ROLE_ARRENDATARIO	3	Sheila Angeles	Done
TS-04	Pruebas de API - Catálogo de Vehículos (`vehicles.feature`)	T6	Pruebas de consulta de catálogo	Pruebas Karate para listado general de vehículos y validación de estructura de datos	4	Sergio Landa	Done
		T7	Validación de vehículo inexistente y seguridad	Pruebas Karate para vehículo 404 y restricción de creación sin cabecera de autenticación	4	Sheila Angeles	Done

Sprint #	**Sprint 2**						
User Story		**Work-Item/task**					
ID	**Title**	**Id**	**Title**	**Description**	**Estimation(Hrs)**	**Assigned To**	**Status**
US-01	Registro de Usuario	T001	Creacion de usuario	Crea un perfil para el registro del usuario	4	Bruce Via	Done
		T002	Seleccion de rol	Al crear el usuario este tiene la capacidad de elegir el rol que desea y acceder a las funcionalidades adecuadas	5	Estefano Solis	Done
US-07	Publicaciones en el Dashboard	T0011	Ver Publicaciones disponibles	Toda publicacion publicada se mostrara en el dashboard correspondiendte con acceso a dicha publciacion	4	Sergio Landa	Done
		T0012	Separar publicaciones por roles	Separa las secciones exclusivas para los roles especificos	5	Cesar Linares	Done
		T0013	Separacion de dashboard	Muestra un dashboard diferente para cada rol con sus publciaciones respectivas	4	Sheila Angeles	Done
US-20	Confirmar Solicitud de Reserva	T011	Reservar correctamente un vehiculo	Al momento de realizar la rerva y se cumplan los requisitos se confirma y se inicia el proceso	7	Estefano Solis	Done
US-26	Iniciar Simulación Telemática en Ruta	T021	Visualizar la simulacion	Ver la ubicacion de auto en un mapa	8	Sheila Angeles	Done
		T022	Visualizar simulacion	Muestra al auto moviendose por el mapa de Lima	8	Bruce Via	Done

Sprint #	Sprint 2						
TS-05	Pruebas de API - Gestión de Usuarios (`users.feature`)	T8	Automatización de catálogo de usuarios	Pruebas Karate para GET /api/v1/users y validación de respuesta	4	Sergio Landa	Done
		T9	Pruebas de consulta de usuario por ID	Pruebas Karate para registro dinámico y consulta de usuario por ID propio	4	Bruce Via	Done
TS-06	Pruebas de API - Gestión de Reservas (`bookings.feature`)	T10	Pruebas de seguridad en reservas	Pruebas Karate para validar rechazo 401/403 en /my-bookings y /my-requests sin auth	4	Estefano Solis	Done
		T11	Pruebas de creación y búsqueda de reservas	Pruebas Karate para POST reserva no autenticada y consulta por ID inexistente (404)	4	Cesar Linares	Done
TS-07	Pruebas de API - Telemetría y Rutas IoT (`telemetry.feature`)	T12	Automatización de ruta GPS simulada	Pruebas Karate para consulta y verificación de /api/v1/simulation/route	4	Sheila Angeles	Done
		T13	Pruebas de seguridad en telemetría	Pruebas Karate para restringir POST /telemetry y consulta de posición sin token	4	Bruce Via	Done
TS-08	Pruebas de API - Calificaciones y Reseñas (`reviews.feature`)	T14	Pruebas de consulta de opiniones	Pruebas Karate para GET reseñas por vehículo y cálculo de valoraciones	4	Sergio Landa	Done
		T15	Pruebas de seguridad en publicación	Pruebas Karate para restricción de creación de reseña sin credenciales autorizadas	3	Sheila Angeles	Done

5.2.2. Implemented Landing Page Evidence

La Landing Page de **RentiCar** fue diseñada para brindar una primera impresión confiable, moderna y atractiva, maximizando la conversión de visitantes en usuarios registrados.


Logo
Feature Feature Feature Feature Feature
Feature
POST, RENT, DRIVE.
ALL IN ONE CLICK
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
How it work
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.
Find or List
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Set Dates
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Book or Earn
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Most popular car rental deals
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Why choose us
Feature
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Feature
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Feature
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Feature
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Feature
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Feature
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Subscribe for the latest updates from Renti Car
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
What people say about us?
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
Car Rental
Lorem ipsum dolor sit amet,
Our Services
Home
Vehicle Model
Toyota Corolla
Contact Us
Feature

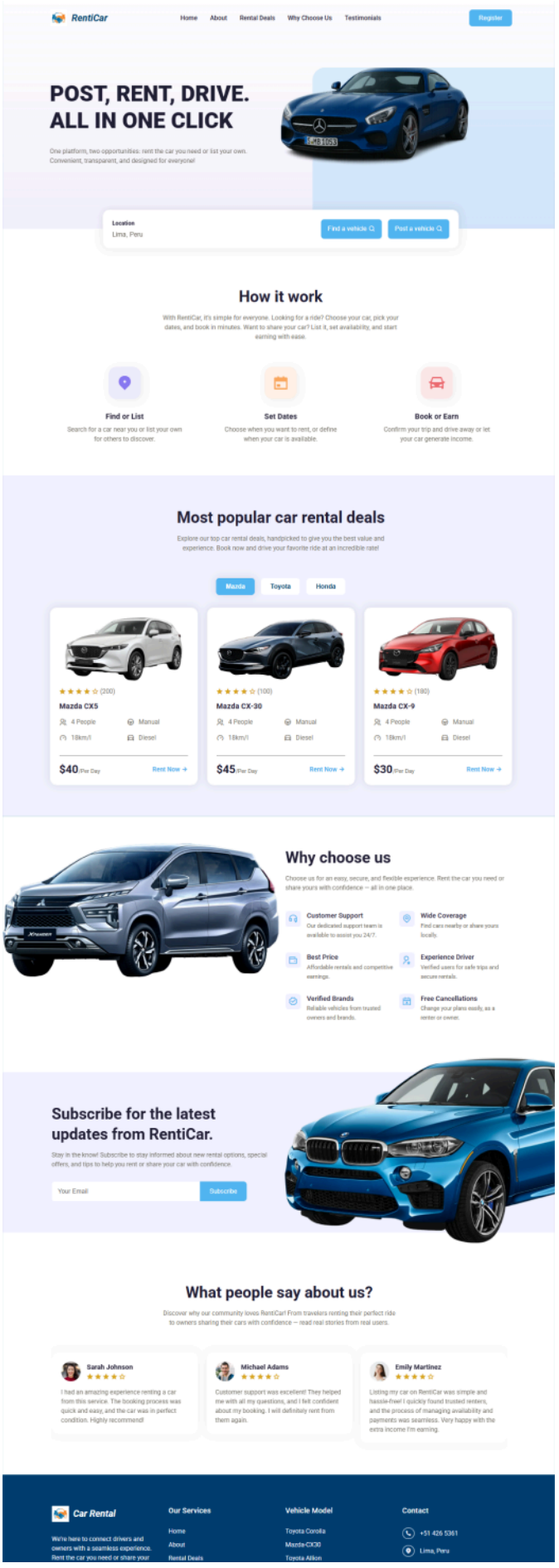

RentiCar
Home
About
Rental Deals
Why Choose Us
Testimonials
Register
POST, RENT, DRIVE.
ALL IN ONE CLICK
One platform, two opportunities: rent the car you need or list your own. Convenient, transparent, and designed for everyone!
Location
Lima, Peru
Find a vehicle
Post a vehicle
How it work
With RentiCar, it's simple for everyone. Looking for a ride? Choose your car, pick your dates, and book in minutes. Want to share your car? List it, set availability, and start earning with ease.
Find or List
Search for a car near you or list your own for others to discover.
Set Dates
Choose when you want to rent, or define when your car is available.
Book or Earn
Confirm your trip and drive away or let your car generate income.
Most popular car rental deals
Explore our top car rental deals, handpicked to give you the best value and experience. Book now and drive your favorite ride at an incredible rate!
Mazda
Toyota
Honda
Mazda CX5
4 People
Manual
18km/l
Diesel
$40/Per Day
Rent Now
Mazda CX-30
4 People
Manual
18km/l
Diesel
$45/Per Day
Rent Now
Mazda CX-9
4 People
Manual
18km/l
Diesel
$30/Per Day
Rent Now
Why choose us
Choose us for an easy, secure, and flexible experience. Rent the car you need or share yours with confidence — all in one place.
Customer Support
Our dedicated support team is available to assist you 24/7.
Wide Coverage
Find cars nearby or share yours locally.
Best Price
Affordable rentals and competitive earnings.
Experience Driver
Verified users for safe trips and secure rentals.
Verified Brands
Reliable vehicles from trusted owners and brands.
Free Cancellations
Change your plans easily, as a renter or owner.
Subscribe for the latest updates from RentiCar.
Stay in the know! Subscribe to stay informed about new rental options, special offers, and tips to help you rent or share your car with confidence.
Your Email
Subscribe
What people say about us?
Discover why our community loves RentiCar! From travelers renting their perfect ride to owners sharing their cars with confidence — read real stories from real users.
Sarah Johnson
I had an amazing experience renting a car from this service. The booking process was quick and easy, and the car was in perfect condition. Highly recommend!
Michael Adams
Customer support was excellent! They helped me with all my questions, and I felt confident about my booking. I will definitely rent from them again.
Emily Martinez
Listing my car on RentiCar was simple and hassle-free! I quickly found trusted renters, and the process of managing availability and payments was seamless. Very happy with the extra income I'm earning.
Car Rental
We're here to connect drivers and owners with a seamless experience. Rent the car you need or share your
Our Services
Home
About
Rental Deals
Vehicle Model
Toyota Corolla
Mazda CX30
Toyota Allion
Contact
+51 426 5361
Lima, Peru

Figura 5.2.2.1: Evolución de Wireframe a Mock-up de la Landing Page

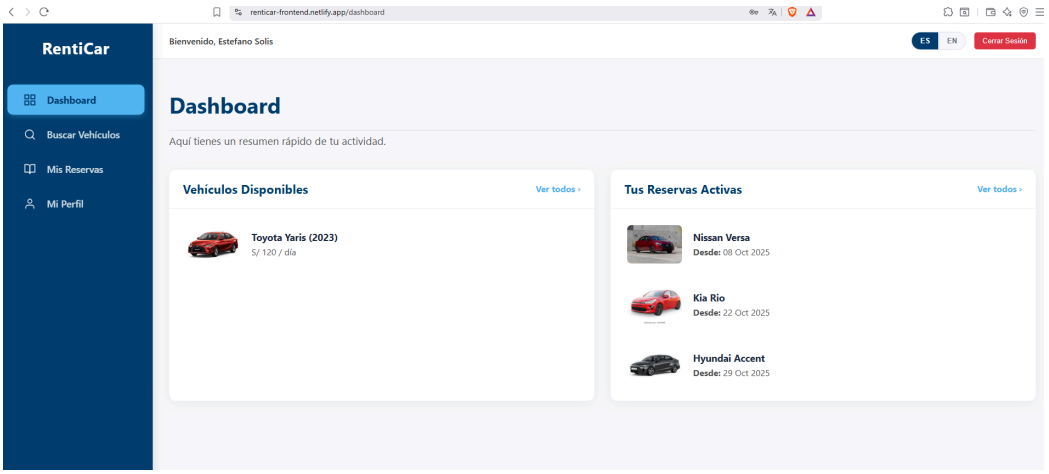


Figura 5.2.2.2: Landing Page implementada y desplegada en producción

Características Principales Implementadas en la Landing Page:

- **Encabezado y Navegación Adaptable:** Barra de navegación fija con logotipo corporativo, enlaces a las secciones ("Inicio", "Por qué elegirnos", "Autos populares", "Opiniones") y selector dinámico de idioma (i18n).
- **Hero Section con Llamado a la Acción:** Propuesta de valor clara orientada a la movilidad compartida en el Perú con botones directos hacia el registro y la exploración del catálogo.
- **Catálogo de Modelos Populares:** Carrusel interactivo desarrollado con la librería Swiper.js, permitiendo navegar entre los automóviles más destacados.
- **Despliegue Continuo en Vercel:** Vinculación directa con la rama `main` del repositorio `LandingPage`, logrando un pipeline automatizado con tiempos de carga optimizados.

Enlace a la Landing Page Desplegada

https://codexa-team.github.io/Landing-Page/

5.2.3. Implemented Frontend-Web Application Evidence

La aplicación web de **RentiCar** fue construida en **Angular**, utilizando Angular Material para los controles de interfaz de usuario y Leaflet para la visualización cartográfica.

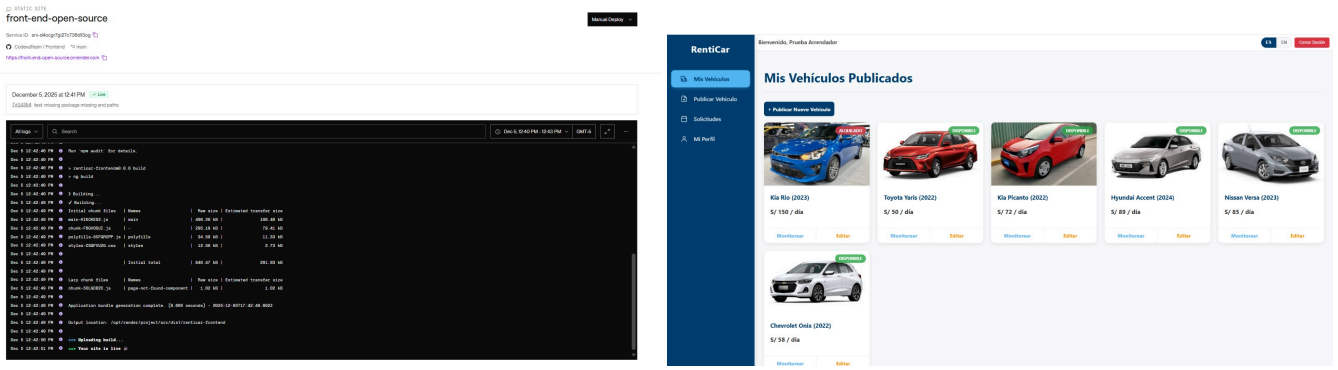


Figura 5.2.3.1: Evidencia de compilación y ejecución de la aplicación web en producción

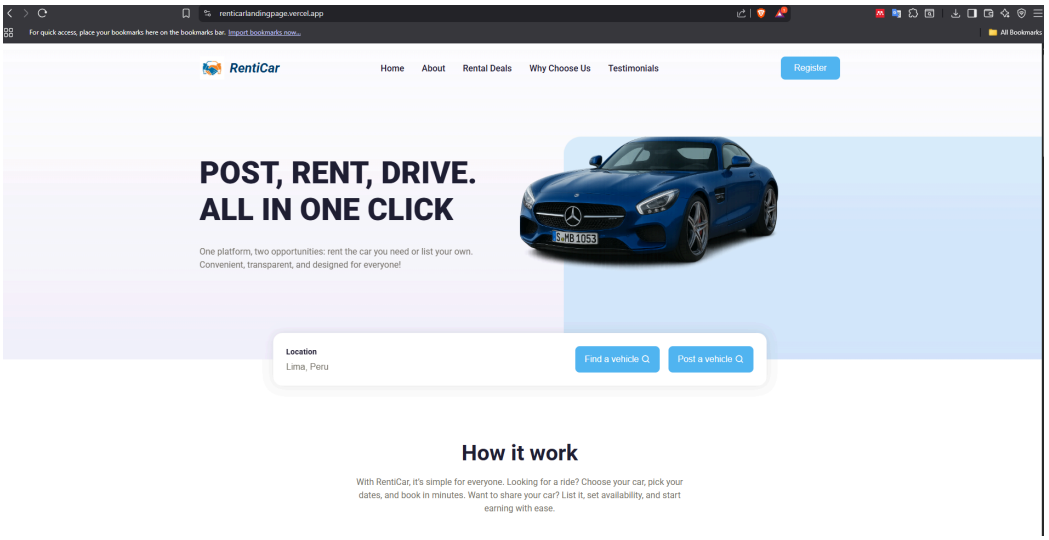


Figura 5.2.3.2: Navegación por el catálogo interactivo de vehículos y panel de reservas

Módulos Clave del Frontend Verificados:

- **Routing y Guards de Seguridad:** Rutas protegidas mediante `auth.guard.ts` para restringir el acceso a paneles privados sin token JWT válido.
- **Manejo Centralizado de Estados:** Servicios de Angular gestionando reactivamente el estado de la sesión, la cesta de reserva y los filtros de búsqueda.
- **Visualización Telemática Interactiva:** Integración con mapas vectoriales de código abierto permitiendo seguir la trayectoria simulada del vehículo durante el viaje.

Enlace al Frontend Desplegado

https://front-end-open-source.onrender.com/

5.2.4. Acuerdo de Servicio - SaaS

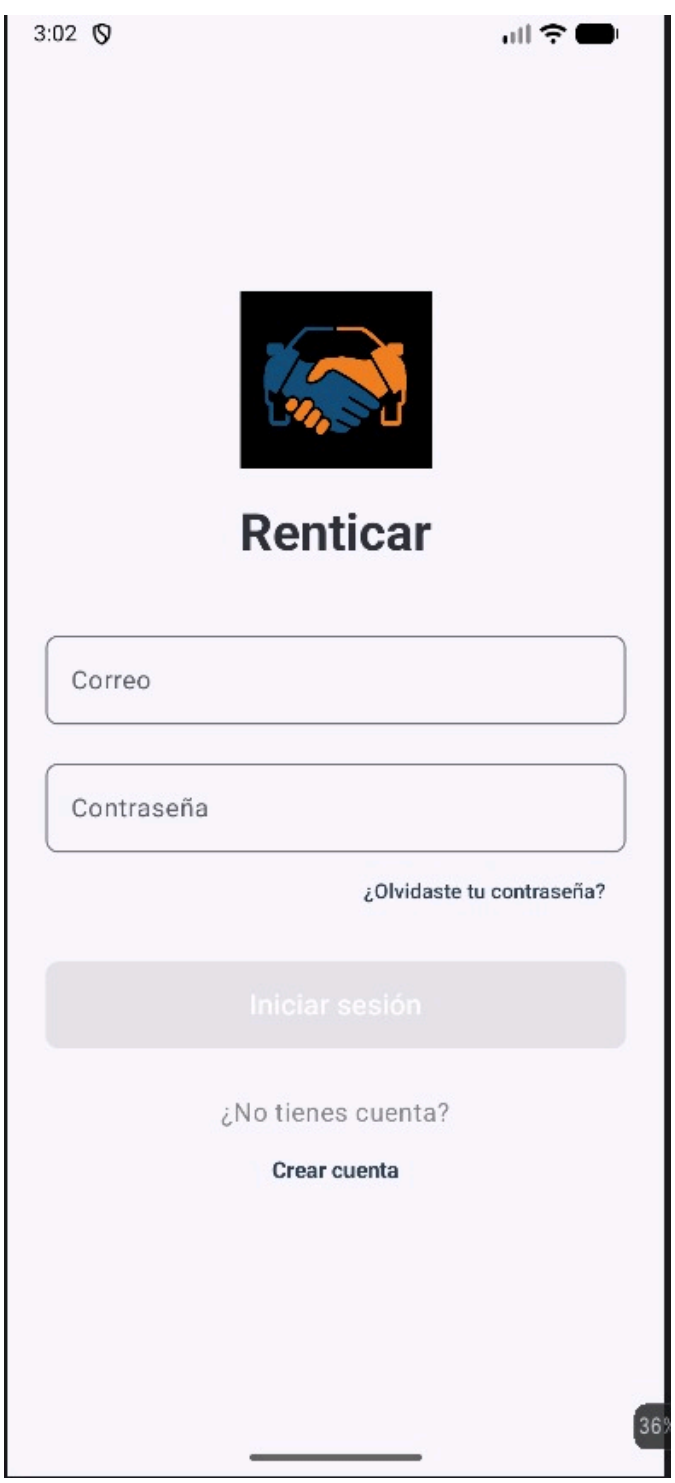
La plataforma **RentiCar** se distribuye bajo el modelo de **Software as a Service (SaaS)**. Con el objetivo de garantizar la confianza y transparencia entre la startup **Codexa-Team**, los propietarios de vehículos (arrendadores) y los usuarios clientes (arrendatarios), se establece el siguiente Acuerdo de Nivel de Servicio (SLA):

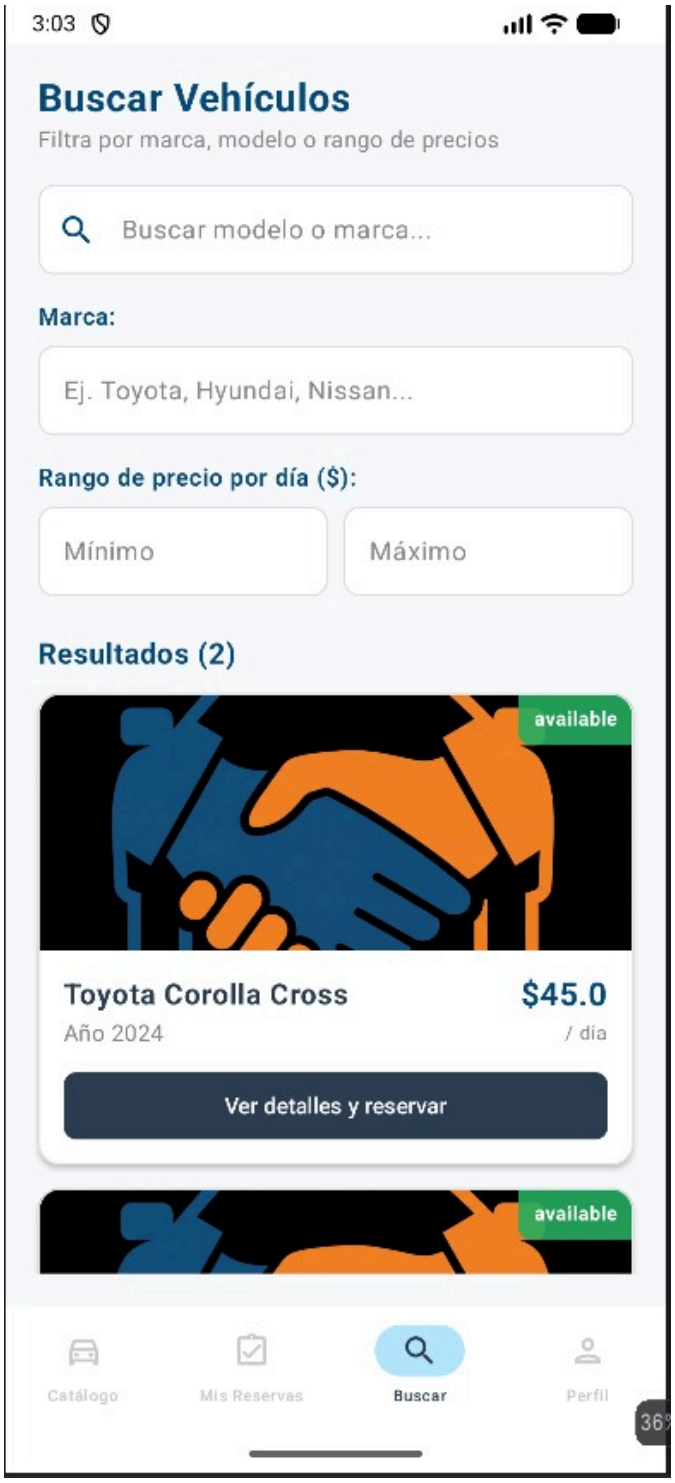
1. **Disponibilidad del Servicio (Uptime SLA):**
   Codexa-Team garantiza una disponibilidad operativa mensual de la plataforma de al menos el **99.5%**, excluyendo periodos de mantenimiento preventivo programados con antelación.
2. **Seguridad y Confidencialidad de la Información:**
   Toda la comunicación de datos se transmite cifrada mediante protocolo seguro TLS/HTTPS. Las contraseñas de los usuarios se almacenan con hashing irreversible BCrypt, y los datos personales son resguardados en cumplimiento estricto con la Ley de Protección de Datos Personales (Ley N° 29733).
3. **Mecanismo de Retención y Comisiones:**
   La plataforma opera bajo un esquema de comisión transaccional transparente sobre cada reserva completada, reteniendo los fondos de forma segura hasta que el vehículo es devuelto satisfactoriamente.
4. **Soporte y Tiempos de Respuesta ante Incidentes:**
   Se establecen canales de atención técnica con respuesta máxima de 4 horas para incidentes críticos (fallas en confirmación de reservas) y 24 horas para consultas ordinarias.

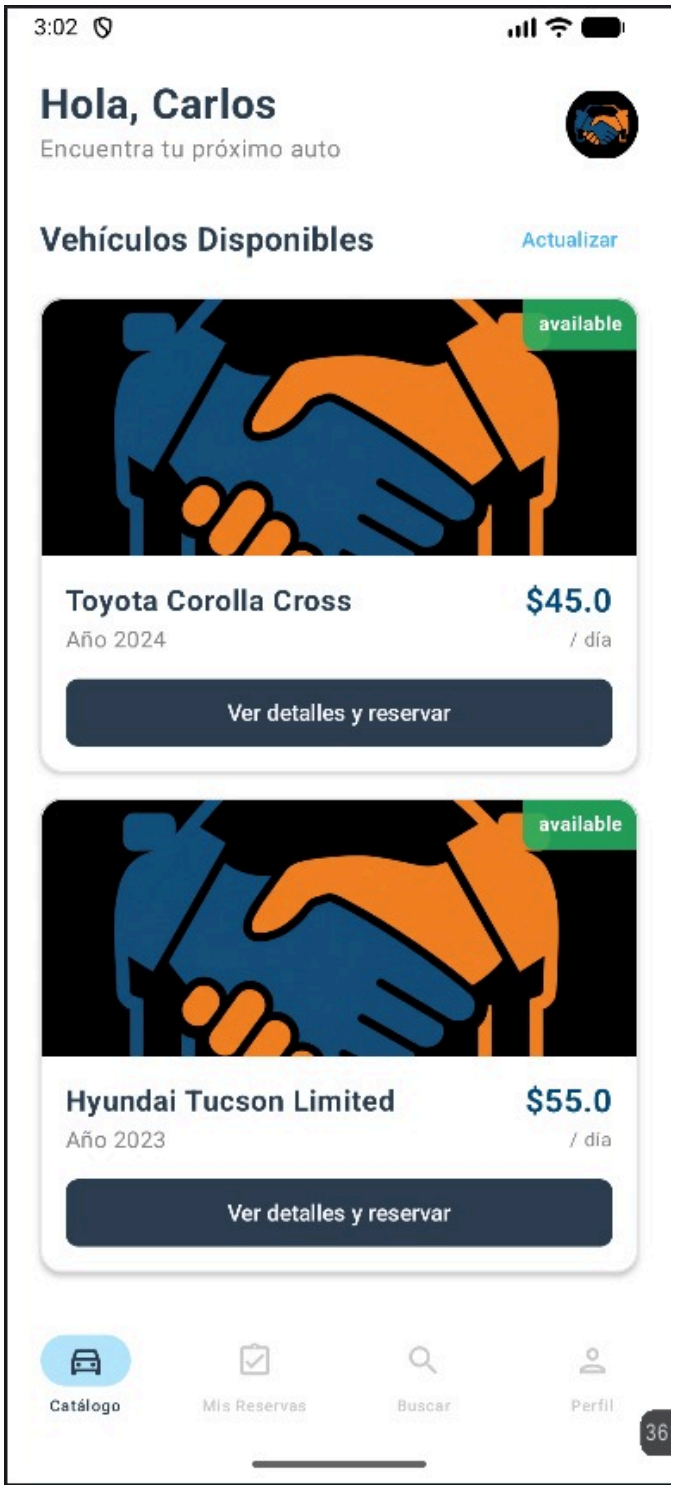
5.2.5. Implemented Native-Mobile Application Evidence

Gracias a los diseños desarrollados en las secciones anteriores tenemos evidencia del desarrollo de la aplicacion movil nativa para el proyecto

Renticar es una plataforma móvil y backend para el alquiler colaborativo de vehículos que conecta a arrendadores (propietarios que publican sus autos, fijan tarifas y gestionan solicitudes de alquiler) con arrendatarios (clientes que exploran el catálogo y reservan vehículos en tiempo real); está desarrollada con una aplicación nativa en Android (Kotlin) utilizando Jetpack Compose y arquitectura MVVM, respaldada por un backend robusto en Java con Spring Boot protegido mediante autenticación JWT, y una base de datos relacional en MySQL para la gestión persistente y segura de usuarios, vehículos y reservas.


3:02
Renticar
Correo
Contraseña
¿Olvidaste tu contraseña?
Iniciar sesión
¿No tienes cuenta?
Crear cuenta


3:03
Buscar Vehículos
Filtra por marca, modelo o rango de precios
Buscar modelo o marca...
Marca:
Ej. Toyota, Hyundai, Nissan...
Rango de precio por día ($):
Mínimo
Máximo
Resultados (2)
available
Toyota Corolla Cross
$45.0
Año 2024
/ día
Ver detalles y reservar
available
Catálogo
Mis Reservas
Buscar
Perfil



5.2.6. Implemented RESTful API and/or Serverless Backend Evidence

El backend fue desplegado en la plataforma cloud **Render**, conectado de forma persistente a una instancia gestionada de **TiDB Cloud**:

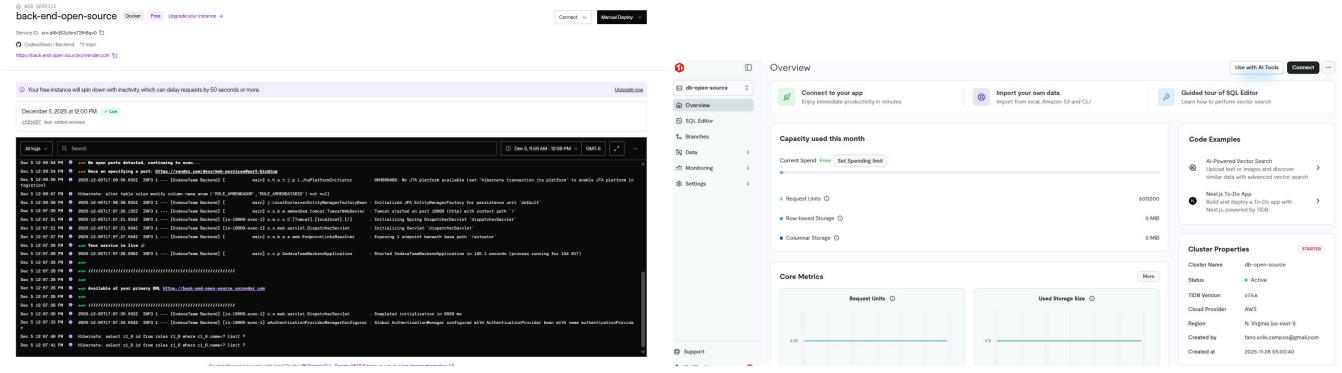


Figura 5.2.5.1: Despliegue del servicio RESTful en Render y conexión exitosa a TiDB Cloud

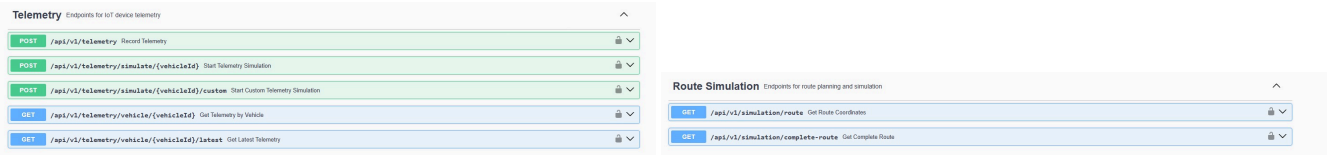


Figura 5.2.5.2: Logs de compilación Maven exitosa e inicialización de esquemas JPA

Configuración Técnica de Despliegue:

- **Entorno de Ejecución:** OpenJDK 17 / 21 sobre contenedor Linux en Render.
- **Variables de Entorno Clave:** `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME`, `SPRING_DATASOURCE_PASSWORD`, `JWT_SECRET`.
- **Integración Continua:** Cada push a la rama `main` del repositorio `Backend` dispara automáticamente la compilación Maven, ejecución de pruebas y nuevo despliegue sin caída de servicio.

Enlace al Backend Desplegado
https://back-end-open-source.onrender.com/

5.2.6. RESTful API Documentation

La documentación interactiva de la API fue implementada mediante **Swagger UI / OpenAPI 3.0**, permitiendo inspeccionar los esquemas DTO de entrada y salida, así como ejecutar peticiones de prueba en vivo contra el servidor desplegado:

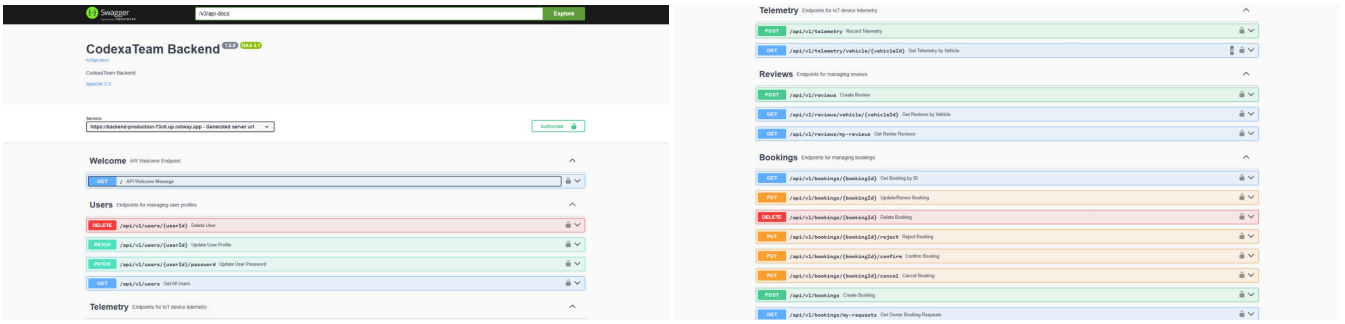


Figura 5.2.6.1: Vistas de la documentación interactiva en Swagger UI de RentiCar

Matriz Oficial de Endpoints RESTful Implementados en Spring Boot

La siguiente matriz detalla con rigor técnico los controladores, métodos HTTP, rutas oficiales, esquemas de transferencia de datos y códigos de estado HTTP implementados en la solución:

Bounded Context IAM

Bounded Context el servicio de identificacion y registro e usuarios

Bounded Context	Método	Ruta del Endpoint	Descripción del Servicio
IAM	`POST`	`/api/v1/authentication/sign-up`	Registra una nueva cuenta de usuario asignando roles (`ROLE_RENTER`, `ROLE_OWNER`).
IAM	`POST`	`/api/v1/authentication/sign-in`	Valida credenciales de acceso y emite el token Bearer JWT con claims de usuario.
IAM	`GET`	`/api/v1/authentication`	Verifica el estado general de autenticación del sistema.
IAM	`GET`	`/api/v1/users`	Lista los usuarios registrados en el sistema.
IAM	`PATCH`	`/api/v1/users/{userId}`	Actualiza la información básica del perfil del usuario.
IAM	`PATCH`	`/api/v1/users/{userId}/password`	Restablece la contraseña cifrada del usuario.
IAM	`GET`	`/api/v1/roles`	Consulta el catálogo de roles admitidos en la plataforma.

Bounded Context Listing

Bounded Context	Método	Ruta del Endpoint	Descripción del Servicio
Listings	`GET`	`/api/v1/vehicles`	Retorna el catálogo completo de vehículos disponibles para alquiler.
Listings	`GET`	`/api/v1/vehicles/{vehicleId}`	Retorna la ficha técnica detallada de un vehículo por su ID.
Listings	`GET`	`/api/v1/vehicles/{vehicleId}/image`	Descarga/renderiza la fotografía binaria almacenada del vehículo.
Listings	`GET`	`/api/v1/vehicles/my-listings`	Retorna la lista de vehículos publicados por el arrendador autenticado.
Listings	`POST`	`/api/v1/vehicles`	Publica un nuevo vehículo en la flota adjuntando datos y archivo fotográfico.
Listings	`PUT`	`/api/v1/vehicles/{vehicleId}`	Modifica las especificaciones, tarifa diaria o fotografía del vehículo.
Listings	`DELETE`	`/api/v1/vehicles/{vehicleId}`	Da de baja una publicación de vehículo del catálogo.

Bounded Context Booking

Bounded Context	Método	Ruta del Endpoint	Descripción del Servicio
Booking	POST	/api/v1/bookings	Registra una nueva solicitud de reserva en estado transaccional PENDING.
Booking	GET	/api/v1/bookings/my-bookings	Consulta el historial completo de reservas solicitadas por el cliente autenticado.
Booking	GET	/api/v1/bookings/my-requests	Retorna la bandeja de solicitudes de reserva recibidas por el dueño autenticado.
Booking	GET	/api/v1/bookings/{bookingId}	Obtiene el comprobante y detalle de una reserva específica.
Booking	PUT	/api/v1/bookings/{bookingId}/confirm	Confirma la reserva transicionándola a CONFIRMED y bloquea disponibilidad.
Booking	PUT	/api/v1/bookings/{bookingId}/reject	Declina la solicitud transicionándola a REJECTED y libera fechas.
Booking	PUT	/api/v1/bookings/{bookingId}/cancel	Cancela la reserva por iniciativa del cliente transicionándola a CANCELLED.

Bounded Context IoT

Bounded Context	Método	Ruta del Endpoint	Descripción del Servicio
IoT	POST	/api/v1/telemetry	Registra una lectura telemática emitida por los sensores vehiculares.
IoT	GET	/api/v1/telemetry/vehicle/{vehicleId}	Retorna el historial cronológico de telemetría de un vehículo.
IoT	GET	/api/v1/telemetry/vehicle/{vehicleId}/latest	Obtiene la última lectura telemática (latitud, longitud, velocidad, combustible).
IoT	GET	/api/v1/simulation/route	Retorna la secuencia de coordenadas GPS simuladas de la ruta en curso.

Bounded Context Reviews

Bounded Context	Método	Ruta del Endpoint	Descripción del Servicio
Reviews	POST	/api/v1/reviews	Registra una calificación de 1 a 5 estrellas y comentario tras culminar alquiler.
Reviews	GET	/api/v1/reviews/vehicle/{vehicleId}	Retorna todías las opiniones y promedio de valoración de un vehículo.
Reviews	GET	/api/v1/reviews/my-reviews	Retorna las reseñas emitidas por el usuario autenticado.

Bounded Context Shared

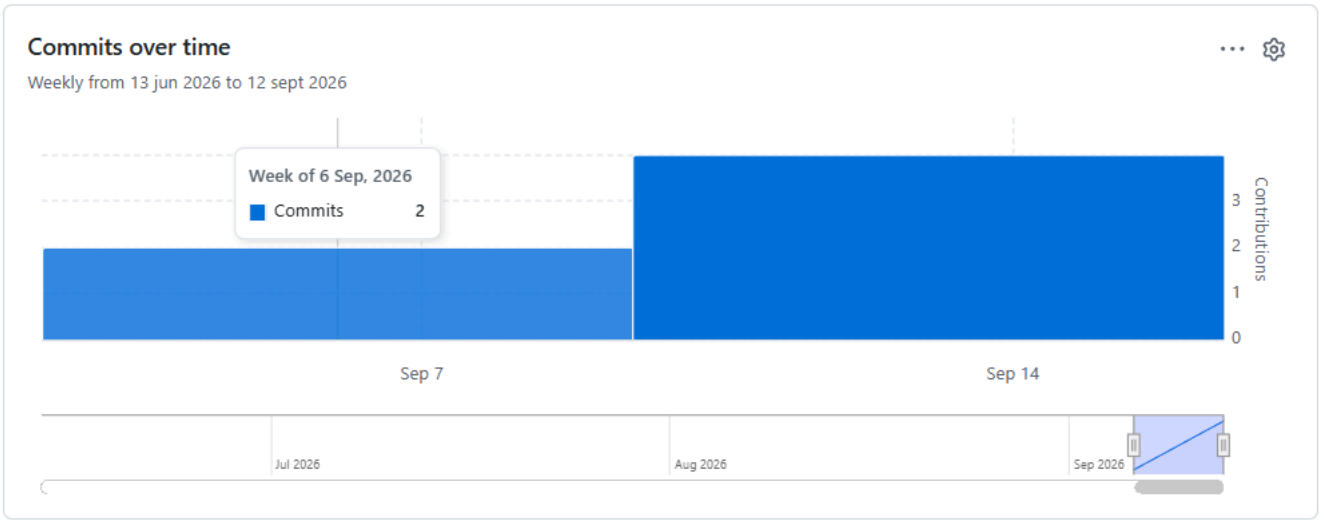
Bounded Context	Método	Ruta del Endpoint	Descripción del Servicio
Shared	GET	/	Endpoint raíz que expone el estado y versión de la API REST.

5.2.7. Team Collaboration Insights

Durante todo el ciclo de desarrollo, **Codexa-Team** mantuvo una disciplina estricta de colaboración ágil y control de versiones utilizando **GitHub**:

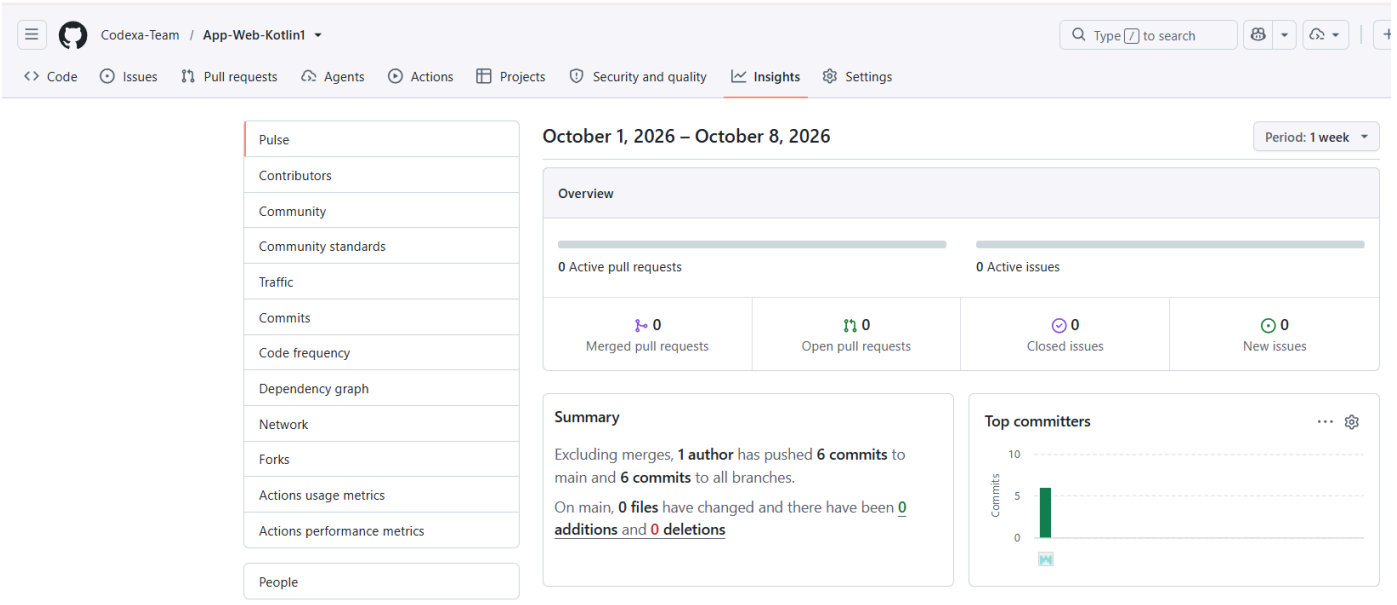
Sprint 1

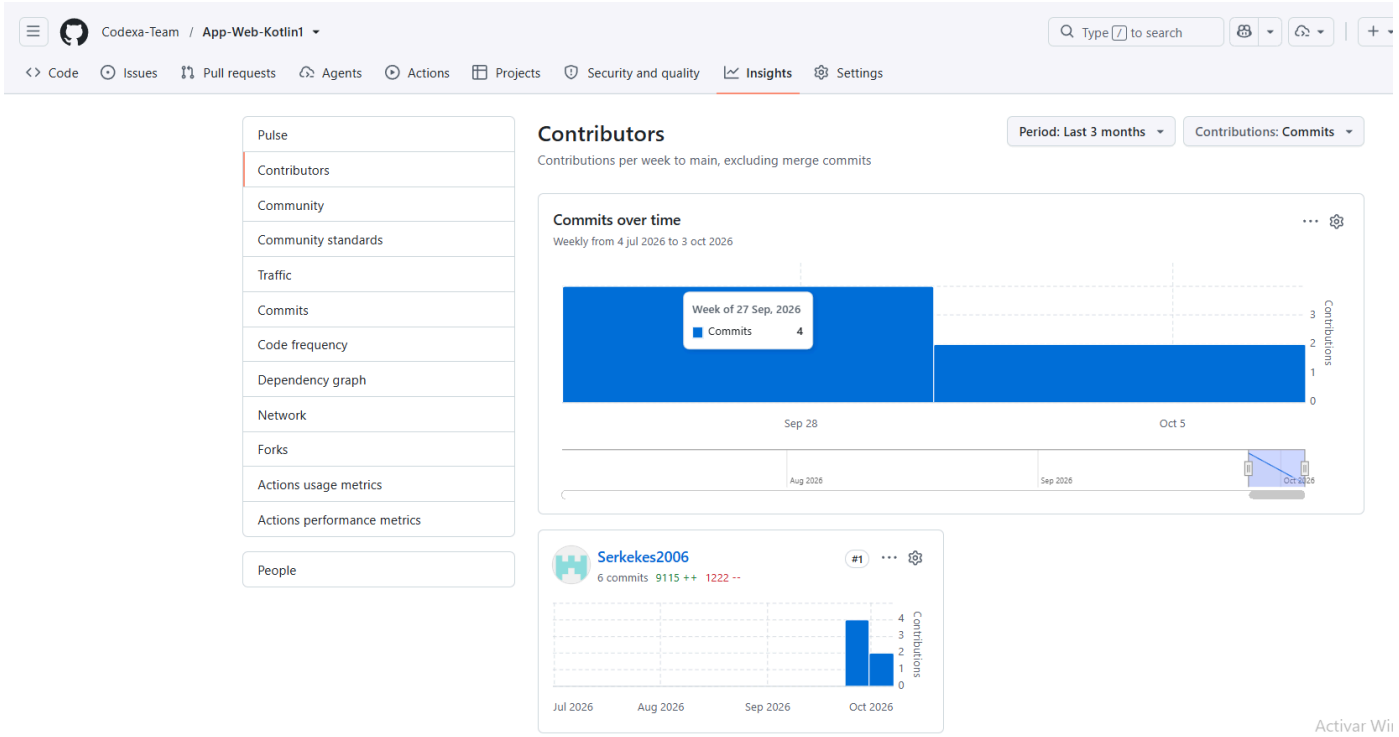
Collaboration Landing Page:



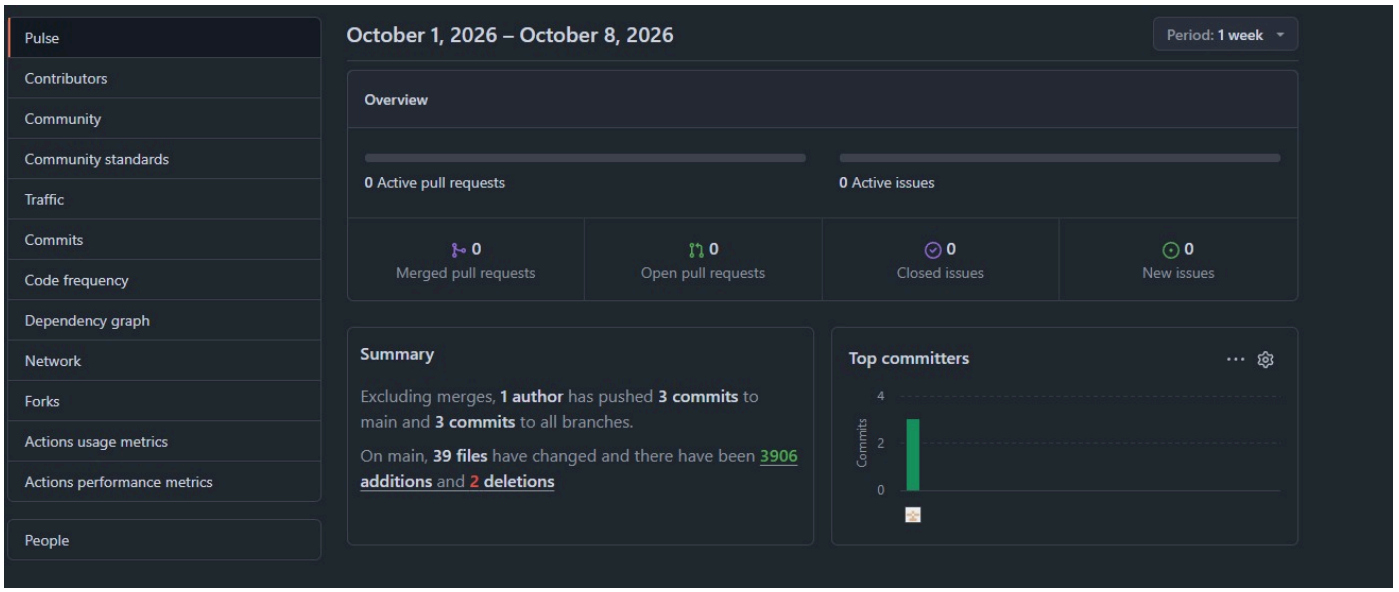
Sprint 2

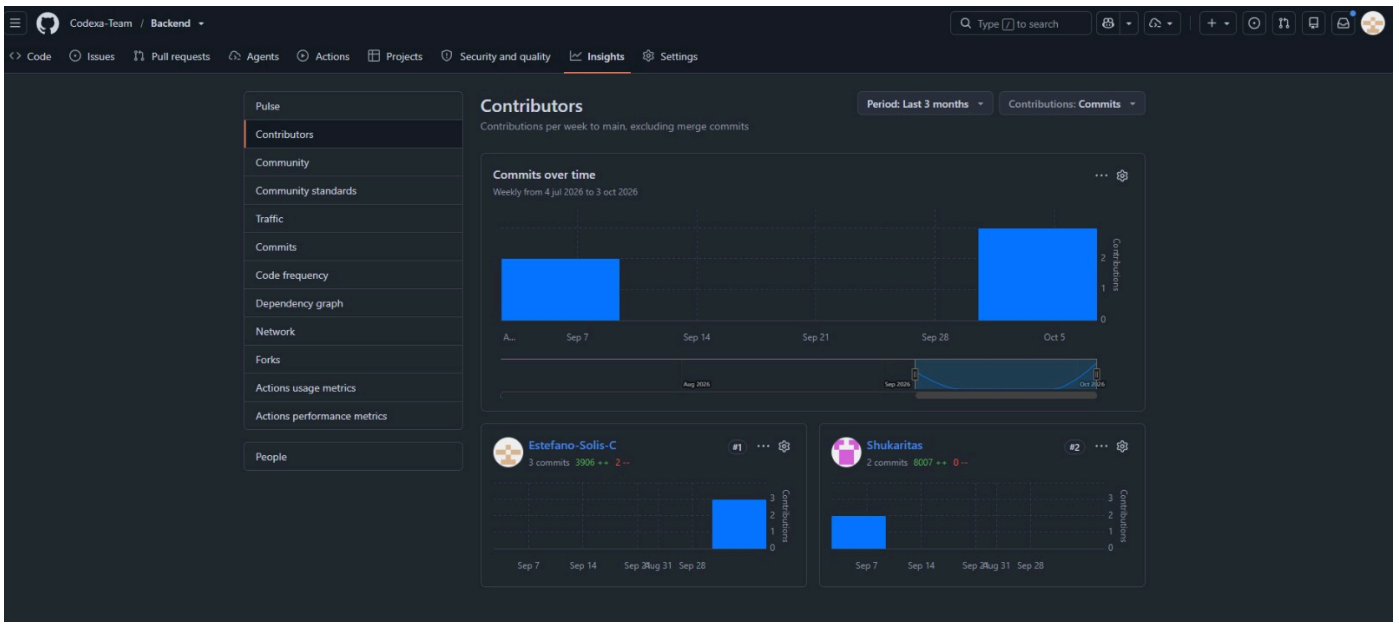
Collaboration App Mobile:



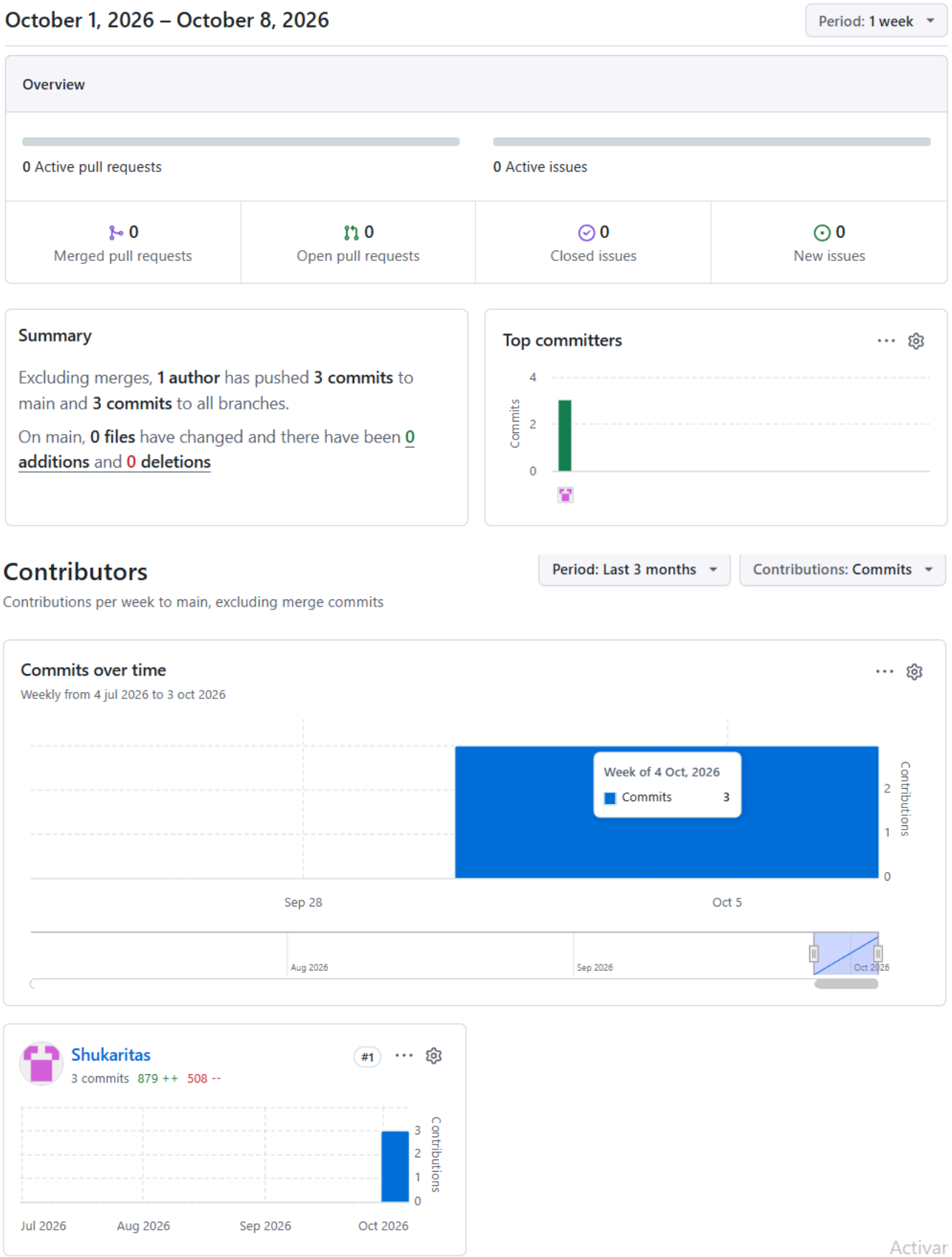


Collaboration Backend:





Collaboration Test Karate:



5.3. Video About-the-Product

En este apartado se documenta el video promocional y técnico elaborado por **Codexa-Team**, en el cual se demuestra el funcionamiento real e integrado de la plataforma **RentiCar**:

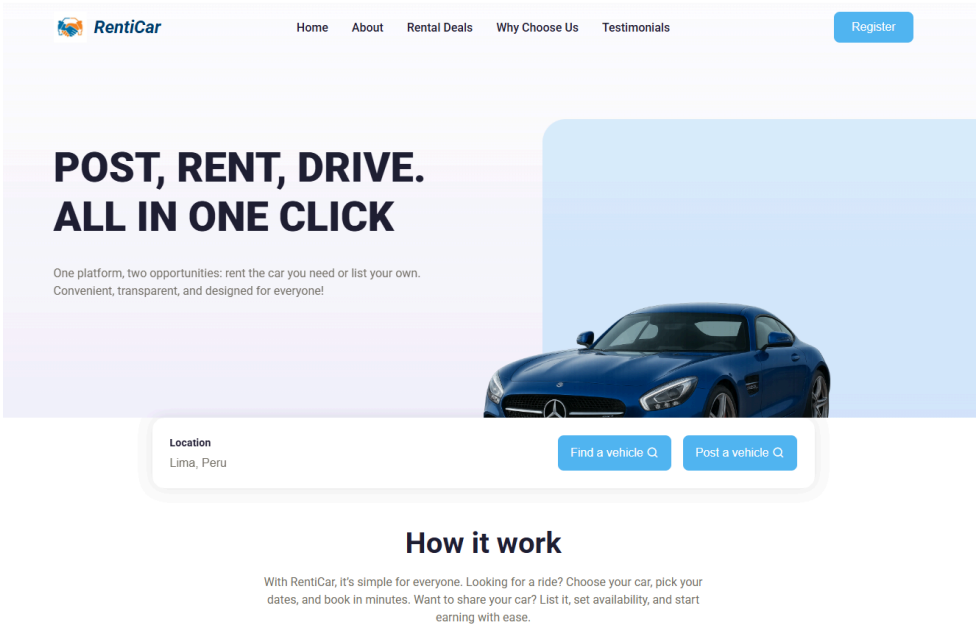


Figura 5.3: Presentación oficial del Video About-the-Product de RentiCar

Ficha Técnica del Video About-the-Product

Campo	Detalle Informativo
Título del Video:	About The Product
Startup Responsable:	Codexa-Team
Producto Presentado:	RentiCar (Aplicación Web Angular + Backend Spring Boot + TiDB)
Integrantes Participantes:	Bruce Vía Luna, César Linares, Sergio Landa, Estefano Solís, Sheila Angeles
Duración Aproximada:	9:27 minutos
Enlace Oficial del Video:	Ver Video About-the-Product en YouTube (https://youtu.be/t_m6u7wnkCs)

Objetivos del Video Demostrativo:

1. Presentar la propuesta de valor de **RentiCar** y la problemática de movilidad informal resuelta en el Perú.
2. Demostrar en vivo el flujo completo del **Arrendatario**: registro, login JWT, navegación en el catálogo, filtros avanzados, selección de fechas en calendario y confirmación de reserva.
3. Demostrar en vivo el flujo del **Arrendador**: dashboard, publicación de un vehículo con especificaciones y fotos, recepción de solicitudes de alquiler y aprobación.
4. Exponer la funcionalidad de **Monitoreo IoT**: visualización de coordenadas GPS en tiempo real sobre mapa Leaflet y lectura de telemetría de velocidad y combustible.
5. Evidenciar el proceso de **Calificación y Reseñas** que consolida la confianza comunitaria en la plataforma.

Part II: Verification, Validation & Pipeline

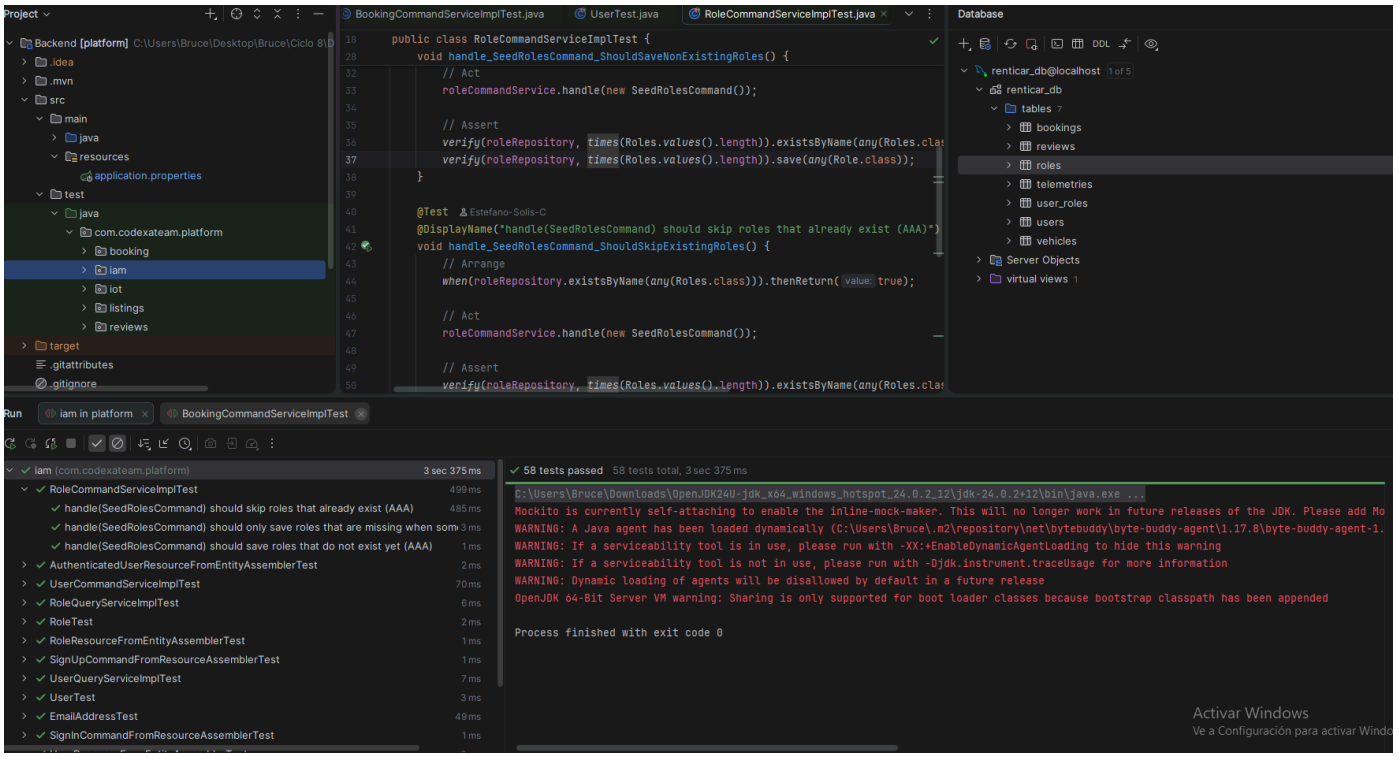
Capítulo VI: Product Verification & Validation

6.1. Testing Suites & Validation

6.1.1. Core Entities Unit Tests.

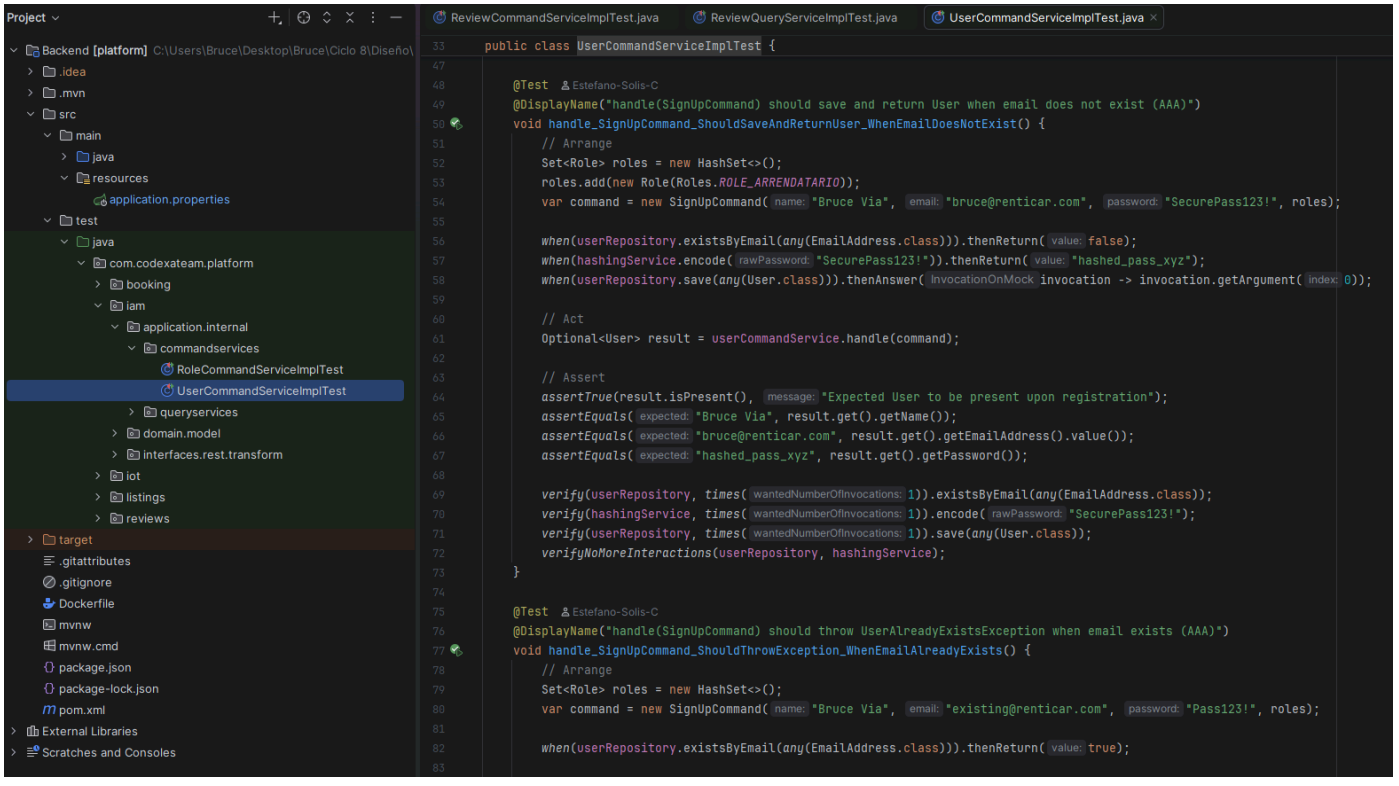
Unit Test en IAM Bounded Context

Dentro del Bounded Context de IAM se realizaron un total de XX pruebas unitarias en base a las funciones establecidas dentro del bounded context

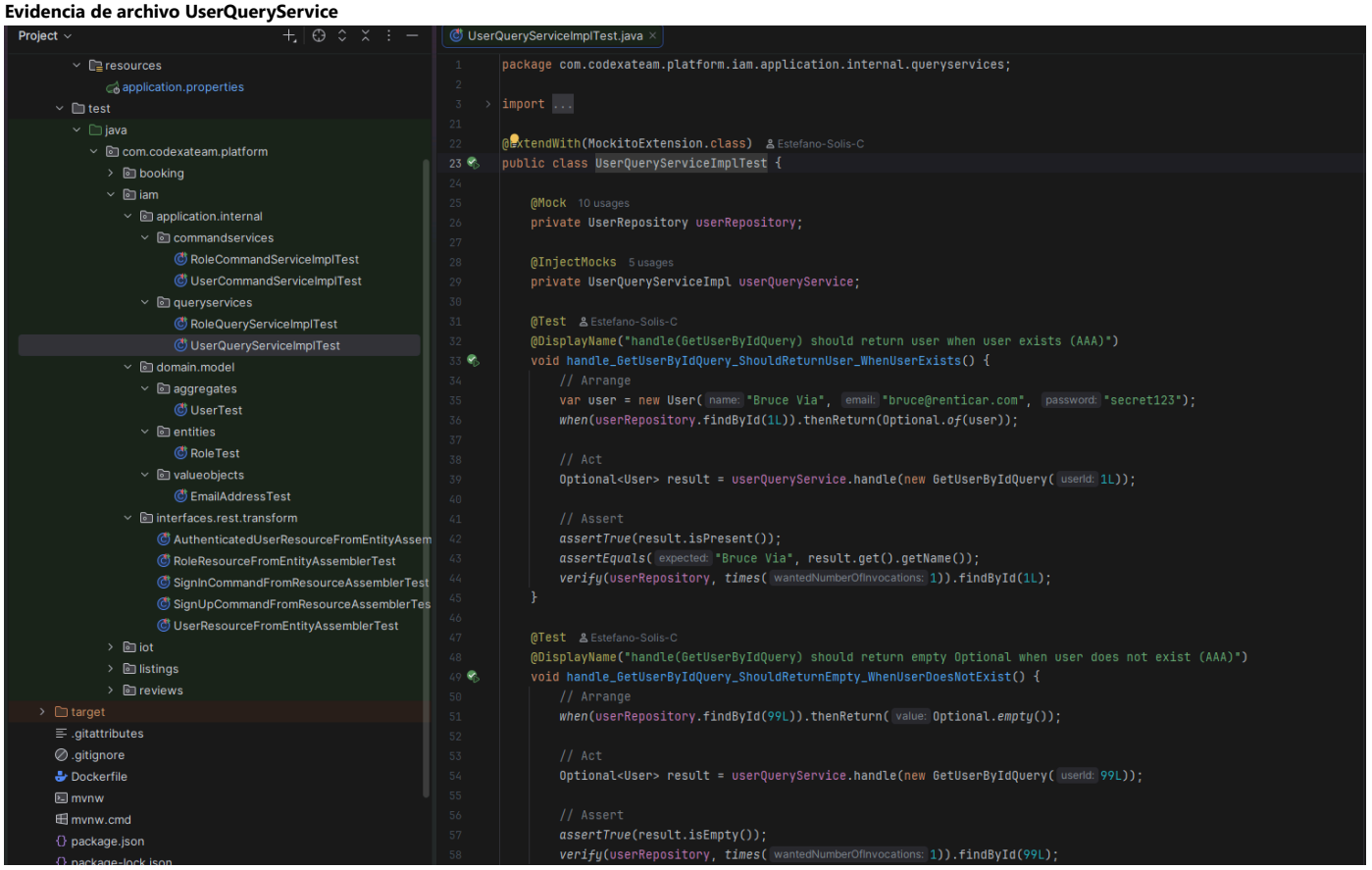


Asi tambien se puede evidenciar como tiene pruebas del tipo Command y Query Service

Evidencia de archivo UserCommandService

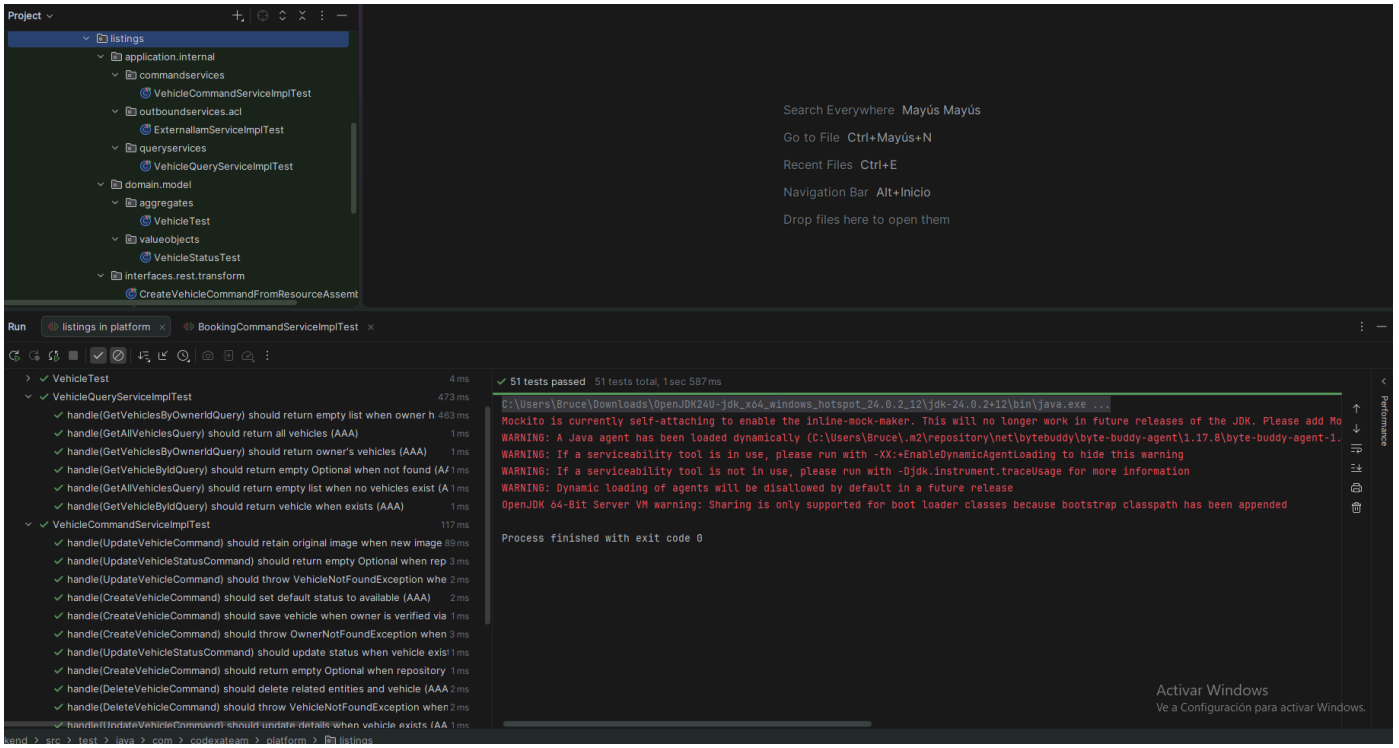


Evidencia de archivo UserQueryService

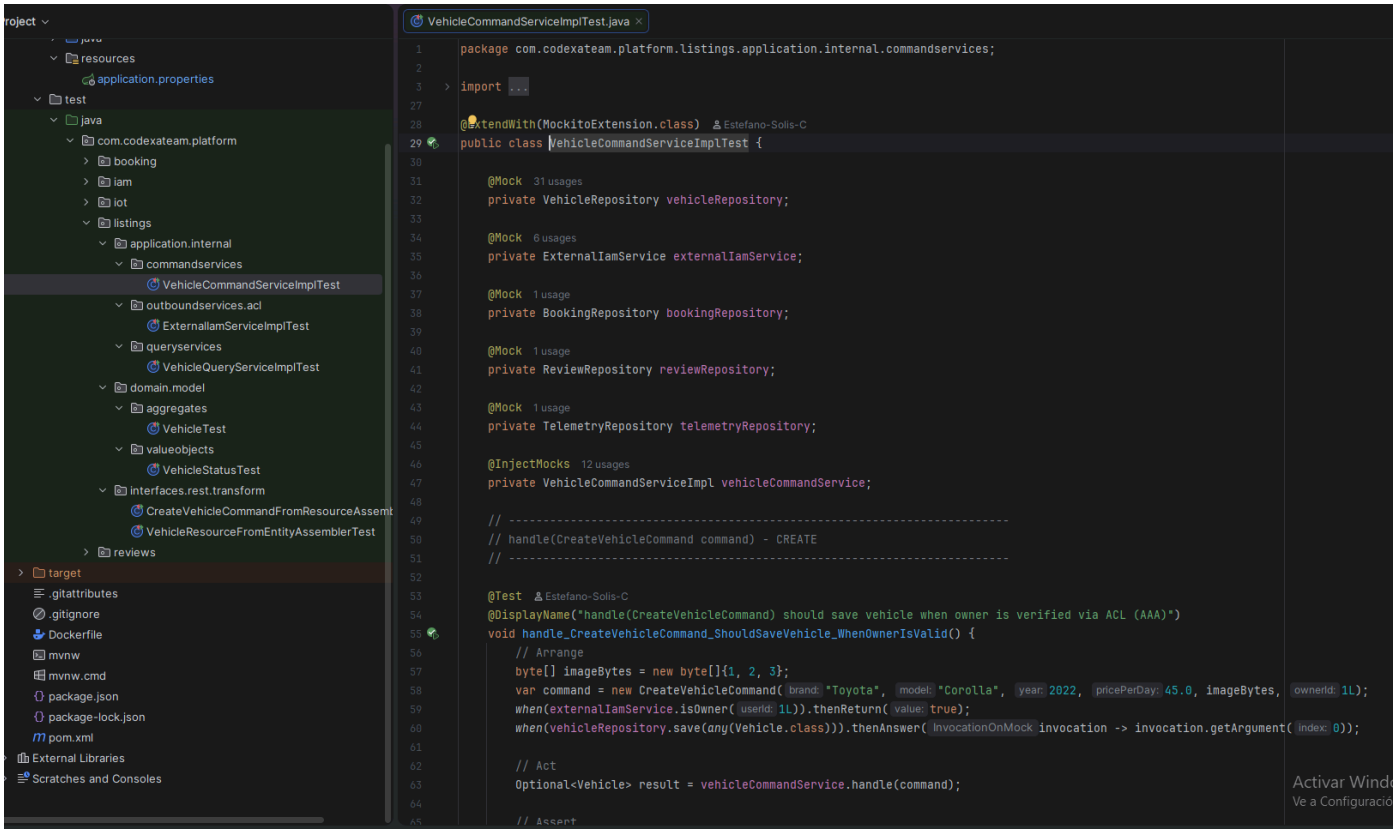


Unit Test en Listings & Fleet Management Bounded Context

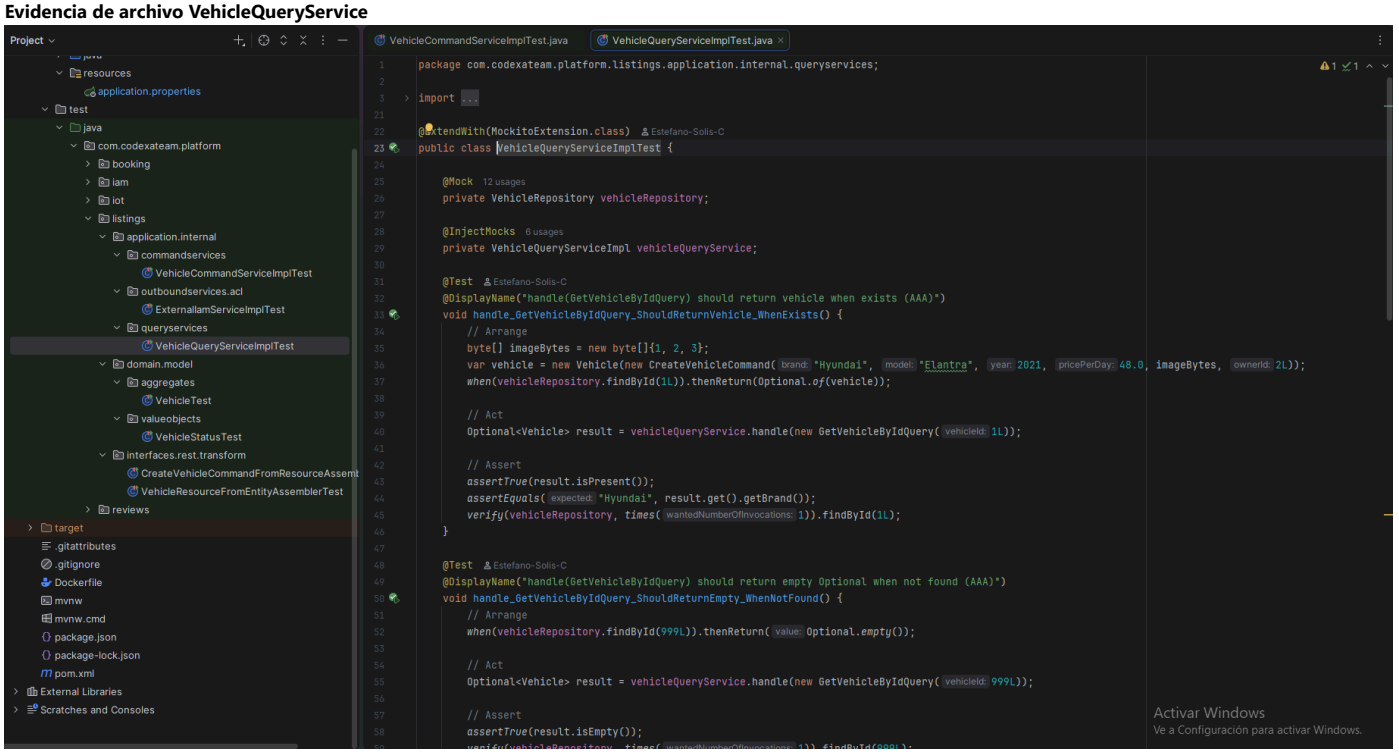
Dentro del Bounded Context de Listing se realizaron un total de XX pruebas unitarias en base a las funciones establecidas dentro del bounded context



Evidencia de archivo VehicleCommandService

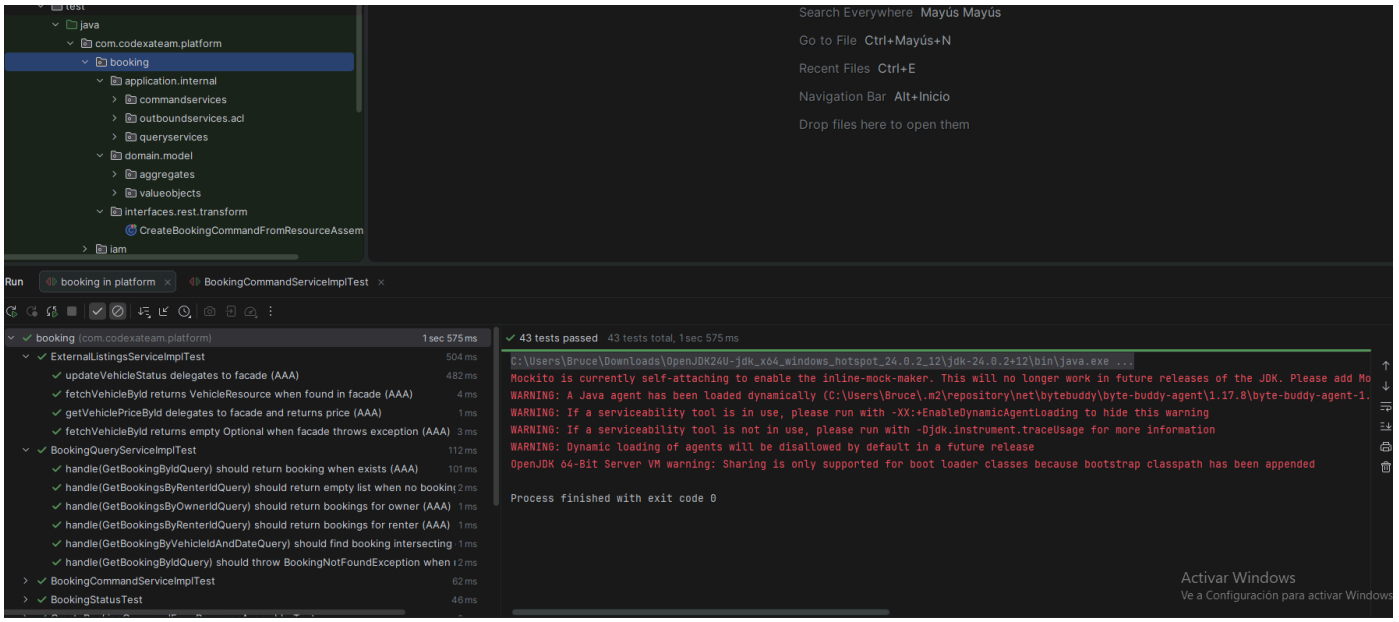


Evidencia de archivo VehicleQueryService

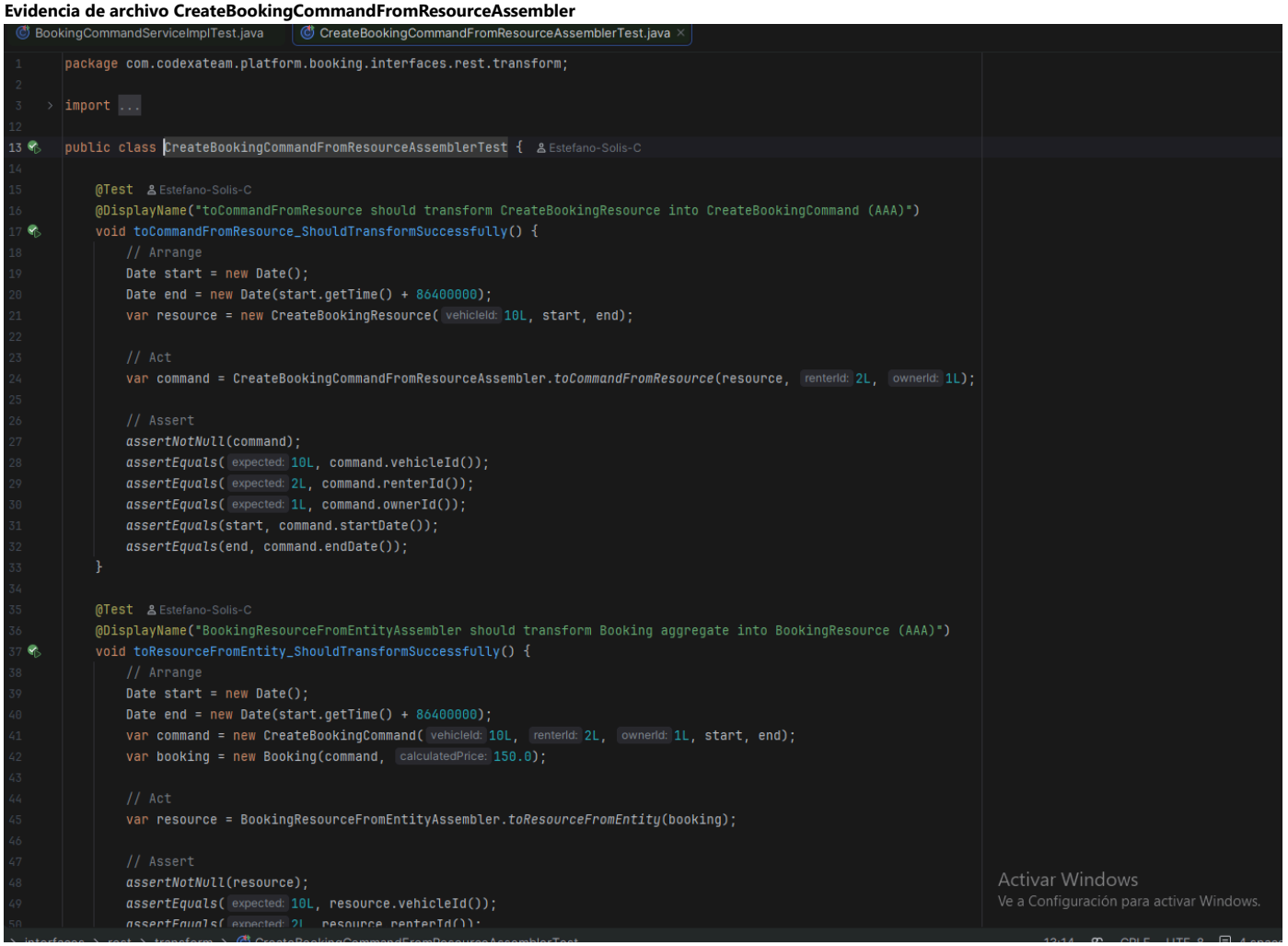


Unit Test en Booking & Digital Contract Bounded Context

Dentro del Bounded Context de Booking se realizaron un total de XX pruebas unitarias en base a las funciones establecidas dentro del bounded context

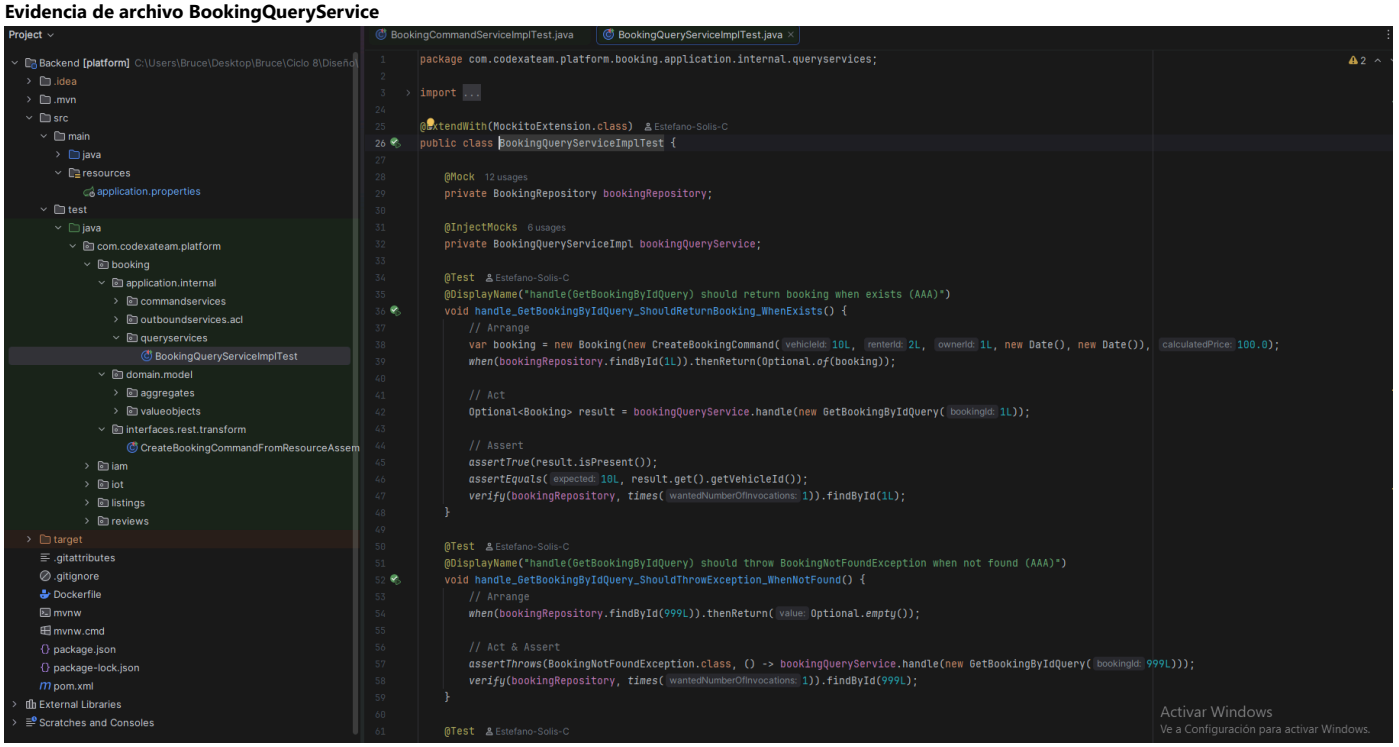


Evidencia de archivo CreateBookingCommandFromResourceAssembler



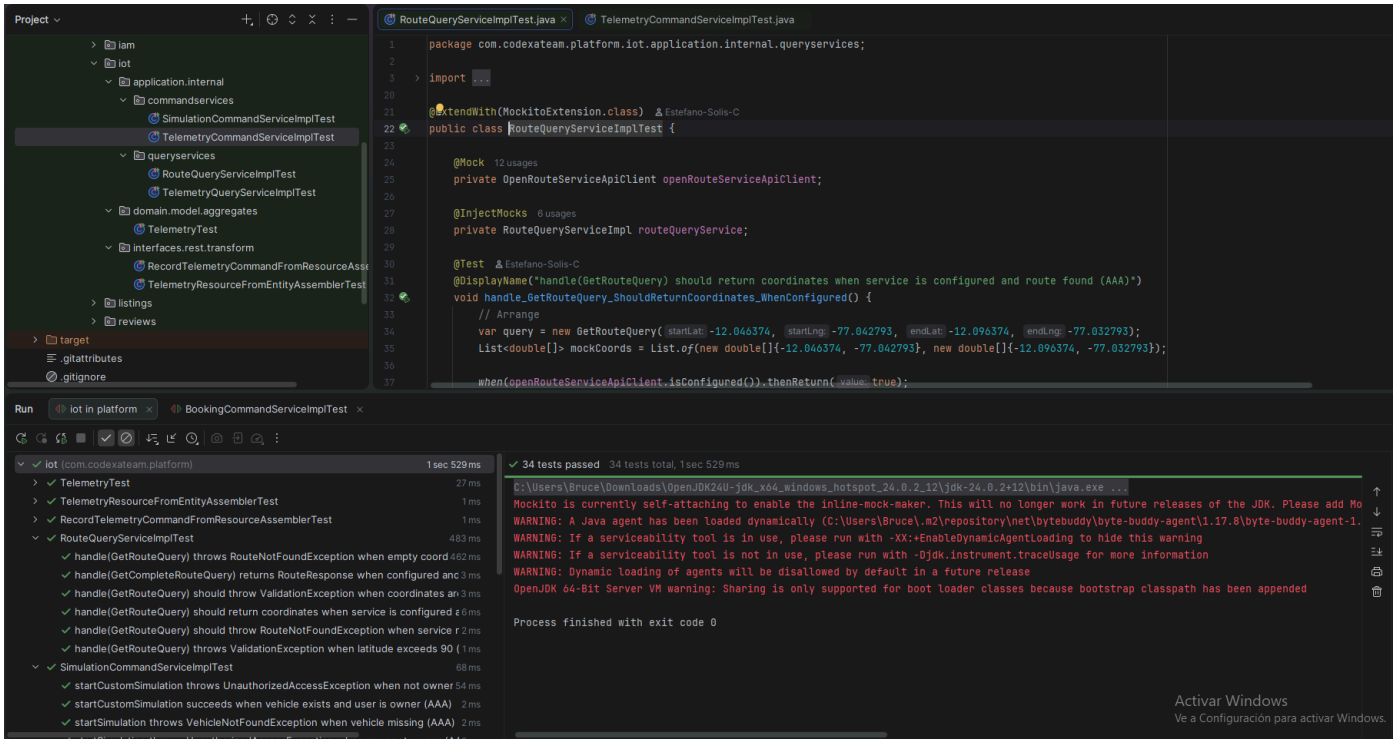

```java
package com.codexateam.platform.booking.interfaces.rest.transform;

import ...

public class CreateBookingCommandFromResourceAssemblerTest {

    @Test
    @DisplayName("toCommandFromResource should transform CreateBookingResource into CreateBookingCommand (AAA)")
    void toCommandFromResource_ShouldTransformSuccessfully() {
        // Arrange
        Date start = new Date();
        Date end = new Date(start.getTime() + 86400000);
        var resource = new CreateBookingResource( vehicleId: 10L, start, end);

        // Act
        var command = CreateBookingCommandFromResourceAssembler.toCommandFromResource(resource, renterId: 2L, ownerId: 1L);

        // Assert
        assertNotNull(command);
        assertEquals( expected: 10L, command.vehicleId());
        assertEquals( expected: 2L, command.renterId());
        assertEquals( expected: 1L, command.ownerId());
        assertEquals(start, command.startDate());
        assertEquals(end, command.endDate());
    }

    @Test
    @DisplayName("BookingResourceFromEntityAssembler should transform Booking aggregate into BookingResource (AAA)")
    void toResourceFromEntity_ShouldTransformSuccessfully() {
        // Arrange
        Date start = new Date();
        Date end = new Date(start.getTime() + 86400000);
        var command = new CreateBookingCommand( vehicleId: 10L, renterId: 2L, ownerId: 1L, start, end);
        var booking = new Booking(command, calculatedPrice: 150.0);

        // Act
        var resource = BookingResourceFromEntityAssembler.toResourceFromEntity(booking);

        // Assert
        assertNotNull(resource);
        assertEquals( expected: 10L, resource.vehicleId());
```

Evidencia de archivo BookingQueryService

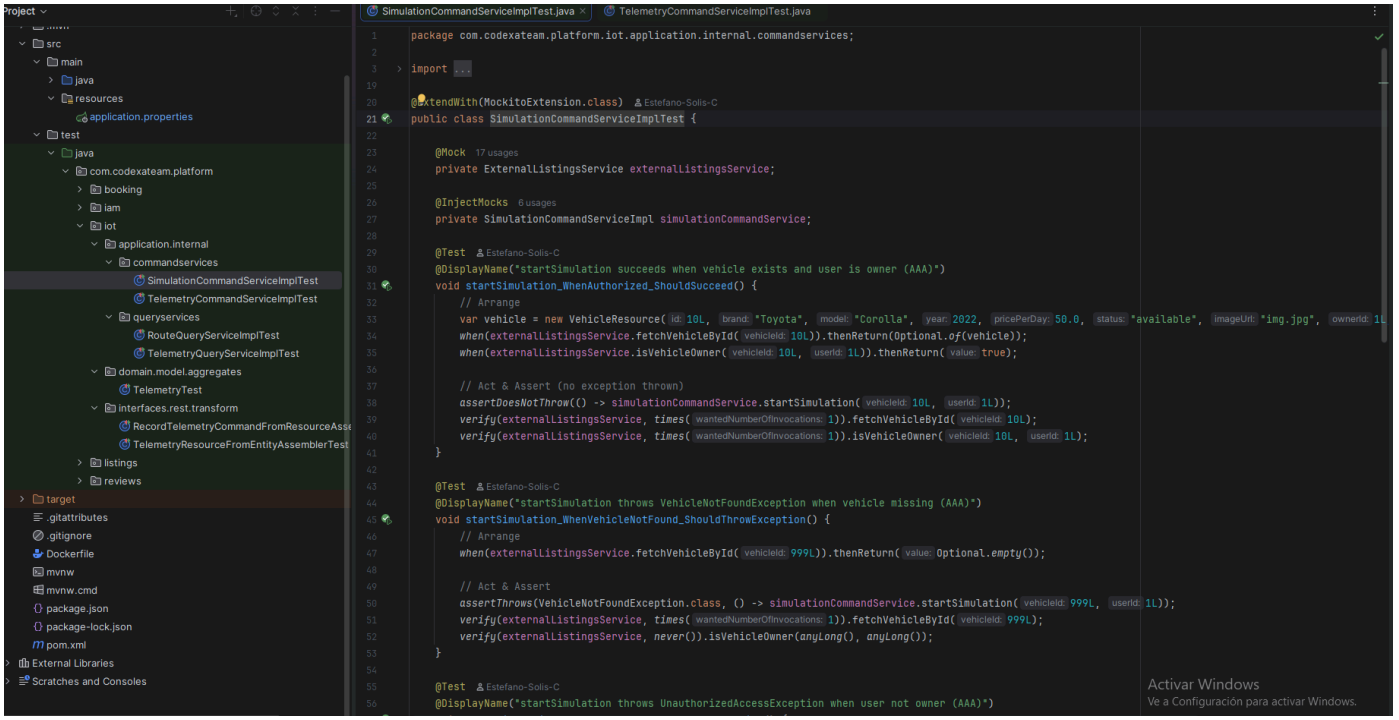


Unit Test en IoT Monitoring & Route Simulation Bounded Context

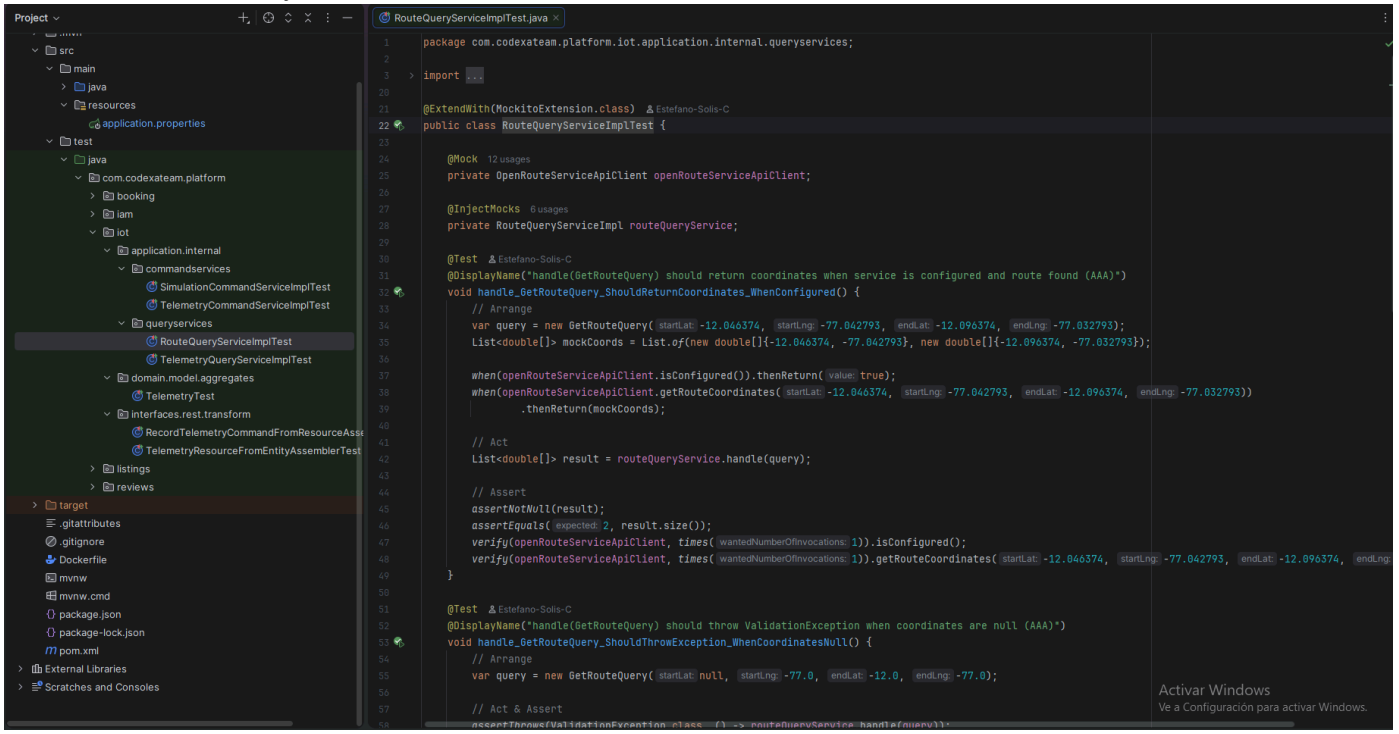
Dentro del Bounded Context de IoT Monitoring se realizaron un total de XX pruebas unitarias en base a las funciones establecidas dentro del bounded context



Evidencia de archivo SimulationCommandService

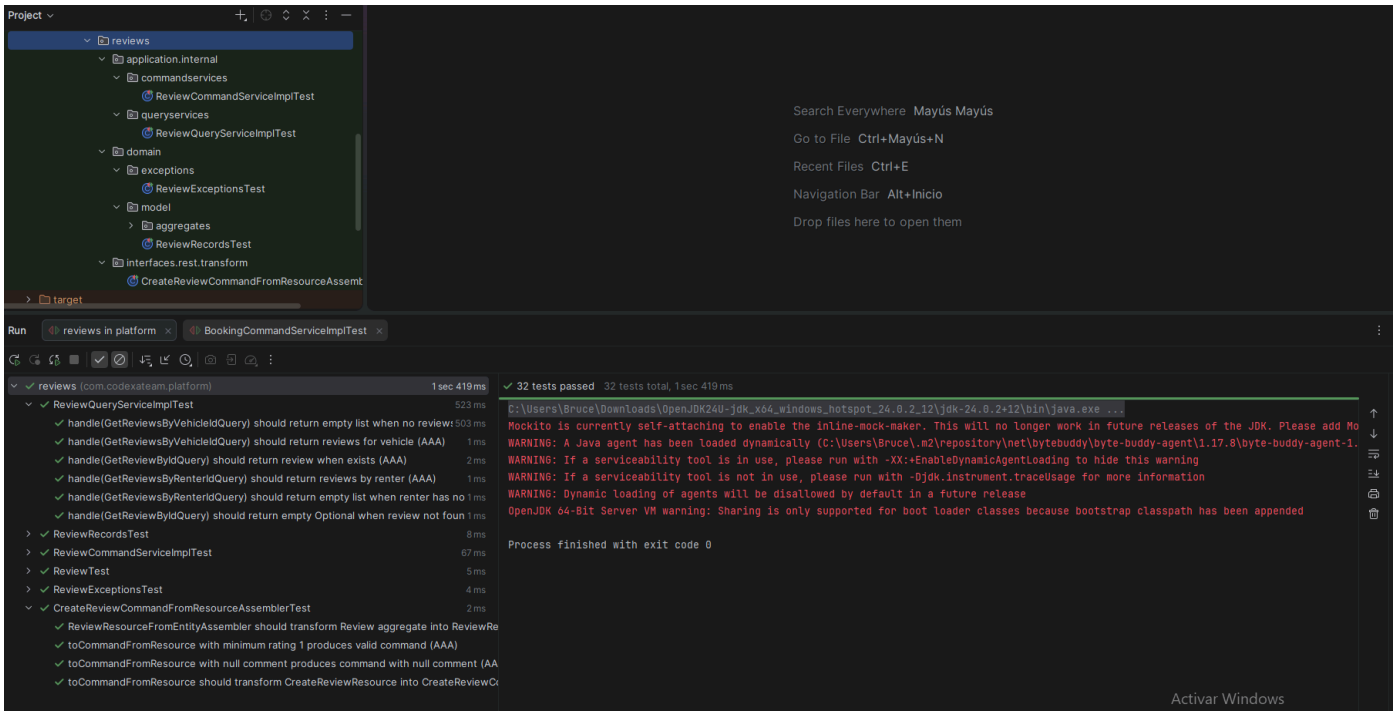


Evidencia de archivo RouteQueryService

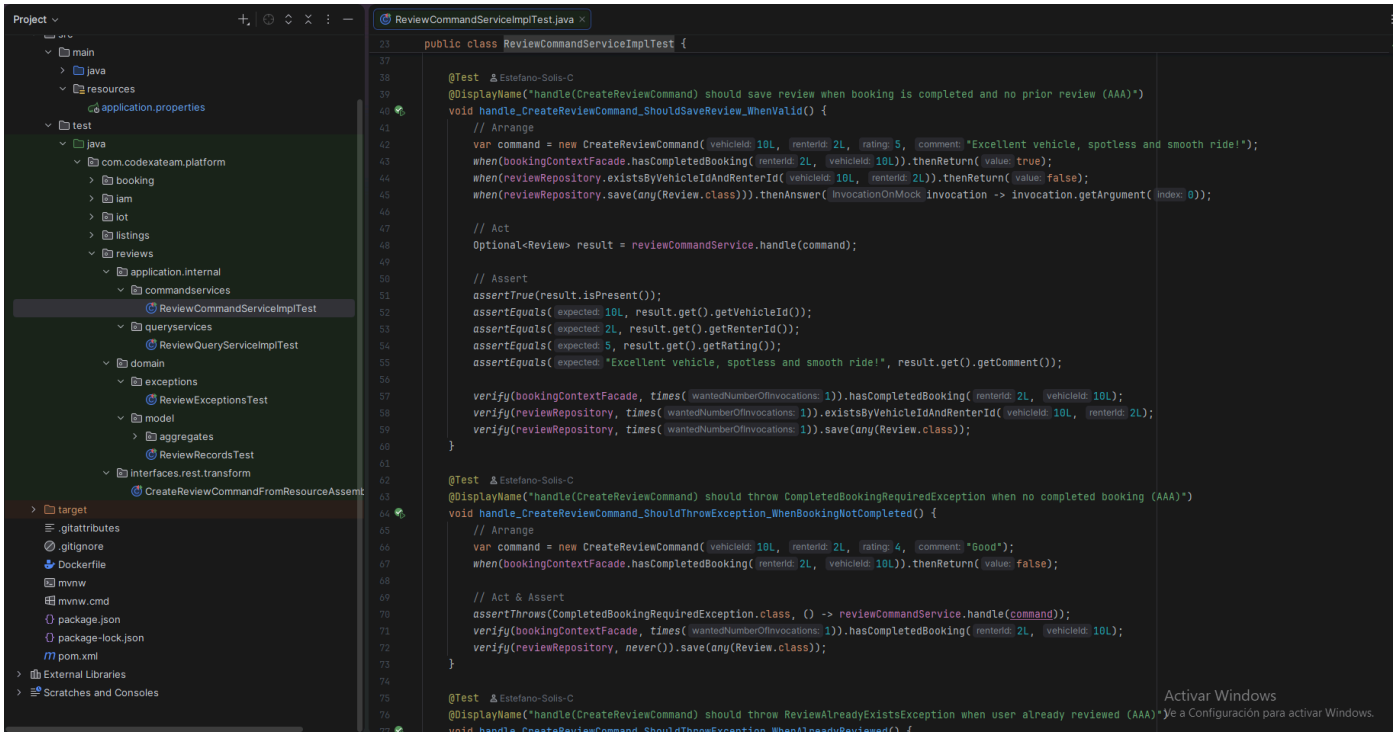


Unit Test en Reviews & Reputation Bounded Context

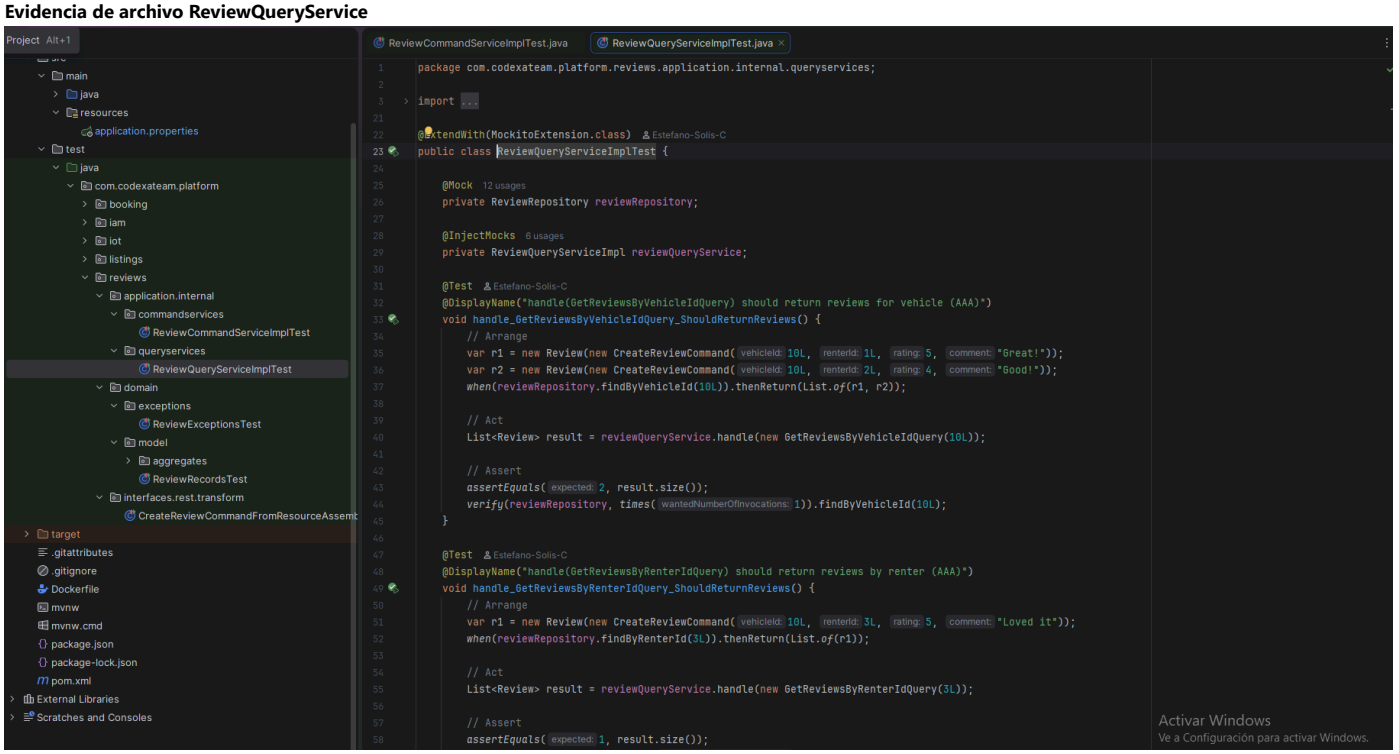
Dentro del Bounded Context de Reviews se realizaron un total de XX pruebas unitarias en base a las funciones establecidas dentro del bounded context



Evidencia de archivo ReviewCommandService



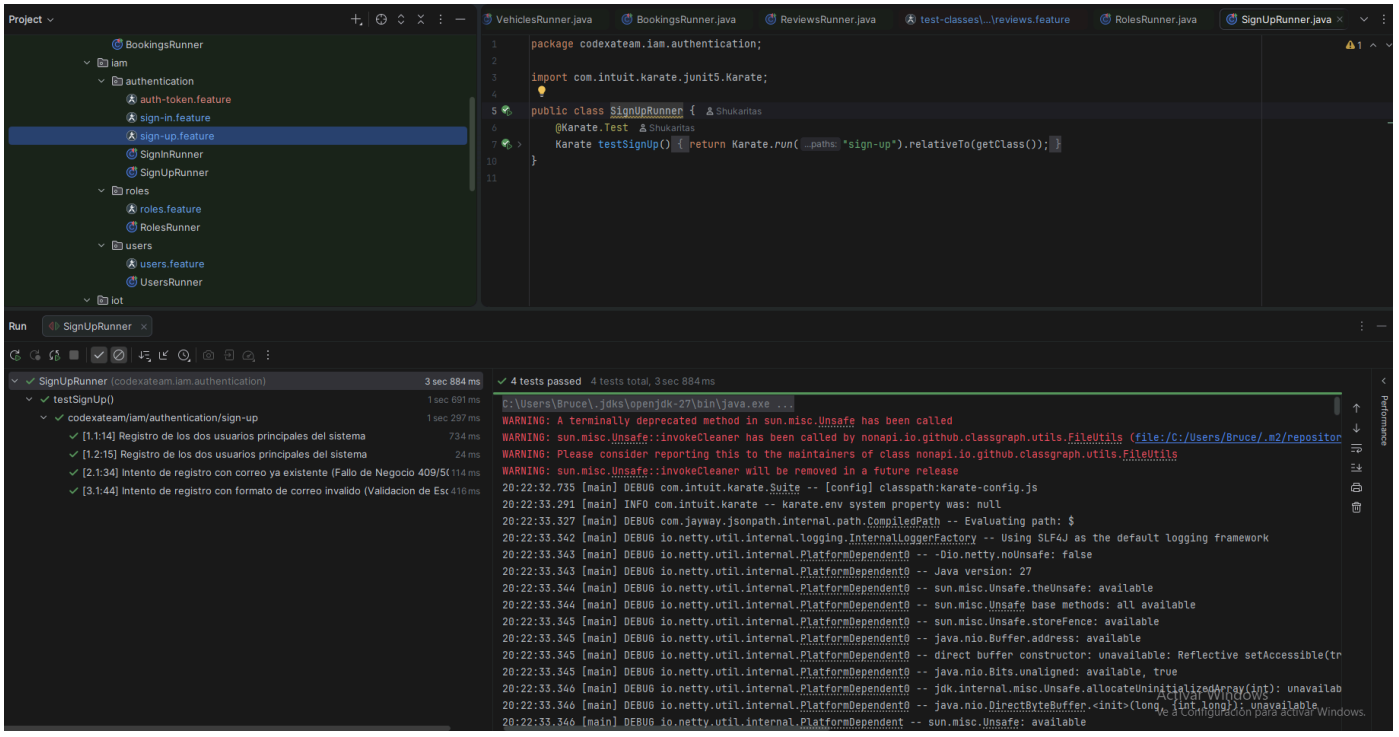
Evidencia de archivo ReviewQueryService

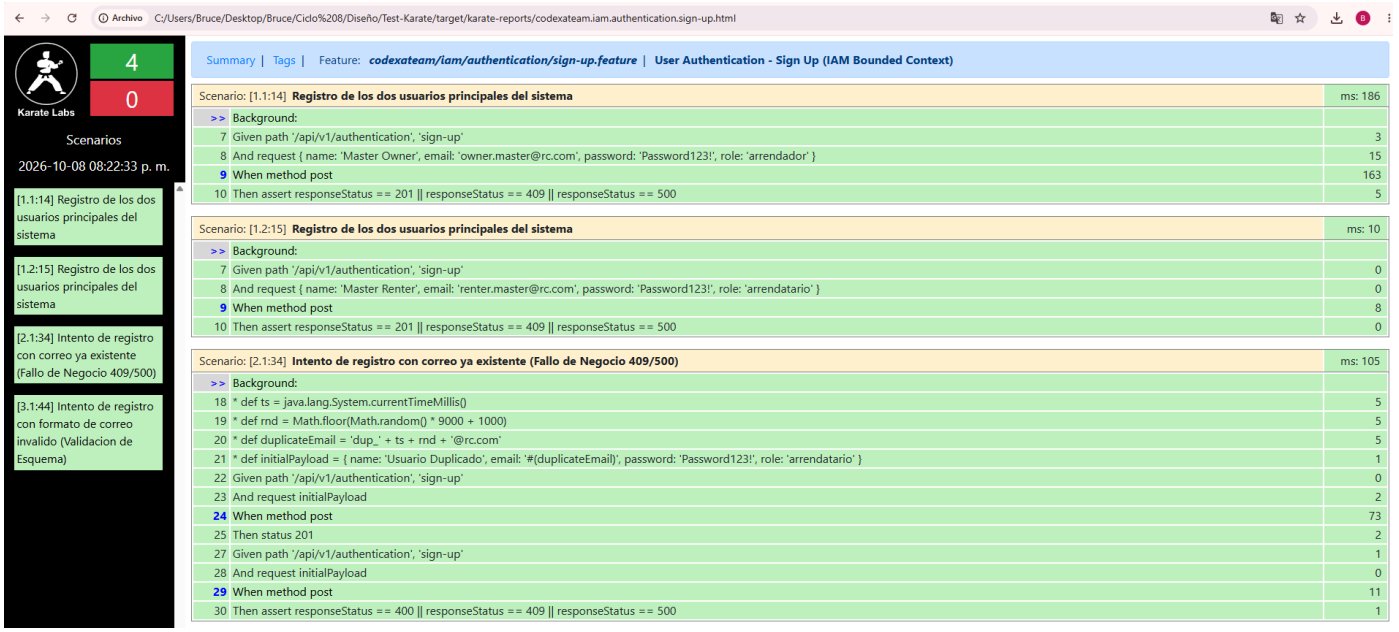


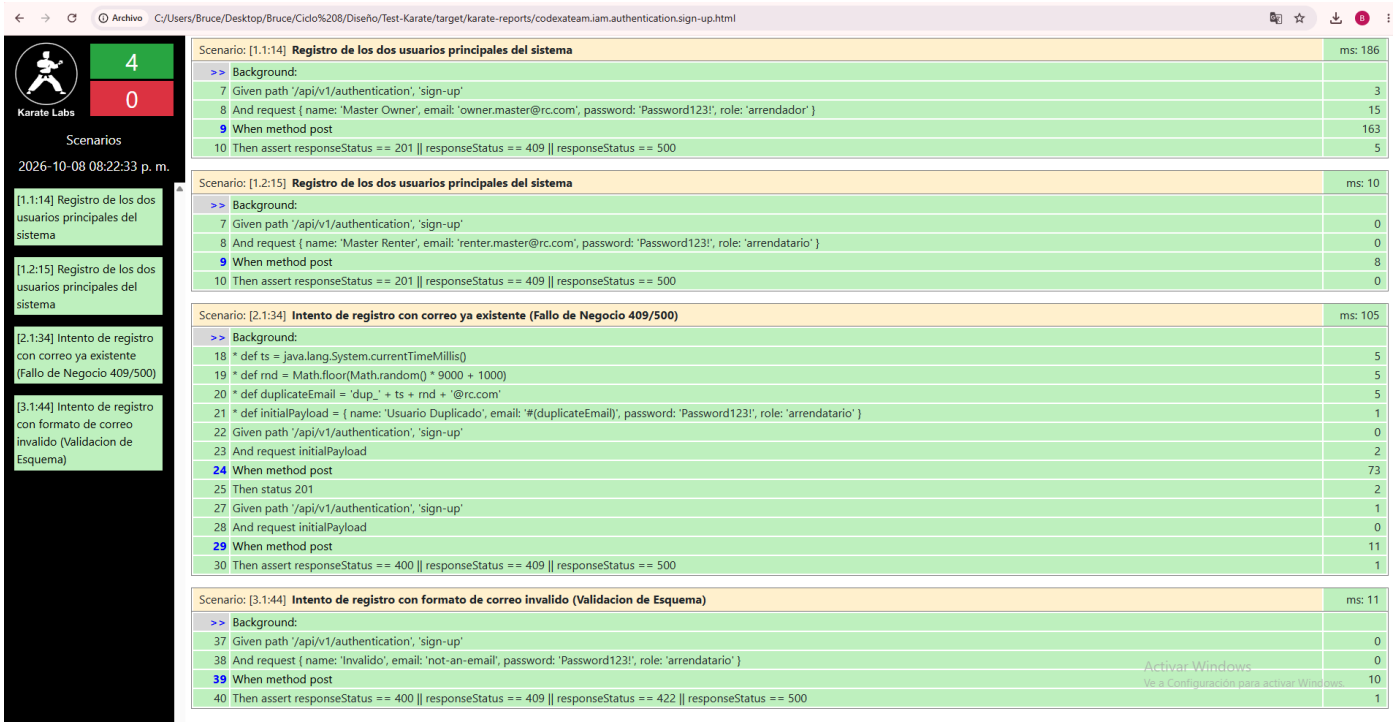
6.1.2. Core Integration Tests.

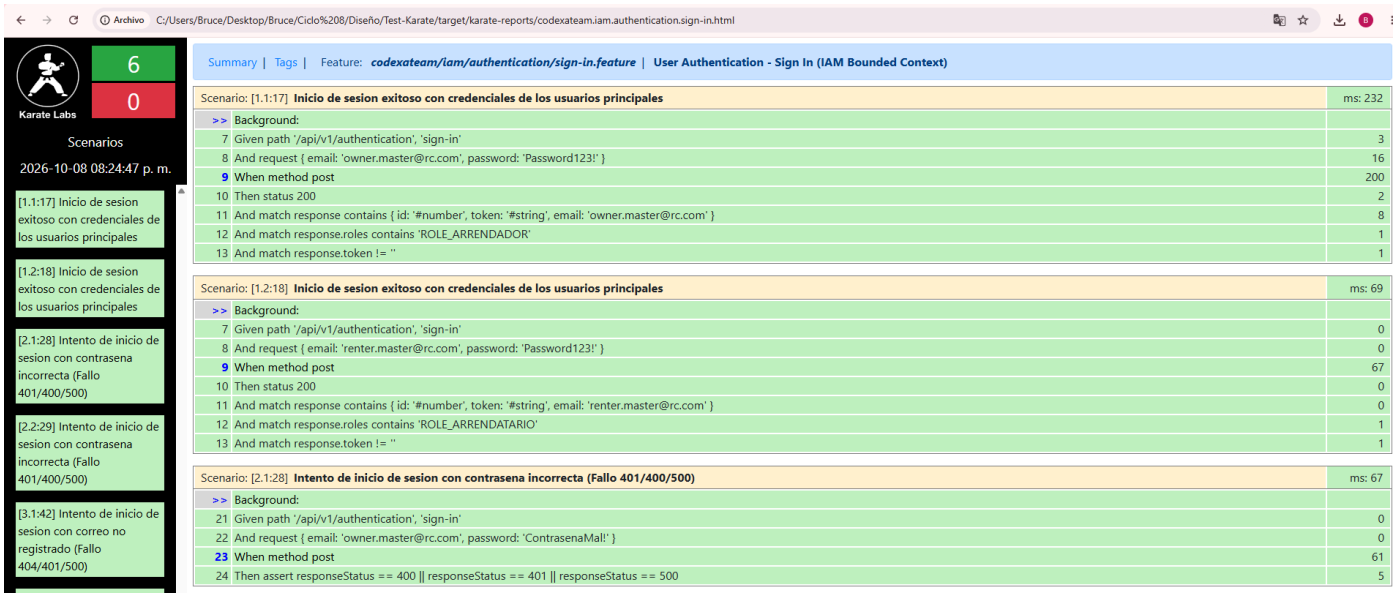
En esta seccion podemos evidenciar las pruebas de intregacion hechas en Karate para los diferentes bounded context del proyecto

Integration Test en IAM Bounded Context

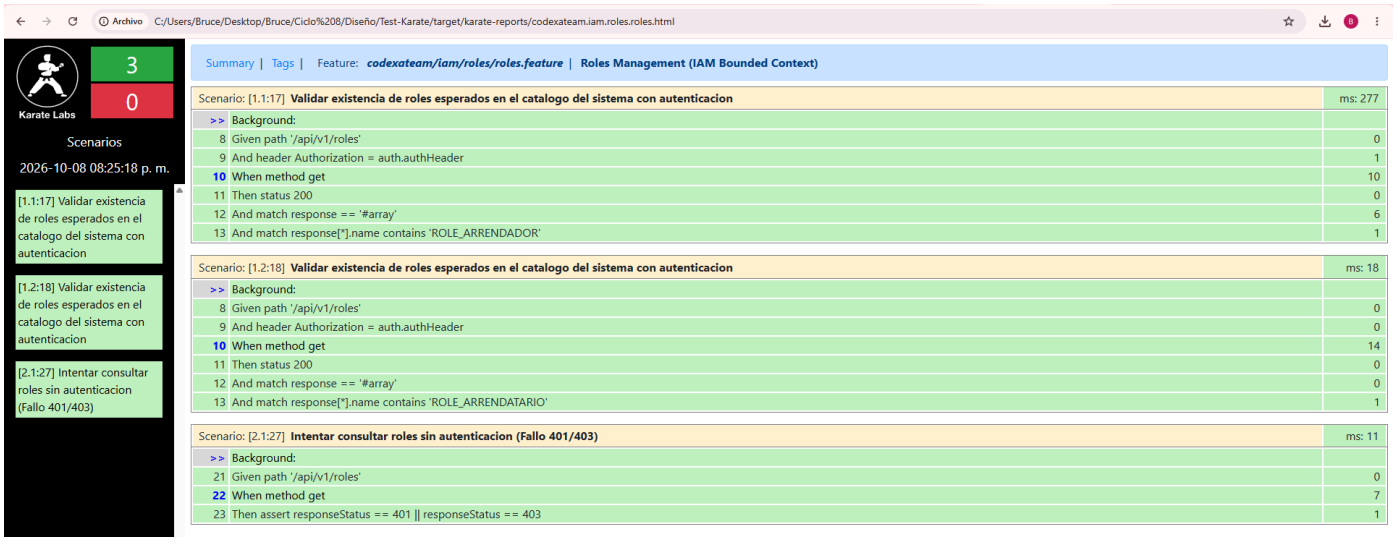


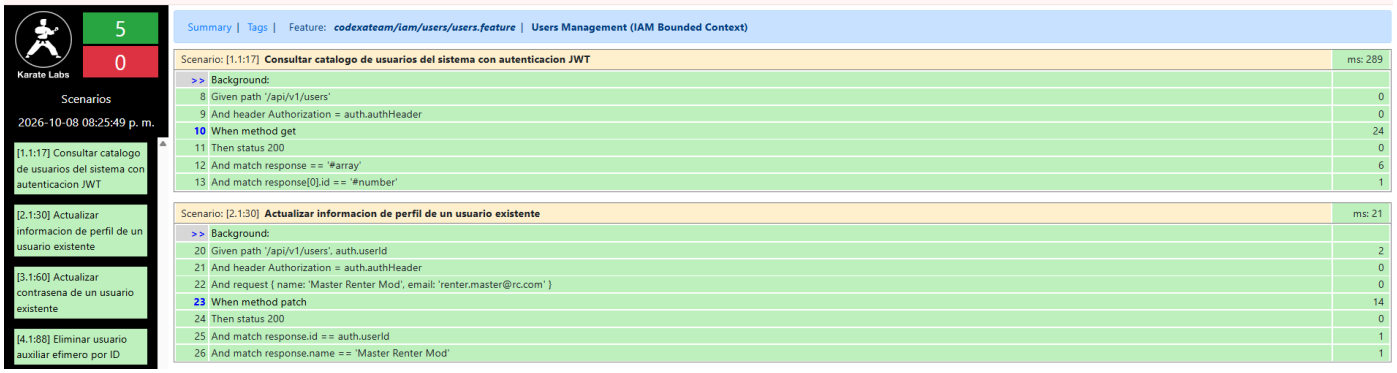

Archivo C:/Users/Bruce/Desktop/Bruce/Ciclo%208/Diseño/Test-Karate/target/karate-reports/codexateam.iam.authentication.sign-up.html
Karate Labs
4
0
Scenarios
2026-10-08 08:22:33 p. m.
[1.1:14] Registro de los dos usuarios principales del sistema
[1.2:15] Registro de los dos usuarios principales del sistema
[2.1:34] Intento de registro con correo ya existente (Fallo de Negocio 409/500)
[3.1:44] Intento de registro con formato de correo invalido (Validacion de Esquema)
Summary | Tags | Feature: codexateam/iam/authentication/sign-up.feature | User Authentication - Sign Up (IAM Bounded Context)
Scenario: [1.1:14] Registro de los dos usuarios principales del sistema ms: 186
>> Background:
7 Given path '/api/v1/authentication', 'sign-up' 3
8 And request { name: 'Master Owner', email: 'owner.master@rc.com', password: 'Password123!', role: 'arrendador' } 15
9 When method post 163
10 Then assert responseStatus == 201 || responseStatus == 409 || responseStatus == 500 5
Scenario: [1.2:15] Registro de los dos usuarios principales del sistema ms: 10
>> Background:
7 Given path '/api/v1/authentication', 'sign-up' 0
8 And request { name: 'Master Renter', email: 'renter.master@rc.com', password: 'Password123!', role: 'arrendatario' } 0
9 When method post 8
10 Then assert responseStatus == 201 || responseStatus == 409 || responseStatus == 500 0
Scenario: [2.1:34] Intento de registro con correo ya existente (Fallo de Negocio 409/500) ms: 105
>> Background:
18 * def ts = java.lang.System.currentTimeMillis() 5
19 * def rnd = Math.floor(Math.random() * 9000 + 1000) 5
20 * def duplicateEmail = 'dup_' + ts + rnd + '@rc.com' 5
21 * def initialPayload = { name: 'Usuario Duplicado', email: '#(duplicateEmail)', password: 'Password123!', role: 'arrendatario' } 1
22 Given path '/api/v1/authentication', 'sign-up' 0
23 And request initialPayload 2
24 When method post 73
25 Then status 201 2
27 Given path '/api/v1/authentication', 'sign-up' 1
28 And request initialPayload 0
29 When method post 11
30 Then assert responseStatus == 400 || responseStatus == 409 || responseStatus == 500 1

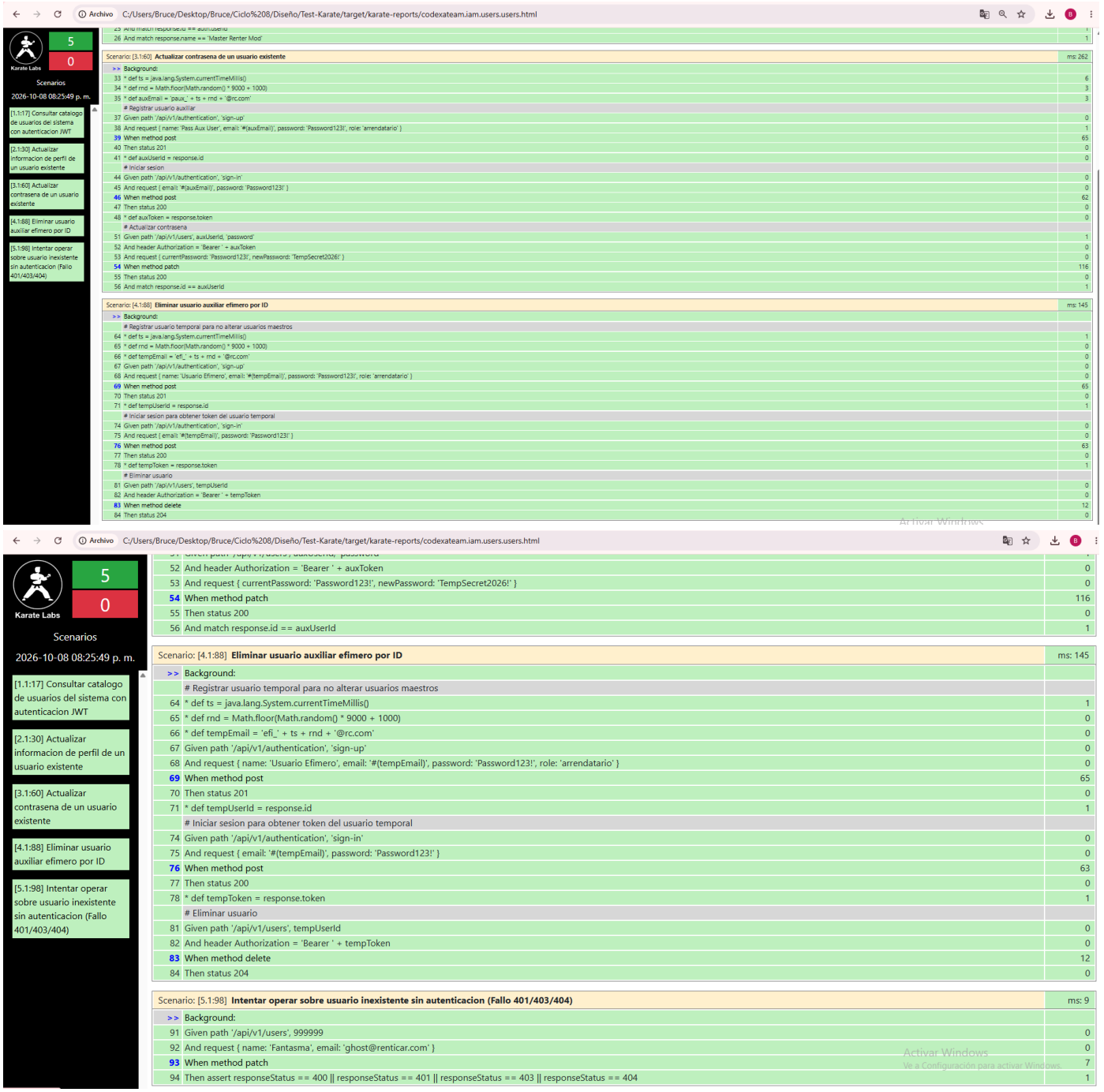

Archivo C:/Users/Bruce/Desktop/Bruce/Ciclo%208/Diseño/Test-Karate/target/karate-reports/codexateam.iam.authentication.sign-up.html
Karate Labs
4
0
Scenarios
2026-10-08 08:22:33 p. m.
[1.1:14] Registro de los dos usuarios principales del sistema
[1.2:15] Registro de los dos usuarios principales del sistema
[2.1:34] Intento de registro con correo ya existente (Fallo de Negocio 409/500)
[3.1:44] Intento de registro con formato de correo invalido (Validacion de Esquema)
Scenario: [1.1:14] Registro de los dos usuarios principales del sistema ms: 186
>> Background:
7 Given path '/api/v1/authentication', 'sign-up' 3
8 And request { name: 'Master Owner', email: 'owner.master@rc.com', password: 'Password123!', role: 'arrendador' } 15
9 When method post 163
10 Then assert responseStatus == 201 || responseStatus == 409 || responseStatus == 500 5
Scenario: [1.2:15] Registro de los dos usuarios principales del sistema ms: 10
>> Background:
7 Given path '/api/v1/authentication', 'sign-up' 0
8 And request { name: 'Master Renter', email: 'renter.master@rc.com', password: 'Password123!', role: 'arrendatario' } 0
9 When method post 8
10 Then assert responseStatus == 201 || responseStatus == 409 || responseStatus == 500 0
Scenario: [2.1:34] Intento de registro con correo ya existente (Fallo de Negocio 409/500) ms: 105
>> Background:
18 * def ts = java.lang.System.currentTimeMillis() 5
19 * def rnd = Math.floor(Math.random() * 9000 + 1000) 5
20 * def duplicateEmail = 'dup_' + ts + rnd + '@rc.com' 5
21 * def initialPayload = { name: 'Usuario Duplicado', email: '#(duplicateEmail)', password: 'Password123!', role: 'arrendatario' } 1
22 Given path '/api/v1/authentication', 'sign-up' 0
23 And request initialPayload 2
24 When method post 73
25 Then status 201 2
27 Given path '/api/v1/authentication', 'sign-up' 1
28 And request initialPayload 0
29 When method post 11
30 Then assert responseStatus == 400 || responseStatus == 409 || responseStatus == 500 1
Scenario: [3.1:44] Intento de registro con formato de correo invalido (Validacion de Esquema) ms: 11
>> Background:
37 Given path '/api/v1/authentication', 'sign-up' 0
38 And request { name: 'Invalido', email: 'not-an-email', password: 'Password123!', role: 'arrendatario' } 0
39 When method post 10
40 Then assert responseStatus == 400 || responseStatus == 409 || responseStatus == 422 || responseStatus == 500 1
Activar Windows
Ve a Configuración para activar Windows.


Archivo C:/Users/Bruce/Desktop/Bruce/Ciclo%208/Diseño/Test-Karate/target/karate-reports/codexateam.iam.authentication.sign-in.html
Karate Labs
6
0
Scenarios
2026-10-08 08:24:47 p. m.
[1.1:17] Inicio de sesion exitoso con credenciales de los usuarios principales
[1.2:18] Inicio de sesion exitoso con credenciales de los usuarios principales
[2.1:28] Intento de inicio de sesion con contrasena incorrecta (Fallo 401/400/500)
[2.2:29] Intento de inicio de sesion con contrasena incorrecta (Fallo 401/400/500)
[3.1:42] Intento de inicio de sesion con correo no registrado (Fallo 404/401/500)
Summary | Tags | Feature: codexateam/iam/authentication/sign-in.feature | User Authentication - Sign In (IAM Bounded Context)
Scenario: [1.1:17] Inicio de sesion exitoso con credenciales de los usuarios principales ms: 232
>> Background:
7 Given path '/api/v1/authentication', 'sign-in' 3
8 And request { email: 'owner.master@rc.com', password: 'Password123!' } 16
9 When method post 200
10 Then status 200 2
11 And match response contains { id: '#number', token: '#string', email: 'owner.master@rc.com' } 8
12 And match response.roles contains 'ROLE_ARRENDADOR' 1
13 And match response.token != '' 1
Scenario: [1.2:18] Inicio de sesion exitoso con credenciales de los usuarios principales ms: 69
>> Background:
7 Given path '/api/v1/authentication', 'sign-in' 0
8 And request { email: 'renter.master@rc.com', password: 'Password123!' } 0
9 When method post 67
10 Then status 200 0
11 And match response contains { id: '#number', token: '#string', email: 'renter.master@rc.com' } 0
12 And match response.roles contains 'ROLE_ARRENDATARIO' 1
13 And match response.token != '' 1
Scenario: [2.1:28] Intento de inicio de sesion con contrasena incorrecta (Fallo 401/400/500) ms: 67
>> Background:
21 Given path '/api/v1/authentication', 'sign-in' 0
22 And request { email: 'owner.master@rc.com', password: 'ContrasenaMal!' } 0
23 When method post 61
24 Then assert responseStatus == 400 || responseStatus == 401 || responseStatus == 500 5

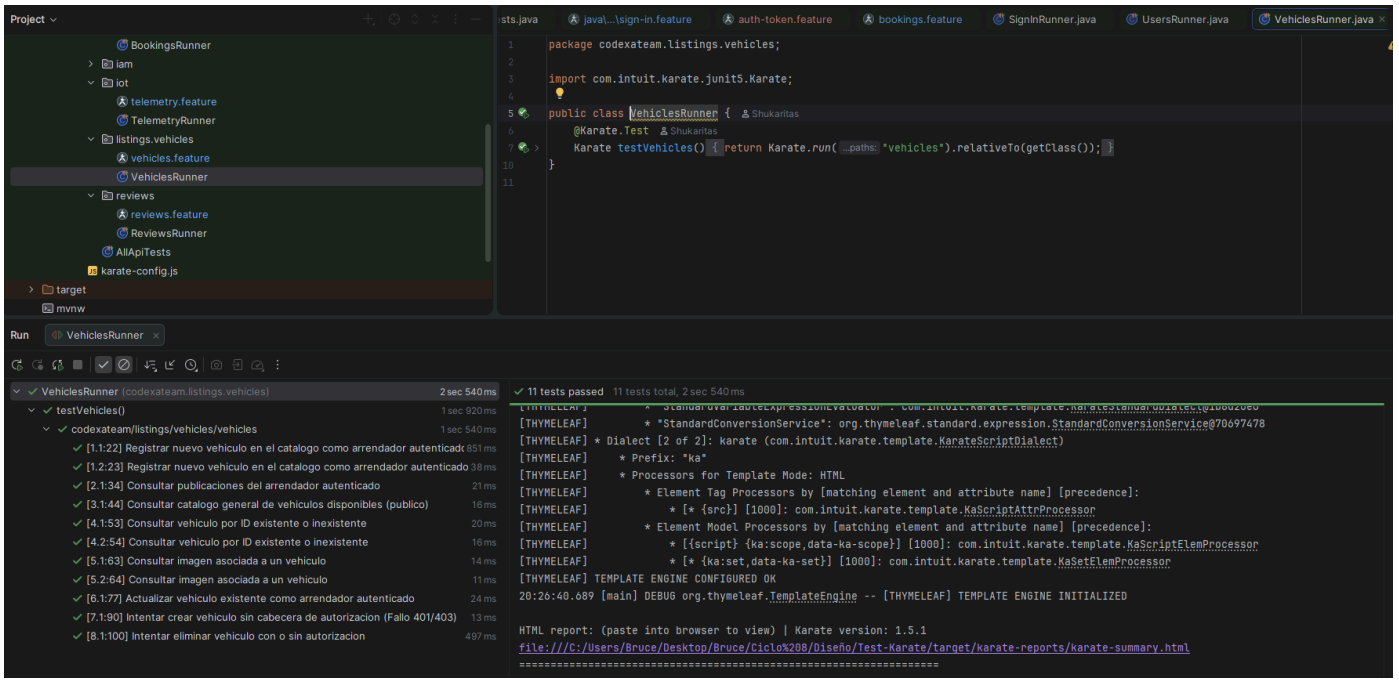
Archivo C:/Users/Bruce/Desktop/Bruce/Ciclo%208/Diseño/Test-Karate/target/karate-reports/codexateam.iam.authentication.sign-in.html
Karate Labs
6
0
Scenarios
2026-10-08 08:24:47 p. m.
[1.1:17] Inicio de sesion exitoso con credenciales de los usuarios principales
[1.2:18] Inicio de sesion exitoso con credenciales de los usuarios principales
[2.1:28] Intento de inicio de sesion con contrasena incorrecta (Fallo 401/400/500)
[2.2:29] Intento de inicio de sesion con contrasena incorrecta (Fallo 401/400/500)
[3.1:42] Intento de inicio de sesion con correo no registrado (Fallo 404/401/500)
[4.1:52] Consultar informacion general del servicio de autenticacion
10 Then status 200 0
11 And match response contains { id: '#number', token: '#string', email: 'renter.master@rc.com' } 0
12 And match response.roles contains 'ROLE_ARRENDATARIO' 1
13 And match response.token != '' 1
Scenario: [2.1:28] Intento de inicio de sesion con contrasena incorrecta (Fallo 401/400/500) ms: 67
>> Background:
21 Given path '/api/v1/authentication', 'sign-in' 0
22 And request { email: 'owner.master@rc.com', password: 'ContrasenaMal!' } 0
23 When method post 61
24 Then assert responseStatus == 400 || responseStatus == 401 || responseStatus == 500 5
Scenario: [2.2:29] Intento de inicio de sesion con contrasena incorrecta (Fallo 401/400/500) ms: 66
>> Background:
21 Given path '/api/v1/authentication', 'sign-in' 0
22 And request { email: 'renter.master@rc.com', password: 'BadPassword99!' } 0
23 When method post 65
24 Then assert responseStatus == 400 || responseStatus == 401 || responseStatus == 500 0
Scenario: [3.1:42] Intento de inicio de sesion con correo no registrado (Fallo 404/401/500) ms: 22
>> Background:
32 * def ts = java.lang.System.currentTimeMillis() 4
33 * def rnd = Math.floor(Math.random() * 9000 + 1000) 4
34 * def nxEmail = 'nx_' + ts + rnd + '@rc.com' 5
35 Given path '/api/v1/authentication', 'sign-in' 0
36 And request { email: '#(nxEmail)', password: 'AnyPassword123' } 1
37 When method post 8
38 Then assert responseStatus == 400 || responseStatus == 401 || responseStatus == 404 || responseStatus == 500 0
Scenario: [4.1:52] Consultar informacion general del servicio de autenticacion ms: 9
>> Background:
45 Given path '/api/v1/authentication' 0
46 When method get 8
47 Then status 200 0
48 And match response.message contains 'Authentication API' 1
Activar Windows
Ve a Configuración para activar Windows.

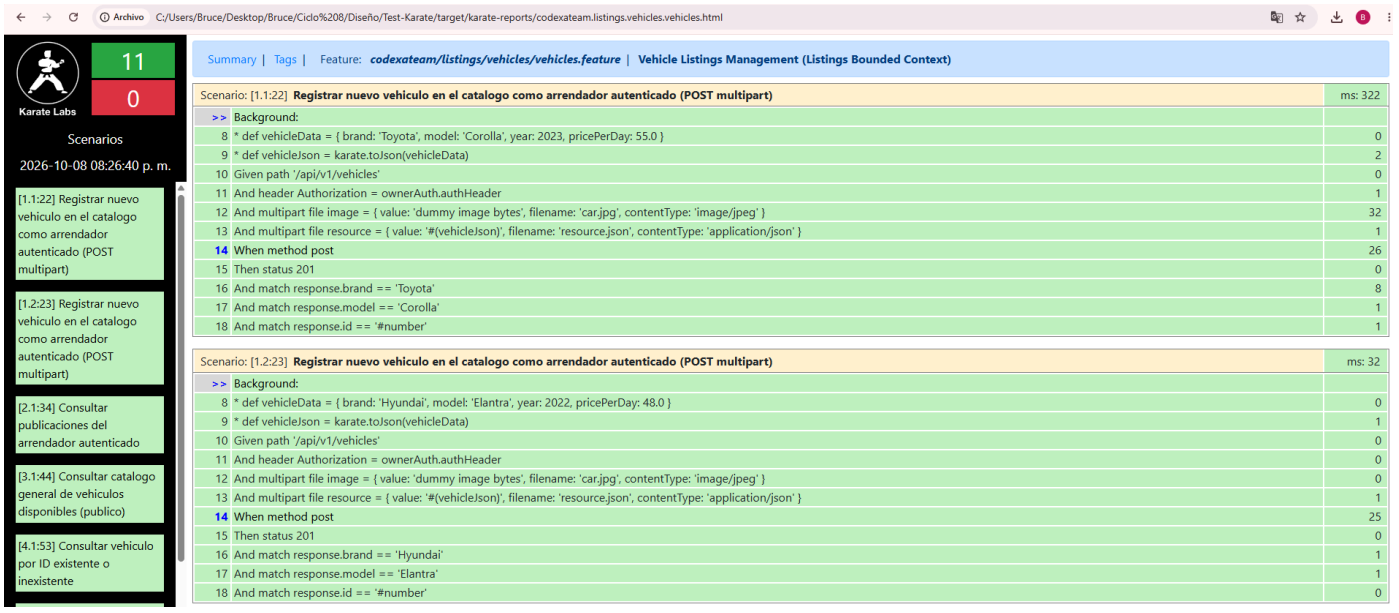

Archivo C:/Users/Bruce/Desktop/Bruce/Ciclo%208/Diseño/Test-Karate/target/karate-reports/codexateam.iam.roles.roles.html
Karate Labs
3
0
Scenarios
2026-10-08 08:25:18 p. m.
[1.1:17] Validar existencia de roles esperados en el catalogo del sistema con autenticacion
[1.2:18] Validar existencia de roles esperados en el catalogo del sistema con autenticacion
[2.1:27] Intentar consultar roles sin autenticacion (Fallo 401/403)
Summary | Tags | Feature: codexateam/iam/roles/roles.feature | Roles Management (IAM Bounded Context)
Scenario: [1.1:17] Validar existencia de roles esperados en el catalogo del sistema con autenticacion ms: 277
>> Background:
8 Given path '/api/v1/roles' 0
9 And header Authorization = auth.authHeader 1
10 When method get 10
11 Then status 200 0
12 And match response == '#array' 6
13 And match response[*].name contains 'ROLE_ARRENDADOR' 1
Scenario: [1.2:18] Validar existencia de roles esperados en el catalogo del sistema con autenticacion ms: 18
>> Background:
8 Given path '/api/v1/roles' 0
9 And header Authorization = auth.authHeader 0
10 When method get 14
11 Then status 200 0
12 And match response == '#array' 0
13 And match response[*].name contains 'ROLE_ARRENDATARIO' 1
Scenario: [2.1:27] Intentar consultar roles sin autenticacion (Fallo 401/403) ms: 11
>> Background:
21 Given path '/api/v1/roles' 0
22 When method get 7
23 Then assert responseStatus == 401 || responseStatus == 403 1


Karate Labs
5
0
Scenarios
2026-10-08 08:25:49 p. m.
[1.1:17] Consultar catalogo de usuarios del sistema con autenticacion JWT
[2.1:30] Actualizar informacion de perfil de un usuario existente
[3.1:60] Actualizar contrasena de un usuario existente
[4.1:88] Eliminar usuario auxiliar efimero por ID
Summary | Tags | Feature: codexateam/iam/users/users.feature | Users Management (IAM Bounded Context)
Scenario: [1.1:17] Consultar catalogo de usuarios del sistema con autenticacion JWT ms: 289
>> Background:
8 Given path '/api/v1/users' 0
9 And header Authorization = auth.authHeader 0
10 When method get 24
11 Then status 200 0
12 And match response == '#array' 6
13 And match response[0].id == '#number' 1
Scenario: [2.1:30] Actualizar informacion de perfil de un usuario existente ms: 21
>> Background:
20 Given path '/api/v1/users', auth.userId 2
21 And header Authorization = auth.authHeader 0
22 And request { name: 'Master Renter Mod', email: 'renter.master@rc.com' } 0
23 When method patch 14
24 Then status 200 0
25 And match response.id == auth.userId 1
26 And match response.name == 'Master Renter Mod' 1

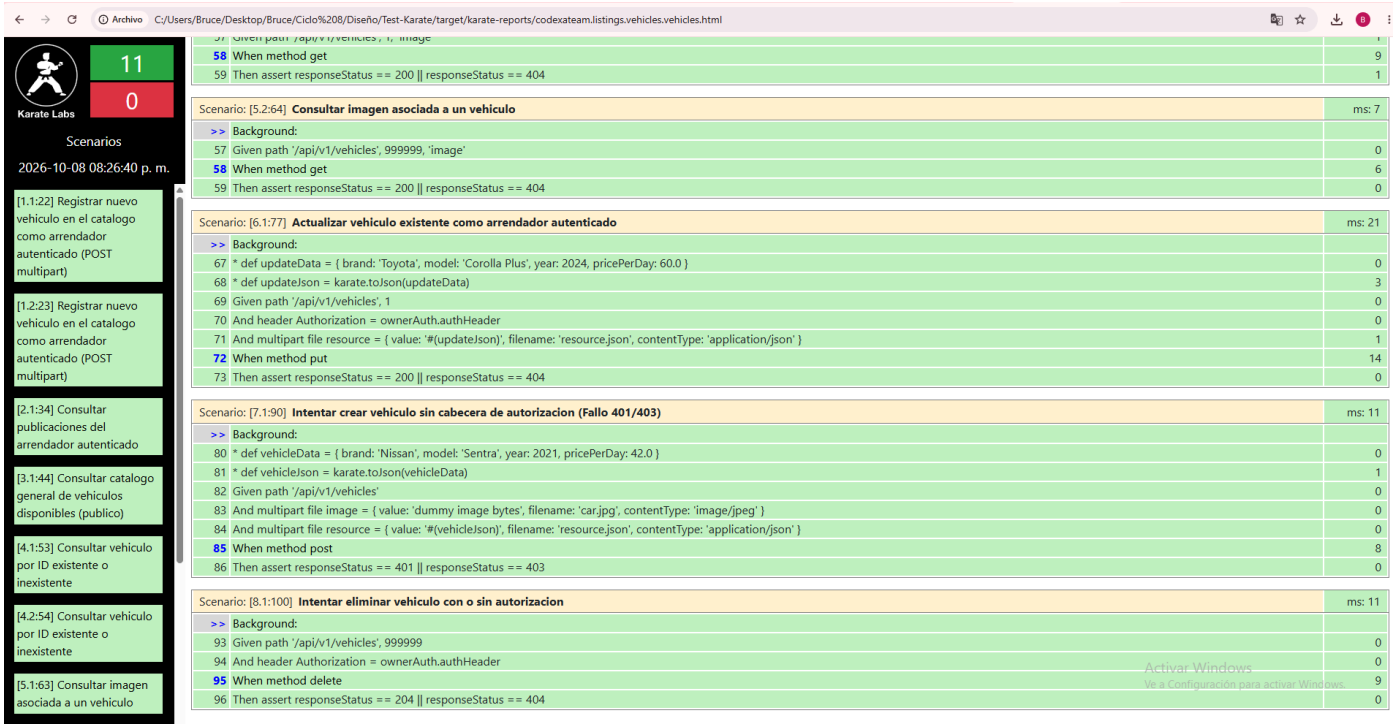


Integration Test en Listings & Fleet Management Bounded Context

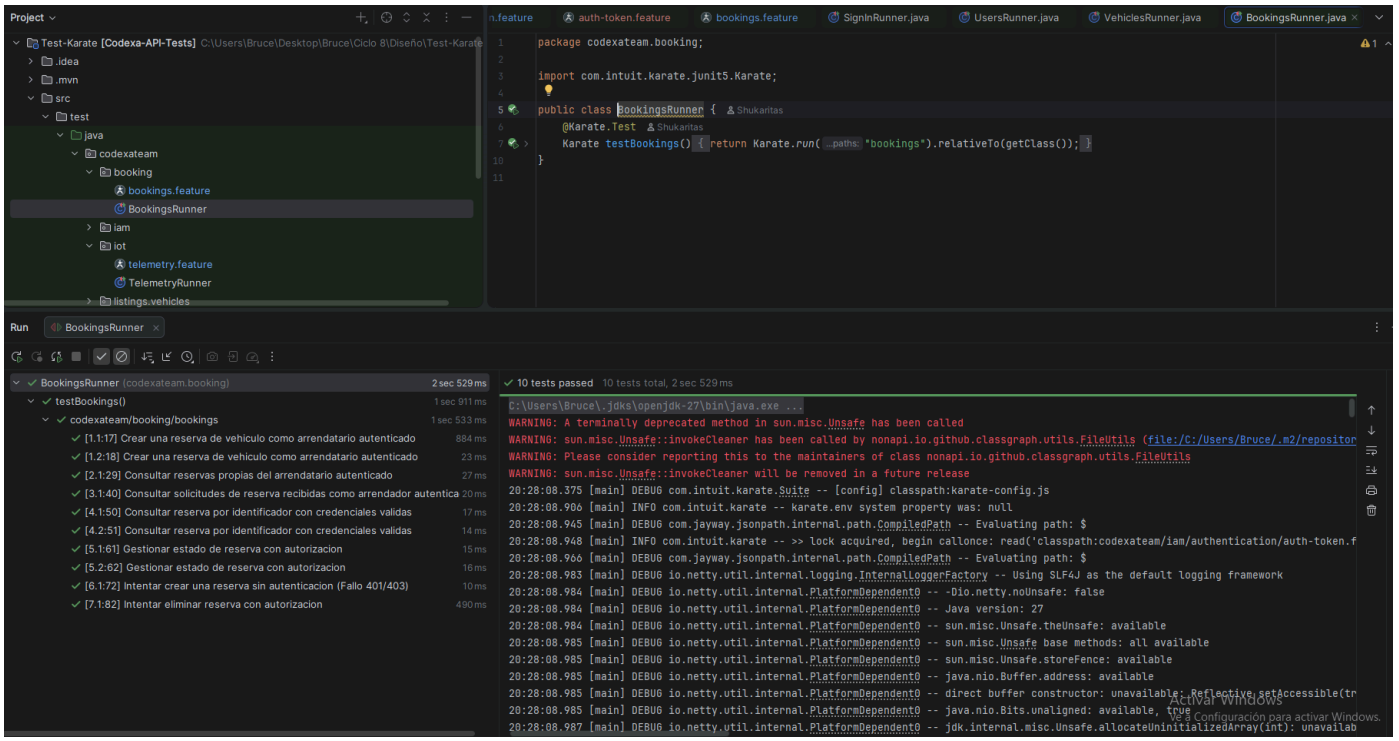

Project
BookingsRunner
iam
iot
telemetry.feature
TelemetryRunner
listings.vehicles
vehicles.feature
VehiclesRunner
reviews
reviews.feature
ReviewsRunner
AllApiTests
karate-config.js
target
mvnw
sts.java
java\...\sign-in.feature
auth-token.feature
bookings.feature
SignInRunner.java
UsersRunner.java
VehiclesRunner.java
package codexateam.listings.vehicles;
import com.intuit.karate.junit5.Karate;
public class VehiclesRunner {
@Karate.Test
Karate testVehicles() { return Karate.run("vehicles").relativeTo(getClass()); }
}
Run
VehiclesRunner
VehiclesRunner (codexateam.listings.vehicles) 2 sec 540 ms
testVehicles() 1 sec 920 ms
codexateam/listings/vehicles/vehicles 1 sec 540 ms
[1.1:22] Registrar nuevo vehiculo en el catalogo como arrendador autenticad 851 ms
[1.2:23] Registrar nuevo vehiculo en el catalogo como arrendador autenticado 38 ms
[2.1:34] Consultar publicaciones del arrendador autenticado 21 ms
[3.1:44] Consultar catalogo general de vehiculos disponibles (publico) 16 ms
[4.1:53] Consultar vehiculo por ID existente o inexistente 20 ms
[4.2:54] Consultar vehiculo por ID existente o inexistente 16 ms
[5.1:63] Consultar imagen asociada a un vehiculo 14 ms
[5.2:64] Consultar imagen asociada a un vehiculo 11 ms
[6.1:77] Actualizar vehiculo existente como arrendador autenticado 24 ms
[7.1:90] Intentar crear vehiculo sin cabecera de autorizacion (Fallo 401/403) 13 ms
[8.1:100] Intentar eliminar vehiculo con o sin autorizacion 497 ms
11 tests passed 11 tests total, 2 sec 540 ms
[THYMELEAF] * "StandardConversionService": org.thymeleaf.standard.expression.StandardConversionService@70697478
[THYMELEAF] * Dialect [2 of 2]: karate (com.intuit.karate.template.KarateScriptDialect)
[THYMELEAF] * Prefix: "ka"
[THYMELEAF] * Processors for Template Mode: HTML
[THYMELEAF] * Element Tag Processors by [matching element and attribute name] [precedence]:
[THYMELEAF] * [* {src}] [1000]: com.intuit.karate.template.KaScriptAttrProcessor
[THYMELEAF] * Element Model Processors by [matching element and attribute name] [precedence]:
[THYMELEAF] * [{script} {ka:scope,data-ka-scope}] [1000]: com.intuit.karate.template.KaScriptElemProcessor
[THYMELEAF] * [* {ka:set,data-ka-set}] [1000]: com.intuit.karate.template.KaSetElemProcessor
[THYMELEAF] TEMPLATE ENGINE CONFIGURED OK
20:20:40.689 [main] DEBUG org.thymeleaf.TemplateEngine -- [THYMELEAF] TEMPLATE ENGINE INITIALIZED
HTML report: (paste into browser to view) | Karate version: 1.5.1
file:///C:/Users/Bruce/Desktop/Bruce/Ciclo%208/Diseño/Test-Karate/target/karate-reports/karate-summary.html

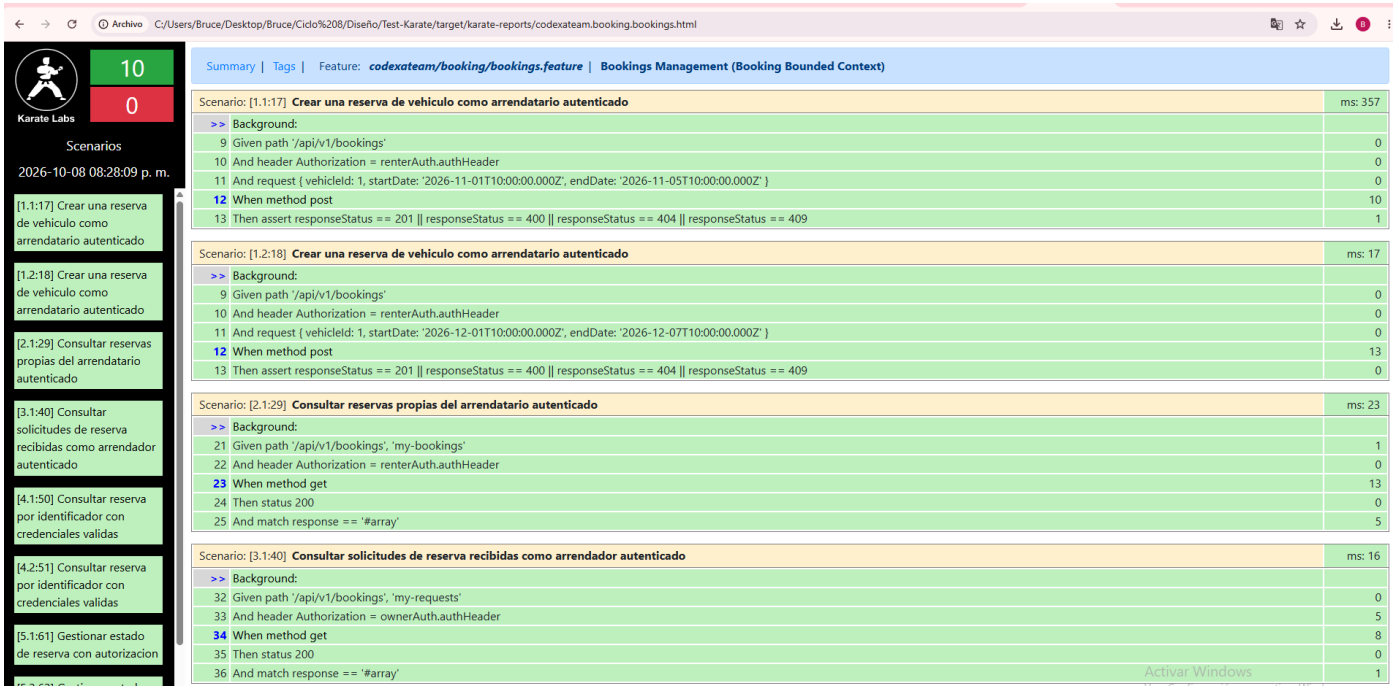

Archivo C:/Users/Bruce/Desktop/Bruce/Ciclo%208/Diseño/Test-Karate/target/karate-reports/codexateam.listings.vehicles.vehicles.html
11
0
Karate Labs
Scenarios
2026-10-08 08:26:40 p. m.
[1.1:22] Registrar nuevo vehiculo en el catalogo como arrendador autenticado (POST multipart)
[1.2:23] Registrar nuevo vehiculo en el catalogo como arrendador autenticado (POST multipart)
[2.1:34] Consultar publicaciones del arrendador autenticado
[3.1:44] Consultar catalogo general de vehiculos disponibles (publico)
[4.1:53] Consultar vehiculo por ID existente o inexistente
Summary | Tags | Feature: codexateam/listings/vehicles/vehicles.feature | Vehicle Listings Management (Listings Bounded Context)
Scenario: [1.1:22] Registrar nuevo vehiculo en el catalogo como arrendador autenticado (POST multipart) ms: 322
>> Background:
8 * def vehicleData = { brand: 'Toyota', model: 'Corolla', year: 2023, pricePerDay: 55.0 } 0
9 * def vehicleJson = karate.toJson(vehicleData) 2
10 Given path '/api/v1/vehicles' 0
11 And header Authorization = ownerAuth.authHeader 1
12 And multipart file image = { value: 'dummy image bytes', filename: 'car.jpg', contentType: 'image/jpeg' } 32
13 And multipart file resource = { value: '#(vehicleJson)', filename: 'resource.json', contentType: 'application/json' } 1
14 When method post 26
15 Then status 201 0
16 And match response.brand == 'Toyota' 8
17 And match response.model == 'Corolla' 1
18 And match response.id == '#number' 1
Scenario: [1.2:23] Registrar nuevo vehiculo en el catalogo como arrendador autenticado (POST multipart) ms: 32
>> Background:
8 * def vehicleData = { brand: 'Hyundai', model: 'Elantra', year: 2022, pricePerDay: 48.0 } 0
9 * def vehicleJson = karate.toJson(vehicleData) 1
10 Given path '/api/v1/vehicles' 0
11 And header Authorization = ownerAuth.authHeader 0
12 And multipart file image = { value: 'dummy image bytes', filename: 'car.jpg', contentType: 'image/jpeg' } 0
13 And multipart file resource = { value: '#(vehicleJson)', filename: 'resource.json', contentType: 'application/json' } 1
14 When method post 25
15 Then status 201 0
16 And match response.brand == 'Hyundai' 1
17 And match response.model == 'Elantra' 1
18 And match response.id == '#number' 0

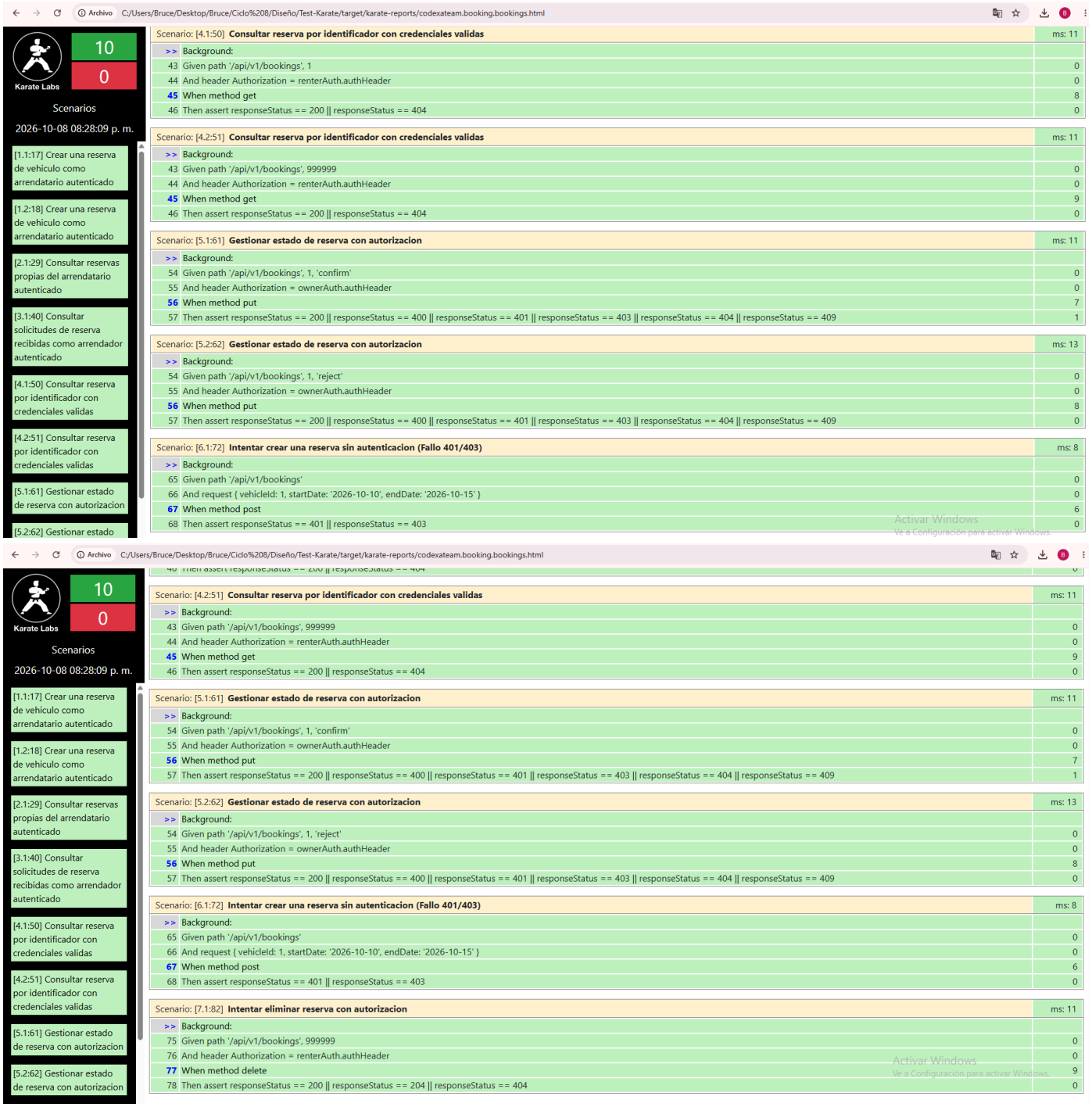
Archivo C:/Users/Bruce/Desktop/Bruce/Ciclo%208/Diseño/Test-Karate/target/karate-reports/codexateam.listings.vehicles.vehicles.html
11
0
Karate Labs
Scenarios
2026-10-08 08:26:40 p. m.
[1.1:22] Registrar nuevo vehiculo en el catalogo como arrendador autenticado (POST multipart)
[1.2:23] Registrar nuevo vehiculo en el catalogo como arrendador autenticado (POST multipart)
[2.1:34] Consultar publicaciones del arrendador autenticado
[3.1:44] Consultar catalogo general de vehiculos disponibles (publico)
[4.1:53] Consultar vehiculo por ID existente o inexistente
[4.2:54] Consultar vehiculo por ID existente o
Scenario: [2.1:34] Consultar publicaciones del arrendador autenticado ms: 13
>> Background:
26 Given path '/api/v1/vehicles', 'my-listings' 0
27 And header Authorization = ownerAuth.authHeader 0
28 When method get 11
29 Then status 200 0
30 And match response == '#array' 0
Scenario: [3.1:44] Consultar catalogo general de vehiculos disponibles (publico) ms: 12
>> Background:
37 Given path '/api/v1/vehicles' 0
38 When method get 10
39 Then status 200 0
40 And match response == '#array' 0
Scenario: [4.1:53] Consultar vehiculo por ID existente o inexistente ms: 14
>> Background:
47 Given path '/api/v1/vehicles', 1 1
48 When method get 10
49 Then assert responseStatus == 200 || responseStatus == 404 || responseStatus == 400 1
Scenario: [4.2:54] Consultar vehiculo por ID existente o inexistente ms: 11
>> Background:
47 Given path '/api/v1/vehicles', 999999 0
48 When method get 9
49 Then assert responseStatus == 200 || responseStatus == 404 || responseStatus == 400 0
Scenario: [5.1:63] Consultar imagen asociada a un vehiculo ms: 11
>> Background:
57 Given path '/api/v1/vehicles', 1, 'image' 1
58 When method get 9
59 Then assert responseStatus == 200 || responseStatus == 404 1



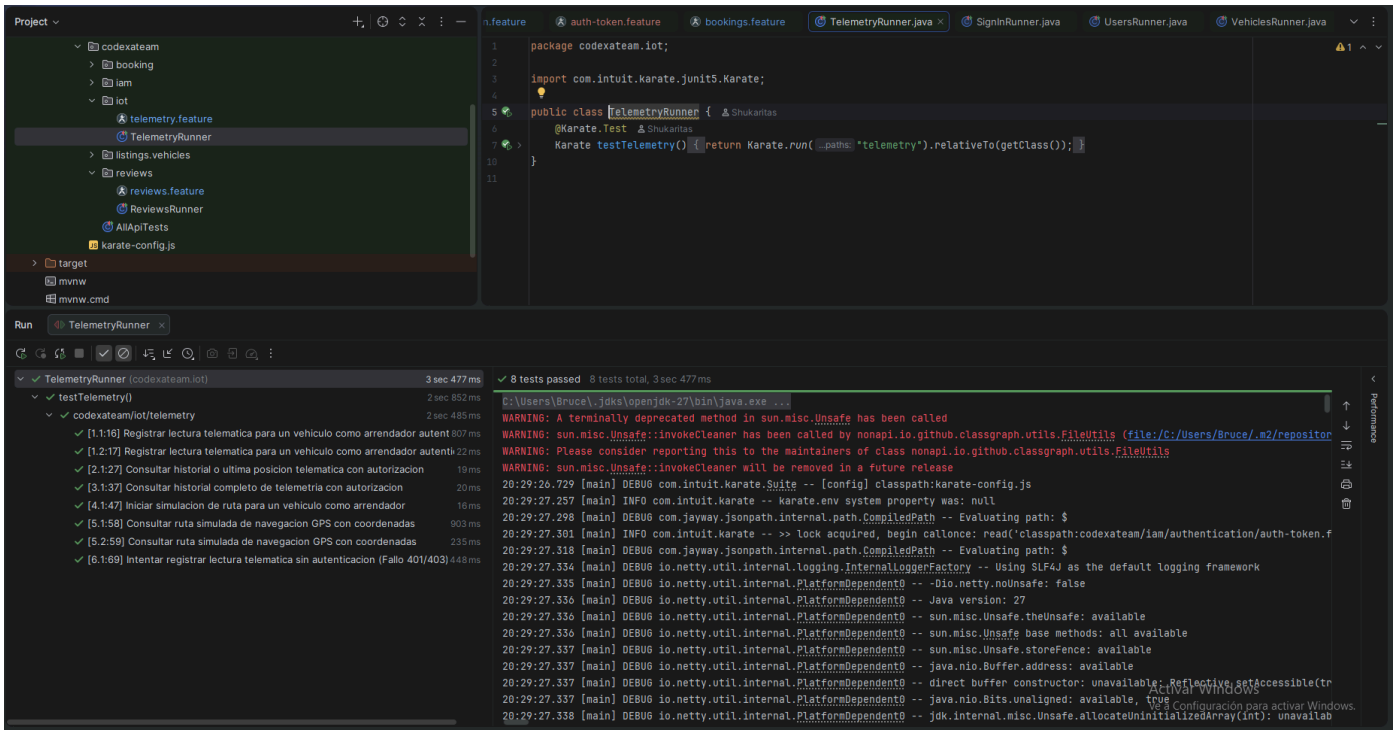
Integration Test en Booking & Digital Contract Bounded Context

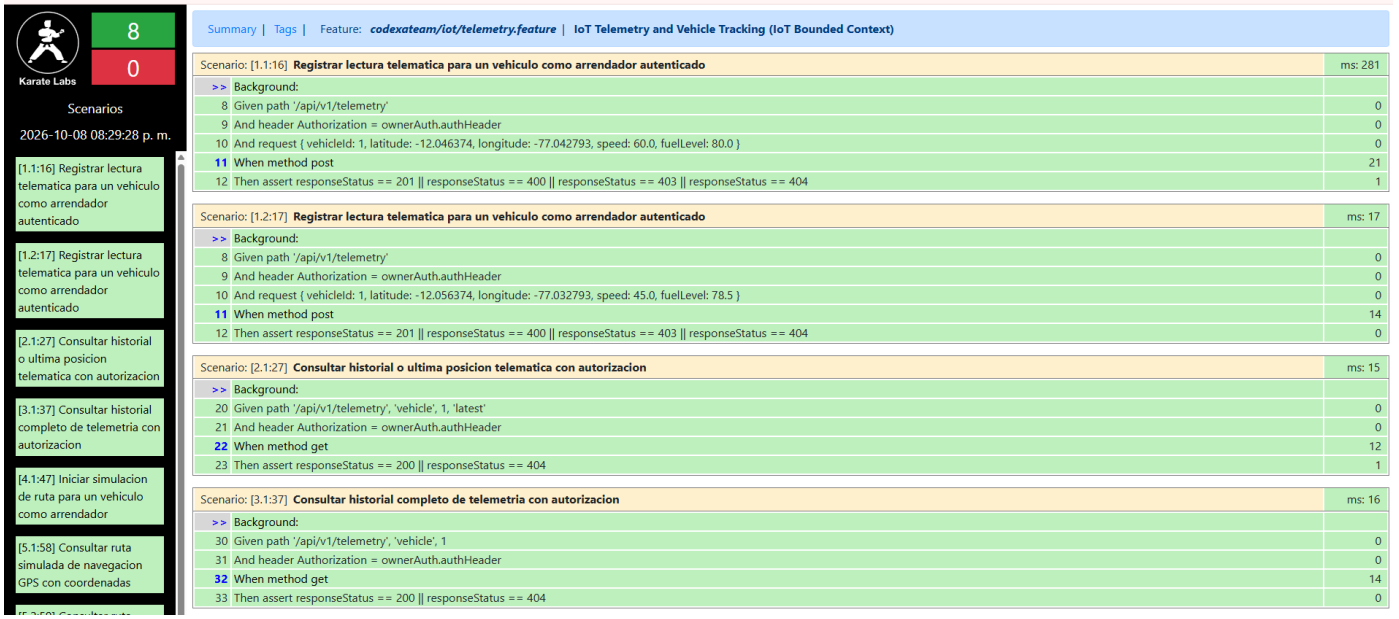

Project
Test-Karate [Codexa-API-Tests] C:\Users\Bruce\Desktop\Bruce\Ciclo 8\Diseño\Test-Karate
.idea
.mvn
src
test
java
codexateam
booking
bookings.feature
BookingsRunner
iam
iot
telemetry.feature
TelemetryRunner
listings.vehicles
auth-token.feature
bookings.feature
SignInRunner.java
UsersRunner.java
VehiclesRunner.java
BookingsRunner.java
package codexateam.booking;
import com.intuit.karate.junit5.Karate;
public class BookingsRunner {
@Karate.Test
Karate testBookings() { return Karate.run("bookings").relativeTo(getClass()); }
Run
BookingsRunner
BookingsRunner (codexateam.booking) 2 sec 529 ms
testBookings() 1 sec 911 ms
codexateam/booking/bookings 1 sec 533 ms
[1.1:17] Crear una reserva de vehiculo como arrendatario autenticado 884 ms
[1.2:18] Crear una reserva de vehiculo como arrendatario autenticado 23 ms
[2.1:29] Consultar reservas propias del arrendatario autenticado 27 ms
[3.1:40] Consultar solicitudes de reserva recibidas como arrendador autentica 20 ms
[4.1:50] Consultar reserva por identificador con credenciales validas 17 ms
[4.2:51] Consultar reserva por identificador con credenciales validas 14 ms
[5.1:61] Gestionar estado de reserva con autorizacion 15 ms
[5.2:62] Gestionar estado de reserva con autorizacion 16 ms
[6.1:72] Intentar crear una reserva sin autenticacion (Fallo 401/403) 10 ms
[7.1:82] Intentar eliminar reserva con autorizacion 490 ms
10 tests passed 10 tests total, 2 sec 529 ms
C:\Users\Bruce\.jdks\openjdk-27\bin\java.exe ...
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::invokeCleaner has been called by nonapi.io.github.classgraph.utils.FileUtils (file:/C:/Users/Bruce/.m2/repositor
WARNING: Please consider reporting this to the maintainers of class nonapi.io.github.classgraph.utils.FileUtils
WARNING: sun.misc.Unsafe::invokeCleaner will be removed in a future release
20:28:08.375 [main] DEBUG com.intuit.karate.Suite -- [config] classpath:karate-config.js
20:28:08.906 [main] INFO com.intuit.karate -- karate.env system property was: null
20:28:08.945 [main] DEBUG com.jayway.jsonpath.internal.path.CompiledPath -- Evaluating path: $
20:28:08.948 [main] INFO com.intuit.karate -- >> lock acquired, begin callonce: read('classpath:codexateam/iam/authentication/auth-token.f
20:28:08.966 [main] DEBUG com.jayway.jsonpath.internal.path.CompiledPath -- Evaluating path: $
20:28:08.983 [main] DEBUG io.netty.util.internal.logging.InternalLoggerFactory -- Using SLF4J as the default logging framework
20:28:08.984 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- -Dio.netty.noUnsafe: false
20:28:08.984 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- Java version: 27
20:28:08.984 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- sun.misc.Unsafe.theUnsafe: available
20:28:08.985 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- sun.misc.Unsafe base methods: all available
20:28:08.985 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- sun.misc.Unsafe.storeFence: available
20:28:08.985 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- java.nio.Buffer.address: available
20:28:08.985 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- direct buffer constructor: unavailable: Reflective setAccessible(tr
20:28:08.985 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- java.nio.Bits.unaligned: available, true
20:28:08.987 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- jdk.internal.misc.Unsafe.allocateUninitializedArray(int): unavailab
Activar Windows
Ve a Configuración para activar Windows.


Archivo C:/Users/Bruce/Desktop/Bruce/Ciclo%208/Diseño/Test-Karate/target/karate-reports/codexateam.booking.bookings.html
Karate Labs
10
0
Scenarios
2026-10-08 08:28:09 p. m.
[1.1:17] Crear una reserva de vehiculo como arrendatario autenticado
[1.2:18] Crear una reserva de vehiculo como arrendatario autenticado
[2.1:29] Consultar reservas propias del arrendatario autenticado
[3.1:40] Consultar solicitudes de reserva recibidas como arrendador autenticado
[4.1:50] Consultar reserva por identificador con credenciales validas
[4.2:51] Consultar reserva por identificador con credenciales validas
[5.1:61] Gestionar estado de reserva con autorizacion
Summary | Tags | Feature: codexateam/booking/bookings.feature | Bookings Management (Booking Bounded Context)
Scenario: [1.1:17] Crear una reserva de vehiculo como arrendatario autenticado ms: 357
>> Background:
9 Given path '/api/v1/bookings' 0
10 And header Authorization = renterAuth.authHeader 0
11 And request { vehicleId: 1, startDate: '2026-11-01T10:00:00.000Z', endDate: '2026-11-05T10:00:00.000Z' } 0
12 When method post 10
13 Then assert responseStatus == 201 || responseStatus == 400 || responseStatus == 404 || responseStatus == 409 1
Scenario: [1.2:18] Crear una reserva de vehiculo como arrendatario autenticado ms: 17
>> Background:
9 Given path '/api/v1/bookings' 0
10 And header Authorization = renterAuth.authHeader 0
11 And request { vehicleId: 1, startDate: '2026-12-01T10:00:00.000Z', endDate: '2026-12-07T10:00:00.000Z' } 0
12 When method post 13
13 Then assert responseStatus == 201 || responseStatus == 400 || responseStatus == 404 || responseStatus == 409 0
Scenario: [2.1:29] Consultar reservas propias del arrendatario autenticado ms: 23
>> Background:
21 Given path '/api/v1/bookings', 'my-bookings' 1
22 And header Authorization = renterAuth.authHeader 0
23 When method get 13
24 Then status 200 0
25 And match response == '#array' 5
Scenario: [3.1:40] Consultar solicitudes de reserva recibidas como arrendador autenticado ms: 16
>> Background:
32 Given path '/api/v1/bookings', 'my-requests' 0
33 And header Authorization = ownerAuth.authHeader 5
34 When method get 8
35 Then status 200 0
36 And match response == '#array' 1
Activar Windows

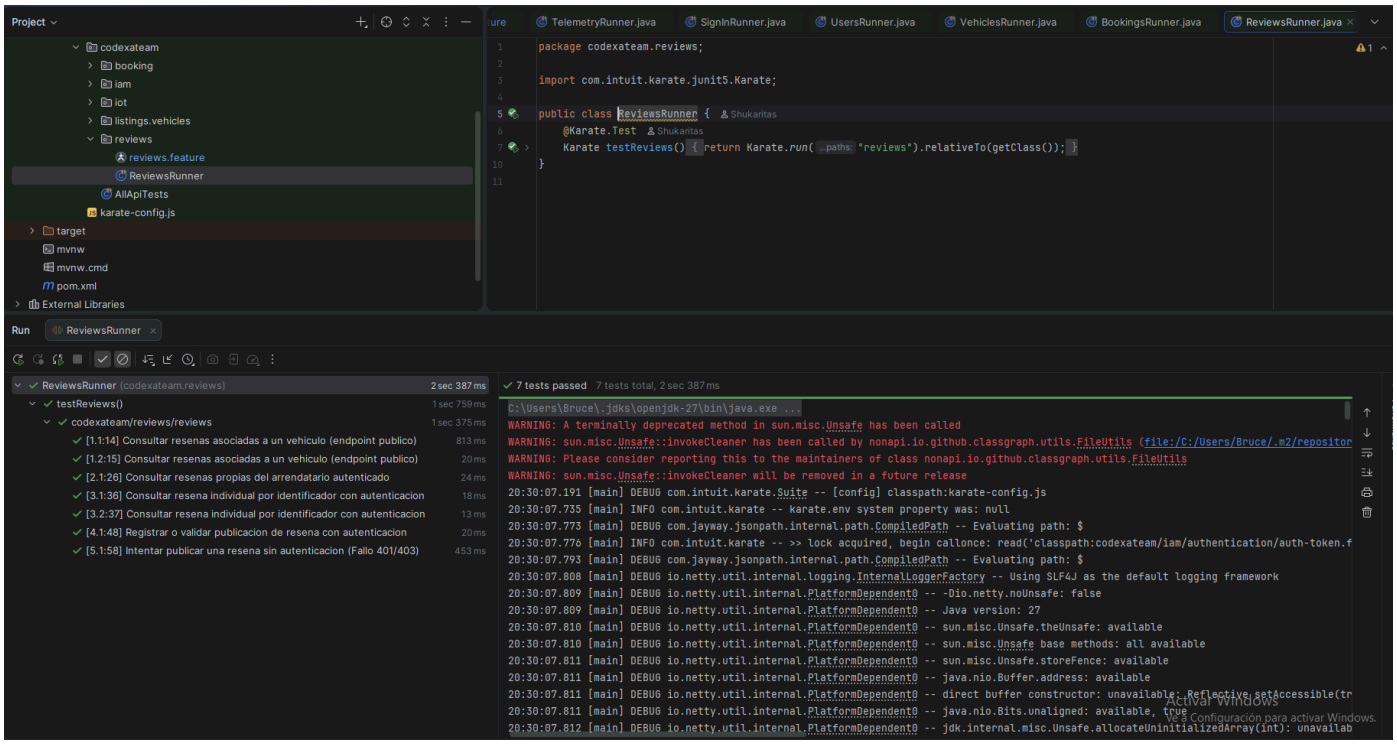


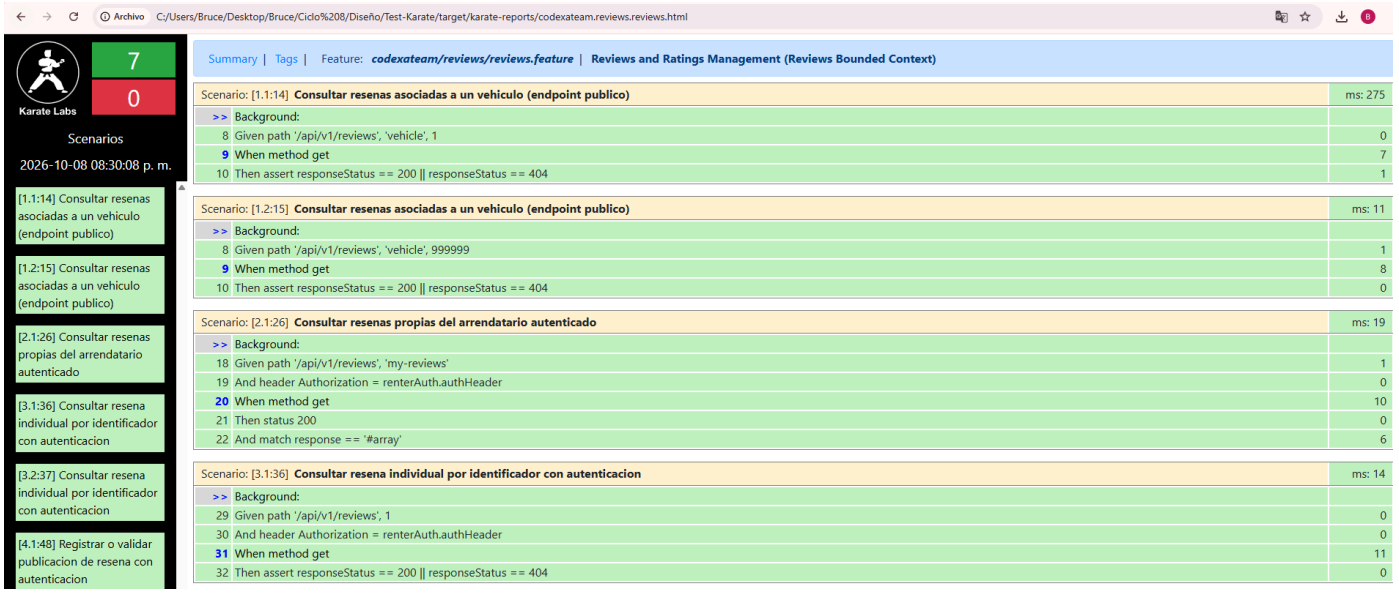
Integration Test en IoT Monitoring & Route Simulation Bounded Context


Project
codexateam
booking
iam
iot
telemetry.feature
TelemetryRunner
listings.vehicles
reviews
reviews.feature
ReviewsRunner
AllApiTests
karate-config.js
target
mvnw
mvnw.cmd
auth-token.feature
bookings.feature
TelemetryRunner.java
SignInRunner.java
UsersRunner.java
VehiclesRunner.java
package codexateam.iot;
import com.intuit.karate.junit5.Karate;
public class TelemetryRunner {
@Karate.Test
Karate testTelemetry() { return Karate.run(...paths: "telemetry").relativeTo(getClass()); }
Run
TelemetryRunner
TelemetryRunner (codexateam.iot) 3 sec 477 ms
testTelemetry() 2 sec 852 ms
codexateam/iot/telemetry 2 sec 485 ms
[1.1:16] Registrar lectura telematica para un vehiculo como arrendador autent 807 ms
[1.2:17] Registrar lectura telematica para un vehiculo como arrendador autenti 22 ms
[2.1:27] Consultar historial o ultima posicion telematica con autorizacion 19 ms
[3.1:37] Consultar historial completo de telemetria con autorizacion 20 ms
[4.1:47] Iniciar simulacion de ruta para un vehiculo como arrendador 16 ms
[5.1:58] Consultar ruta simulada de navegacion GPS con coordenadas 903 ms
[5.2:59] Consultar ruta simulada de navegacion GPS con coordenadas 235 ms
[6.1:69] Intentar registrar lectura telematica sin autenticacion (Fallo 401/403) 448 ms
8 tests passed 8 tests total, 3 sec 477 ms
C:\Users\Bruce\.jdks\openjdk-27\bin\java.exe ...
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::invokeCleaner has been called by nonapi.io.github.classgraph.utils.FileUtils (file:C:/Users/Bruce/.m2/repositor
WARNING: Please consider reporting this to the maintainers of class nonapi.io.github.classgraph.utils.FileUtils
WARNING: sun.misc.Unsafe::invokeCleaner will be removed in a future release
20:29:26.729 [main] DEBUG com.intuit.karate.Suite -- [config] classpath:karate-config.js
20:29:27.257 [main] INFO com.intuit.karate -- karate.env system property was: null
20:29:27.298 [main] DEBUG com.jayway.jsonpath.internal.path.CompiledPath -- Evaluating path: $
20:29:27.301 [main] INFO com.intuit.karate -- >> lock acquired, begin callonce: read('classpath:codexateam/iam/authentication/auth-token.f
20:29:27.318 [main] DEBUG com.jayway.jsonpath.internal.path.CompiledPath -- Evaluating path: $
20:29:27.334 [main] DEBUG io.netty.util.internal.logging.InternalLoggerFactory -- Using SLF4J as the default logging framework
20:29:27.335 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- -Dio.netty.noUnsafe: false
20:29:27.336 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- Java version: 27
20:29:27.336 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- sun.misc.Unsafe.theUnsafe: available
20:29:27.336 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- sun.misc.Unsafe base methods: all available
20:29:27.337 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- sun.misc.Unsafe.storeFence: available
20:29:27.337 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- java.nio.Buffer.address: available
20:29:27.337 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- direct buffer constructor: unavailable: Reflective setAccessible(tr
20:29:27.337 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- java.nio.Bits.unaligned: available, true
20:29:27.338 [main] DEBUG io.netty.util.internal.PlatformDependent0 -- jdk.internal.misc.Unsafe.allocateUninitializedArray(int): unavailab
Activar Windows
Ve a Configuración para activar Windows.
Performance


Karate Labs
8
0
Scenarios
2026-10-08 08:29:28 p. m.
[1.1:16] Registrar lectura telematica para un vehiculo como arrendador autenticado
[1.2:17] Registrar lectura telematica para un vehiculo como arrendador autenticado
[2.1:27] Consultar historial o ultima posicion telematica con autorizacion
[3.1:37] Consultar historial completo de telemetria con autorizacion
[4.1:47] Iniciar simulacion de ruta para un vehiculo como arrendador
[5.1:58] Consultar ruta simulada de navegacion GPS con coordenadas
Summary | Tags | Feature: codexateam/iot/telemetry.feature | IoT Telemetry and Vehicle Tracking (IoT Bounded Context)
Scenario: [1.1:16] Registrar lectura telematica para un vehiculo como arrendador autenticado ms: 281
>> Background:
8 Given path '/api/v1/telemetry' 0
9 And header Authorization = ownerAuth.authHeader 0
10 And request { vehicleId: 1, latitude: -12.046374, longitude: -77.042793, speed: 60.0, fuelLevel: 80.0 } 0
11 When method post 21
12 Then assert responseStatus == 201 || responseStatus == 400 || responseStatus == 403 || responseStatus == 404 1
Scenario: [1.2:17] Registrar lectura telematica para un vehiculo como arrendador autenticado ms: 17
>> Background:
8 Given path '/api/v1/telemetry' 0
9 And header Authorization = ownerAuth.authHeader 0
10 And request { vehicleId: 1, latitude: -12.056374, longitude: -77.032793, speed: 45.0, fuelLevel: 78.5 } 0
11 When method post 14
12 Then assert responseStatus == 201 || responseStatus == 400 || responseStatus == 403 || responseStatus == 404 0
Scenario: [2.1:27] Consultar historial o ultima posicion telematica con autorizacion ms: 15
>> Background:
20 Given path '/api/v1/telemetry', 'vehicle', 1, 'latest' 0
21 And header Authorization = ownerAuth.authHeader 0
22 When method get 12
23 Then assert responseStatus == 200 || responseStatus == 404 1
Scenario: [3.1:37] Consultar historial completo de telemetria con autorizacion ms: 16
>> Background:
30 Given path '/api/v1/telemetry', 'vehicle', 1 0
31 And header Authorization = ownerAuth.authHeader 0
32 When method get 14
33 Then assert responseStatus == 200 || responseStatus == 404 0

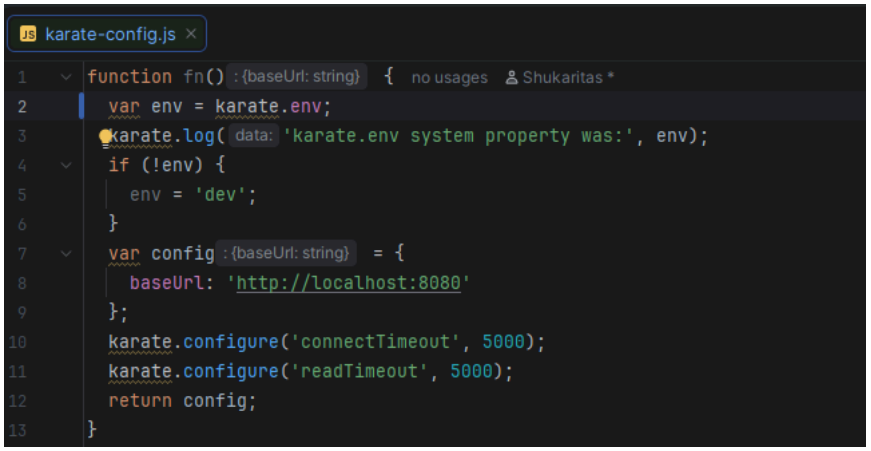
Integration Test en Reviews & Reputation Bounded Context





6.1.3. Core Behavior-Driven Development

Aqui se ubican los archivos .feature que sirven para el funcionamiento de las pruebas de integracion, se inicio definiendo un archivo tipo karate.js para configurar las variables globlales dentro del proyecto



Behavior-Driven Development para IAM Bounded Context

- Validacion de token

auth-token.feature | bookings.feature | TelemetryRunner.java | SignInRunner.java | UsersRunner.java | VehiclesRunner.java

```
@ignore
Feature: Reusable Token Authentication Service

  Background:
    * url baseUrl

  Scenario: Obtener token y credenciales segun rol solicitado
    * def isOwner = role == 'arrendador' || role == 'ROLE_ARRENDADOR'
    * def email = isOwner ? 'owner.master@rc.com' : 'renter.master@rc.com'
    * def name = isOwner ? 'Master Owner' : 'Master Renter'
    * def roleName = isOwner ? 'arrendador' : 'arrendatario'
    * def password = 'Password123!'

    # Asegurar registro de usuario si no existe previamente
    Given path '/api/v1/authentication', 'sign-up'
    And request { name: '#(name)', email: '#(email)', password: '#(password)', role: '#(roleName)' }
    When method post

    # Autenticar para obtener el token JWT vigente
    Given path '/api/v1/authentication', 'sign-in'
    And request { email: '#(email)', password: '#(password)' }
    When method post
    Then status 200
    * def token = response.token
    * def userId = response.id
    * def userEmail = response.email
    * def authHeader = 'Bearer ' + response.token
```

- Pruebas de registro de usuario

asses\...\reviews.feature | RolesRunner.java | SignUpRunner.java | test-classes\...\sign-in.feature | java\...\reviews.feature | sign

```
Feature: User Authentication - Sign Up (IAM Bounded Context)

  Background:
    * url baseUrl

  Scenario Outline: Registro de los dos usuarios principales del sistema
    Given path '/api/v1/authentication', 'sign-up'
    And request { name: '<name>', email: '<email>', password: '<password>', role: '<role>' }
    When method post
    Then assert responseStatus == 201 || responseStatus == 409 || responseStatus == 500

    Examples:
      | name          | email               | password     | role         |
      | Master Owner  | owner.master@rc.com | Password123! | arrendador   |
      | Master Renter | renter.master@rc.com| Password123! | arrendatario |

  Scenario Outline: Intento de registro con correo ya existente (Fallo de Negocio 409/500)
    * def ts = java.lang.System.currentTimeMillis()
    * def rnd = Math.floor(Math.random() * 9000 + 1000)
    * def duplicateEmail = '<prefix>' + ts + rnd + '@rc.com'
    * def initialPayload = { name: '<name>', email: '#(duplicateEmail)', password: '<password>', role: '<role>' }
    Given path '/api/v1/authentication', 'sign-up'
    And request initialPayload
    When method post
    Then status 201

    Given path '/api/v1/authentication', 'sign-up'
    And request initialPayload
    When method post
    Then assert responseStatus == 400 || responseStatus == 409 || responseStatus == 500

    Examples:
      | name             | prefix | password     | role         |
      | Usuario Duplicado | dup_   | Password123! | arrendatario |
```

- Pruebas de inicio de sesion

java\...\sign-in.feature × | auth-token.feature | bookings.feature | TelemetryRunner.java | SignInRunner.java

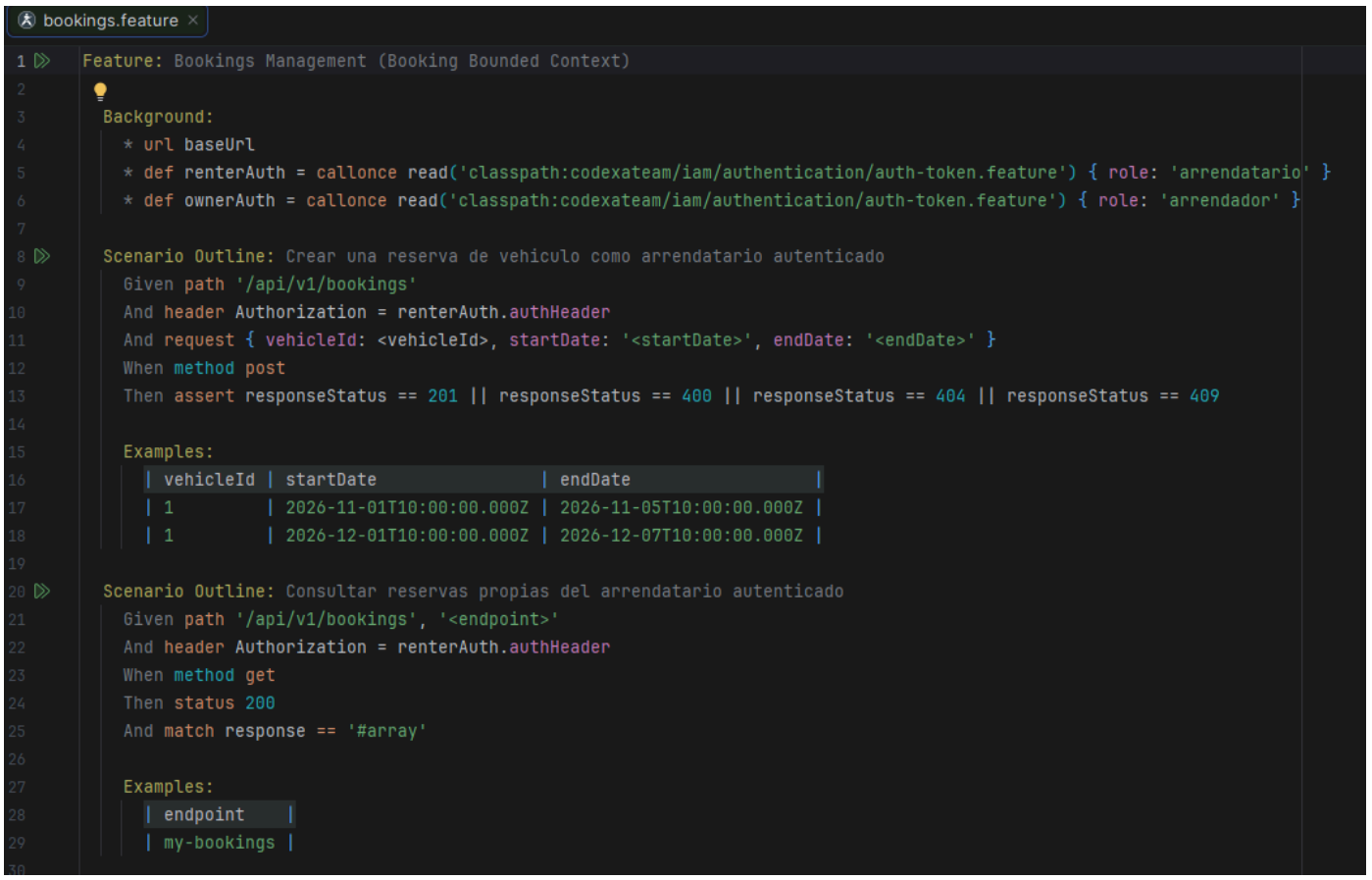
```
Feature: User Authentication - Sign In (IAM Bounded Context)

  Background:
    * url baseUrl

  Scenario Outline: Inicio de sesion exitoso con credenciales de los usuarios principales
    Given path '/api/v1/authentication', 'sign-in'
    And request { email: '<email>', password: '<password>' }
    When method post
    Then status 200
    And match response contains { id: '#number', token: '#string', email: '<email>' }
    And match response.roles contains '<expectedRole>'
    And match response.token != ''

    Examples:
      | email                | password     | expectedRole      |
      | owner.master@rc.com | Password123! | ROLE_ARRENDADOR   |
      | renter.master@rc.com| Password123! | ROLE_ARRENDATARIO |

  Scenario Outline: Intento de inicio de sesion con contrasena incorrecta (Fallo 401/400/500)
    Given path '/api/v1/authentication', 'sign-in'
    And request { email: '<email>', password: '<wrongPassword>' }
    When method post
    Then assert responseStatus == 400 || responseStatus == 401 || responseStatus == 500

    Examples:
      | email                | wrongPassword  |
      | owner.master@rc.com | ContrasenaMal! |
      | renter.master@rc.com| BadPassword99! |
```

Behavior-Driven Development para Listings & Fleet Management Bounded Context

vehicles.feature ×

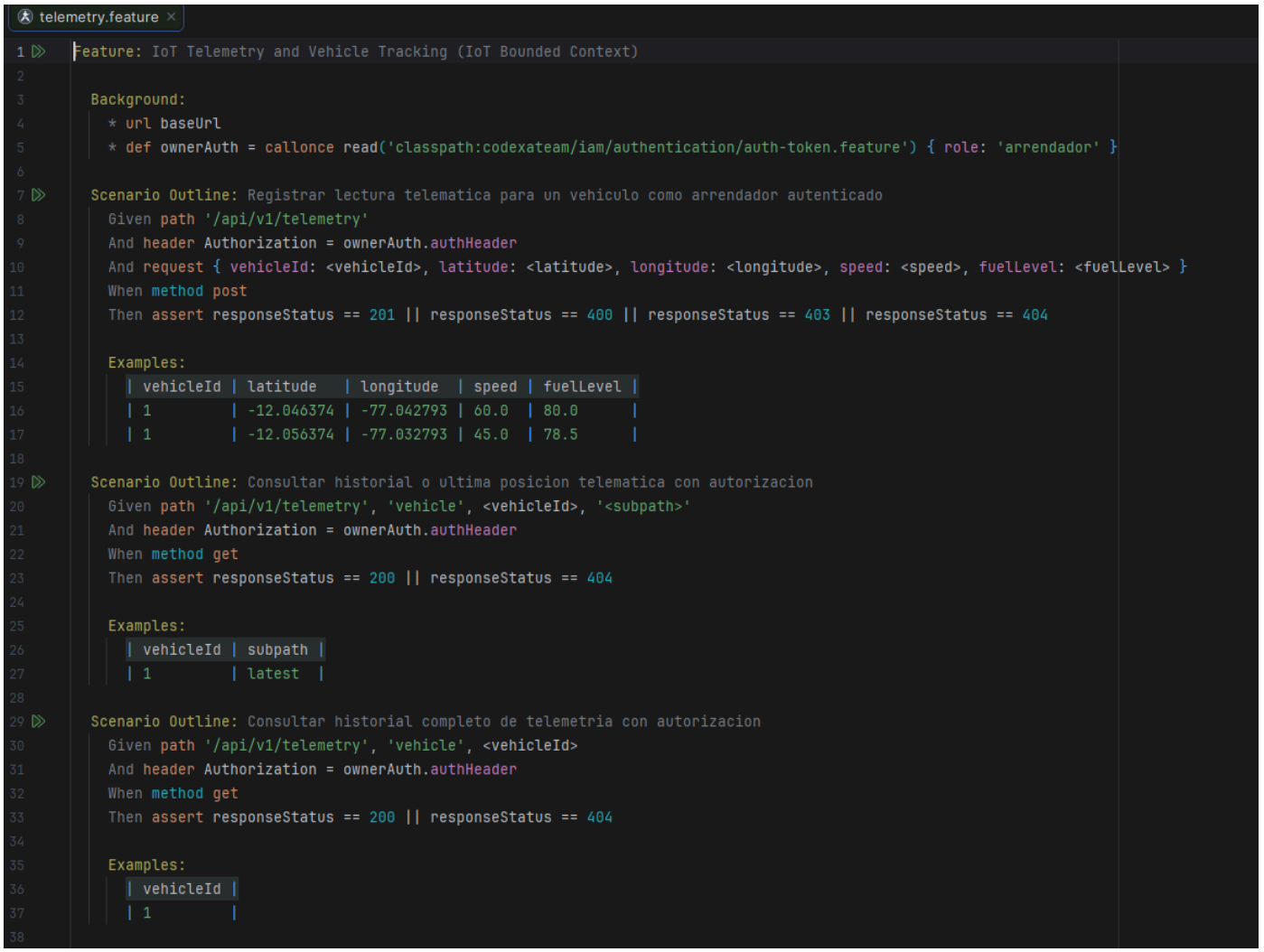
```
Feature: Vehicle Listings Management (Listings Bounded Context)

  Background:
    * url baseUrl
    * def ownerAuth = callonce read('classpath:codexateam/iam/authentication/auth-token.feature') { role: 'arrendador' }

  Scenario Outline: Registrar nuevo vehiculo en el catalogo como arrendador autenticado (POST multipart)
    * def vehicleData = { brand: '<brand>', model: '<model>', year: <year>, pricePerDay: <pricePerDay> }
    * def vehicleJson = karate.toJson(vehicleData)
    Given path '/api/v1/vehicles'
    And header Authorization = ownerAuth.authHeader
    And multipart file image = { value: 'dummy image bytes', filename: 'car.jpg', contentType: 'image/jpeg' }
    And multipart file resource = { value: '#(vehicleJson)', filename: 'resource.json', contentType: 'application/json' }
    When method post
    Then status 201
    And match response.brand == '<brand>'
    And match response.model == '<model>'
    And match response.id == '#number'

    Examples:
      | brand   | model   | year | pricePerDay |
      | Toyota  | Corolla | 2023 | 55.0        |
      | Hyundai | Elantra | 2022 | 48.0        |

  Scenario Outline: Consultar publicaciones del arrendador autenticado
    Given path '/api/v1/vehicles', '<subpath>'
    And header Authorization = ownerAuth.authHeader
    When method get
    Then status 200
    And match response == '#array'

    Examples:
      | subpath     |
      | my-listings |
```

Behavior-Driven Development para Booking & Digital Contract Bounded Context



```
bookings.feature
Feature: Bookings Management (Booking Bounded Context)

  Background:
    * url baseUrl
    * def renterAuth = callonce read('classpath:codexateam/iam/authentication/auth-token.feature') { role: 'arrendatario' }
    * def ownerAuth = callonce read('classpath:codexateam/iam/authentication/auth-token.feature') { role: 'arrendador' }

  Scenario Outline: Crear una reserva de vehiculo como arrendatario autenticado
    Given path '/api/v1/bookings'
    And header Authorization = renterAuth.authHeader
    And request { vehicleId: <vehicleId>, startDate: '<startDate>', endDate: '<endDate>' }
    When method post
    Then assert responseStatus == 201 || responseStatus == 400 || responseStatus == 404 || responseStatus == 409

    Examples:
      | vehicleId | startDate                | endDate                  |
      | 1         | 2026-11-01T10:00:00.000Z | 2026-11-05T10:00:00.000Z |
      | 1         | 2026-12-01T10:00:00.000Z | 2026-12-07T10:00:00.000Z |

  Scenario Outline: Consultar reservas propias del arrendatario autenticado
    Given path '/api/v1/bookings', '<endpoint>'
    And header Authorization = renterAuth.authHeader
    When method get
    Then status 200
    And match response == '#array'

    Examples:
      | endpoint    |
      | my-bookings |
```

Behavior-Driven Development para IoT Monitoring & Route Simulation Bounded Context



```
telemetry.feature

Feature: IoT Telemetry and Vehicle Tracking (IoT Bounded Context)

  Background:
    * url baseUrl
    * def ownerAuth = callonce read('classpath:codexateam/iam/authentication/auth-token.feature') { role: 'arrendador' }

  Scenario Outline: Registrar lectura telematica para un vehiculo como arrendador autenticado
    Given path '/api/v1/telemetry'
    And header Authorization = ownerAuth.authHeader
    And request { vehicleId: <vehicleId>, latitude: <latitude>, longitude: <longitude>, speed: <speed>, fuelLevel: <fuelLevel> }
    When method post
    Then assert responseStatus == 201 || responseStatus == 400 || responseStatus == 403 || responseStatus == 404

    Examples:
      | vehicleId | latitude   | longitude  | speed | fuelLevel |
      | 1         | -12.046374 | -77.042793 | 60.0  | 80.0      |
      | 1         | -12.056374 | -77.032793 | 45.0  | 78.5      |

  Scenario Outline: Consultar historial o ultima posicion telematica con autorizacion
    Given path '/api/v1/telemetry', 'vehicle', <vehicleId>, '<subpath>'
    And header Authorization = ownerAuth.authHeader
    When method get
    Then assert responseStatus == 200 || responseStatus == 404

    Examples:
      | vehicleId | subpath |
      | 1         | latest  |

  Scenario Outline: Consultar historial completo de telemetria con autorizacion
    Given path '/api/v1/telemetry', 'vehicle', <vehicleId>
    And header Authorization = ownerAuth.authHeader
    When method get
    Then assert responseStatus == 200 || responseStatus == 404

    Examples:
      | vehicleId |
      | 1         |
```

Behavior-Driven Development para Reviews & Reputation Bounded Context

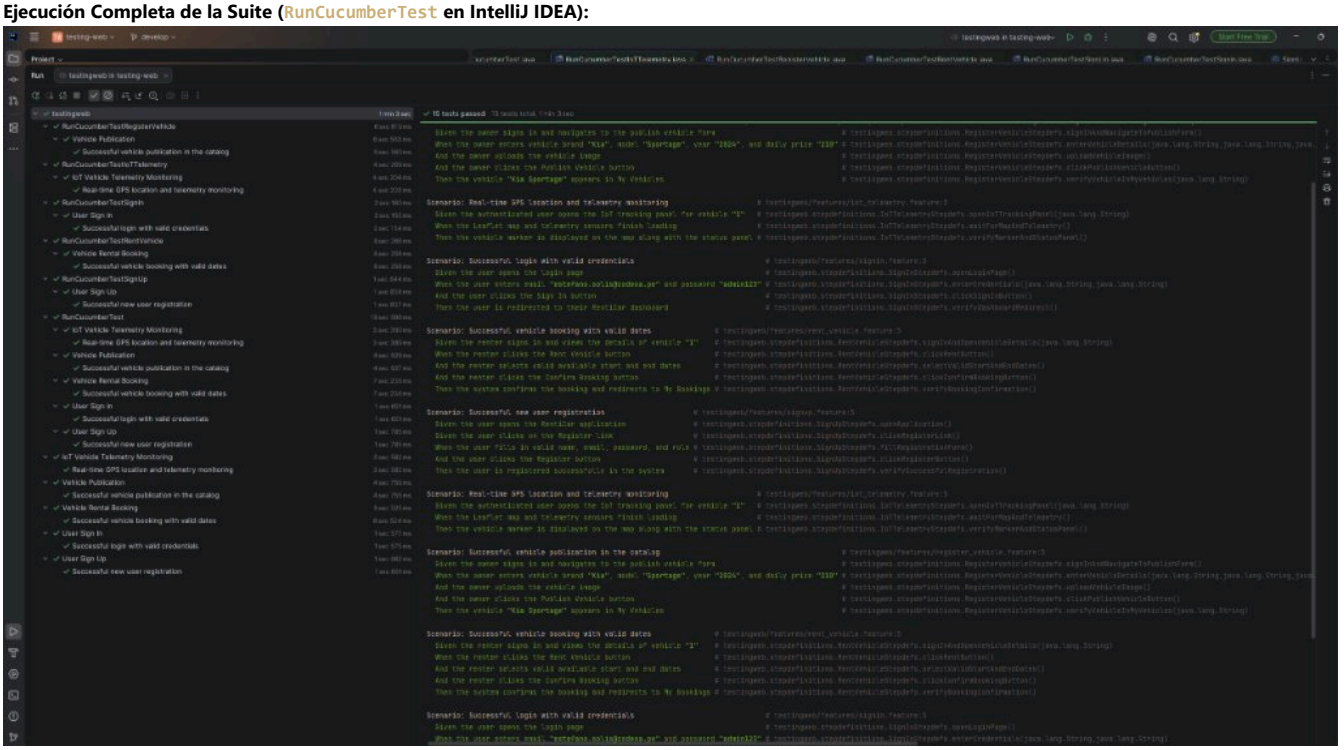
reviews.feature

```
    Examples:
      | subpath    |
      | my-reviews |

  Scenario Outline: Consultar resena individual por identificador con autenticacion
    Given path '/api/v1/reviews', <reviewId>
    And header Authorization = renterAuth.authHeader
    When method get
    Then assert responseStatus == 200 || responseStatus == 404

    Examples:
      | reviewId |
      | 1        |
      | 999999   |

  Scenario Outline: Registrar o validar publicacion de resena con autenticacion
    Given path '/api/v1/reviews'
    And header Authorization = renterAuth.authHeader
    And request { vehicleId: <vehicleId>, rating: <rating>, comment: '<comment>' }
    When method post
    Then assert responseStatus == 201 || responseStatus == 400 || responseStatus == 403 || responseStatus == 404 || responseStatus == 409

    Examples:
      | vehicleId | rating | comment                          |
      | 1         | 5      | Excelente experiencia de alquiler! |

  Scenario Outline: Intentar publicar una resena sin autenticacion (Fallo 401/403)
    Given path '/api/v1/reviews'
    And request { vehicleId: <vehicleId>, rating: <rating>, comment: '<comment>' }
    When method post
    Then assert responseStatus == 401 || responseStatus == 403

    Examples:
      | vehicleId | rating | comment                    |
      | 1         | 4      | Sin autorizacion no es valido |
```

6.1.4. Core System Tests

En este apartado nos encargamos de implementar las pruebas de sistema automatizadas de extremo a extremo (End-to-End) para **RentiCar**, utilizando **Selenium WebDriver** para interactuar con la aplicación web en Angular y **Cucumber BDD** con sintaxis **Gherkin** en inglés para verificar los flujos principales de negocio.

- **Ejecución Completa de la Suite (`RunCucumberTest` en IntelliJ IDEA):**

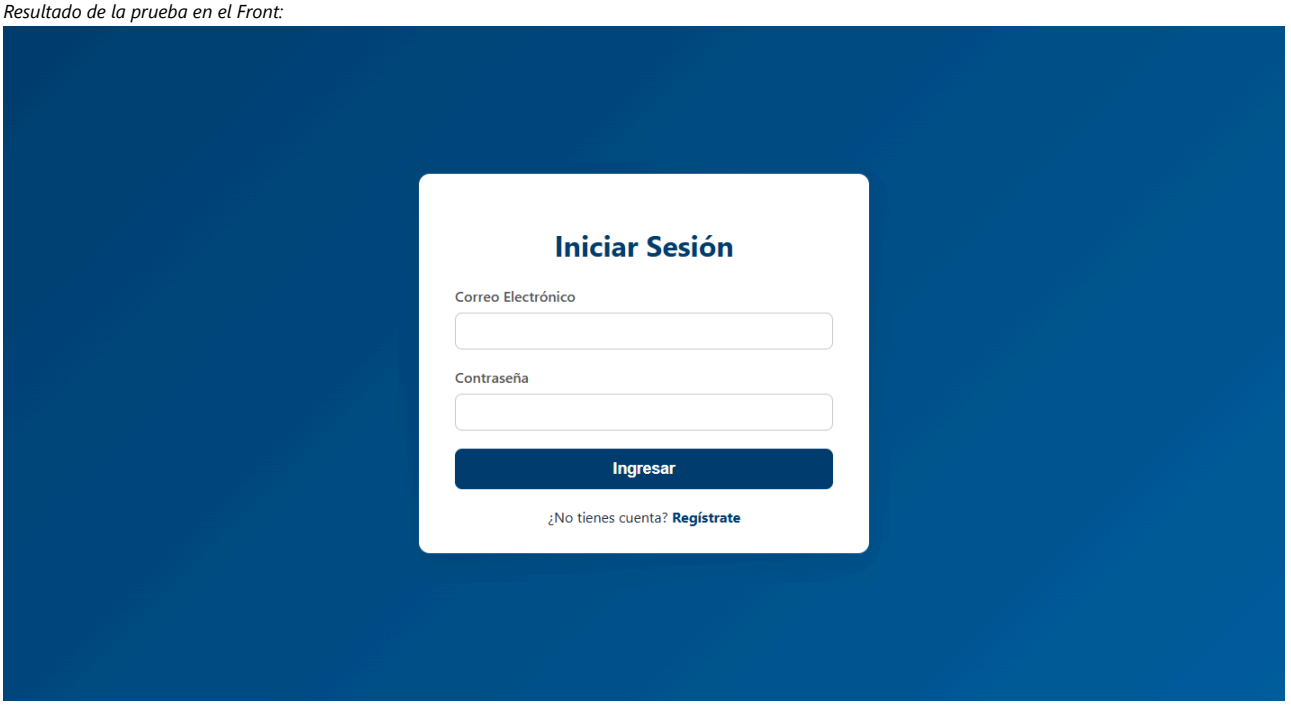


---

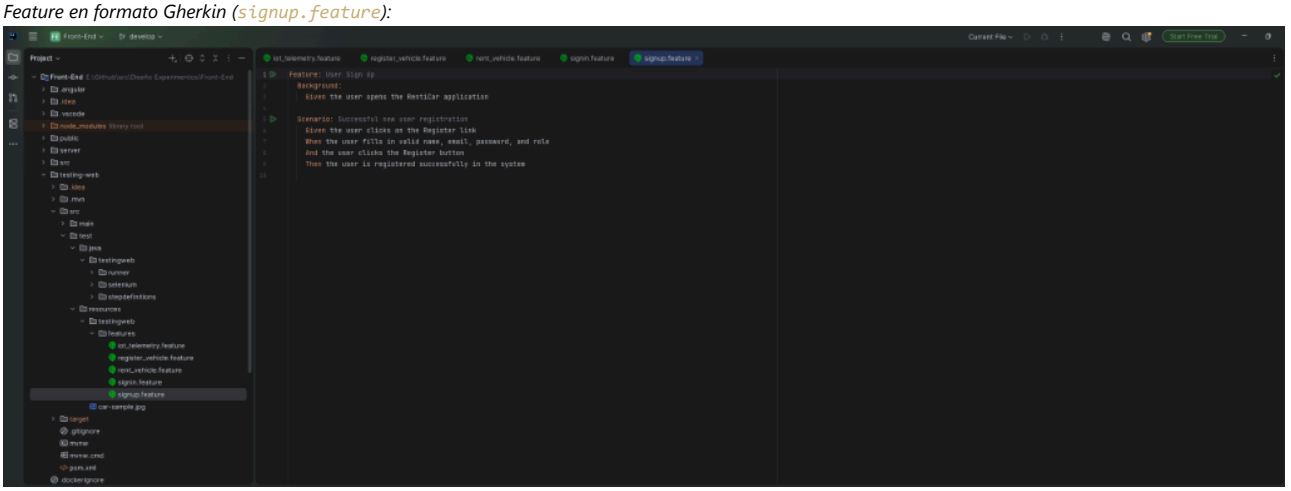
1. Flujo de Registro de Usuario (SignUp)

- **Evidencias de Ejecución:**

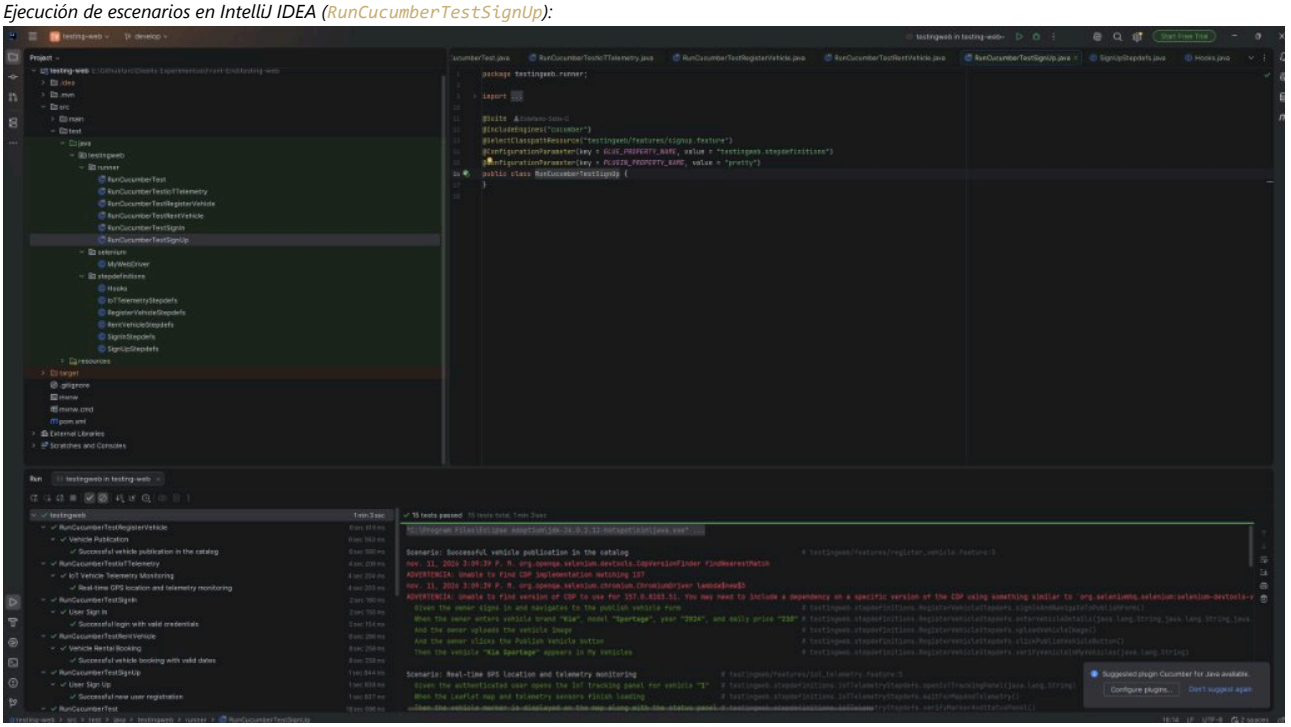
- *Resultado de la prueba en el Front:*



- *Feature en formato Gherkin (`signup.feature`):*

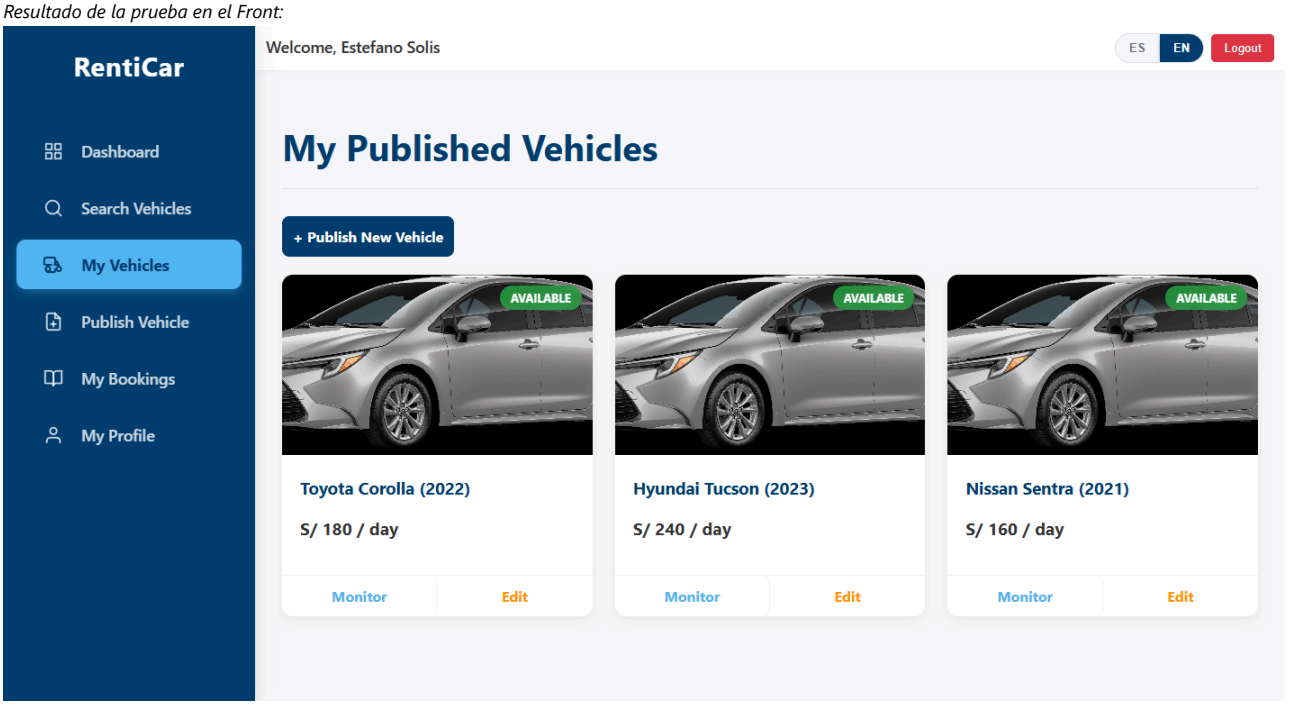


- *Ejecución de escenarios en IntelliJ IDEA (`RunCucumberTestSignUp`):*

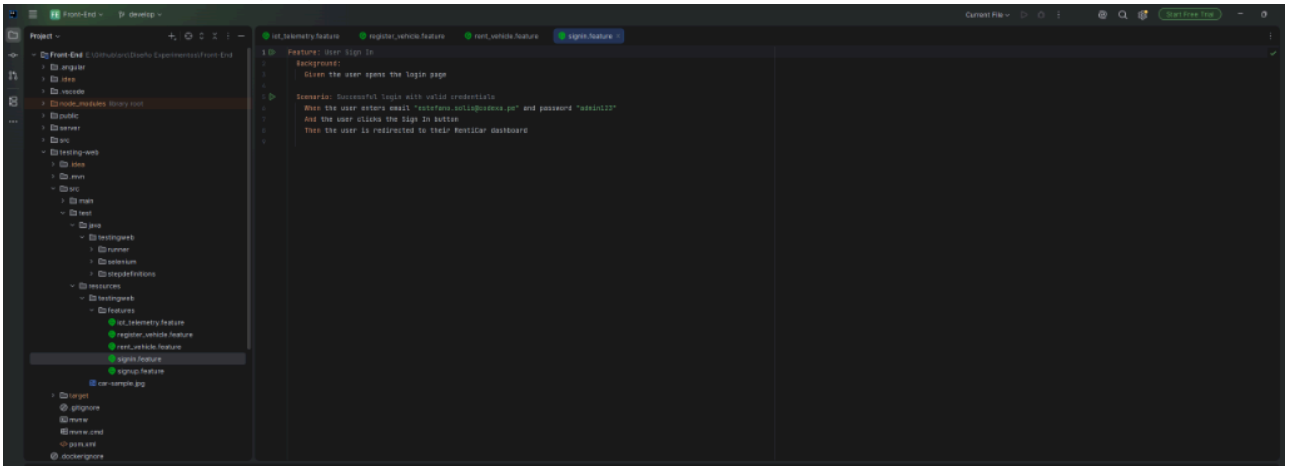


2. Flujo de Autenticación de Usuario (SignIn)

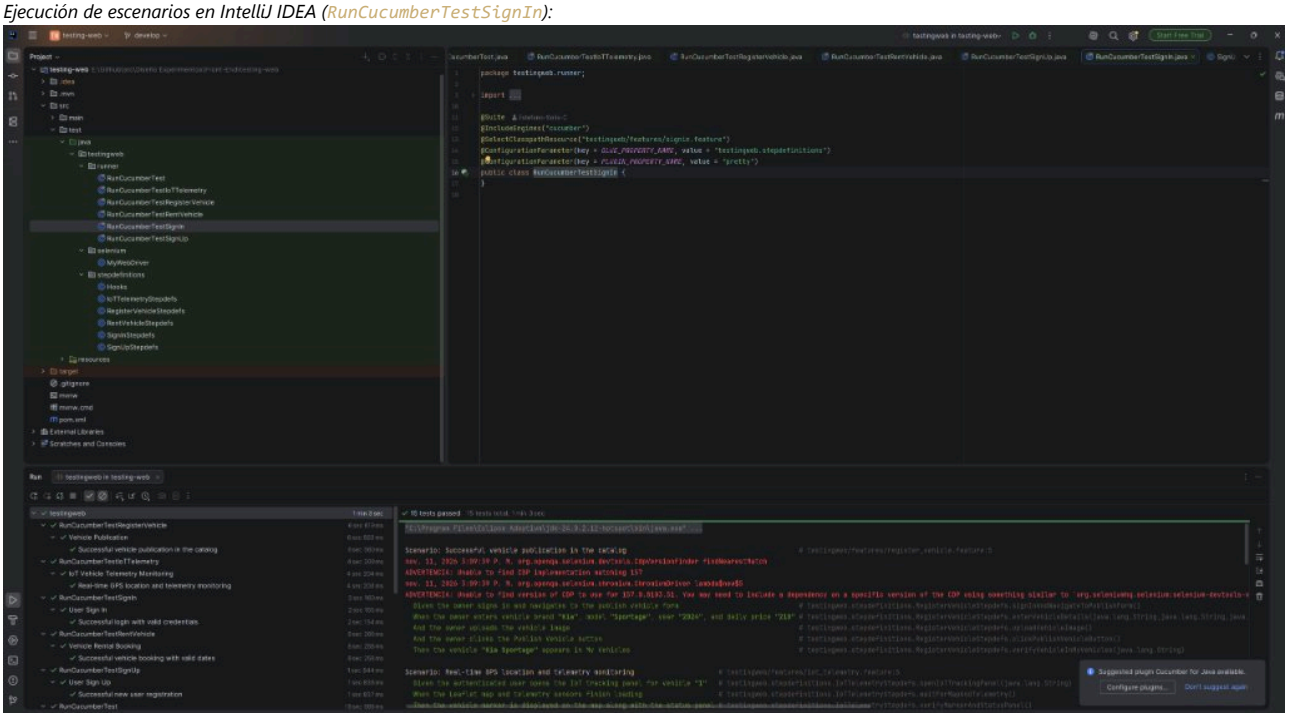
- **Evidencias de Ejecución:**
  - *Resultado de la prueba en el Front:*



  - *Feature en formato Gherkin (`signin.feature`):*

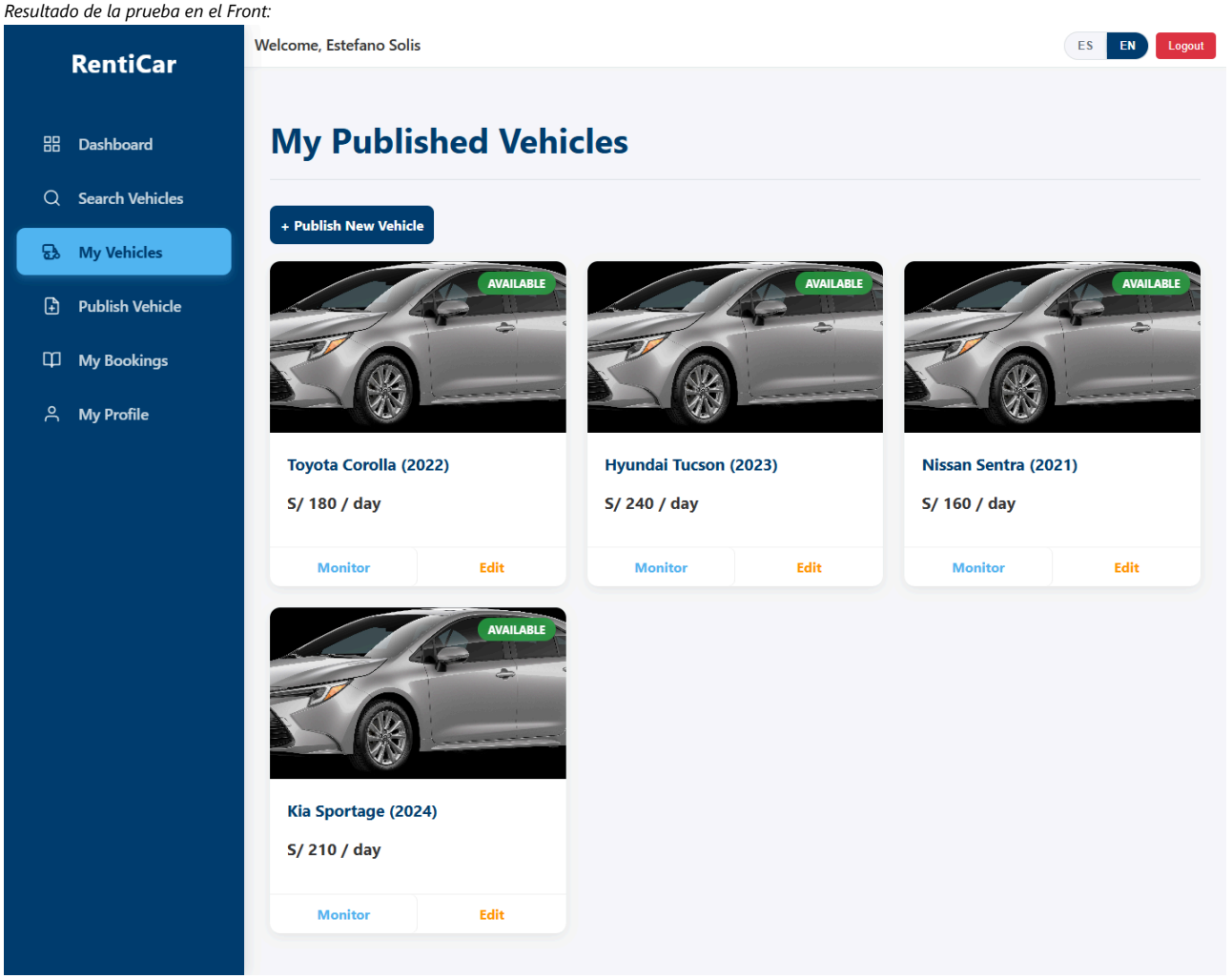


  - *Ejecución de escenarios en IntelliJ IDEA (`RunCucumberTestSignIn`):*

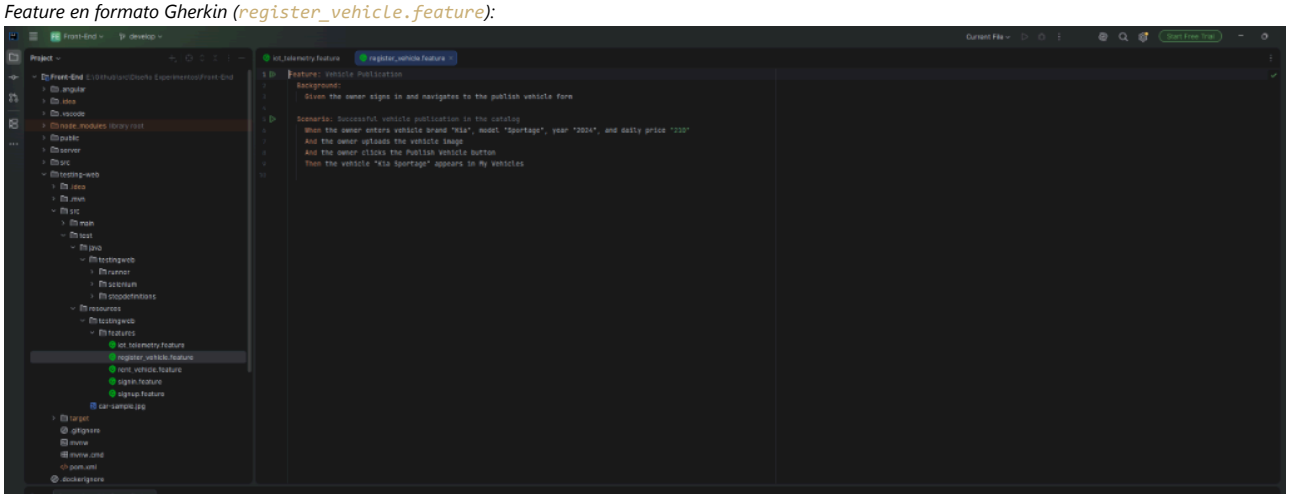


3. Flujo de Publicación de Vehículo (RegisterVehicle)

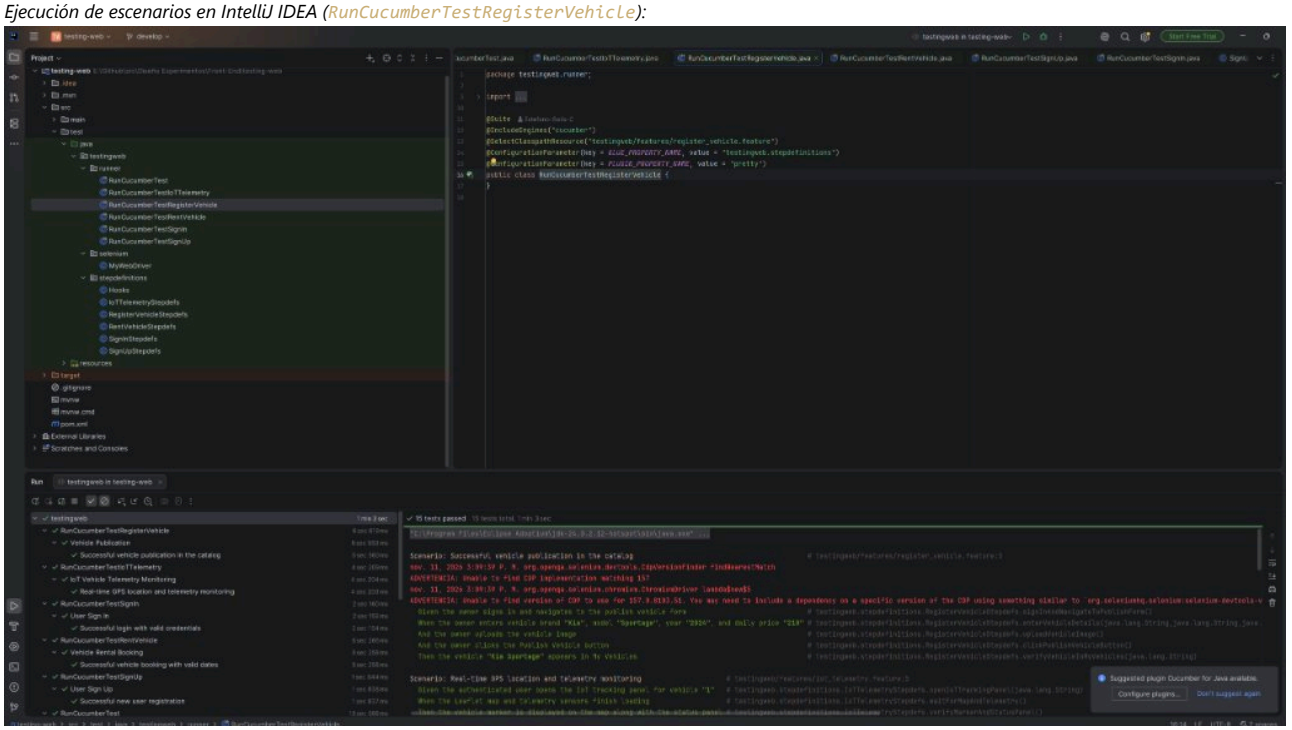
- **Evidencias de Ejecución:**
  - *Resultado de la prueba en el Front:*



  - *Feature en formato Gherkin (`register_vehicle.feature`):*



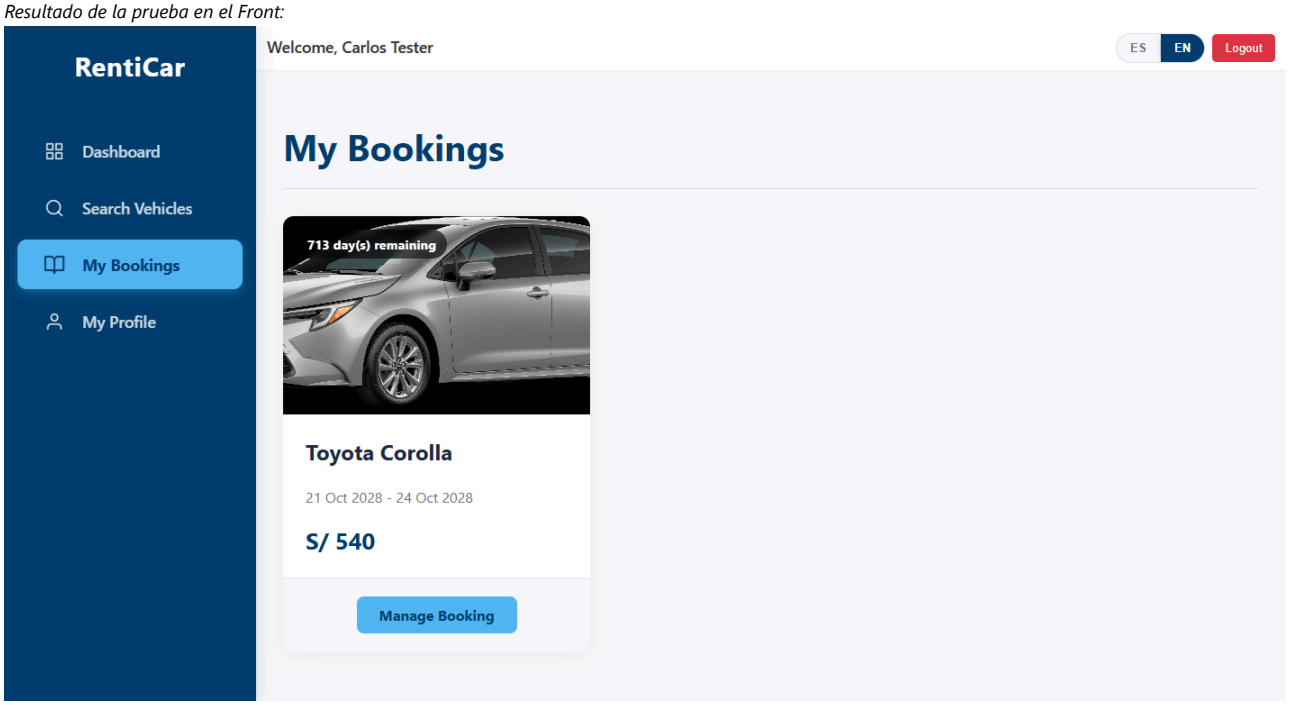
- *Ejecución de escenarios en IntelliJ IDEA (`RunCucumberTestRegisterVehicle`):*



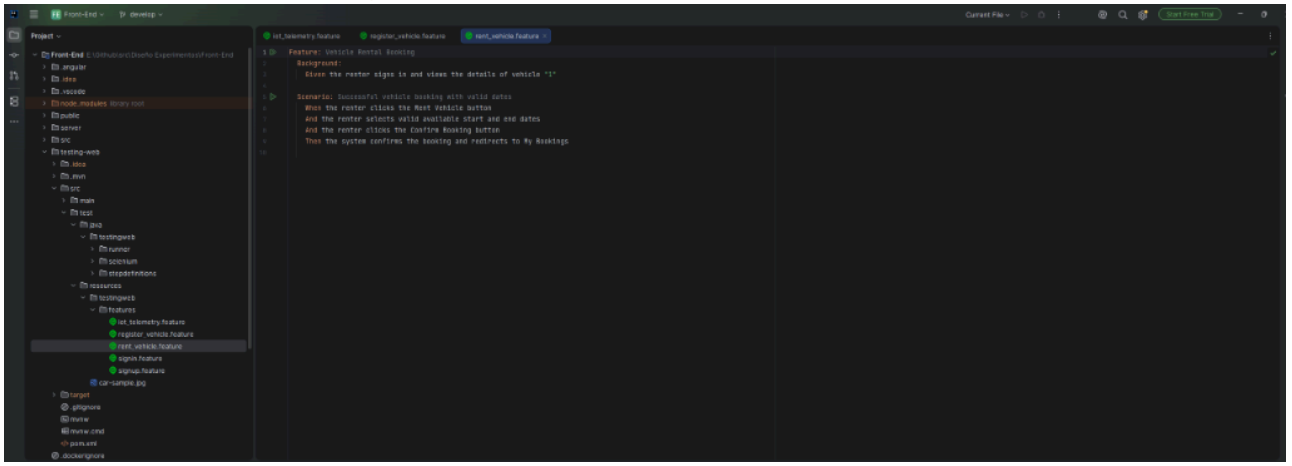
---

4. Flujo de Reserva de Vehículo (RentVehicle)

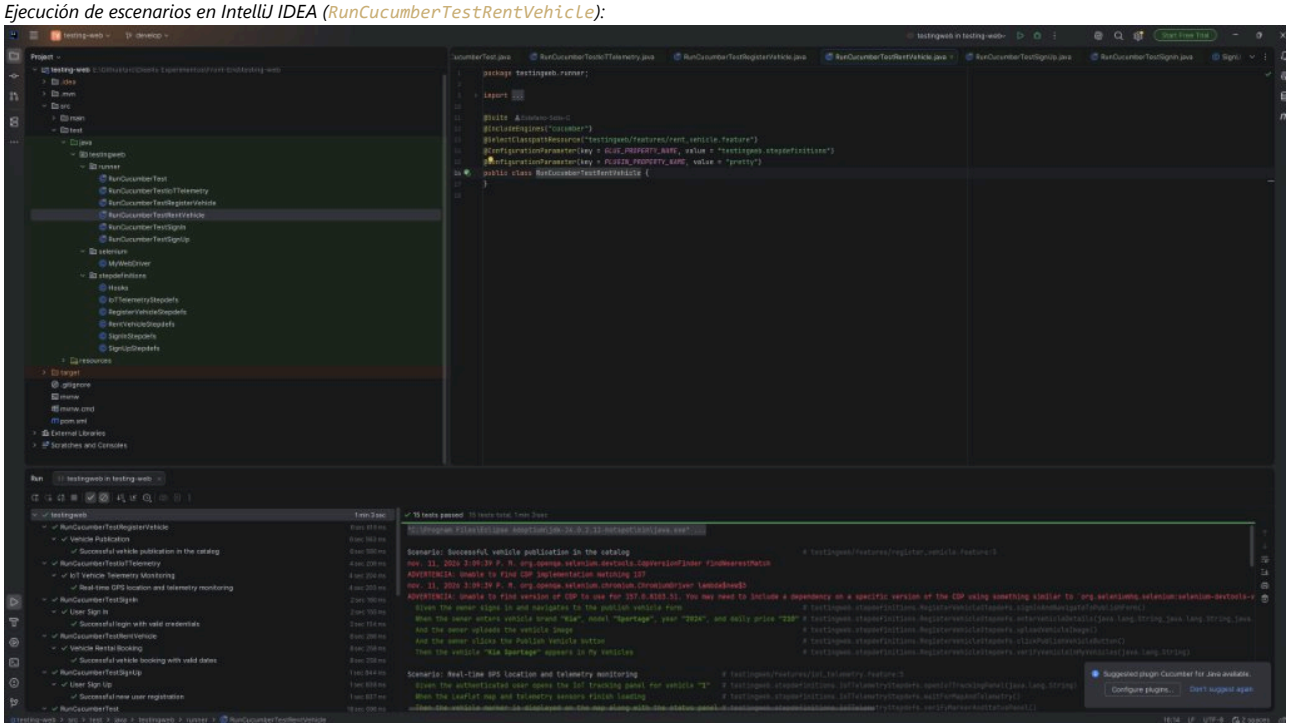
- **Evidencias de Ejecución:**
  - *Resultado de la prueba en el Front:*



- *Feature en formato Gherkin (`rent_vehicle.feature`):*



- *Ejecución de escenarios en IntelliJ IDEA (`RunCucumberTestRentVehicle`):*

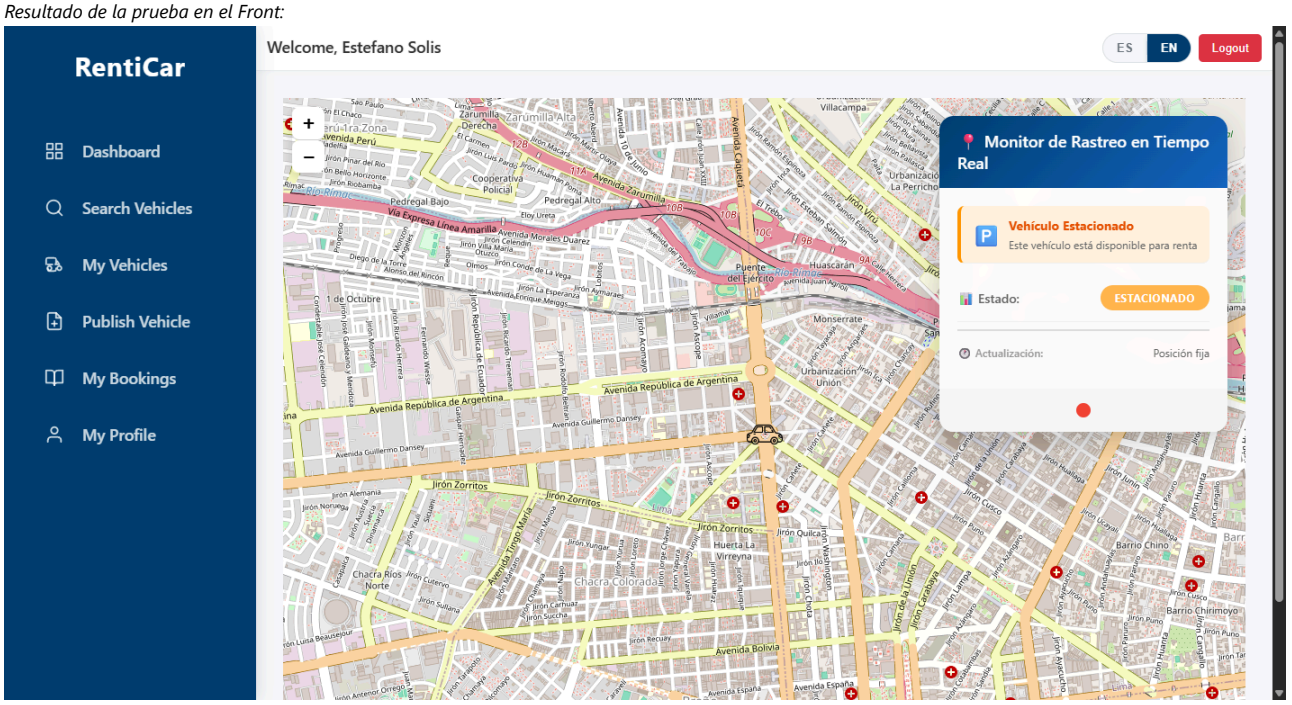


---

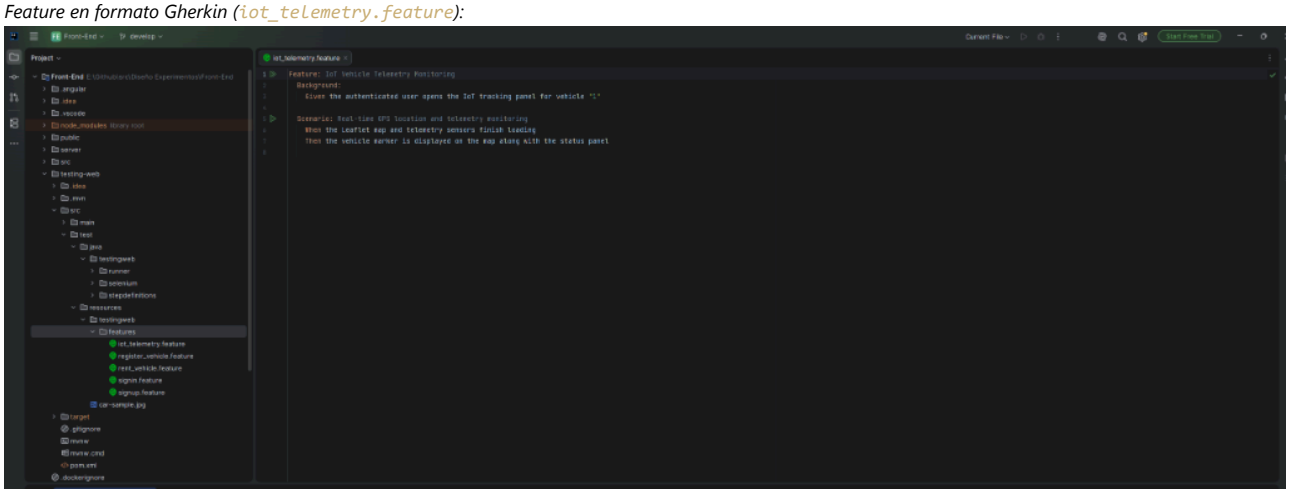
5. Flujo de Monitoreo de Telemetría IoT (IoTTelemetry)

- **Evidencias de Ejecución:**

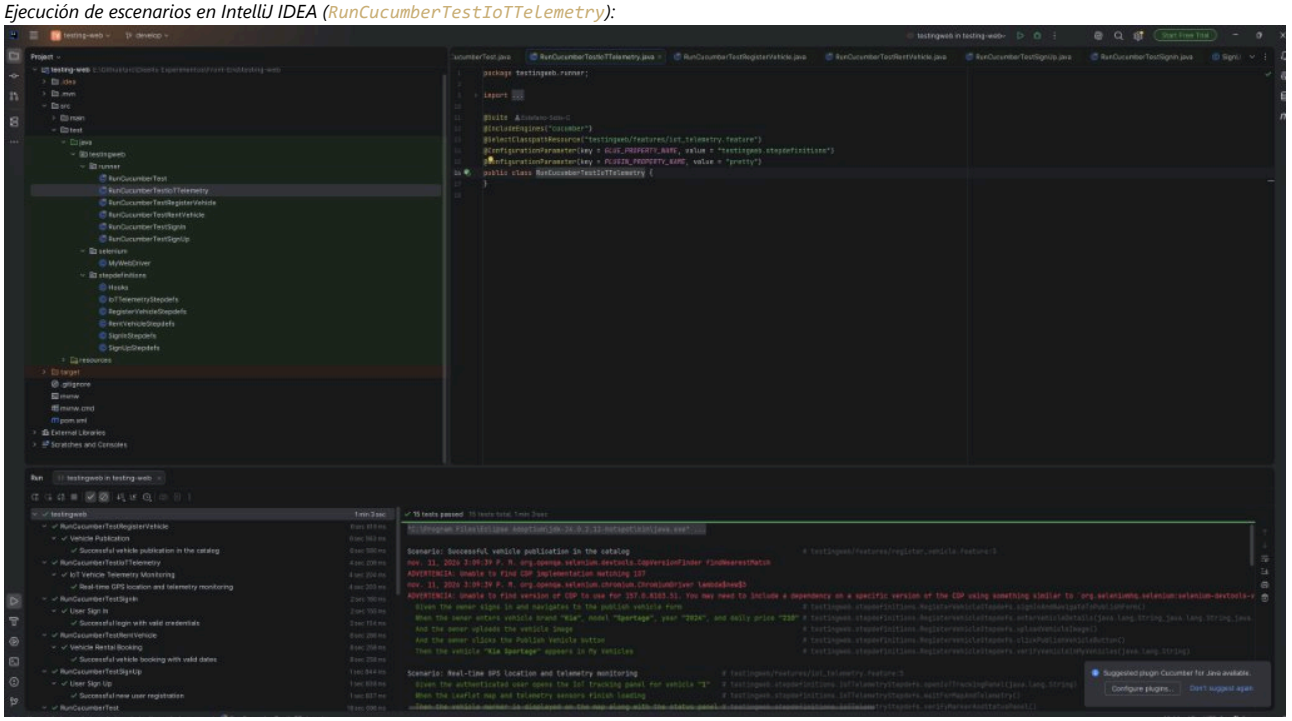
- *Resultado de la prueba en el Front:*



- *Feature en formato Gherkin (`iot_telemetry.feature`):*



- *Ejecución de escenarios en IntelliJ IDEA (`RunCucumberTestIoTTelemetry`):*



6.2. Static testing & Verification

Esta sección reúne las prácticas de verificación estática aplicadas al código fuente de RentiCar para detectar errores de formato, convenciones y diseño antes de compilar o integrar cambios.

6.2.1. Static Code Analysis

El análisis estático en RentiCar se configura directamente en la fase de validación de Maven (`validate`) y en los editores del equipo antes de enviar cambios al repositorio.

6.2.1.1. Coding standard & Code conventions

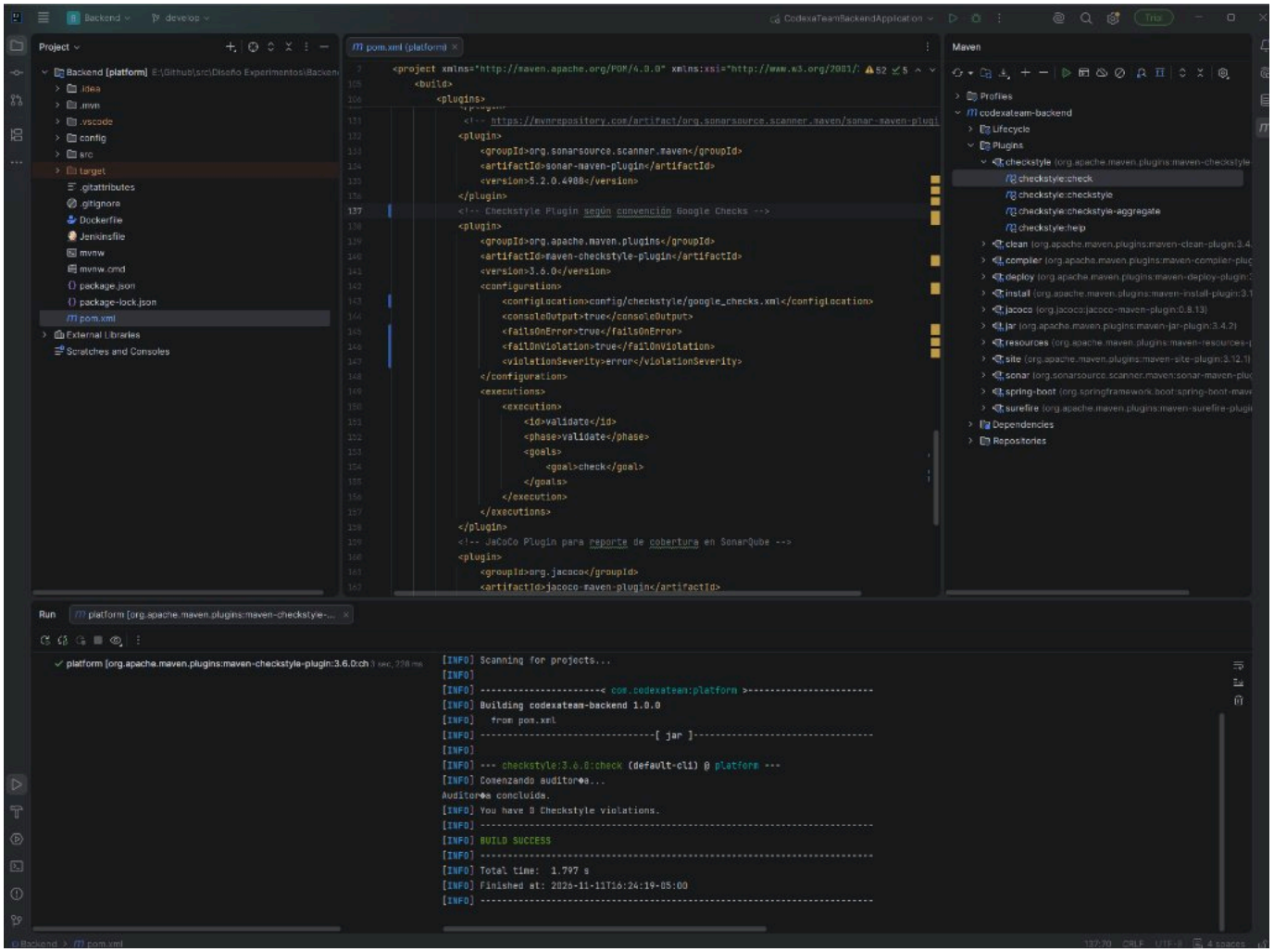
Las convenciones de código definen reglas uniformes de formato, nomenclatura y estructura entre el backend en Java y el frontend en Angular.

Lenguaje / Stack	Estándar / Convención Adoptada	Herramienta de Aplicación	IDE / Editores Usados
Java 21 (Spring Boot 3.5.7)	**Google Java Style Guide** (`config/checkstyle/google_checks.xml`, 4 espacios y máx. 120 caracteres)	**CheckStyle** (`maven-checkstyle-plugin:3.6.0`)	**IntelliJ IDEA**
TypeScript / HTML (Angular 20)	Convenciones de formato de Angular (`printWidth: 100`, comillas simples en `.ts`)	**EditorConfig** (`.editorconfig`) y **Prettier** (`package.json`)	**IntelliJ IDEA**, **Visual Studio Code**

Principios de Codificación Clave:

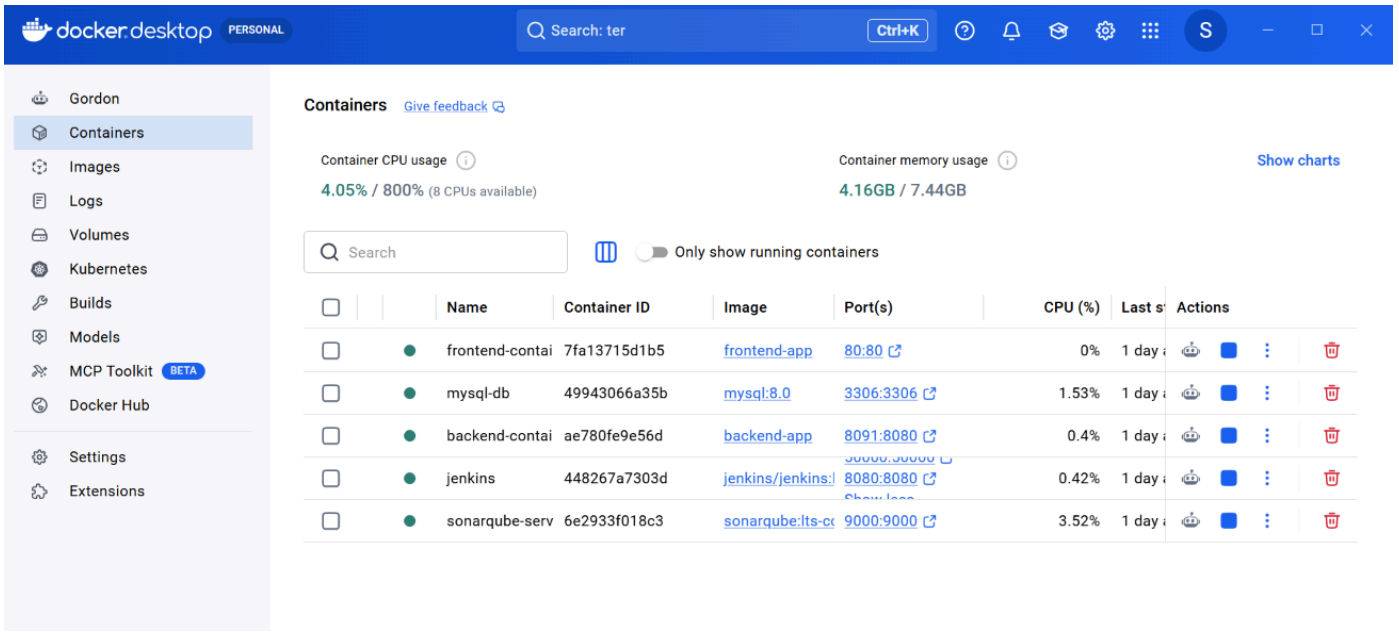
- **Clean Code:** Se utilizan nombres descriptivos (`camelCase` en variables y métodos, `PascalCase` en clases, `SCREAMING_SNAKE_CASE` en constantes) y se evita código muerto o importaciones con comodín.
- **Domain-Driven Design (DDD) (Backend Java):** La lógica sigue el lenguaje ubicuo del negocio en cinco bounded contexts (`iam`, `listings`, `booking`, `iot`, `reviews`), con separación CQRS entre `*CommandServiceImpl` y `*QueryServiceImpl` y comunicación entre contextos mediante fachadas ACL (`ListingsContextFacade`, `BookingContextFacade`).
- **Convenciones en Archivos:** Las clases Java siguen `PascalCase` (`Vehicle.java`, `Booking.java`, `VehiclesController.java`) y los paquetes siguen `lowercase` bajo `com.codexateam.platform`.

En la siguiente captura se observa la configuración de `maven-checkstyle-plugin` (`3.6.0`) en `Backend/pom.xml` apuntando a `config/checkstyle/google_checks.xml` y la ejecución de `checkstyle:check` en IntelliJ IDEA con `0` violaciones (`BUILD SUCCESS`):



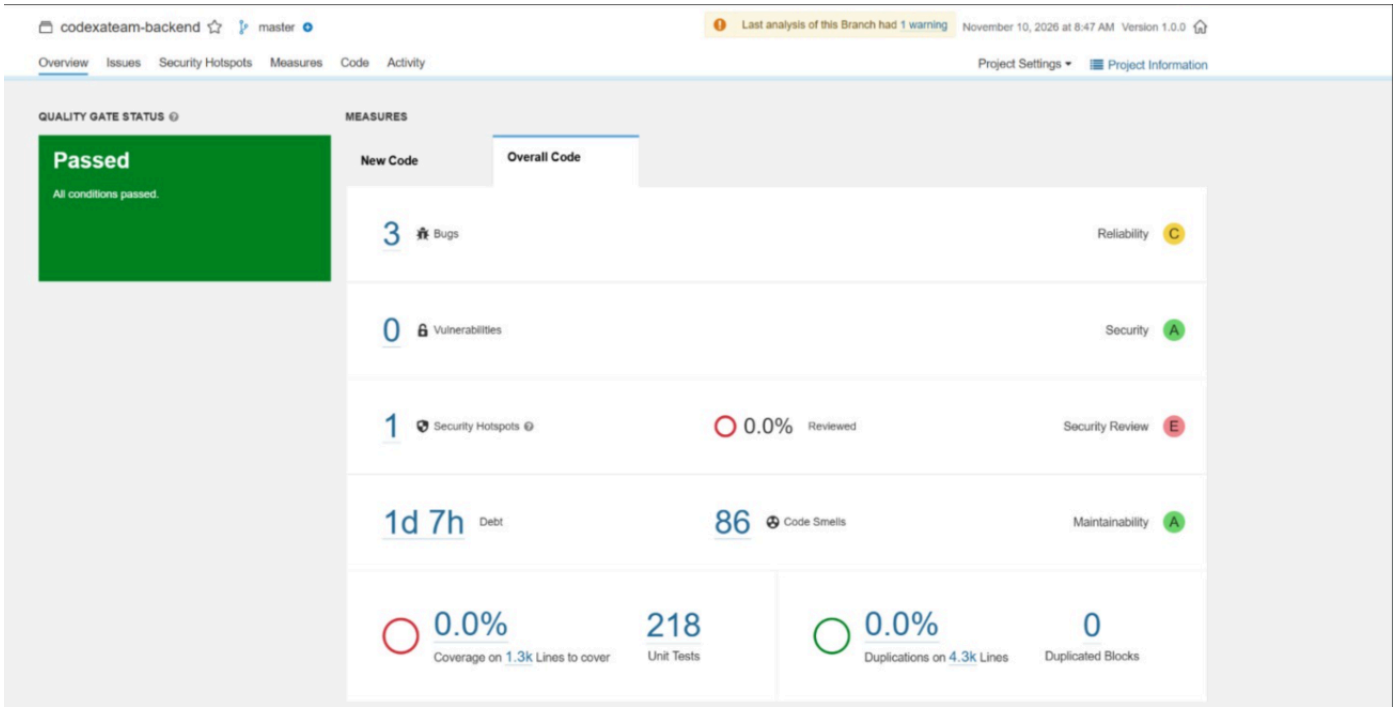
6.2.1.2. Code Quality & Code Security

Para asegurar la mantenibilidad, seguridad y calidad del código en el proyecto, se integró SonarQube. El proceso se estructuró en dos fases principales: primero, la contenedorización de los servicios mediante Docker, y posteriormente la ejecución del análisis de calidad para validar métricas de código limpio, vulnerabilidades y cobertura.



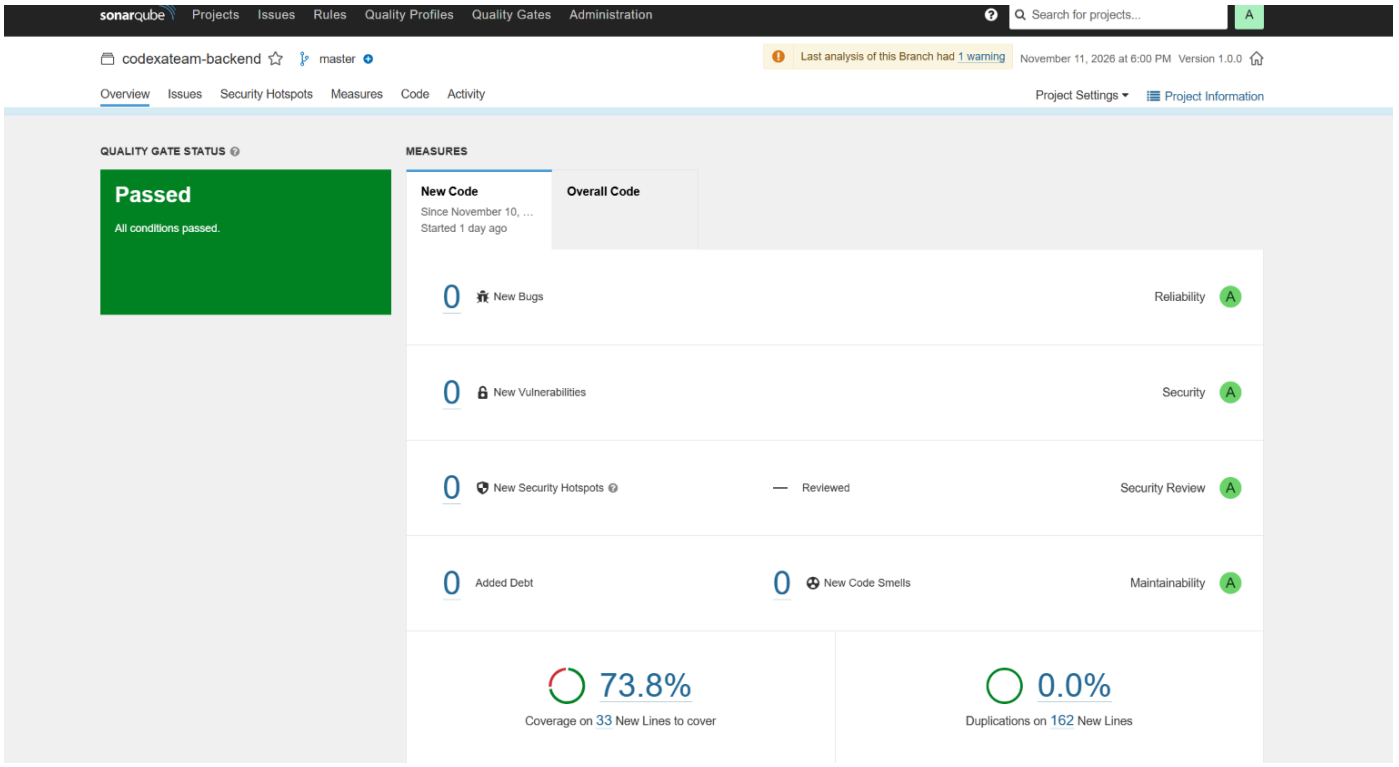
Como se muestra en la imagen dockerizamos tanto el backend que sera nuestro codigo a analizar y el SonarQube, para garantizar que funcione exactamente igual en tu máquina de desarrollo local, en un servidor de pruebas o en un pipeline de CI/CD en la nube.

Primera ejecuciòn en SonarQube



Como se muestra en la imagen el SonarQube analiza el codigo del backend y nos muestra errores encontrados, como es el caso de 3 bugs y 1 Security Hostpost.

Segunda ejecuciòn en SonarQube



Como se muestra en la imagen el SonarQube analiza el codigo del backend despues de haber corregido los errores previamente mostrados, SonarQube analizo el check Style y cumplio con cubrir el minimo requerido del còdigo. Ademas ya no se muestran los errores previamente encontrados, lo cual nos confirma que si cumple con todos los estandares de calidad altamente requeridos.

6.2.2. Reviews

Las revisiones de código de Codexa-Team se realizan en GitHub sobre el flujo GitFlow definido en la sección 5.1.2, validando los cambios de las ramas `feat/*` y `fix/*` antes de integrarlos en `develop` y `main`. Para garantizar que ninguna integración degrade la calidad del software, el equipo combina **revisiones manuales entre pares** con **verificación estática automatizada** mediante **CheckStyle** y **SonarQube**.

Tipos de Revisiones y Herramientas

Tipo de Revisión	Descripción y Propósito	Herramientas Utilizadas
Verificación Estática Local (Estilo)	Validación de reglas de estilo Google Java Checks mediante Maven (`.\mvnw.cmd checkstyle:check`) previa al commit o apertura del Pull Request.	CheckStyle (`maven-checkstyle-plugin:3.6.0`), IntelliJ IDEA
Inspección de Calidad y Seguridad	Análisis estático profundo ejecutado en contenedores Docker para detectar *bugs*, *code smells*, vulnerabilidades, *Security Hotspots* y validar cobertura de pruebas antes del merge.	SonarQube Community Build (Docker), `sonar-maven-plugin`, JaCoCo
Revisión Manual entre Pares (Peer Review)	Un integrante revisa el *diff* en GitHub durante la integración entre ramas (`feat/* -> develop -> main`), verificando legibilidad, lenguaje ubicuo del dominio y separación DDD/CQRS.	GitHub (Pull Requests / Commits)

Criterios de Aceptación para Integrar Cambios

Para que un *Pull Request* sea aprobado y fusionado hacia `develop` o `main`, debe cumplir obligatoriamente con las siguientes condiciones:

- **Estilo sin violaciones en Backend (CheckStyle):** El comando `.\mvnw.cmd checkstyle:check` debe finalizar en `BUILD SUCCESS` con 0 errores bajo `config/checkstyle/google_checks.xml` (sin importaciones con comodín `*`, indentación de 4 espacios y longitud máxima de 120 caracteres).
- **Quality Gate Aprobado en SonarQube:** El escaneo estático (`.\mvnw.cmd sonar:sonar`) debe reportar estado **Passed**, con **0 Bugs**, **0 Vulnerabilidades**, **0 Security Hotspots** sin revisar y cumpliendo el umbral mínimo de cobertura de código generado por JaCoCo.
- **Compilación y pruebas en verde:** El código integrado no debe romper la compilación en Maven (Backend) o Angular (Front-End) ni las pruebas unitarias/integración asociadas al cambio.
- **Frecuencia:** La revisión se ejecuta de forma incremental al concluir cada historia de usuario o tarea técnica antes de realizar el *merge*.

Flujo de Revisión Aplicado (Caso de Corrección e Integración)

Durante el ciclo de desarrollo, este proceso permitió detectar y corregir incidencias antes de consolidar los cambios en la rama principal:

1. **Detección en revisión previa al merge:** Al analizar una rama de funcionalidad, CheckStyle validó el formato del código, mientras que SonarQube identificó **3 bugs** y **1 Security Hotspot** en el backend.
2. **Subsanación y re-validación:** El autor corrigió las observaciones en su rama de trabajo y volvió a ejecutar el análisis estático, logrando **0 violaciones de CheckStyle** y el estado **Passed en SonarQube**.
3. **Aprobación en GitHub:** Con las métricas limpias verificadas, el revisor aprobó e integró el Pull Request hacia `develop`.

6.3. Validation Interviews

6.3.1. Diseño de Entrevistas

Entrevistas por Segmento

Segmento 1: Dueño del Automovil

1. ¿Qué tan fácil te resultó registrarte y entender para qué servía RentiCar como cliente?
2. ¿Cómo fue tu experiencia al navegar desde el Home hasta subir su automovil?
3. ¿Te sirvió el módulo de solicitudes para saber cuándo ocuparan el automovil?
4. ¿Te parecio util la opcion de editar los datos y precios de tu automovil?
5. ¿Pudiste identificar sin problemas las diferentes secciones y sus funcionalidades dentro de la pagina?
6. ¿Sentiste que el diseño de la app era confiable y profesional desde el primer momento?
7. ¿Qué tan intuitiva le parecio la distribucion de botones? ¿Qué funcion añadiria?
8. ¿La función de geolocalización y mapa te ayudó verificar mejor el estado de tu automovil?
9. ¿Te sentiste cómodo aceptando a alguien por internet directamente desde la plataforma?
10. ¿Te fue util el como se comunican las solicitudes entres los dos usuarios para el manejo de la reserva?
11. ¿Después de recibir el servicio, sentiste necesaria una opcion de calificar al arrendador?
12. ¿Qué cambiarías en la experiencia visual o funcional para que RentiCar sea aún más clara o rápida de usar?

Segmento 2: Usuario que busca alquilar un vehiculo

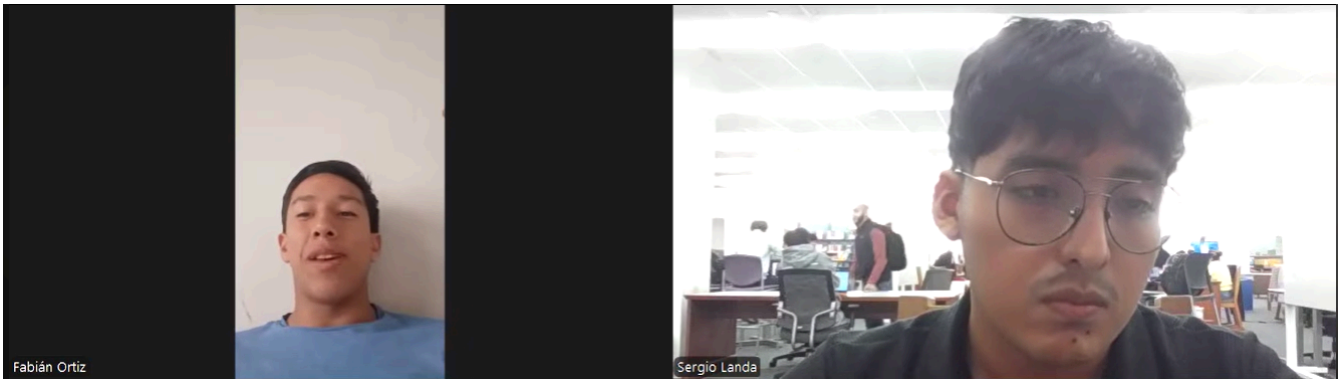
1. ¿Qué tan fácil te resultó registrarte y entender para qué servía RentiCar como cliente?
2. ¿Cómo fue tu experiencia al navegar de navegacion entre las secciones?
3. ¿Te sirvió el módulo automoviles disponibles para seleccionar el que mas se acomode a tus necesidades?
4. ¿Te parecio util la opcion de ver a mas detalles los automoviles de su interes?
5. ¿Pudiste identificar sin problemas las diferentes secciones y sus funcionalidades dentro de la pagina?
6. ¿Sentiste que el diseño de la app era confiable y profesional desde el primer momento?
7. ¿Qué tan intuitiva le parecio la distribucion de botones? ¿Qué funcion añadiria?
8. ¿La función de actualziar el estado de la reserva le parecio util?
9. ¿Te sentiste cómodo aceptando el automovil de alguien por internet a travez de la plataforma?
10. ¿Te sirvió la opcion para la renovacion de la reserva?
11. ¿Después de recibir el servicio, sentiste util la opcion de calificar al arrendador?
12. ¿Qué cambiarías en la experiencia visual o funcional para que RentiCar sea aún más clara o rápida de usar?

6.3.2. Registro de Entrevistas

Entrevista a arrendador de vehiculos

	Entrevista 1
Entrevistado	Fabian Ortiz
Edad	22 años
Distrito	San Miguel
Timing	0:00-6:16
URL	https://youtu.be/0rgB42iOyd0

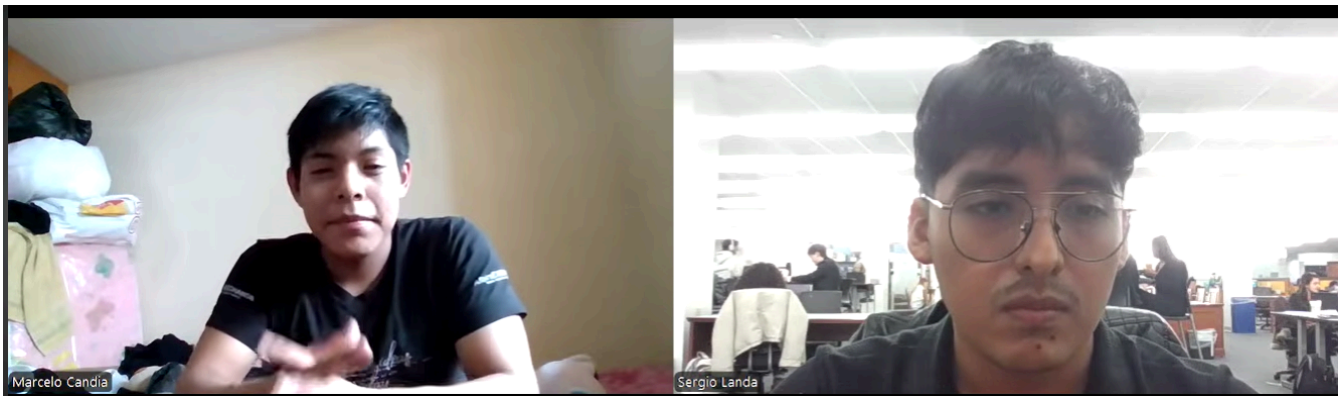
Resumen:
El entrevistado destacó que el proceso de registro y la validación de identidad generan un entorno seguro y de confianza. En términos de usabilidad, resaltó la fluidez de la navegación móvil, la eficacia de los filtros de búsqueda y el diseño limpio de la interfaz, elementos que transmiten profesionalismo al momento de gestionar métodos de pago. Además, valoró positivamente la autonomía para monitorear o extender reservas directamente desde la aplicación y el sistema de calificaciones mutuas que fortalece a la comunidad. Finalmente, como oportunidades de mejora para optimizar la herramienta, sugirió implementar una función de favoritos, permitir la comparación de vehículos y visualizar las tarifas diarias directamente en los pines del mapa interactivo.



Entrevista a arrendatario de vehiculo

Entrevista 1	
Entrevistado	Marcelo Candia
Edad	25 años
Distrito	Pueblo Libre
Timing	0:00-6:58
URL	https://youtu.be/_PdoxDvojJM

Resumen:
Marcelo Candia, un arrendador de 25 años e ingeniero de sistemas, validó la plataforma "RentiCar" destacando la fluidez de su interfaz y el registro en pasos cortos para publicar vehículos, así como la utilidad del calendario gráfico para organizar sus reservas. En términos de seguridad y confianza, resaltó la tranquilidad que le brinda la geolocalización en tiempo real, la validación de identidad de los conductores, las reseñas previas y la necesidad del sistema de calificación al arrendatario para constatar las condiciones de devolución, además de valorar el chat interno para coordinar las entregas. Como oportunidades de mejora, sugirió habilitar la carga múltiple de fotos, incorporar un botón rápido para pausar temporalmente la publicación de un auto por mantenimiento y añadir un panel que muestre los ingresos mensuales acumulados de un solo vistazo.



6.3.3. Evaluaciones según heurísticas.

Evaluación de Heurísticas y Principios de UX

Curso: 1ASI0729 - Desarrollo de Aplicaciones Open Source
Profesorado: Bautista Ubillús, Efraín Ricardo; Castro Veramendi, Rafael Oswaldo; Espinoza Bravo, Wilder Julio; Flores Moroco, Juan Antonio; Mori Paiva, Hugo Allan; Robles Fernández, Iván; Velásquez Núñez, Ángel Augusto
Cliente: RentiCar

Tareas Evaluadas:

Esta evaluación se enfoca en la usabilidad y experiencia de usuario de las siguientes tareas en la aplicación:

Landing Page (Desktop)

- Presentación general de la plataforma y sus funcionalidades.
- Descripción del equipo de trabajo.
- Funcionalidad de los elementos de navegación (header y footer).
- Eficiencia de la llamada a la acción (Call to Action - CTA).

Aplicación Web (Desktop)

- Gestión de inventarios: Adición de elementos.
- Visualización del dashboard de arrendadores y arrendatarios.
- Agregar y gestionar asistencias de vehículos.
- Visualización de horarios de los empleados.

Exclusiones para Esta Evaluación:

Landing Page (Desktop):

- Enlaces a términos y condiciones.
- Política de privacidad y condiciones de uso.

Aplicación Web (Desktop):

- Visualización del perfil de usuario.
- Descripción detallada del proceso de creación de asistencias.
- Gestión del inventario.
- Registro de nuevos usuarios.

Escala de Severidad de los Problemas Identificados:

Los problemas identificados durante la evaluación de usabilidad se clasificaron de acuerdo con la siguiente escala:

Nivel	Descripción
1	Problema superficial: puede ser fácilmente superado por el usuario y ocurre raramente.
2	Problema menor: ocurre con algo más de frecuencia y es moderadamente difícil de superar.
3	Problema mayor: ocurre con frecuencia y los usuarios no pueden resolverlo por sí mismos.
4	Problema grave: error de alto impacto que impide el uso de la plataforma.

Tabla Resumen de Problemas y Heurísticas Violadas:

Aplicación Web

#	Problema	Severidad	Heurística/Principio Violado
1	Los filtros de búsqueda son limitados y no permiten una búsqueda avanzada de vehículos.	2	Usabilidad - Flexibilidad y eficiencia de uso
2	La visualización de la disponibilidad de los vehículos no es clara ni en tiempo real.	3	Usabilidad - Retroalimentación y estado del sistema
3	Los arrendadores tienen dificultades para modificar los detalles de sus vehículos.	2	Usabilidad - Consistencia y estándares
4	Falta un mapa interactivo para visualizar la ubicación y disponibilidad de los vehículos.	3	IA - ¿Es encontrable?

Landing Page

#	Problema	Severidad	Heurística/Principio Violado
1	El texto descriptivo sobre la plataforma no tiene suficiente claridad, dificultando la comprensión.	1	IA - ¿Es claro?
2	Los botones de call to action no comunican claramente la acción esperada, afectando la interacción.	2	IA - ¿Es usable?
3	Las cards de los vehículos no incluyen información completa, como el número de placa.	2	Usabilidad - Consistencia y estándares
4	El diseño visual no tiene una jerarquía clara, dificultando la navegación de los usuarios.	3	IA - ¿Es encontrable?

Descripción de Problemas y Recomendaciones:

Landing Page

Problema 1:

- *Severidad:* 1
- *Heurística violada:* IA - ¿Es claro?
- *Descripción:* El texto de la página de aterrizaje es ambiguo, dificultando que los usuarios comprendan de inmediato la propuesta de valor de la plataforma.
- *Recomendación:* Mejorar la estructura y claridad del texto, utilizando títulos más claros y descripciones concisas. Usar listas y puntos destacados para facilitar la comprensión rápida.

Problema 2:

- *Severidad:* 2
- *Heurística violada:* IA - ¿Es usable?
- *Descripción:* Los botones de acción ("Reservar ahora") no son lo suficientemente claros ni atractivos, lo que genera dudas sobre su función.
- *Recomendación:* Rediseñar los botones de call to action con textos más específicos y visibles, como “Reserva tu vehículo ahora”, para transmitir claramente lo que ocurrirá al hacer clic.

Problema 3:

- *Severidad:* 2
- *Heurística violada:* Usabilidad - Consistencia y estándares
- *Descripción:* Las cards de los vehículos no contienen todos los detalles relevantes (por ejemplo, el número de placa), lo que puede generar confusión o desconfianza en los usuarios.
- *Recomendación:* Asegurarse de que cada card incluya toda la información básica, como el número de placa, el modelo del vehículo, y otros datos clave, de manera coherente en todas las opciones.

Problema 4:

- *Severidad:* 3
- *Heurística violada:* IA - ¿Es encontrable?
- *Descripción:* La jerarquía visual no está bien definida, lo que dificulta la navegación y la identificación rápida de los elementos más importantes.
- *Recomendación:* Rediseñar la página para establecer una jerarquía visual clara, asegurando que las secciones más importantes, como el call to action, sean fácilmente accesibles.

Aplicación Web

Problema 1:

- *Severidad:* 2
- *Heurística violada:* Usabilidad - Flexibilidad y eficiencia de uso
- *Descripción:* Los filtros de búsqueda disponibles no permiten una búsqueda avanzada por características específicas del vehículo, lo que dificulta a los usuarios encontrar opciones específicas.
- *Recomendación:* Mejorar el sistema de búsqueda añadiendo filtros más detallados, como tipo de vehículo, precio, ubicación, y características específicas.

Problema 2:

- *Severidad:* 3
- *Heurística violada:* Usabilidad - Retroalimentación y estado del sistema
- *Descripción:* La plataforma no muestra la disponibilidad de los vehículos en tiempo real, lo que causa incertidumbre en los arrendatarios.
- *Recomendación:* Implementar un sistema de retroalimentación en tiempo real que actualice constantemente la disponibilidad de los vehículos y muestre mensajes claros cuando no haya vehículos disponibles.

Problema 3:

- *Severidad:* 2
- *Heurística violada:* Usabilidad - Consistencia y estándares
- *Descripción:* Los arrendadores tienen dificultades para modificar los detalles de sus vehículos debido a la falta de consistencia en la interfaz.
- *Recomendación:* Estandarizar los formularios y procesos para la edición de vehículos, asegurándose de que el flujo sea sencillo y consistente con el resto de la plataforma.

Problema 4:

- *Severidad:* 3
- *Heurística violada:* IA - ¿Es encontrable?
- *Descripción:* La falta de un mapa interactivo impide a los arrendatarios localizar los vehículos disponibles fácilmente.
- *Recomendación:* Implementar un mapa interactivo para que los arrendatarios puedan visualizar los vehículos disponibles en su área, mejorando así la accesibilidad y la facilidad de uso de la plataforma.

Estas mejoras y recomendaciones están diseñadas para optimizar la experiencia del usuario, mejorar la accesibilidad y garantizar la eficiencia de las funcionalidades clave de la plataforma. La implementación de estas sugerencias fortalecerá la plataforma, haciéndola más intuitiva y eficiente para todos los usuarios. Con estos cambios, la evaluación de las heurísticas estará más alineada con los estándares profesionales y será clara y efectiva en su presentación.

6.4. Auditoría de Experiencias de Usuario

6.4.1. Auditoría realizada

En el marco de la colaboración inter-equipos de la carrera, el equipo de **Codexa-Team** llevó a cabo una auditoría heurística completa sobre el producto digital desarrollado por el grupo homólogo asignado.

6.4.1.1. Información del grupo auditado

A continuación se documenta la revisión interna de la aplicación web de ElectroLink.

Campo	Detalle informativo
Grupo evaluador	G2 ElectroLink (autoauditoría).
Producto evaluado	ElectroLink Web, aplicación Angular ejecutada localmente en http://127.0.0.1:4200/.
Grupo auditado	G2 ElectroLink.
Alcance	Navegación entre ingreso y registro, identificación de campos y control de visibilidad de contraseña.
Método	Inspección heurística de las plantillas y rutas, más comprobación del enlace de registro en el DOM renderizado.

6.4.1.2. Cronograma de auditoría realizada

Hito	Fecha	Participantes	Objetivo y resultado
Revisión de navegación	11/11/2026	Equipo G2	Comparar los enlaces visibles con las rutas declaradas en app.routes.ts; se detectaron dos destinos inexistentes.
Inspección de formularios	11/11/2026	Equipo G2	Revisar etiquetas y controles de ingreso y registro; se identificaron problemas de accesibilidad y un botón sin acción.
Subsanación y verificación técnica	11/11/2026	Equipo G2	Corregir los tres hallazgos y ejecutar npm run build, que terminó sin errores.

6.4.1.3. Contenido de auditoría realizada

La severidad se expresa en una escala de 1 a 3: *1* corresponde a una molestia menor, *2* a una dificultad que afecta la tarea y *3* a un obstáculo importante para completarla. La revisión se limitó a la aplicación web y a las pantallas indicadas en el alcance.

1. Evaluación de la navegación entre ingreso y registro

#	Problema identificado	Severidad	Heurística o principio afectado
AU-01	"Regístrate" llevaba a /register e "Inicia sesión" a /login; ninguna de esas rutas existe. La redirección general devolvía al usuario al inicio.	3	Consistencia y estándares; control y libertad del usuario.

2. Evaluación de los formularios de la aplicación web

#	Problema identificado	Severidad	Heurística o principio afectado
AU-02	Las etiquetas visibles no estaban asociadas a los campos mediante for e id, lo que dificultaba identificar los controles con tecnologías de asistencia.	2	Accesibilidad; reconocimiento de la información.
AU-03	El botón con icono de visibilidad de contraseña no cambiaba el estado del campo.	2	Consistencia y estándares; correspondencia entre control y resultado.

Descripción de problemas y recomendaciones

- *AU-01 — Navegación (severidad 3):* app.routes.ts define /sign-up y /sign-in, mientras que las plantillas enlazaban a /register y /login. El enlace de registro se confirmó también en el DOM de la aplicación local. *Recomendación:* enlazar cada acción con su ruta existente.
- *AU-02 — Etiquetas de campos (severidad 2):* los nombres de los campos aparecían como etiquetas visuales sin asociación explícita con el control correspondiente. *Recomendación:* usar for e id en los campos de texto y un nombre accesible para el selector de rol.
- *AU-03 — Visibilidad de contraseña (severidad 2):* el botón mostraba un icono de ojo, pero carecía de una acción que alternara el tipo del campo. *Recomendación:* permitir mostrar y ocultar la contraseña y comunicar el estado del botón con una etiqueta accesible.

6.4.2. Auditoría recibida

A continuación, se documenta la auditoría heurística recibida por **Codexa-Team** respecto a nuestra plataforma **RentiCar**, evaluando tanto la Landing Page como la Aplicación Web (SPA).

6.4.2.1. Información del grupo auditor

Campo	Detalle Informativo
Grupo Evaluador:	Equipo Evaluador Par - Curso 1ASI0729
Producto Evaluado:	**RentiCar** (Landing Page Desktop y Aplicación Web Angular)
Cliente / Startup:	Codexa-Team
Evaluadores:	Estudiantes asignados para la auditoría cruzada
Alcance:	Usabilidad, consistencia de interfaz, claridad en la propuesta de valor y flujos de reserva

6.4.2.2. Cronograma de auditoría recibida

Hito / Sesión	Fecha	Participantes	Objetivo
Sesión de Demostración	16/10/2026	Codexa-Team y Grupo Auditor	Presentación guiada de los flujos principales de arrendatario y arrendador
Recepción del Informe Heurístico	22/10/2026	Estefano Solís, Sergio Landa	Recepción del reporte con los 8 problemas identificados y su severidad
Reunión de Análisis y Plan de Subsanación	25/10/2026	Equipo Codexa-Team	Evaluación técnica de factibilidad e incorporación al backlog de desarrollo

6.4.2.3. Contenido de auditoría recibida

1. Evaluación de la Landing Page

#	Problema Identificado	Severidad	Heurística / Principio Violado
1	El texto descriptivo sobre la plataforma no tiene suficiente claridad, dificultando la comprensión rápida de la propuesta de valor.	1	IA - ¿Es claro?
2	Los botones de *call to action* no comunican claramente la acción esperada, generando dudas sobre la interacción.	2	IA - ¿Es usable?
3	Las cards de los vehículos en la vista previa no incluyen información completa, como el número de placa y transmisión.	2	Usabilidad - Consistencia y estándares
4	El diseño visual no tiene una jerarquía clara, dificultando la navegación de los usuarios hacia los planes de alquiler.	3	IA - ¿Es encontrable?

2. Evaluación de la Aplicación Web

#	Problema Identificado	Severidad	Heurística / Principio Violado
1	Los filtros de búsqueda disponibles son limitados y no permiten una búsqueda avanzada por características específicas del vehículo.	2	Usabilidad - Flexibilidad y eficiencia de uso
2	La visualización de la disponibilidad de los vehículos no es clara ni en tiempo real, lo que causa incertidumbre en los arrendatarios.	3	Usabilidad - Retroalimentación y estado del sistema
3	Los arrendadores tienen dificultades para modificar los detalles de sus vehículos debido a la falta de consistencia en los formularios.	2	Usabilidad - Consistencia y estándares
4	La falta de un mapa interactivo impide a los arrendatarios localizar los vehículos disponibles en su área geográfica.	3	IA - ¿Es encontrable?

Descripción Detallada de Problemas y Recomendaciones Recibidas:

Landing Page:

- **Problema 1 (Severidad 1 - IA: ¿Es claro?):** El texto descriptivo inicial resulta genérico, sin enfatizar el valor diferencial de la telemetría IoT y los contratos seguros.
  - *Recomendación:* Mejorar la estructura y claridad del texto, utilizando titulares orientados al beneficio y listas de viñetas claras.
- **Problema 2 (Severidad 2 - IA: ¿Es usable?):** El botón de llamada a la acción tenía un texto genérico ("Entrar") que no motivaba la conversión.
  - *Recomendación:* Rediseñar los botones CTA con microcopia directa y atractiva, como "Reserva tu vehículo ahora".
- **Problema 3 (Severidad 2 - Usabilidad: Consistencia y estándares):** Las tarjetas de exhibición omitían metadatos relevantes como año del modelo y placa.
  - *Recomendación:* Estandarizar las cards vehiculares asegurando que contengan marca, modelo, año, precio por día y placa.
- **Problema 4 (Severidad 3 - IA: ¿Es encontrable?):** La jerarquía visual carecía de separación evidente entre beneficios para arrendadores y para arrendatarios.
  - *Recomendación:* Segmentar la página en dos bloques visualmente diferenciados con llamadas a la acción personalizadas por rol.

Aplicación Web:

- **Problema 1 (Severidad 2 - Usabilidad: Flexibilidad y eficiencia de uso):** El buscador solo permitía filtrar por marca.
  - *Recomendación:* Expandir el panel de filtros para permitir selección de rango de precios, tipo de transmisión, combustible y número de asientos.
- **Problema 2 (Severidad 3 - Usabilidad: Retroalimentación y estado del sistema):** Los vehículos alquilados seguían mostrándose disponibles hasta que el usuario intentaba seleccionarlos.
  - *Recomendación:* Actualizar el estado visual del vehículo de inmediato a "ALQUILADO" con indicador de color y deshabilitar el botón de reserva.
- **Problema 3 (Severidad 2 - Usabilidad: Consistencia y estándares):** Los formularios de creación y edición de flota utilizaban componentes visuales distintos.
  - *Recomendación:* Unificar el diseño utilizando componentes consistentes de Angular Material (`mat-form-field`, `mat-select`).
- **Problema 4 (Severidad 3 - IA: ¿Es encontrable?):** Los arrendatarios no podían visualizar la cercanía geográfica de los vehículos para la recogida.
  - *Recomendación:* Implementar un mapa interactivo con Leaflet que renderice pines de ubicación precisa de cada unidad disponible.

6.4.2.4. Resumen de modificaciones para subsanar hallazgos

A partir de los hallazgos recibidos en la auditoría, **Codexa-Team** priorizó e implementó las siguientes mejoras en la plataforma **RentiCar**:

1. **Incorporación de Mapa Interactivo con Leaflet:** Se integró un mapa interactivo sobre OpenStreetMap tanto en el catálogo de búsqueda como en el panel de telemetría IoT, permitiendo a los arrendatarios ubicar de inmediato las unidades vehiculares más cercanas a su ubicación actual.
2. **Sistema Avanzado de Filtros Multi-Criterio:** Se enriqueció la barra de búsqueda en Angular con filtros dinámicos por rango de tarifa diaria, tipo de transmisión, modelo y estado de disponibilidad en tiempo real.
3. **Estandarización UI en Flujos de Gestión de Flota:** Se rediseñaron los formularios de publicación y edición de vehículos bajo la directiva de Angular Material, garantizando consistencia en etiquetas, botones de guardado y mensajes de confirmación.
4. **Optimización de Microcopia y Jerarquía Visual:** En la Landing Page y en la aplicación, se actualizaron los botones principales hacia textos de acción claros ("Reserva tu vehículo ahora", "Publicar mi automóvil"), y se añadieron insignias visuales de placa y disponibilidad en cada tarjeta vehicular.

Capítulo VII: DevOps Practices

7.1. Continuous Integration

La Integración Continua (CI) es una práctica que consiste en unir automáticamente los cambios de código a una rama común de forma frecuente. Busca encontrar y resolver problemas de integración lo antes posible, para que el proyecto se mantenga estable y con buena calidad.

7.1.1. Tools and Practices.

Dentro de nuestro proceso, la Integración Continua se apoya en dos metodologías clave, BDD y TDD, que nos ayudan a asegurar que el código no solo cumpla con lo que el negocio necesita, sino también con los estándares técnicos exigidos.

Herramienta	Tipo	Descripción	Propósito en el Proceso
JUnit 5	Pruebas Unitarias (TDD)	*Framework* estándar de Java para escribir y ejecutar pruebas que validan el comportamiento de pequeñas unidades de código (métodos, clases).	Asegurar la **calidad interna y funcional** de los componentes del *backend* Java.
Mockito	Simulaciones (TDD)	Librería de *mocking* que permite crear *mocks* de dependencias externas (bases de datos, servicios, etc.).	Facilitar las pruebas unitarias aislando la lógica de negocio para ejecutarlas de forma rápida y confiable.
Karate	Pruebas de Integración y E2E (BDD)	*Framework* que combina la sintaxis BDD (Gherkin) con pruebas API/Web. Se usa en archivos .feature para validar la integración de *endpoints* y flujos de negocio.	Validar el comportamiento del **API REST** contra casos de uso definidos en lenguaje Gherkin.
Gherkin	Metodología BDD	Lenguaje estructurado para describir el comportamiento del *software* en términos de negocio (utilizado a través de Karate).	Garantizar que el desarrollo esté **alineado con las necesidades del negocio** al escribir los escenarios de aceptación.
IntelliJ IDEA	IDEs principales	Entornos de Desarrollo Integrado que, junto a *plugins* de BDD y Java, facilitan la escritura, depuración y ejecución local de las pruebas.	Aumentar la **productividad del desarrollador** y la ejecución inmediata de pruebas TDD/BDD.

7.1.2. Build & Test Suite Pipeline Components.

Aqui definimos y documentamos la arquitectura técnica, los componentes y la especificación del pipeline de Integración Continua (CI) implementado mediante el servidor de automatización Jenkins para nuestro proyecto Renticar. La automatización de este pipeline garantiza que cada incremento de código fuente enviado al repositorio central (Backend) sea sometido a un proceso riguroso de compilación, verificación estática y ejecución de suites de prueba automatizadas antes de su eventual integración en las ramas de despliegue.

Arquitectura y Fases del Pipeline de CI en Jenkins.

El pipeline de compilación y pruebas está configurado como un **Jenkins Pipeline** integrado en la raíz del repositorio. Las etapas (*stages*) que componen el flujo de CI en Jenkins se detallan a continuación:

1. **Disparador del Pipeline (Pipeline Trigger):** El job en Jenkins se activa automáticamente mediante un *GitHub Webhook* configurado ante eventos de tipo `push` o `pull_request` sobre las ramas.
2. **Aprovisionamiento del Entorno (Checkout & Environment Setup):** Jenkins descarga el código fuente del repositorio y prepara el entorno de ejecución dentro del agente (*node/runner*) configurando el JDK en su versión **Java 17 (Eclipse Temurin)** y mapeando el directorio de caché local de **Apache Maven**, optimizando los tiempos de construcción.
3. **Compilación y Verificación Sintáctica (Stage: Build):** Se ejecuta la fase de compilación del proyecto Spring Boot. Este paso valida que la sintaxis, las importaciones entre los Bounded Contexts y el tipado estático del código no presenten errores.
4. **Ejecución de Pruebas Unitarias (Unit Tests - TDD):** Se ejecutan los tests unitarios diseñados bajo el enfoque **Test-Driven Development (TDD)**. Esto aísla la lógica de negocio de las entidades de dominio y servicios de aplicación simulando sus dependencias mediante **Mockito**.
5. **Ejecución de Pruebas de Integración y BDD (Stage: Integration & BDD Tests):** Se procesan los escenarios de prueba escritos en lenguaje natural **Gherkin** mediante **Karate**. Esta suite evalúa el comportamiento real de las interfaces RESTful, validando las respuestas HTTP y las estructuras JSON retornadas por los endpoints de la API.
6. **Generación e Inspección de Cobertura (Stage: Code Coverage & Artifacts):** Aqui se calcula la métrica de cobertura de código alcanzada por las suites de prueba. Jenkins procesa los reportes con el plugin de JaCoCo/JUnit para publicar los resultados en el tablero del job y archiva los reportes HTML resultantes como **artefactos de la ejecución**.

7.2. Continuous Delivery

El objetivo de la Entrega Continua (CD) es automatizar la integración y pruebas del código, manteniendo la aplicación lista para un despliegue manual a producción en cualquier momento. A diferencia del Despliegue Continuo (Deployment), esta práctica introduce una aprobación final.

7.2.1. Tools and Practices

Herramienta	Tipo	Propósito en Continuous Delivery
Github Actions	Automatizacion CI/CD	Se usa para ejecutar el pipeline completo de build, test y staging. Se configura una etapa donde el despliegue final a producción es manual ademas de server junta a configuraciones y mejoras en la comunicacion y compartimiento de codigo o procesos durante el CI/CD

Herramienta	Tipo	Propósito en Continuous Delivery
Render	Plataforam de despliegue Backend y Frontend	Se utiliza para el despliegue del backend RESTful API y el frontend. Garantizando un entorno de producción estable y fácil de gestionar.
Docker	Contenerizacion	Aseguramiento de consistencia del entorno durante las fases del desarrollo
Trello	Gestion de desarrollo individual del proyecto	Se usa para gestionar el proceso de aprobación. Después de la validación del pipeline en staging, un Product Owner debe revisar el build y autorizar manualmente el movimiento del ticket a la columna de Despliegue a Producción

Practicas Clave:

- Feature Branching y Merge Requests: Los cambios se desarrollan en ramas separadas (feature/login-ui) y se integran a una rama estable (main o feature) solo después de pasar todas las pruebas de CI.
- Pipeline de Validación en Staging: Tras la CI exitosa, los artefactos se despliegan automáticamente a un entorno de Staging. Aquí, el equipo puede ejecutar pruebas exploratorias y los interesados pueden validar la funcionalidad, replicando las condiciones de producción (Render).
- Despliegue Semiautomático: El pipeline prepara la aplicación y las imágenes de Docker/artefactos para el despliegue final, pero no lo ejecuta. La acción de desplegar a producción se dispara únicamente cuando un desarrollador o administrador aprueba la build validada en el entorno de staging.
- Aprobación Manual: Antes del despliegue en producción, el pipeline requiere una aprobación explícita (Merge Request a main) para reducir el riesgo de lanzar código no deseado.
- Rollback Manual: Aunque la infraestructura de Render puede facilitar rollbacks rápidos, la decisión y ejecución de un rollback en producción son manuales y controladas por el equipo, garantizando la supervisión total ante una incidencia.

7.2.2. Stages Deployment Pipeline Components

El pipeline de Continuous Delivery está dividido en etapas claras para garantizar la calidad antes de la aprobación final. Utilizamos Jenkins para la orquestación de la fase de Integración Continua (CI), que es el pilar de este flujo.

Flujo de Integración (Jenkins)

Etapa del Pipeline	Objetivo
Checkout SCM	Obtener la version del codigo mas reciente
Compile Stage	Construir la aplicación Java/Spring Boot
Testing Stage	Ejecutar la suite de pruebas (JUnit, Mockito, Karate, Selenium)
Package Stage	Generar el artefacto final (Imagen de Docker) para el despliegue

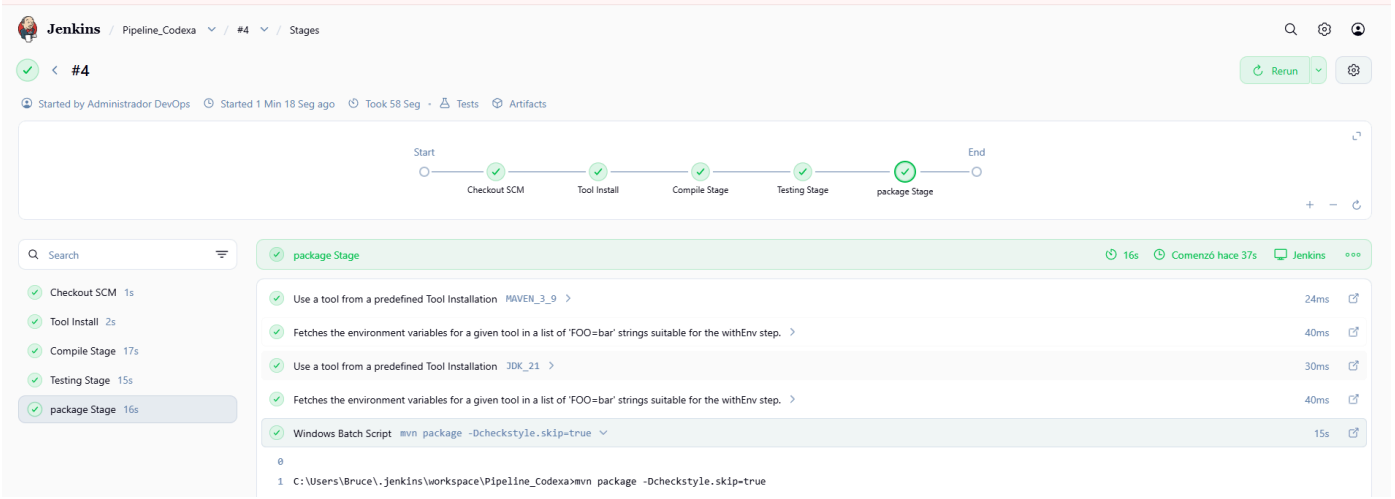
Componentes de Entrega Continua (Post-CI)

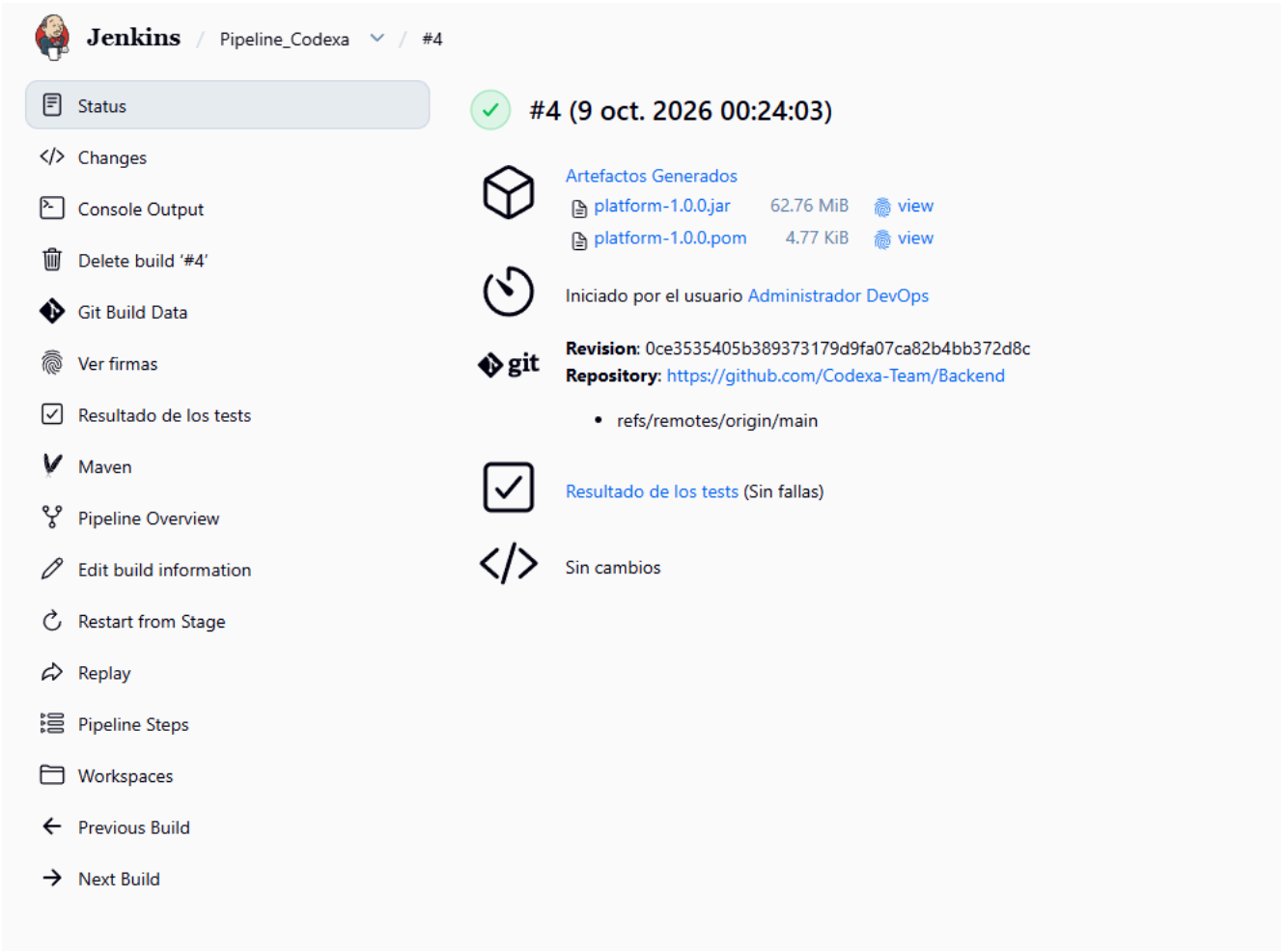
Una vez que el build es exitoso en Jenkins, el pipeline avanza a las siguientes etapas, orquestadas por GitHub Actions en conjunto con las plataformas de despliegue:

Componente	Descripcion
Despliegue a Staging	El backend y el front-end se despliegan automáticamente al entorno de Staging para la validación de pre-producción
Validacion en Staging	Se realizan pruebas de sistema y validaciones manuales/exploratorias para verificar el comportamiento de extremo a extremo
Aprobación Manual de Despliegue	Se deben revisar los resultados de las pruebas en Staging y dar la aprobación explícita para avanzar a produccion
Despliegue Controlado a Producción	Solo tras la aprobación manual, se ejecuta la acción para desplegar la build validada a los entornos de producción

Evidencias de Ejecución del Pipeline en Jenkins

Para constatar la operatividad del pipeline de CI, se documentan las evidencias de compilación exitosa y el reporte consolidado de pruebas obtenido directamente desde la interfaz web de Jenkins para el proyecto RentiCar:





7.3. Continuous deployment

El Despliegue Continuo (CD) se enfoca en que los cambios aprobados en el código pasen automáticamente desde el desarrollo hasta la producción, garantizando que cada nueva versión sea entregada sin intervención manual, siempre y cuando pase todas las pruebas

7.3.1. Tools and Practices

Herramienta	Propósito en el Despliegue automatico
Github Actions	Para automatizar el pipeline de CI/CD. Permite configurar workflows que incluyen la ejecución de pruebas y el despliegue automático a entornos
Render	Se utiliza para la automatizacion del backend RESTful API y el frontend
TiDB	Plataforma para la base de datos MySQL. Permite gestionar el despliegue de forma automatizada, con soporte para migraciones y backups
Docker	Para contenerizar la aplicación backend (Spring Boot), asegura consistencia en los entornos de desarrollo y producción

Practicas Clave:

- Feature Branching: Utilizamos ramas separadas para el desarrollo. Una vez completadas y probadas, se fusionan a la rama feature y luego main.

- Commit-based Deployment: Cada vez que se realiza un commit en la rama feat/ (o rama de integración), el pipeline de CI/CD se activa automáticamente para el despliegue a Staging.
- Rollback Automático: En caso de detectar fallos post-despliegue en entornos automáticos, el pipeline está configurado para realizar un rollback automático, restaurando la versión anterior.

7.3.2. Production Deployment Pipeline Components

Estos componentes detallan cómo se automatiza el despliegue, incluyendo la Base de Datos, el Backend y el Frontend.

Herramienta	Flujo de automatizacion
Base de Datos (TiDB)	**Gestion de migraciones:** Al modificar, agregar o hacer consultas aplica los cambios automaticamente y los guarda **Backup Automatico:** Se crean copias de seguridad antes de migraciones criticas
Front-end (Render)	**Compilacion:** Construye y compila la aplicacion en la fase o seccion de produccion solicitada **Pruebas automatizadas:** Ejecuta pruebas unitarias, E2E y Selenium **Despliegue:** Implementa la version mas reciente del codigo
Backend (Render)	**Integracion continua:** Render toma la rama a eleccion (main) y la compila **Construccion de versiones:** Crea la imagen en docker y selenium para las pruebas **Despliegue:** Implementa la version mas reciente del codigo

7.4. Continuous Monitoring

El monitoreo continuo en RentiCar supervisa la actividad del sitio web publicado en producción y define la base técnica de observabilidad y alertas del proyecto.

7.4.1. Tools and Practices

La estrategia actual combina analítica web activa en el frontend con herramientas de telemetría backend contempladas para futuras iteraciones:

Herramienta	Capa y Estado	Propósito en el Proyecto
Google Tag Manager & Google Analytics 4 (GA4)	Frontend Web (Activo)	Registra visitas y eventos base (`page_view`, `first_visit`, `scroll`, `session_start`) en el flujo `RentiCar Web` (`G-7T0SJBF5L2`) mediante `GTM-5P5DVR9F`.
Spring Boot Actuator	Backend Java (Dependencia base)	Incluido mediante `spring-boot-starter-actuator` en `Backend/pom.xml` para soporte de estado y métricas internas de la aplicación (aún sin exposición pública ni reglas conectadas).
Prometheus, Alertmanager y Grafana	Alertas y Visualización (Planteado)	Stack contemplado para recolección de métricas, evaluación de umbrales y paneles operativos del backend (pendiente de configuración y despliegue).

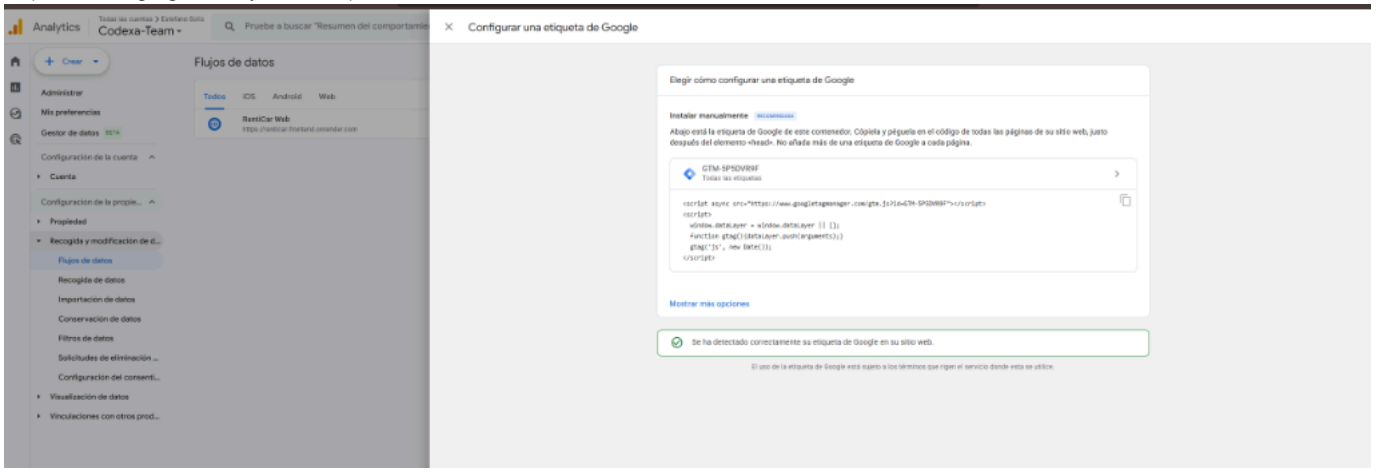
Prácticas Clave:

- Revisar el panel en tiempo real de Google Analytics 4 y los logs del servicio en Render después de desplegar cambios en `main`.
- Observar cambios bruscos en visitas o eventos base (`page_view`, `first_visit`, `scroll`, `session_start`) tras una actualización publicada.
- Contrastar el uso real registrado en GA4 con el plan de experimentación del Capítulo VIII para priorizar mejoras sobre evidencia observada.
- Mantener `spring-boot-starter-actuator` como base en el backend para acoplar Prometheus, Alertmanager y Grafana en etapas posteriores.

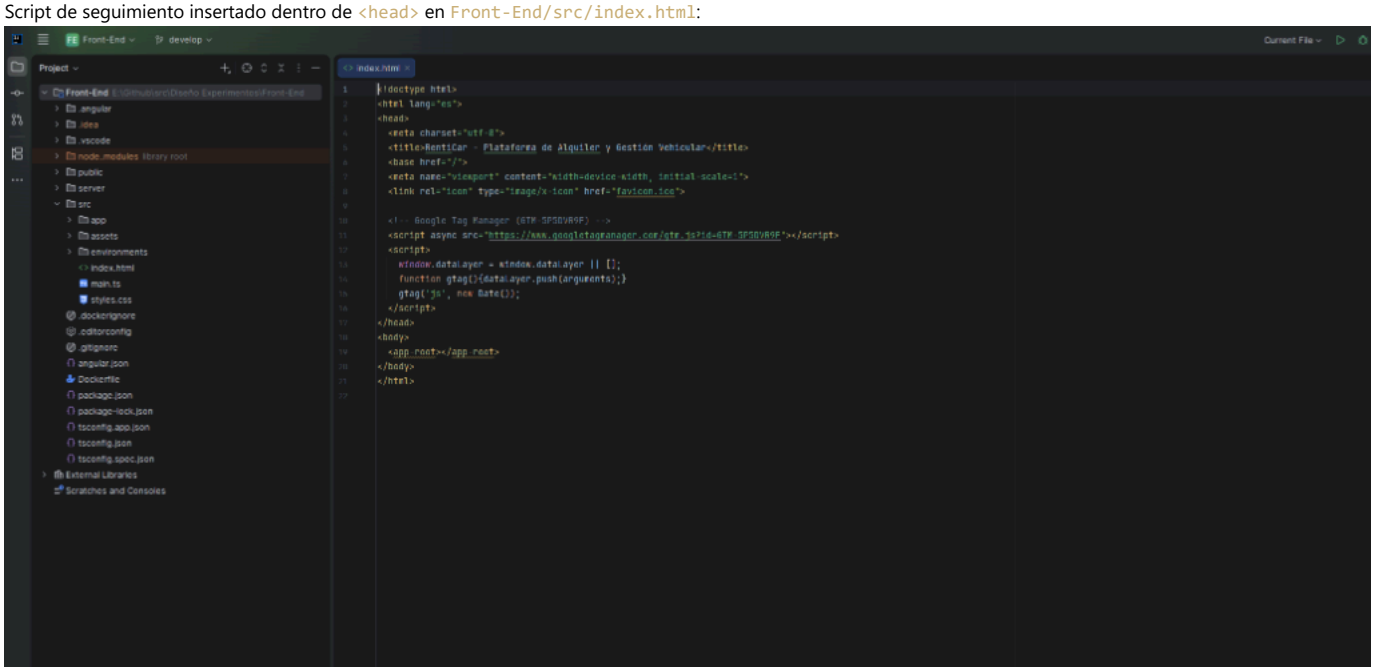
7.4.2. Monitoring Pipeline Components

El monitoreo de uso web en RentiCar se implementa con Google Tag Manager (`GTM-5P5DVR9F`) conectado al flujo **RentiCar Web** en Google Analytics 4 (`G-7T0SJBF5L2`).

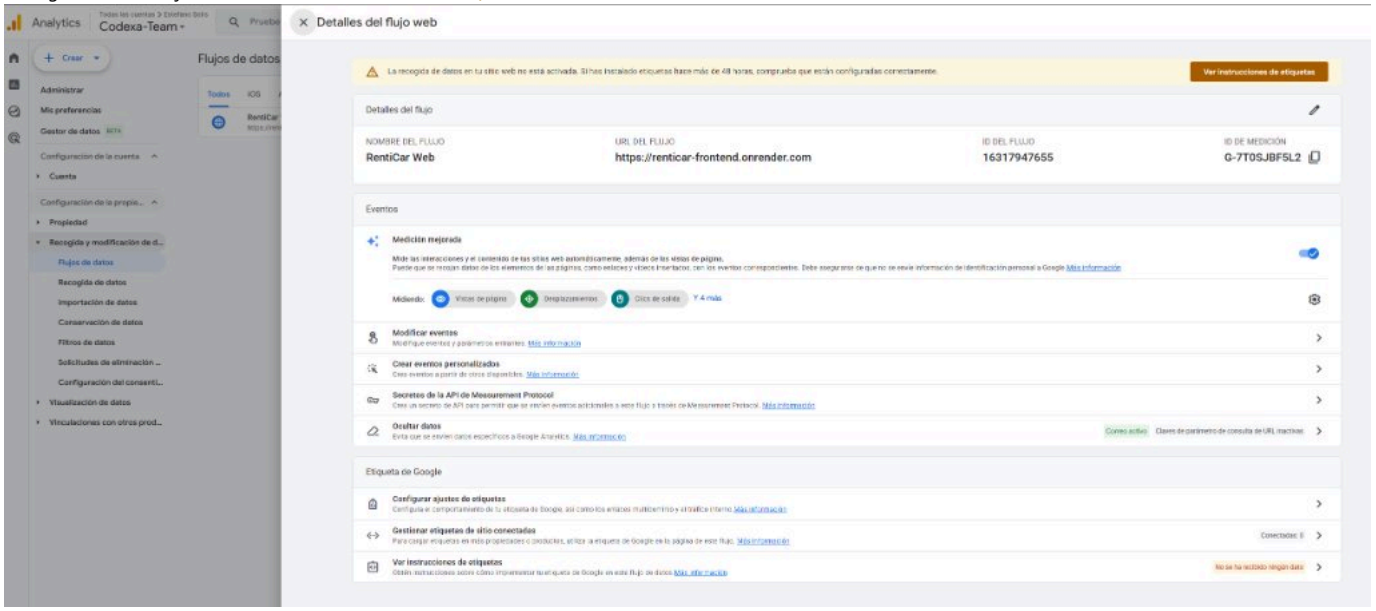
Etiqueta de Google generada y verificada para su instalación en el sitio web (`GTM-5P5DVR9F`):



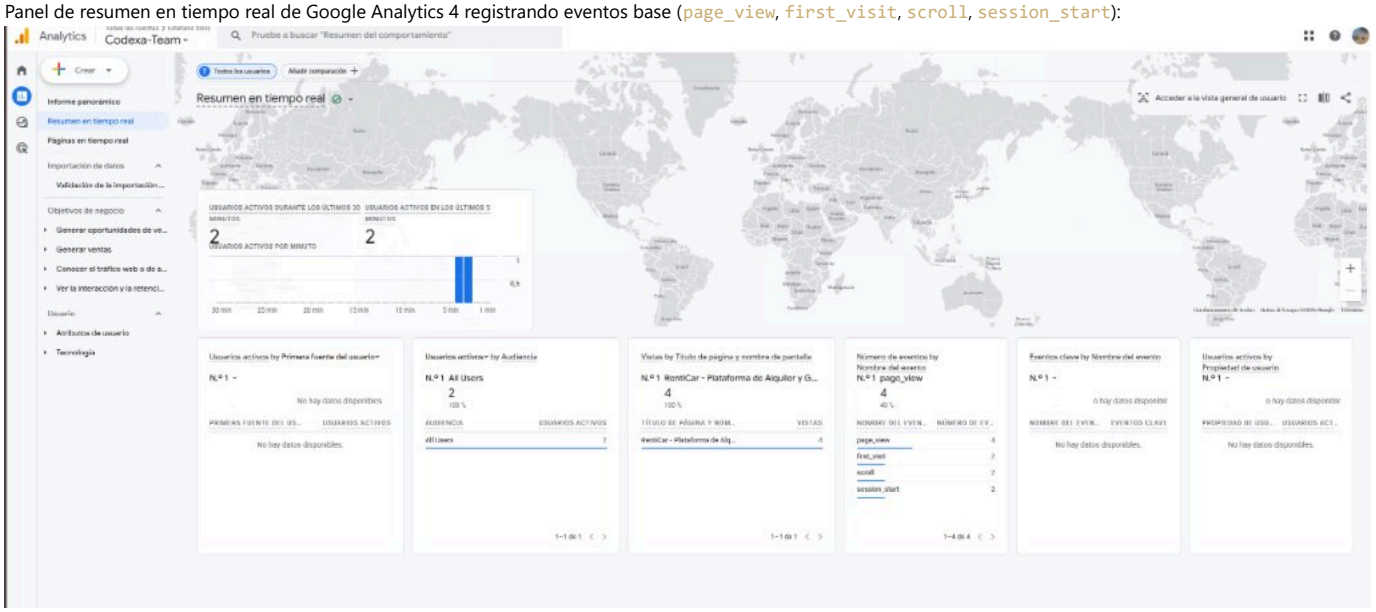
Script de seguimiento insertado dentro de `<head>` en `Front-End/src/index.html`:



Configuración del flujo de datos web `RentiCar Web` (`https://renticar-frontend.onrender.com`, ID de medición `G-7T0SJBF5L2`):



Panel de resumen en tiempo real de Google Analytics 4 registrando eventos base (`page_view`, `first_visit`, `scroll`, `session_start`):



7.4.3. Alerting Pipeline Components

El componente de alertas define la detección de fallos de disponibilidad y rendimiento a partir de las métricas expuestas por el backend (`spring-boot-starter-actuator` en `Backend/pom.xml`), planteado sobre las siguientes herramientas (aún sin archivo de reglas o despliegue en infraestructura):

- **Prometheus con Alertmanager:** Recopila métricas desde los endpoints de Actuator y evalúa umbrales de error o latencia para enrutar alertas según su gravedad hacia los canales del equipo.
- **Grafana:** Consolida la visualización de series temporales y la definición de alertas visuales sobre paneles operativos del backend Spring Boot.

7.4.4. Notification Pipeline Components

El componente de notificaciones articula la comunicación del estado del pipeline de CI y de los eventos críticos hacia los canales de coordinación del equipo (`Discord` y `WhatsApp`, según la sección 5.1.1):

- **Jenkins (`Backend/Jenkinsfile`):** Ejecuta las etapas de compilación, pruebas y empaquetado (`Compile Stage`, `Testing Stage`, `package Stage`) y permite acoplar notificaciones automáticas ante fallos de build o regresión (webhooks externos pendientes de configuración).
- **Alertas Operativas en Tiempo Real:** Define el envío de avisos automáticos ante anomalías detectadas en los despliegues de Render o caídas de tráfico observadas en Google Analytics 4.

Capítulo VIII: Experiment-Driven Development

8.1. Experiment Planning

Esta sección detalla la planificación de los experimentos diseñados para validar las hipótesis del proyecto RentiCar. El objetivo es transformar los supuestos de negocio y las suposiciones de los usuarios en conocimiento validado, permitiendo un desarrollo guiado por datos.

8.1.1. As-Is Summary

El estado actual del mercado de alquiler de vehículos en el Perú se caracteriza por una alta informalidad y una marcada desconfianza entre los actores. Los propietarios de vehículos que buscan generar ingresos adicionales se ven obligados a utilizar canales poco seguros como redes sociales y contactos directos, sin garantías contractuales ni herramientas para verificar la identidad de los arrendatarios. Por su parte, los arrendatarios enfrentan procesos engorrosos, falta de transparencia en los precios y un alto riesgo al entregar depósitos a desconocidos. Esta situación, descrita en el análisis de entrevistas del Capítulo II, genera una fricción constante que limita el crecimiento del mercado y expone a ambas partes a riesgos financieros y de seguridad.

8.1.2. Raw Material: Assumptions, Knowledge Gaps, Ideas, Claims

A partir del Lean UX Process (sección 1.2.2), se identificaron los siguientes elementos como material base para la experimentación:

1. Assumptions (Suposiciones):

- Confianza: Los arrendatarios valoran más un sistema de reputación y perfiles verificados que un precio ligeramente inferior ofrecido en canales informales.
- Seguridad del Propietario: Los propietarios estarán dispuestos a listar sus vehículos si cuentan con herramientas de monitoreo (IoT) y un sistema de contratos digitales que les brinde respaldo legal.
- Adopción: La interfaz de la plataforma será lo suficientemente intuitiva como para que los usuarios migren desde sus métodos tradicionales (Facebook Marketplace, WhatsApp) a RentiCar.

2. Knowledge Gaps (Brechas de Conocimiento):

- No se conoce el grado de influencia que tiene la presencia de telemetría IoT en la decisión final del propietario para alquilar su auto.
- Se desconoce la tarifa mínima (comisión) que los propietarios están dispuestos a pagar por la seguridad y visibilidad que ofrece la plataforma.
- No está claro qué tan dispuestos están los arrendatarios a pagar una tarifa premium por un seguro de cobertura total ofrecido dentro de la plataforma.

3. Ideas:

- Implementar un sistema de calificaciones bidireccional (propietario a arrendatario y viceversa) para fomentar la responsabilidad.
- Ofrecer un panel de control con telemetría en tiempo real (ubicación GPS, nivel de combustible) como diferenciador clave.
- Integrar contratos digitales automáticos al confirmar una reserva.

4. Claims (Declaraciones):

- RentiCar formalizará el mercado de alquiler de vehículos en Perú, reduciendo el fraude y aumentando la confianza.
- La digitalización del proceso de reserva y contrato reducirá significativamente el tiempo de gestión para ambas partes.

8.1.3. Experiment-Ready Questions

De las brechas de conocimiento, se formularon las siguientes preguntas listas para experimentación:

- Q1 (Confianza/Cliente): ¿Aumentará la tasa de conversión de reservas la implementación de un sistema de reseñas verificadas y perfiles de usuario detallados?
- Q2 (Valor/Propietario): ¿Qué valor relativo otorgan los propietarios a las funcionalidades de monitoreo IoT frente a una reducción en la comisión por servicio?
- Q3 (Retención/Propietario): ¿Estarán los propietarios dispuestos a mantener sus vehículos publicados en la plataforma tras la primera experiencia de alquiler si se les ofrece un seguro de daños?

- Q4 (Adopción/Cliente): ¿Incrementará la implementación de filtros de búsqueda por marca, precio y fechas la tasa de solicitudes de reserva en comparación con una búsqueda simple por lista?

8.1.4. Question Backlog

Priorizando las preguntas según su impacto en el modelo de negocio, se estableció el siguiente backlog:

Prioridad	ID	Pregunta(Experiment-Ready Question)
Alta	*Q1*	¿Aumentará la tasa de conversión de reservas la implementación de un sistema de reseñas verificadas y perfiles de usuario detallados?
Alta	*Q4*	¿Incrementará la implementación de filtros de búsqueda (marca, precio, fechas) la tasa de solicitudes de reserva en comparación con una búsqueda simple por lista?
Media	*Q2*	¿Qué valor relativo otorgan los propietarios a las funcionalidades de monitoreo IoT frente a una reducción en la comisión por servicio?
Baja	*Q3*	¿Estarán los propietarios dispuestos a mantener sus vehículos publicados en la plataforma tras la primera experiencia de alquiler si se les ofrece un seguro de daños?

Para priorizar las preguntas de experimentación, se utilizó un enfoque de scoring basado en cuatro criterios: Confianza (C), Riesgo (R), Impacto (I) e Interés (IN). Cada criterio se evaluó en una escala del 1 al 5, donde 5 representa el valor más alto. El puntaje total se calculó como la suma de los cuatro criterios, y la prioridad se asignó en orden descendente. Adicionalmente, se definió un criterio de desempate en caso de puntajes iguales.

Tabla de Priorización del Backlog de Preguntas

ID	Pregunta Corta	Confianza (C)	Riesgo (R)	Impacto (I)	Interés (IN)	Total (S_total)	Prioridad	Criterio de Orden / Desempate
Q1	¿Aumentará la tasa de conversión de reservas la implementación de un sistema de reseñas verificadas y perfiles de usuario detallados?	3	5	5	5	18	1	Mayor puntaje total absoluto.
Q4	¿Incrementará la implementación de filtros de búsqueda (marca, precio, fechas) la tasa de solicitudes de reserva en comparación con una búsqueda simple por lista?	3	4	5	5	17	2	Empate a 17. Mayor R = 4.
Q2	¿Qué valor relativo otorgan los propietarios a las funcionalidades de monitoreo IoT frente a una reducción en la comisión por servicio?	2	5	5	4	16	3	Empate a 16. Mayor R = 5.
Q3	¿Estarán los propietarios dispuestos a mantener sus vehículos publicados en la plataforma tras la primera experiencia de alquiler si se les ofrece un seguro de daños?	3	4	4	4	15	4	Empate a 15. Mayor R = 4.

8.1.5. Experiment Cards

Basados en las preguntas establecidas y ordenadas por orden de importancia, diseñamos los siguientes experimentos:

Elemento	Experiment Card #1: Validación de Confianza del Arrendatario
Hypothesis	*(Basándonos en Q1)*
Experiment	Creemos que los arrendatarios (Clientes) se sentirán más seguros y aumentarán su intención de reserva si la plataforma muestra perfiles de propietarios con reseñas verificadas, en lugar de solo información básica. Para verificar esto, realizaremos una prueba A/B con un prototipo funcional del catálogo de vehículos. Les mostraremos al Grupo A (control) una tarjeta con la información del vehículo y el precio. Les mostraremos al Grupo B (experimental) la misma tarjeta, pero con la adición de una insignia de "Propietario Verificado" y su calificación promedio en estrellas.
Metrics	Mediremos la Tasa de Clics (CTR) en el botón "Reservar" y la Tasa de Conversión (solicitud de reserva completada) por cada variante. También se realizarán encuestas de salida para medir la confianza percibida (escala 1-5).
Success Criteria	Sabremos que la hipótesis es válida si el Grupo B registra una tasa de conversión al menos un 20% superior a la del Grupo A y si la puntuación de confianza promedio reportada es de 4 o superior.
Elemento	**Experiment Card #2: Validación de la Usabilidad del Catálogo (Arrendatario)**
Hypothesis	*(Basándonos en Q4)*
Experiment	Creemos que los arrendatarios completarán el proceso de reserva más rápidamente si el catálogo incluye filtros por marca, precio y fechas, en lugar de una lista simple. Para verificar esto, realizaremos una prueba de usabilidad con 10 participantes del segmento de arrendatarios, divididos en dos grupos. Les pediremos que encuentren un vehículo de una marca específica para un rango de fechas determinado. El Grupo de Control usará una versión del catálogo con una barra de búsqueda simple; el Grupo Experimental usará la versión con filtros dinámicos.

Elemento	Experiment Card #2: Validación de la Usabilidad del Catálogo (Arrendatario)
Metrics	Mediremos el Tiempo Promedio para completar la tarea (en segundos) y la Tasa de Éxito (porcentaje de usuarios que completan la reserva correctamente). Se recopilarán comentarios cualitativos sobre la facilidad de uso.
Success Criteria	Sabremos que la hipótesis es válida si el tiempo promedio de la tarea se reduce en un 30% en comparación con el grupo de control y si la tasa de éxito es superior al 80%.
Elemento	**Experiment Card #3: Validación del Valor del Monitoreo IoT (Propietario)**
Hypothesis	*(Basándonos en Q2)*
Experiment	Creemos que los propietarios de vehículos (Arrendadores) se sentirán más seguros y dispuestos a publicar sus autos si la plataforma les ofrece funcionalidades de monitoreo telemático IoT en tiempo real, en lugar de solo un contrato digital. Para verificar esto, realizaremos una prueba de concepto con dos grupos de propietarios. Les presentaremos al Grupo A (control) la plataforma con las funcionalidades básicas de publicación y contrato digital. Les presentaremos al Grupo B (experimental) la misma plataforma, pero con una demostración interactiva del panel de monitoreo IoT (visualización de GPS, odómetro y nivel de combustible en un mapa). Tras la demostración, les pediremos que completen el proceso de publicación de un vehículo ficticio.
Metrics	Mediremos la Tasa de Publicación (porcentaje de usuarios que completan el formulario de publicación) en cada grupo. Adicionalmente, se realizará una encuesta para medir la "percepción de seguridad" (escala 1-5) y la "intención de listar un vehículo real" en los próximos 30 días.
Success Criteria	Sabremos que la hipótesis es válida si el Grupo B presenta una Tasa de Publicación al menos un 25% superior a la del Grupo A, y si la percepción de seguridad promedio es de 4 o superior, demostrando que el IoT es un diferenciador clave para la captación de oferta.
Elemento	**Experiment Card #4: Validación de la Retención de Propietarios (Propietario)**
Hypothesis	*(Basándonos en Q3)*
Experiment	Creemos que los propietarios que han completado su primer alquiler en RentiCar mantendrán sus vehículos publicados si la plataforma les ofrece un seguro de daños integrado y una gestión de disputas simplificada, en lugar de solo el sistema de reseñas. Para verificar esto, realizaremos una prueba de concepto con dos grupos de propietarios. Les presentaremos al Grupo A (control) la plataforma con las funcionalidades básicas de publicación y contrato digital. Les presentaremos al Grupo B (experimental) la misma plataforma, pero con una demostración interactiva del panel de monitoreo IoT (visualización de GPS, odómetro y nivel de combustible en un mapa). Tras la demostración, se les pedirá que completen el proceso de publicación de un vehículo ficticio.
Metrics	Mediremos la Tasa de Retención (porcentaje de propietarios que reactivan su publicación en los 3 meses siguientes) para cada grupo. También registraremos el NPS (Net Promoter Score) y el número de tickets de soporte relacionados con daños o disputas.
Success Criteria	Sabremos que la hipótesis es válida si el Grupo B muestra una Tasa de Retención al menos un 15% superior a la del Grupo A y si el NPS del Grupo B es significativamente mayor, confirmando que un ecosistema de seguridad post-alquiler es un factor decisivo para la fidelización de los propietarios.

8.2. Experiment Design

Esta fase consiste en detallar el diseño técnico de los experimentos que surgen de las Experiment-Ready Questions de la sección anterior. Se busca así construir un marco metodológico sólido que permita confirmar o descartar las hipótesis críticas del negocio con significancia estadística.

8.2.1. Hypotheses

A continuación, se formalizan las hipótesis nulas (H0) y alternativas (H1) para las preguntas establecidas, definiendo así la base para la validación estadística.

- **Hipótesis 1 (Adopción del Cliente - Confianza):**
  - **Creemos que** al implementar un sistema de insignias de "Propietario Verificado" (basado en la validación de identidad y documentos) y mostrar las reseñas de forma prominente en el perfil del arrendador, los arrendatarios (Persona: Sebastián Altamirano) experimentarán una mayor confianza en la plataforma.
  - **Resultará en** un incremento en la tasa de conversión de visitas a solicitudes de reserva, reduciendo la percepción de riesgo asociada a los canales informales.
  - **Sabremos que esto es cierto si** la Variante B (con insignias y reseñas destacadas) logra una tasa de conversión de "visita a perfil" a "solicitud de reserva" al menos un 20% mayor que la Variante A (perfil estándar), en un período de 14 días.
- **Hipótesis 2 (Eficiencia del Catálogo - Usabilidad):**
  - **Creemos que** al implementar un catálogo con filtros avanzados por marca, rango de precio y fechas de disponibilidad, los arrendatarios (Persona: Gino Ábalos) podrán encontrar el vehículo deseado de manera más ágil y eficiente.
  - **Resultará en** una reducción significativa del tiempo requerido para completar una reserva, mejorando la experiencia de usuario y disminuyendo la frustración asociada a la búsqueda manual.
  - **Sabremos que esto es cierto si** los usuarios expuestos a la Variante B (catálogo con filtros) completan la tarea de reserva en un tiempo promedio al menos un 30% menor que el Grupo A (catálogo con búsqueda simple), durante las pruebas de usabilidad.
- **Hipótesis 3 (Captación del Propietario - Valor del IoT):**

- **Creemos que** al ofrecer a los propietarios de vehículos (Persona: Oriana Cáceres) una demostración interactiva del panel de monitoreo telemático IoT (visualización de GPS, odómetro y nivel de combustible), percibirán un mayor valor en la plataforma y se sentirán más seguros al momento de listar sus autos.
- **Resultará en** un incremento en la tasa de publicación de vehículos, ya que el IoT mitiga el miedo al mal uso o robo del patrimonio.
- **Sabremos que esto es cierto si** el Grupo B (expuesto a la funcionalidad IoT) presenta una Tasa de Publicación al menos un 25% superior a la del Grupo A (sin la funcionalidad), y si la percepción de seguridad reportada es de 4 o superior en una escala de 1 a 5.

- **Hipótesis 4 (Retención del Propietario - Ecosistema de Seguridad):**
  - **Creemos que** al proporcionar a los propietarios un paquete de "Protección RentiCar" (que incluye un seguro de cobertura de daños y un canal de soporte prioritario para disputas), se reducirá la incertidumbre post-alquiler y se fomentará la fidelización.
  - **Resultará en** una mayor disposición a mantener los vehículos publicados y a reactivar las publicaciones tras el primer alquiler, consolidando una oferta estable en la plataforma.
  - **Sabremos que esto es cierto si** los propietarios expuestos al paquete de protección (Variante B) muestran una Tasa de Retención (reactivación de publicación) al menos un 15% superior a la del grupo de control (Variante A) en los 3 meses posteriores al primer alquiler.

Hipótesis	Pregunta	Belief (Creencia)	Hypothesis (H1)	Null Hypothesis (H0)
H1 (Adopción del Cliente - Confianza)	Q1	La implementación de insignias de "Propietario Verificado" y reseñas destacadas aumentará la confianza de los arrendatarios.	La Tasa de Conversión (visita a reserva) será al menos un 20% mayor en los perfiles que muestran insignias y reseñas.	La implementación de insignias y reseñas no aumentará la Tasa de Conversión en un 20% o más.
H2 (Eficiencia del Catálogo - Usabilidad)	Q4	Un catálogo con filtros avanzados (marca, precio, fechas) es más eficiente y mejora la experiencia del usuario que una lista simple.	El Tiempo Promedio para completar una reserva se reducirá en al menos un 30% con el uso de filtros avanzados.	La implementación de filtros avanzados no reducirá el Tiempo Promedio para completar una reserva en un 30% (o más).
H3 (Captación del Propietario - Valor del IoT)	Q2	Los propietarios valoran las funcionalidades de monitoreo IoT como un factor clave para sentirse seguros al alquilar su vehículo.	Los propietarios expuestos a la funcionalidad de monitoreo IoT mostrarán una tasa de publicación de vehículos al menos un 25% superior que aquellos que no la conocen.	La exposición a la funcionalidad de monitoreo IoT no aumentará la tasa de publicación de vehículos en un 25% (o más).
H4 (Retención del Propietario - Ecosistema de Seguridad)	Q3	Un ecosistema de seguridad post-alquiler (seguro de daños y gestión de disputas) fidelizará a los propietarios, incentivando la republicación de sus vehículos.	Los propietarios que reciben el paquete "Protección RentiCar" presentarán una Tasa de Retención (reactivación de publicación) al menos un 15% superior a la del grupo de control en los 3 meses posteriores al primer alquiler.	La incorporación del paquete "Protección RentiCar" no incrementará la Tasa de Retención de los propietarios en un 15% (o más) durante los 3 meses posteriores al primer alquiler.

8.2.2. Domain Business Metrics

Las métricas de negocio a nivel de dominio que se verán impactadas por los experimentos son:

- **Tasa de Conversión de Reservas (TCR):** (Número de reservas confirmadas / Número de visitas al catálogo) * 100. Es un indicador directo de la efectividad del catálogo y la confianza que genera.
- **Tasa de Publicación de Vehículos (TPV):** (Número de vehículos publicados / Número de propietarios registrados) * 100. Mide la capacidad de la plataforma para atraer y retener la oferta.
- **Tiempo Medio de Alquiler (TMA):** Tiempo promedio desde que un usuario inicia una búsqueda hasta que confirma una reserva. Un TMA menor indica una plataforma más eficiente.
- **Tasa de Retención de Propietarios:** Porcentaje de propietarios que, tras completar su primer alquiler, publican de nuevo en la plataforma en los siguientes 3 meses.

8.2.3. Measures

Estas son las métricas de datos crudos (cuantitativas y cualitativas) que recopilaremos directamente durante el experimento para evaluar las hipótesis.

- **Para Hipótesis 1 (Confianza en Reservas - Arrendatario):**
- Clics en el botón "Reservar" (por variante A/B).
- Número de impresiones (vistas) del perfil del propietario.
- Tasa de rebote en la página de la ficha del vehículo.
- Cualitativo: Respuestas a encuestas de salida ("¿Qué tan confiable le pareció este propietario y su vehículo?").
- **Para Hipótesis 2 (Eficiencia del Catálogo - Arrendatario):**
- Tiempo promedio (en segundos) para completar la tarea de reserva.
- Número de clics realizados en la interfaz de búsqueda (filtros vs. lista).

- Tasa de éxito de la tarea (porcentaje de usuarios que completan la reserva correctamente).
- Cualitativo: Comentarios en voz alta durante las pruebas de usabilidad sobre la facilidad de uso de los filtros.
- **Para Hipótesis 3 (Captación del Propietario - Valor del IoT):**
- Tasa de Publicación (porcentaje de usuarios que completan el formulario de publicación de un vehículo ficticio).
- Número de propietarios que activan la opción de monitoreo IoT tras la demostración.
- Cualitativo: Resultados de la encuesta de percepción de seguridad (escala 1-5) y la intención de listar un vehículo real en los próximos 30 días.
- **Para Hipótesis 4 (Retención del Propietario - Ecosistema de Seguridad):**
- Tasa de Retención (porcentaje de propietarios que reactivan su publicación en los 3 meses siguientes).
- Número de tickets de soporte relacionados con daños o disputas (por grupo).
- Cualitativo: Net Promoter Score (NPS) obtenido tras la experiencia de alquiler y las entrevistas de seguimiento sobre la percepción del paquete "Protección RentiCar".

Question	Measure
Q1: ¿Aumentará la tasa de conversión de reservas la implementación de un sistema de reseñas verificadas y perfiles de usuario detallados?	Cuantitativo: Clics en el botón "Reservar" por variante. Tasa de Conversión (Visita a Perfil → Solicitud). Tasa de rebote en la página del vehículo. Cualitativo: Respuestas a encuestas de salida ("¿Qué tan confiable le pareció este propietario y su vehículo?").
Q4: ¿Incrementará la implementación de filtros de búsqueda (marca, precio, fechas) la tasa de solicitudes de reserva en comparación con una búsqueda simple por lista?	Cuantitativo: Tiempo promedio para completar la reserva (en segundos). Número de clics en la interfaz de búsqueda. Tasa de éxito de la tarea. Cualitativo: Comentarios en voz alta durante las pruebas de usabilidad sobre la facilidad de uso de los filtros.
Q2: ¿Qué valor relativo otorgan los propietarios a las funcionalidades de monitoreo IoT frente a una reducción en la comisión por servicio?	Cuantitativo: Tasa de Publicación (porcentaje de formularios completados). Número de activaciones de la opción de monitoreo IoT. Cualitativo: Resultados de la encuesta de percepción de seguridad (escala 1-5) y la intención de listar un vehículo real en los próximos 30 días.
Q3: ¿Estarán los propietarios dispuestos a mantener sus vehículos publicados en la plataforma tras la primera experiencia de alquiler si se les ofrece un seguro de daños?	Cuantitativo: Tasa de Retención (reactivación de publicación en 3 meses). Número de tickets de soporte por daños o disputas. Cualitativo: Net Promoter Score (NPS) y entrevistas de seguimiento sobre la percepción del paquete "Protección RentiCar".

8.2.4. Conditions

Definimos las condiciones del experimento (grupos de control y variantes) para aislar el impacto de los cambios introducidos en cada hipótesis, asegurando la validez de los resultados.

Experimento	Condición Experimental (Variante B)	Condición de Control (Grupo A)
Experimento 1 (Confianza en Reservas - H1)	50% de los arrendatarios visualizan el diseño To-Be: la tarjeta del vehículo incluye una insignia de "Propietario Verificado" y la calificación promedio en estrellas, con acceso directo a las reseñas destacadas.	50% de los arrendatarios visualizan el diseño As-Is: la tarjeta del vehículo muestra únicamente la información técnica básica y el precio, sin insignias de verificación ni reseñas visibles.
Experimento 2 (Eficiencia del Catálogo - H2)	50% de los arrendatarios utilizan la versión To-Be del catálogo: una interfaz con filtros dinámicos por marca, rango de precio y fechas de disponibilidad.	50% de los arrendatarios utilizan la versión As-Is del catálogo: una lista simple de vehículos con una barra de búsqueda de texto libre.
Experimento 3 (Captación del Propietario - H3)	50% de los propietarios son expuestos a la demostración interactiva del panel de monitoreo IoT (GPS, odómetro, combustible) antes de completar el formulario de publicación.	50% de los propietarios completan el formulario de publicación sin la demostración interactiva del panel de monitoreo IoT, recibiendo solo información textual sobre la funcionalidad.
Experimento 4 (Retención del Propietario - H4)	50% de los propietarios que han completado un alquiler reciben el paquete "Protección RentiCar", que incluye un seguro de cobertura de daños y un canal de soporte prioritario para disputas.	50% de los propietarios que han completado un alquiler reciben únicamente el sistema de reseñas estándar, sin acceso al paquete de protección adicional.

8.2.5. Scale Calculations and Decisions

Para asegurar la validez estadística de los resultados, se calcularon los parámetros necesarios para cada experimento.

- **Decisión de Nivel de Confianza:** 95% (p-value < 0.05).
- **Decisión de Poder Estadístico:** 80%.
- **Efecto Mínimo Detectable (MDE):**
- Para H1: Se busca detectar un incremento relativo del 20% en la tasa de conversión base, estimada en un 5%.
- Para H2: Se busca detectar una reducción relativa del 30% en el tiempo promedio de la tarea, estimado en 120 segundos.
- Para H3: Se busca detectar un incremento relativo del 25% en la tasa de publicación base, estimada en un 15%.
- Para H4: Se busca detectar una reducción relativa del 15% en la tasa de churn base de propietarios, estimada en un 20%.

- **Cálculo de Muestra:**
- Para H1: Se requiere un mínimo de 3,800 visitantes únicos por variante (Total: 7,600).
- Para H2: Se requiere un mínimo de 30 participantes por grupo en las pruebas de usabilidad (Total: 60).
- Para H3: Se requiere un mínimo de 400 propietarios por variante (Total: 800).
- Para H4: Se requiere un mínimo de 500 propietarios por variante (Total: 1,000).
- **Duración:** Los experimentos cuantitativos (H1, H3, H4) se ejecutarán durante 21 días para capturar ciclos de comportamiento completos y alcanzar el tamaño de muestra requerido.

Factor	Valor de Decisión	Justificación y Nivel
Nivel de Confianza ($1-\alpha$)	95%	Ideal. Estándar de la industria para asegurar la fiabilidad de los resultados.
Poder Estadístico ($1-\beta$)	80%	Aceptable/Ideal. Mínimo requerido para reducir la posibilidad de un Falso Negativo.
Efecto Mínimo Detectable (MDE)	H1: Lift relativo del 20% en TCR. H2: Reducción del 30% en tiempo. H3: Lift del 25% en TPV. H4: Reducción del 15% en Churn.	Ideal. Define el cambio mínimo que se considera valioso para el negocio.
Tamaño Muestral Calculado	H1: 7,600 visitantes. H2: 60 participantes. H3: 800 propietarios. H4: 1,000 propietarios.	Ideal. Muestra calculada para validar el MDE con 95% de confianza.
Duración	21 días (H1, H3, H4); 2 semanas (H2).	Ideal. Periodo suficiente para alcanzar el tamaño de muestra y cubrir ciclos de comportamiento.

8.2.6. Methods Selection

Para la ejecución de los experimentos, se evaluaron diversas herramientas de análisis y experimentación, seleccionando las que mejor se adaptan a los objetivos del proyecto RentiCar.

Herramienta	Precio	Capacidad de Análisis	Ventajas	Documentación	Sencillez
Google Analytics	Plan gratuito/Créditos gratis	Análisis cuantitativo completo y exhaustivo del tráfico y comportamiento del usuario.	Excelente para reportes. Gran capacidad de integración. Medición post-experimento.	Extensa	Aprendizaje sencillo para funciones básicas, complejidad media para análisis avanzados.
Optimizely	Basado en suscripción (Plan Starter gratuito)	Plataforma líder para Test A/B, Multivariante y Feature Flagging (Gestión de liberaciones de funcionalidad).	Ideal para la ejecución de experimentos y garantizar la validez estadística. Analítica de impacto directa.	Extensa y técnica	Requiere configuración técnica inicial, pero la interfaz de diseño de experimentos es intuitiva.
Qualtrics	Plan gratuito básico (Suscripción Enterprise)	Recolección de datos cualitativos/cuantitativos mediante encuestas avanzadas, formularios y análisis de feedback.	Herramienta de nivel empresarial para mediciones de NPS, satisfacción (CSAT) y recolección de feedback específico post-experimento.	Muy Extensa y académica	Interfaz intuitiva para encuestas, requiere curva de aprendizaje para análisis estadístico avanzado.
Hotjar	Plan gratuito con limitaciones	Análisis cualitativo (mapas de calor, grabaciones de sesión, encuestas de feedback visual).	Permite entender el "porqué" detrás del comportamiento del usuario de forma visual, fundamental para la H1 (Confianza).	Buena	Interfaz muy intuitiva y fácil de configurar.

Tras evaluar cada una de las opciones, se seleccionaron Google Analytics y Optimizely como las herramientas principales. Google Analytics permite obtener datos estadísticos relevantes y sencillos de forma gratuita, con una amplia documentación disponible. Optimizely, por su parte, facilitará la ejecución de los test A/B y la gestión de las variantes, garantizando la validez estadística de los resultados. Para la recolección de datos cualitativos, se utilizará Hotjar por su capacidad para visualizar el comportamiento del usuario y recoger feedback en el momento.

8.2.7. Data Analytics: Goals, KPIs and Metrics Selection

El objetivo analítico es determinar si las variantes (B) producen un resultado estadísticamente superior al control (A), utilizando los KPIs definidos para cada hipótesis.

Hipótesis	Objetivo Analítico	KPI Primario (Métrica de Éxito)	Métricas Secundarias (Guardrails)
H1: Confianza en Reservas	Determinar si la confianza visual (insignia y reseñas) impacta la decisión de reserva del arrendatario.	Tasa de Conversión (Solicitudes de Reserva / Visitas a Perfil).	Tasa de rebote en la página del vehículo, Tiempo en página.
H2: Eficiencia del Catálogo	Determinar si los filtros avanzados reducen el tiempo y el esfuerzo para completar una reserva.	Tiempo Promedio de la Tarea (en segundos).	Tasa de éxito de la tarea, Número de clics en la interfaz.
H3: Captación del Propietario	Determinar si la demostración del IoT influye en la decisión del propietario de publicar su vehículo.	Tasa de Publicación de Vehículos (formularios completados).	Percepción de seguridad (encuesta), Intención de listar un vehículo real.

Hipótesis	Objetivo Analítico	KPI Primario (Métrica de Éxito)	Métricas Secundarias (Guardrails)
H4: Retención del Propietario	Determinar si el paquete de protección post-alquiler reduce la incertidumbre y fomenta la fidelización.	Tasa de Retención (reactivación de publicación).	Tasa de tickets de soporte por daños/disputas, NPS.

8.2.8. Web and Mobile Tracking Plan

Para RentiCar, el objetivo es monitorear la plataforma web y móvil para validar si los cambios introducidos en la ficha de vehículos (H1) y el panel de gestión y telemetría para arrendadores (H2) generan la confianza y el valor percibido necesarios para sostener el modelo de negocio.

Estableceremos un plan de seguimiento basado en A/B Testing y analítica con Google Tag Manager (`GTM-5P5DVR9F`) y Google Analytics 4 (`GA4`) para evaluar el impacto causal de las mejoras implementadas.

1. Implementación Inicial Durante esta fase, nos enfocaremos en el lanzamiento de los diseños experimentales y la recolección de datos segmentados para establecer la causalidad de los resultados frente a la Línea Base (Control).

- **Asignación del Grupo:** Todos los eventos críticos incluirán una propiedad que identifica si el usuario pertenece al grupo Control o Experimental para garantizar que los resultados se analicen dentro de su grupo asignado.
- **Métricas de Confianza (H1):** Se capturarán las vistas de detalle de vehículo, los clics en iniciar reserva y las confirmaciones de alquiler para medir la Tasa de Conversión a Reserva.
- **Métricas de Valor/Retención (H2):** Se registrarán la interacción de los arrendadores con el panel de telemetría IoT y las acciones sobre publicaciones activas para medir la adopción de la funcionalidad y el abandono.
- **Feedback de Usuarios:** Se aplicarán encuestas breves post-interacción para obtener datos cualitativos sobre la confianza percibida tras consultar reseñas y datos del vehículo (H1).

2. Seguimiento Continuo Después de la fase inicial de validación, se establecerá un proceso continuo de seguimiento para evaluar el rendimiento a largo plazo.

- **Métricas en Tiempo Real:** Se empleará Google Analytics 4 en web y analítica de eventos en la aplicación móvil para monitorear el comportamiento de los usuarios en tiempo real.
- **Segmentación de Usuarios:** Los datos se segmentarán por tipo de usuario (**Arrendatarios** y **Arrendadores**) para entender cómo cada grupo interactúa con las funcionalidades principales.
- **Tasa de Retención a Largo Plazo:** Se medirá la Tasa de Retención de Arrendadores a lo largo del tiempo para evaluar la efectividad del seguimiento operativo e IoT (H2) en mantener a los propietarios activos en la plataforma.

Este enfoque asegurará que RentiCar tome decisiones informadas y validadas por datos, centrándose en la viabilidad económica del proyecto.

8.3. Experimentation

Esta sección traduce el diseño del experimento en los artefactos de desarrollo (Historias de Usuario y Backlog) necesarios para construir, ejecutar y medir las variantes del experimento

8.3.1. To-Be User Stories

Estas historias de usuario describen las funcionalidades específicas requeridas para implementar las variantes de nuestros experimentos

ID	Titulo	Descripcion	Criterios de aceptacion	Relacionado con (Experimento)
HU-EXP-01	Validación de confianza del propietario del vehiculo	Como propietario del vehiculo, quiero visualizar un promedio calificativo, para saber mi calificacion general como arrendador	Escenario 1: Visualización de información de confianza Given el arrendador accede a su perfil When el arrendador tenga reseñas previas de sus vehiculos alquilados Then el sistema mostrara un promedio de sus calificaciones	H1 (Confianza en Reservas)
HU-EXP-02	Validación de usabilidad del catálogo	Como arrendatario, quiero filtrar los vehículos por marca, precio y fechas disponibles, para encontrar y reservar rápidamente un vehículo adecuado	Escenario 1: Aplicación de filtros Given que el arrendatario se encuentra en el catálogo When selecciona rango de precio And aplica los filtros Then el sistema debe mostrar únicamente los vehículos disponibles que cumplan las condiciones Escenario 2: Búsqueda sin resultados Given que no existen vehículos disponibles When el arrendatario ejecuta la búsqueda	H2 (Eficiencia del Catálogo)

ID	Titulo	Descripcion	Criterios de aceptacion	Relacionado con (Experimento)
			Then el sistema debe mostrar un mensaje informativo	
HU-EXP-03	Validación del valor del monitoreo IoT	Como propietario de un vehículo, quiero visualizar una demostración del monitoreo IoT con GPS y nivel de combustible, para sentir mayor seguridad	Escenario 1: Visualización de la demostración IoT Given el automovil esta en uso When el sistema detecte algun cambio en sus parametros internos (Gasolina) Then el sistema registrara el cambio y lo mostrara	H3 (Captación del Propietario)
HU-EXP-04	Widget sobre uso y beneficios	Como usuario de Renticar, quiero una seccion que me explique el uso de la aplciacion para encontrar formas de usar la aplicacion	Escenario 1: Seccion de "Como usar" en Landing Page Given el usuario accede a la Landing Page When el usuario acceda a la seccion "Como usar" Then se mostrara como se usa la aplicacion a detalle	H4 (Retención del Propietario)

8.3.2. To-Be Product Backlog

Este es el backlog de las historias de usuario (priorizado) requerido para implementar los experimentos diseñados

Orden	User Story ID	Titulo	Story Points 1/2/3/5/8
1	HU-EXP-01	Validación de confianza del propietario del vehiculo	5
2	HU-EXP-02	Validación de usabilidad del catálogo	8
3	HU-EXP-03	Validación del valor del monitoreo IoT	8
4	HU-EXP-04	Widget sobre uso y beneficios	3

Conclusiones

AV1:

El desarrollo del proyecto Renticar permitió aplicar de manera integral los conceptos de ingeniería de software y las metodologías orientadas al diseño centrado en el usuario, integrando prácticas modernas como Event Storming, Impact Mapping y User Personas. A través del análisis de entrevistas se identificaron claramente las necesidades de los dos segmentos objetivos, arrendadores y arrendatarios, evidenciando que ambos valoran la seguridad, la transparencia y la confianza como elementos determinantes en la experiencia de alquiler de vehículos.

La incorporación de un componente IoT ligero representa un diferenciador importante frente a soluciones tradicionales, al brindar información básica sobre el estado y ubicación de los vehículos durante la reserva. De esta forma, se refuerza la confiabilidad del sistema y se atienden los puntos de dolor detectados en el análisis de los flujos actuales. Asimismo, la definición de una arquitectura basada en dominios bajo el modelo C4 (Context, Container, Component) asegura la escalabilidad y mantenibilidad del sistema, alineando la solución tecnológica con los objetivos de negocio.

TB1:

Durante la segunda etapa del proyecto se fortaleció la calidad y confiabilidad de RentiCar mediante la implementación de pruebas unitarias, pruebas de integración y prácticas de Behavior-Driven Development. Las pruebas unitarias con JUnit 5 y Mockito permitieron validar de manera aislada la lógica de los principales bounded contexts, como IAM, Listings, Booking, IoT y Reviews. Asimismo, las pruebas de integración desarrolladas con Karate y Gherkin verificaron el comportamiento de los endpoints REST, la autenticación JWT, la gestión de vehículos, las reservas, la telemetría y las reseñas, incluyendo escenarios exitosos y casos de error relacionados con permisos, datos inválidos y recursos inexistentes.

La configuración del pipeline de Integración Continua en Jenkins permitió automatizar la compilación, ejecución de pruebas, generación de reportes y validación de cada cambio incorporado al backend. Esto contribuyó a detectar errores de forma temprana, reducir riesgos durante el despliegue y mantener una mayor estabilidad del producto. La integración de prácticas TDD y BDD también permitió relacionar la implementación técnica con los criterios de aceptación definidos para las user stories y con las necesidades reales de los usuarios.

AV2:

RentiCar evidencia un proceso progresivo de verificación, automatización y experimentación. La aplicación de pruebas unitarias, de integración, BDD y pruebas de sistema permitió comprobar el funcionamiento de los principales componentes y flujos de la plataforma, mientras que las entrevistas y evaluaciones de experiencia ayudaron a identificar oportunidades de mejora desde la perspectiva de los usuarios.

Además, la incorporación de prácticas DevOps fortaleció la integración, entrega, despliegue y monitoreo del producto. La automatización de pruebas, el análisis de calidad, el uso de pipelines y las herramientas de analítica establecieron una base para mantener la estabilidad del sistema y observar su comportamiento en entornos reales.

Bibliografía

- Cohn, M. (2004). User Stories Applied: For Agile Software Development. Addison-Wesley Professional.
- Evans, E. (2004). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.
- Adzic, G. (2012). Impact Mapping: Making a Big Impact with Software Products and Projects. Provoking Thoughts.
- Fowler, M. (2004). Patterns of Enterprise Application Architecture. Addison-Wesley.
- Stickdorn, M., Hormess, M., Lawrence, A., & Schneider, J. (2018). This is Service Design Doing: Applying Service Design Thinking in the Real World. O'Reilly Media.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Anexos

- Enlace a la organizacion
- Enlace al repositorio del Frontend
- Enlace del Frontend Desplegado
- Enlace al repositorio del Backend
- Enlace del Backend Desplegado
- Enlace al repositorio de la landing Page
- Enlace de la Landing Page Desplegada
- Enlace de al repositorio del Reporte
- Enlace de al Canva de presentacion